Machine Learning Foundations

The full teaching manual for the 11-session Machine Learning Foundations kit (Modules 0 to 6; one 3-hour session, two lessons each). Timing, live-demo scripts, the mistakes to pre-empt, lab facilitation, and checks for understanding, for every session.

11 sessions · 21 lessons | 11 sessions × 3 h · 33 h | Instructor copy · not for students

CONTENTS

S01 **Your ML Toolkit and the Estimator API**

- 0.1 Your ML Toolkit
- 1.1 The scikit-learn Estimator API

S02 **Splits, Leakage and Cross-Validation**

- 1.2 Splits, Baselines and Leakage
- 1.3 Cross-Validation

S03 **Metrics and Preprocessing**

- 1.4 Metrics for Classification and Regression
- 2.1 Impute, Scale and Encode

S04 **Pipelines and Creating Features**

- 2.2 Pipelines
- 2.3 Creating Features

S05 **Feature Selection and Imbalanced Classes**

- 2.4 Feature Selection
- 2.5 Imbalanced Classes

S06 **Linear Models and Regularization**

- 3.1 Linear and Logistic Regression
- 3.2 Regularization and the Bias-Variance Trade-off

S07 **Nearest Neighbors and Naive Bayes**

- 4.1 k-Nearest Neighbors

4.2 Naive Bayes

S08 Support Vector Machines and Decision Trees

4.3 Support Vector Machines

5.1 Decision Trees

S09 Random Forests and Gradient Boosting

5.2 Bagging and Random Forests

5.3 Gradient Boosting

S10 Ensembles and Model Selection

5.4 Voting and Stacking

6.1 Model Selection and Learning Curves

S11 Hyperparameter Tuning and Final Project Studio

6.2 Hyperparameter Tuning

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 1: Your ML Toolkit and the Estimator API

How to teach the first session of Machine Learning Foundations. Welcome a room of people who already use pandas; prove that every machine has a working machine-learning environment (scikit-learn and PyTorch) and the course data; find out what they remember; then read Pinecrest's data the way a data scientist does (gaps, targets, rare classes) and train the first models with the four method names every scikit-learn model shares. Learners leave having seen that doing nothing scores 83.5% on churn and 99.1% on fraud, and that a model barely better on accuracy can rank customers several times better than a random list.

`2 lessons` `3 hrs session` `Lessons 0.1–1.1` `Session 1 of 11`

HOW TO USE THIS GUIDE

- Same rhythm for both lessons: frame (the goal and one mental model) → live demo (you type, they type along) → guided lab (a kit lab, new practice on the lesson's skills) → start the lesson's own project. The lesson exercises, quiz and journal are homework.
- Type, do not paste. Every code block in a demo script below was run to produce the output printed under it (Python 3.12.14, pandas 3.0.6, scikit-learn 1.9.1, PyTorch 2.14.1 on 4 CPU threads). Type it live, ask for a prediction before you run it, and read errors from the last line up, out loud.
- The lessons are the reference. Keep `getting_started.html` and `basics_sklearn.html` open in browser tabs for the pictures (the setup map, the workflow loop, the estimator API diagram, the risk-decile chart). Labs live in `labs/module-01/`: `_starter.py` files are handouts, `_solution.py` files are yours.
- Session 1 is different in one way: it opens with 30 minutes of welcome, environment check and diagnostic instead of the usual 10-minute retrieval warm-up. Lesson 0.1's installation sections are done before the session from the setup guide; the room spends its 0.1 time using the toolkit, not installing it.
- Teal blocks are hands-on, amber blocks are watch-outs. Lesson codes (0.1, 1.1) match the course site.

THE #1 RISK OF SESSION 1: THE INSTALL EATS THE SESSION

The machine-learning environment is big: about 2 GB on disk once PyTorch, XGBoost, LightGBM, SHAP and MLflow are in. On Linux the default PyTorch download adds several gigabytes of GPU libraries nobody in this course needs. Install it together on venue Wi-Fi and you will still be fixing laptops at the break, and Lesson 1.1, the lesson that makes people want to come back, will not happen. **Before:** send the setup guide a week ahead and ask every learner to reply with the last line of `check_environment.py`; chase the ones who do not reply. **In the room:** the check gets 12 minutes and no more; a machine that is not ready gets a partner, not your attention. **After:** the NOT READY output is a numbered list the learner can follow at home, and you check it again at the start of Session 2. The second risk is quieter: a notebook kernel that is not the course environment. That learner gets through today on an older scikit-learn and then sees slightly different numbers from the lesson pages for weeks. The version lines in the check and in demo Step 1 are there to catch exactly that.

BEFORE THE SESSION

- ☐ **A week before:** email the setup guide (the kit's Setup Guide, which follows Lesson 0.1: Python 3.12 or newer, a `pinecrest-ml` folder, `.venv`, on Linux the CPU-only PyTorch line FIRST, then `python -m pip install -r requirements.txt`, then `pinecrest-ml-data.zip` unzipped into the folder). Attach `labs/module-01/l0_1-orientation/check_environment.py` and ask learners to run it and reply with its last line. No reply means an install risk: chase it.
- ☐ **Files to hand out** (shared folder or USB stick, in case the network fails): `requirements.txt`, `pinecrest-ml-data.zip` (1.4 MB), the whole `labs/` folder **without** the `*_solution.py` files (keep `labs_paths.py` in it: every lab imports it), and the workbook. Learners put `labs/` inside `pinecrest-ml`, next to `data/`, so the labs find the data on their own.
- ☐ **Run the kit check once:** `python3 labs/check_labs.py module-01` from this kit's folder. Every line should say PASS (10 files, about 20 seconds). Run `python labs/module-01/l0_1-orientation/check_environment.py` on the projector machine so you can show what READY looks like (the first run takes up to a minute: PyTorch and MLflow import slowly).
- ☐ **Downloads:** none today, and none for the whole Foundations kit. The image datasets and pretrained networks (about 175 MB) are only needed from Module 9 on, in the Advanced kit.
- ☐ **Projector machine:** a terminal with `(.venv)` active in your own `pinecrest-ml`, JupyterLab (or VS Code) started from that folder, font size 18 pt or more, two empty notebooks ready: `lesson_0_1` and `lesson_1_1`. Browser tabs: the course home page, `getting_started.html`, `basics_sklearn.html`, and `data/README.md`.
- ☐ **Board, before learners arrive:** the five-step routine (terminal → `cd pinecrest-ml` → activate, see `(.venv)` → start JupyterLab or VS Code here → kernel is `.venv`); the six workflow steps with the arrow from Evaluate back to Prepare; and an empty box headed "Every estimator" for the 1.1 verbs.

- ☐ **Print or share:** the workbook (its Start Here page has the diagnostic), and sticky notes for help flags and the exit ticket.

KNOWN ENVIRONMENT TRAPS

SYMPTOM	CAUSE	FIX
On Linux, `pip install -r requirements.txt` starts downloading files named `nvidia-...` or `...cuda...`, several GB in all	pip chose the GPU build of PyTorch	Ctrl+C. Run the CPU line from Lesson 0.1 first (`python -m pip install torch~=2.14.0 torchvision~=0.29.0 --index-url https://download.pytorch.org/whl/cpu`), then the requirements line again.
`Activate.ps1 cannot be loaded because running scripts is disabled on this system`	PowerShell's default execution policy	Once per user: `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`, answer Y, activate again.
`ModuleNotFoundError: No module named 'sklearn'` in a terminal	The environment is not active in this window	Activate it; look for `(.venv)`. Activation is per terminal window.
`ModuleNotFoundError` in a notebook although the terminal works; or scikit-learn 1.8 / pandas 2.x in the version check	The notebook kernel is a different Python	JupyterLab: start it from the activated terminal. VS Code: Select Kernel ▸ the entry that mentions `.venv`.
`ModuleNotFoundError: No module named 'scikit_learn'`	The install name was used as the import name	`import sklearn` (install `scikit-learn`, import `sklearn`).
`FileNotFoundError: ... 'data/customers_churn.csv'`	Python is working in another folder, or the zip was extracted into a nested folder	`print(Path.cwd())`. Start JupyterLab from `pinecrest-ml`; move files so that `pinecrest-ml/data/customers_churn.csv` exists.
`ERROR: Could not find a version that satisfies the requirement torch~=2.14.0`	Python older than 3.12, a 32-bit Python, or a very new Python the pins do not cover yet	`python --version`. Install 64-bit Python 3.12 (the version the course is tested with), delete `.venv`, create it again, reinstall.
`check_environment.py` prints nothing for 30 seconds or more	The first import of PyTorch and MLflow builds caches	Wait. The second run is much faster.

SYMPTOM	CAUSE	FIX
The install is very slow or fails halfway (venue Wi-Fi)	Several hundred megabytes of downloads	Do not install in the room. This is why the setup guide goes out a week ahead. Today: pair up; install at home.
A locked work laptop: nothing can be installed	No administrator rights	Pair today. Before Session 2: the python.org installer's per-user option usually needs no admin rights; otherwise their IT team. A hosted notebook works if they upload the data folder, but its versions differ and some numbers will not match the pages.

SESSION 1 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	30	Welcome, environment check and diagnostic	Introductions, how the course runs, `check_environment.py` on every machine, a six-question diagnostic of Python and pandas (not graded).
0:30	10	**0.1** Frame and mental model	The workshop and its routine; the six-step workflow on the board, with its loop.
0:40	20	**0.1** Live demo	Versions, the data folder, the customer table, its gaps, three targets of three kinds, three errors on purpose.
1:00	15	**0.1** Guided lab	`labs/module-01/l0_1-orientation/missing_values_starter.py`: why is it missing?
1:15	10	**0.1** Start the lesson project	The data card, Steps 1 and 2; Steps 3 to 6 go home.
1:25	10	Break	Fix NOT READY machines now. Leave the workflow and the routine on the board.
1:35	10	**1.1** Frame and mental model	The new hire and the case files: fit, predict, score; grade on unseen files.
1:45	30	**1.1** Live demo	X and y, the split, a baseline, the API, a scaler, logistic regression, probabilities and deciles, four models in one loop.
2:15	20	**1.1** Guided lab	`labs/module-01/l1_1-estimator-api/fraud_baseline_starter.py`, then `labs/module-01/l1_1-estimator-api/new_customers_starter.py` in pairs.

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
2:35	15	**1.1** Start the lesson project	The churn-risk shortlist, Steps 1 to 3; Steps 4 to 6 go home.
2:50	10	Wrap-up	Exit ticket (3 questions), homework, bring list for Session 2.
Total	180		Ends at 3:00.

In the room. The lesson pages estimate 90 + 90 = 180 minutes for a solo learner, and after the 30-minute opening, the break and the wrap-up the room has 130. So the room does, for each lesson: the mental model, the live demo, one guided kit lab, and the first steps of the lesson project. Lesson 0.1's installation sections (the folder, the environment, the packages, the data download) are prework, verified by the environment check, and are not repeated in the room.

Goes home (2 to 3 hours). The data card Steps 3 to 6 and the churn-risk shortlist Steps 4 to 6; each lesson's Hands-on Exercises (three, plus a stretch); both Quick Quizzes; both journal entries; Lesson 1.1's "Before the Next Lesson" list. Anyone whose check said NOT READY also fixes their machine.

Protect, in this order: the environment check (nobody leaves without knowing whether their machine is ready and what to do if not); the 1.1 demo from the baseline to the deciles (Steps 3 and 8 to 10: the baseline and "barely better on accuracy, very useful as a ranking" is the idea the whole course leans on); Step 4 of the missing-values lab (the leak). **Cut first:** the 0.1 demo Steps 11 to 13 (the date error and the script; say one sentence about each), then the 0.1 project start (send all six steps home), then 1.1 demo Step 12 (the weights).

OPENING SEGMENT

0:00 Welcome, Environment Check and Diagnostic — 30 min

TIMING

5m	Welcome and introductions
5m	How the course runs: 11 sessions, two lessons each, the room and home split, the journal
12m	Environment check: `check_environment.py` ends with READY on every machine (or the machine gets a helper)
8m	Diagnostic: six predict-the-output questions on Python and pandas, not graded

SEQUENCE AND TALKING POINTS

- **Welcome (5m).** One minute on yourself, including a time a model (or a rule of thumb) turned out to be wrong in an instructive way. Then a fast round: name, where they use Python today, and one thing they would like a model to predict by the end of the course. Write three or four of those on the board; they come back in today's first journal prompt. Housekeeping in one breath: break at 1:25, where the lab files are, and "if you are stuck, put a sticky note on your laptop lid and keep trying while I get to you."
- **How the course runs (5m).** Show the course home page. Say: "Eleven sessions of three hours, two lessons a session: the workflow and honest evaluation first, then preparing data, then the main families of models, and a final project you choose. Every lesson uses the same made-up company, Pinecrest Outfitters, so by the end you will know its customers, products and payments well." Then the deal: "Each lesson was written for someone working alone for about 90 to 105 minutes, so two do not fit in three hours. In this room we do what is hard alone: the mental model, a live demo, a guided lab, and the first steps of each project. At home you finish the projects and do the exercises, quiz and journal: plan on two to three hours a week." Scroll once through `getting_started.html`: objectives, sections with real output, common mistakes, a project in steps, exercises with hidden solutions, a quiz, a journal prompt.
- **Environment check (12m).** Say: "Before any new ideas, we prove every machine is ready." On the projector, then theirs: **1** open a terminal; **2** `cd pinecrest-ml`; **3** activate and look for `(.venv)`; **4** `python labs/module-01/l0_1-orientation/check_environment.py`. Warn them it can sit silent for up to a minute the first time. Done = the last line says `READY for Python Machine Learning`. Anyone who saw READY at home runs it once more and becomes a helper. A NOT READY machine prints a numbered fix list: activation and the data folder you fix now (one minute each); a missing Python 3.12 or a full package install you do not start on venue Wi-Fi: pair that learner with a ready neighbor and fix it at home.
- **Diagnostic (8m).** Say: "Six quick questions on the Python and pandas this course builds on. Not a test, nobody collects it. It tells you what to reread and tells me what to slow down for." Learners answer D1 to

D6 on the workbook's Start Here page (about four minutes), then show fingers (1 to 4 for A to D) as you reveal each on the slide. Settle any argument by running the code. Keep a tally of wrong answers per question on a sticky note.

DIAGNOSTIC ANSWERS AND HOW TO READ THE TALLY

Answers: **D1** A) `[7, 8, 9] [5, 6]`; **D2** C) `{'Bend': 2, 'Boise': 1}`; **D3** B) `72.0 60.0`; **D4** D) `200.0`; **D5** A) `{'Basic': 0.5, 'Gold': 0.0}`; **D6** C) `2 2.0`.

- **D1 (slicing) and D3 (default arguments)** are the shape of today's code: selecting columns for `X`, and creating estimators with a few settings while the rest keep their defaults. Two or more misses on D3: narrate every constructor out loud in the 1.1 demo ("`max_depth=4`: one setting changed, every other one left at its default").
- **D2 (dictionaries)** comes back in the 1.1 demo's model loop. Misses are not urgent.
- **D4 (filtering) and D5 (groupby)** are the pandas the course assumes every day: rates by group, rows that meet a condition. A third of the room missing either: spend an extra minute on demo Step 5 and do TODO 1 and 2 of the lab together on the projector.
- **D6 (missing values)** predicts trouble with today's lab and with the NaN error in 1.1. Misses here: make a point of `isna()` in every step of the lab debrief.
- **Three or more wrong:** not a problem to announce. Pair them with a strong neighbor today, and at the wrap-up hand them `labs/module-01/l0_1-orientation/diagnostic_starter.py` as extra homework: six small functions with a checker, one per question (`diagnostic_solution.py` is your reference). The workbook's Start Here page names what to review for each.

WATCH-OUTS

- **Do not debug one machine in front of the room.** Two minutes per machine, then pair them and move on. The trap table in Before the Session covers nearly everything you will see.
- **Mixed backgrounds.** Some learners finished Python Data Science last week; some learned pandas at work and never met a virtual environment. The diagnostic is for them as much as for you.
- **"I already use scikit-learn."** Welcome them and give them a job: helper during the environment check, and the stretch tasks in each lab. Ask them to watch for the habits today is about (a baseline first, test data untouched), which experienced people skip most.
- **Versions.** The lessons print output from scikit-learn 1.9 and pandas 3.0. A learner on older versions will see some different numbers and some different messages. The check prints every version: compare there first.

LESSON PLANS

0.1 Your ML Toolkit

55 min

OBJECTIVES (FROM THE LESSON PAGE)

- Create a project folder with a virtual environment, install the course packages, and confirm that scikit-learn and PyTorch import
- Choose between a notebook and a script for a task, and run code in both
- Describe the machine-learning workflow (frame, prepare, train, evaluate, explain, deploy) in your own words
- Tell supervised from unsupervised learning and classification from regression, and name the features and target of a table
- Load the seven Pinecrest data files and summarize each one's size, gaps and target

TIMING IN THE ROOM

10m	Frame: the workshop and its routine; the six-step workflow, and why the loop is the point
20m	Live demo: versions, the data folder, the customer table and its gaps, three targets of three kinds, three errors on purpose
15m	Guided lab: `missing_values_starter.py` (why is it missing?)
10m	Start the lesson project: the data card, Steps 1 and 2

MENTAL MODEL TO SAY

Say it while you draw four nested boxes on the board: "Python is the building. A virtual environment is one workshop inside it with its own tool wall: the tools hanging there are for this project only. The packages are the tools. A notebook is the workbench, and the data folder is the raw material. Machine learning adds one thing to the workshop: a **model**, a tool you do not buy but make, by showing it examples."

Then the routine, written on the board and left there all session: **1** open a terminal, **2** `cd pinecrest-ml`, **3** activate (look for `(.venv)`), **4** start JupyterLab or VS Code from that folder, **5** check that the notebook's kernel is the `.venv` one. "Almost every problem in week one is one of these five being skipped."

Then the workflow, pointing at the six boxes: frame, prepare, train, evaluate, explain, deploy. Tap the arrow from Evaluate back to Prepare: "Beginners think machine learning is choosing a clever algorithm. Most of the real work is this arrow: better framing, better data, more honest evaluation. Today is step 1 and step 2. Nobody trains a model until after the break."

LIVE DEMO (YOU TYPE, THEY FOLLOW)

The environment check in the opening already proved that every machine works, so this demo is about using the toolkit and reading the data like a data scientist. Work in a new notebook called `lesson_0_1` in your own `pinecrest-ml`; learners type along in theirs. Steps 8, 9 and 11 are errors you cause on purpose: read the last line out loud each time before you fix anything.

1. **Ask the tools who they are (2m).** "First cell of any machine-learning project: print the versions. Machine-learning results depend on them; when your numbers differ from the page, this is the first place to look."

```
import sys
import numpy as np
import pandas as pd
import sklearn
import torch
print("Python      ", sys.version.split()[0])
print("NumPy       ", np.__version__)
print("pandas      ", pd.__version__)
print("scikit-learn", sklearn.__version__)
print("PyTorch     ", torch.__version__)
print("GPU available:", torch.cuda.is_available())
```

```
Python       3.12.14
NumPy        2.5.4
pandas       3.0.6
scikit-learn 1.9.1
PyTorch      2.14.1+cpu
GPU available: False
```

Last digits may differ; the first two numbers must match (scikit-learn 1.9, pandas 3.0). `GPU available: False` is expected and fine: the whole course runs on the processor. Anyone with scikit-learn 1.8 or older, or pandas 2.x, has the wrong kernel or an old environment: trap table.

2. **Can Python see the data? (1m).** "A relative path starts from the folder Python is working in. Before I open a file I ask whether Python can see it."

```
from pathlib import Path
print(Path("data/customers_churn.csv").exists())
```

```
True
```

Anyone with `False`: `print(Path.cwd())` shows where they really are. Close JupyterLab, `cd pinecrest-ml`, start it again.

3 **One row is one customer (2m).** "This is the table we will use most: one row per customer on June 30, 2025. The last column is what happened in the six months after."

```
pd.set_option("display.width", 120)
customers = pd.read_csv("data/customers_churn.csv")
print(customers.shape)
print(customers[["customer_id", "home_store", "loyalty_tier", "days_since_last
```

```
(6000, 20)
  customer_id home_store loyalty_tier  days_since_last_order  churned
0     C100001       Bend         Gold                     77        0
1     C100002      Boise       Silver                     24        0
2     C100003  Flagstaff        Basic                     24        0
```

Ask: "What is one row?" (a customer at the snapshot). "Which column happened AFTER the others?" (`churned`). That one sentence is the whole idea of a target.

4 **Where are the gaps? (2m).** "Three columns have gaps. Do not count them and move on: ask why each one is missing. That is the lab after the demo."

```
print(customers.isna().sum()[lambda s: s > 0])
```

```
age_group              487
email_opens_90d        537
cancellation_reason   5375
dtype: int64
```

Point at 5,375 in `cancellation_reason` and say only: "Hold that thought. Lesson 1.2 is built around this column."

5 **A target with two values (2m).** "Two values, zero and one: predicting it is classification. And look how unequal they are."

```
print(customers["churned"].value_counts())
print(f"churn rate: {customers['churned'].mean():.1%}")
```

```
churned
0    5008
1     992
Name: count, dtype: int64
churn rate: 16.5%
```

Ask: "If I always say 'stays', how often am I right?" (83.5%). Write 83.5% on the board; Lesson 1.1 measures it.

6 **A target that is a quantity (1m).** "Units sold in a week: a number where values in between make sense. Predicting it is regression."

```
demand = pd.read_csv("data/weekly_demand.csv")
print(demand["units_sold"].nunique(), "distinct values from", demand["units_so
```

```
179 distinct values from 0 to 420
```

Then the trap from the lesson: "`churned` is also stored as a number. Is 0.5 churned meaningful?" (No: a number can still be a category.)

7 **A very rare class (2m).** "One more target, and the number I want you to remember all course."

```
tx = pd.read_csv("data/transactions.csv")
fraud = tx["is_fraud"].mean()
print(f"fraud: {fraud:.2%} of payments; always guessing 'legit' is right {1 -
```

```
fraud: 0.88% of payments; always guessing 'legit' is right 99.12% of the time
```

Say: "A fraud detector that never flags anything is 99.12% accurate and catches nobody. Whenever someone quotes you an accuracy, ask what doing nothing scores."

8 **Error 1: the install name (1m).** "Now I break things on purpose. Read the last line first, always."

```
import scikit_learn
```

```
ModuleNotFoundError: No module named 'scikit_learn'
```

You install `scikit-learn` but import `sklearn`. If the message names the RIGHT module (`No module named 'sklearn'`), the problem is the environment, not the spelling.

9 **Error 2: a name that is almost right (1m).** "A KeyError from a table means: no column with exactly this name."

```
customers["churn"]
```

```
KeyError: 'churn'
```

10 **Ask the table, do not guess (1m).** "Copy the name from here."

```
print(customers.columns.tolist()[-4:])
```

```
['email_opens_90d', 'preferred_category', 'cancellation_reason', 'churned']
```

11 **Error 3: a date that is still text (2m).** "The `.dt` tools only exist for real dates, and `read_csv` leaves dates as text unless you ask."

```
customers["signup_date"].dt.year
```

```
AttributeError: Can only use .dt accessor with datetimelike values
```

12 **Ask for the date when you load (1m).** "`parse_dates` converts the column while the file loads."

```
customers = pd.read_csv("data/customers_churn.csv", parse_dates=["signup_date"
print(customers["signup_date"].dt.year.value_counts().sort_index().tail(3))
```

```
signup_date
2023     618
2024     605
2025    1222
Name: count, dtype: int64
```

A free observation: about a fifth of all customers signed up in 2025. That is why 1,227 of them have under six months of tenure, which matters for churn later.

13 **The same check as a script (2m).** "A notebook is where I work it out; a script is how I keep it. The MLOps lessons turn finished work into scripts." Create `check_setup.py` in the project folder, then run `python check_setup.py` in the terminal (check for `(.venv)` first).

```
from pathlib import Path
import sklearn
import torch
print(f"scikit-learn {sklearn.__version__}, PyTorch {torch.__version__.split('
print("data folder found:", Path("data").is_dir())
```

```
scikit-learn 1.9.1, PyTorch 2.14.1
data folder found: True
```

It starts fresh every time and prints only what you print: that is why the lessons always use `print()`.

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **The kernel or the terminal is not the course environment.** Symptom: `ModuleNotFoundError: No module named 'sklearn'`, or Step 1 prints scikit-learn 1.8 or pandas 2.x. Coach with a question, not a fix: "What does your prompt start with?" and, in a notebook, "Which kernel does the top-right corner name?" Then `import sys; print(sys.executable)`: the path should contain `.venv`.
- **Python is working in a different folder.** Symptom: `FileNotFoundError` for a file they can see, usually because JupyterLab was started from the home folder. Coach: "Ask Python where it is: `print(Path.cwd())`. Is there a `data` folder inside it?" Do not let them paste an absolute path: it works today and breaks every lesson tomorrow.
- **A number taken for a quantity.** Symptom, in conversation or in the project's Step 3: "`churned` is a number, so it is regression", or "ratings are 1 to 5, so regression". Coach with the lesson's test: "Would a value halfway between two others make sense? Is 0.5 churned a thing?" Star ratings are a fair argument either way; the course treats them as categories. Praise the argument, not just the answer.

GUIDED LAB

File: learners open `labs/module-01/l0_1-orientation/missing_values_starter.py`; your reference is `labs/module-01/l0_1-orientation/missing_values_solution.py`. It practices the lesson's moves (load, count gaps, rates by group) on a question the lesson only raises: WHY is a value missing? Four gaps, four different answers.

- **Run it first, as shipped (1m).** Everyone runs the starter untouched and sees the gap counts and four TODO lines. That proves the lab can find the data. A learner who sees "The Pinecrest machine-learning data was not found" has a `labs` folder that is not next to `data`: the message names both fixes.
- **TODO 1 and 2 together on the projector (5m), TODO 3 to 5 in pairs (8m).** Circulate for `isna()` grouped by the wrong thing (`groupby("home_store")` gives five rates, which is fine but noisier: ask what the question was), and for `crosstab` with the columns swapped (also fine: ask them to read it aloud).
- **Expected:** `age_group` missing 12.7% for online shoppers vs 4.4% for store shoppers; churn 26.1% when `email_opens_90d` is missing vs 15.6% when present; all 625 cancellation reasons belong to churners (0 of 5,008 stayers have one); 19,970 of 32,178 interactions (62.1%) have no rating.
- **Debrief (1m):** "Which of these four may a model use?" `age_group`: yes, once the gaps are filled (Lesson 2.1). The email gap: yes, and the fact that it is missing is itself a weak signal. The rating gap means "no review", not "bad product". `cancellation_reason`: never. It is written after the outcome: a leak. Lesson 1.2 shows what it does to a model.
- **Fast finishers:** `labs/module-01/l0_1-orientation/frame_targets_starter.py`: three new requests from Pinecrest teams to frame (table, target, kind, what doing nothing scores). Its sharpest

finding: the fraud rate drifts from 1.00% in January to 0.77% in June, so "always legitimate" gets more accurate every month while catching nobody.

START THE LESSON PROJECT

The lesson project is the **data card**: a one-row-per-file summary of all seven tables (`data_overview.csv`). In the room, do Steps 1 and 2 only (10 minutes).

- Open the lesson page at "Project: A Data Card for the Course Files" and read the brief aloud. Everyone types Step 1 (a dictionary of seven tables) and Step 2 (the gaps per table).
- **Done in the room** = `7 tables loaded`, `106,746 rows in total`, and the gaps list: three tables with gaps (customers_churn, interactions, daily_store_sales).
- **Goes home:** Step 3 (a target and a problem kind for each file, their own judgment), Step 4 (churn 16.53% and fraud 0.88% of rows), Step 5 (`data_overview.csv`, seven rows) and Step 6 (`journal.md`). Tell them the two rates to expect so they can check themselves.

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab (cold-call gently, or think-pair-share):

- You open a new terminal tomorrow and `python check_setup.py` says there is no module named sklearn. What did you skip? **Answer:** activating the environment in that terminal. Nothing was uninstalled.
- Name the features and the target of `customers_churn.csv` in one sentence. **Answer:** the target is `churned` (what happened after the snapshot); the features are columns describing the twelve months before it.
- Pinecrest asks which customers shop alike. Supervised or unsupervised? **Answer:** unsupervised: no column holds the right answer.
- A fraud model is 99.1% accurate. Impressed? **Answer:** not yet: never flagging anything scores 99.12%. Ask what doing nothing scores.

Workbook self-check, items Q1 to Q6 (learners do them at the end of the block or at home).

Answers: **Q1** C) The notebook's kernel is a different Python from the one in `.venv`; **Q2** D) How many dollars will the Boise store take tomorrow?; **Q3** `0.25 3`; **Q4** B) `cancellation_reason`, because it is filled in only after a customer has left; **Q5** A) Evaluate; back to Prepare; **Q6** `str`. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: pair anyone whose check said NOT READY with a ready neighbor for the demo and the lab (one keyboard, two heads) and fix the machine at the break, not in front of the room. For learners who froze on the diagnostic, give them the typing role in the pair for the lab, and keep the five-step routine on the board to point at instead of fixing things for them.

Stretch: `frame_targets_starter.py`; then the lesson's Bonus Challenge (read `data/README.md` and write, for each file, what could go wrong if you trained a model on it). Ask experienced learners to hold back on modeling: today is framing, on purpose.

1.1 The scikit-learn Estimator API

75 min

OBJECTIVES (FROM THE LESSON PAGE)

- Turn a table into a feature matrix `X` and a target `y`, and hold out a test set with `train_test_split`
- Use the estimator methods `fit`, `predict`, `predict_proba` and `score`, and read learned attributes such as `coef_`
- Use a transformer with `fit` and `transform`, fitting it on the training data only
- Measure a `DummyClassifier` baseline and judge whether a real model beats it
- Swap one model for another without changing the rest of your code

TIMING IN THE ROOM

10m	Frame: the new hire and the case files; the four verbs; the grading rule
30m	Live demo: X and y, the split, a baseline, the API, a scaler, logistic regression, probabilities and deciles, four models in one loop, the weights
20m	Guided lab: `fraud_baseline_starter.py`, then `new_customers_starter.py` in pairs
15m	Start the lesson project: the churn-risk shortlist, Steps 1 to 3

MENTAL MODEL TO SAY

Say: "Picture a new hire at Pinecrest's customer-care desk. On day one they get a box of old case files: each describes a customer and ends with a note saying whether that customer left. They read the files and develop a feel for the warning signs. That is **fit**. Then the manager hands over fresh files with the ending torn off: 'Who will leave?' That is **predict**. Finally the manager checks the answers against what really happened and gives a grade. That is **score**."

Then the one rule, written in the "Every estimator" box: **grade with files the new hire has never seen.** Under it, the verbs as you meet them in the demo: create (settings) → `fit` → `predict` / `predict_proba` / `score`; and for transformers, `fit` → `transform`. Why it matters, in one breath: "Every model in scikit-learn, from the simplest to the most elaborate, uses these same names. Learn them once today and cross-validation, pipelines, tuning and explanations later are small variations on a theme."

LIVE DEMO (YOU TYPE, THEY FOLLOW)

New notebook, `lesson_1_1`, same project folder. Ask for a prediction before every run that has a number in it: the lesson is built on two surprises (doing nothing scores 83.5%; a model barely better on accuracy ranks customers far better), and a surprise only teaches if the learner committed to a guess first.

1. **Features and a target (2m).** "`X` is a table, capital because it is a matrix: one row per customer, one column per feature. `y` is one column, the answer. Today only numeric columns with no gaps: text and gaps need Lesson 2.1."

```
import pandas as pd
customers = pd.read_csv("data/customers_churn.csv")
features = ["tenure_months", "orders_12m", "spend_12m", "avg_order_value", "da
            "returns_12m", "return_rate", "support_tickets_12m", "discount_sha
X = customers[features]
y = customers["churned"]
print(X.shape, y.shape)
```

```
(6000, 10) (6000,)
```

Point out what is NOT in X: `cancellation_reason` (the leak), `home_store` (text), `email_opens_90d` (gaps), `customer_id` (a label, not a signal).

2. **Hold out a test set (2m).** "The manager's rule: grade with files the new hire has never seen." Narrate each argument: a quarter for testing; the same churn share on both sides; the same shuffle every time.

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=42
)
print(X_train.shape, X_test.shape)
print(f"churn rate  train {y_train.mean():.3f}  test {y_test.mean():.3f}")
```

```
(4500, 10) (1500, 10)
churn rate  train 0.165  test 0.165
```

Order of the four results: the two X parts, then the two y parts. Have the room say it.

3. **A baseline that knows nothing (3m).** "Before anything clever: how well can we do by learning nothing at all?" Note the first print: `fit` returns the model itself, which is why calls can be chained.

```
from sklearn.dummy import DummyClassifier
baseline = DummyClassifier(strategy="most_frequent")
print(baseline.fit(X_train, y_train))
print(baseline.predict(X_test)[:10])
print(f"baseline accuracy: {baseline.score(X_test, y_test):.3f}")
```

```
DummyClassifier(strategy='most_frequent')
[0 0 0 0 0 0 0 0 0 0]
baseline accuracy: 0.835
```

Say: "83.5% accurate, and it has never found a single customer at risk. If someone tells you their churn model is 83% accurate, the honest reply is 'so is doing nothing'." Write 0.835 next to the 83.5% from the 0.1 demo.

4 **What you chose, and what it learned (2m).** "Settings you chose: `get_params()`. Things it learned during `fit`: names ending in an underscore."

```
print(baseline.get_params())
print(baseline.classes_)
```

```
{'constant': None, 'random_state': None, 'strategy': 'most_frequent'}
[0 1]
```

The underscore is a promise: this attribute exists only after fitting.

5 **Predict before fit, on purpose (1m).** "Creating a model stores settings. It has learned nothing yet."

```
from sklearn.linear_model import LogisticRegression
LogisticRegression().predict(X_test)
```

```
NotFittedError: This LogisticRegression instance is not fitted yet. Call 'fit
```

In a notebook this is what you get after re-running the cell that CREATES the model: the fitted one is replaced by a fresh one.

6 **A transformer: fit learns, transform applies (3m).** "Some estimators change data instead of predicting. `StandardScaler` learns one mean and one standard deviation per column." Ask for `set_output` to keep column names.

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler().set_output(transform="pandas")
scaler.fit(X_train)
print(scaler.mean_[:3].round(2))
print(scaler.scale_[:3].round(2))
```

```
[ 41.48   5.   594.13]
[ 30.53   3.23 414.47]
```

Tenure, orders, spend: averages of about 41 months, 5 orders, $594. Underscores again: learned.

7. **Transform both sets with the TRAINING numbers (2m).** "Fit on train only, transform both. The test rows are next month's customers: the scaler must not peek at them."

```
X_train_s = scaler.transform(X_train)
X_test_s = scaler.transform(X_test)
print(X_train_s[["tenure_months", "days_since_last_order"]].head(3).round(3))
```

```
      tenure_months  days_since_last_order
1556         -1.326                 -0.316
965           0.083                 -0.316
5145         -1.227                 -0.763
```

The index numbers (1556, 965, 5145) are the original rows, shuffled by the split. -1.326 means 1.3 standard deviations below the average tenure.

8. **A first real model (2m).** "Logistic regression: one weight per feature, add them up, squash the total into a probability. Lesson 3.1 opens it up. Today it is just one more estimator: create, fit, score."

```
model = LogisticRegression()
model.fit(X_train_s, y_train)
print(f"logistic regression accuracy: {model.score(X_test_s, y_test):.3f}")
```

```
logistic regression accuracy: 0.848
```

Ask: "0.848 against 0.835. Is this model any good?" Take answers from both camps; do not settle it yet.

9. **More than yes or no (3m).** "`predict_proba` gives the probability of each class, in the order of `classes_`. `predict` just says 1 when column 1 is above 0.5."

```
proba = model.predict_proba(X_test_s)
print(model.classes_)
print(proba[:3].round(3))
print(model.predict(X_test_s)[:3])
```

```
[0 1]
[[0.937 0.063]
 [0.47  0.53 ]
 [0.895 0.105]]
[0 1 0]
```

The second customer gets 0.53: just over the line, so `predict` says 1.

10 **Settle the argument: rank, then check (4m).** "Pinecrest does not need a perfect yes or no; it needs the customers sorted from most to least at risk. Cut the test customers into ten groups by predicted risk and look at what really happened."

```
risk = pd.DataFrame({"p": proba[:, 1], "churned": y_test.to_numpy()})
risk["decile"] = pd.qcut(risk["p"].rank(method="first"), 10, labels=range(1, 1
print(risk.groupby("decile", observed=True)["churned"].mean().round(3).iloc[[0
```

```
decile
1     0.013
5     0.053
10    0.573
Name: churned, dtype: float64
```

Show the lesson page's decile bar chart. Say: "The riskiest tenth churned at 57%, more than three times the average. Barely better on accuracy, very useful as a ranking. Both are true, and Lesson 1.4 gives you the vocabulary."

11 **Swap models, keep the code (4m).** "Here is the payoff of a shared API: four algorithms, one loop. You do not need to know how a forest works to use one; Module 5 opens it up."

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
candidates = {
    "baseline": DummyClassifier(strategy="most_frequent"),
    "logistic regression": LogisticRegression(),
    "decision tree": DecisionTreeClassifier(max_depth=4, random_state=42),
    "random forest": RandomForestClassifier(n_estimators=200, min_samples_leaf
}
for name, estimator in candidates.items():
    estimator.fit(X_train_s, y_train)
    print(f"{name:20s} {estimator.score(X_test_s, y_test):.3f}")
```

```
baseline             0.835
logistic regression  0.848
decision tree        0.864
random forest        0.873
```

The two tree models get `random_state=42` because they use randomness while training. Warn: comparing many models on one test set again and again slowly uses it up (Lessons 1.2 and 1.3).

12 **A first impression of the weights (2m).** "Because the features are scaled, the weights are roughly comparable: positive pushes toward churn."

```
coefs = pd.Series(model.coef_[0], index=features).sort_values()
print(coefs.round(2).iloc[[0, 1, -3, -2, -1]])
```

```
app_user                -0.34
orders_12m              -0.31
support_tickets_12m      0.40
discount_share           0.67
days_since_last_order    0.88
dtype: float64
```

A long gap since the last order, heavy discount use and support tickets raise the risk; app users and frequent buyers are safer. Treat it as a first impression: Module 7 is about trustworthy explanations.

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **X in the wrong shape or with the wrong contents.** Three messages, one habit: read the last line. A single column as a Series: `ValueError: Expected a 2-dimensional container but got <class 'pandas.Series'> instead. Pass a D...` (fix: double brackets). A text column: `ValueError: could not convert string to float: 'Bend'` (leave it out until Lesson 2.1). A gap: `ValueError: Input X contains NaN.` (the full message even suggests an imputer or HistGradientBoosting). Coach: "What does the last line say X contains? Print `X.dtypes` and `X.isna().sum()`."
- **The fitted model is gone, or the scaler peeked.** `NotFittedError` after re-running the cell that creates the model (demo Step 5); or, silently, `fit_transform` on `X_test`. The second raises nothing and changes the scores a little (in `debug_estimator.py`, 0.847 instead of 0.848). Coach: "Which line fitted the scaler? On which rows?" The rule for the board: fit on train, transform everything.
- **Judging the model by accuracy alone.** Either "84.8% is great" or "barely better than the baseline, so useless". Both miss the ranking. Coach: "What did the top decile do?" and, in the lab, "How many frauds are among your 100 riskiest payments, and how many would a random 100 hold?" Lesson 1.4 makes this precise; today it only has to feel wrong to quote accuracy without a baseline.

GUIDED LAB

- **Lab A (12m, everyone):** `labs/module-01/l1_1-estimator-api/fraud_baseline_starter.py`; reference `labs/module-01/l1_1-estimator-api/fraud_baseline_solution.py`. The whole API on a new problem: card fraud in `transactions.csv`, nine numeric features, a stratified 70/30 split, a baseline, a scaled logistic regression, then a ranking. Same moves as the demo, a far rarer class.
- **Expected, Lab A:** 35,000 / 15,000 rows with 307 and 131 frauds; baseline accuracy 0.9913; logistic regression 0.9937; at the 0.5 cut-off it flags 40 payments, 38 of them fraud, and misses 93; the 100 riskiest payments hold 53 frauds against 0.9 for a random 100. The weight for `hour` is negative (-0.31).

- **Circulate in Lab A for:** a split without `stratify` (the fraud counts on each side drift: ask them to compare 307 / 131); `fit_transform` on `X_test`; ranking with `predict` instead of `predict_proba` (every score is 0 or 1, so `nlargest` returns ties). **Debrief:** "The model is 0.24 points more accurate than doing nothing, and its top 100 holds 53 frauds instead of one. Same story as churn, sharper." Then the backwards weight: fraud clusters at night (hours 0 to 5), and a straight line in `hour` can only say "later is safer". Turning hours into a night flag is feature engineering, Lesson 2.3.
- **Lab B (8m, pairs):** `labs/module-01/l1_1-estimator-api/new_customers_starter.py`; reference `labs/module-01/l1_1-estimator-api/new_customers_solution.py`. Three customers described over the phone become rows, go through the SAME fitted scaler and model, and get ranked; on the way, learners check what `mean_` and `scale_` hold and change a setting with `set_params`. One types, the other predicts each answer; swap at TODO 4.
- **Expected, Lab B:** `True`, `True`, `False` (the scaler uses the population standard deviation, `ddof=0`, not pandas' default); C = 1.0, max_iter = 100, accuracy 0.848; Avery 0.808, Casey 0.550, Blake 0.010; columns in another order raise `ValueError: The feature names should match those that were passed during fit.`; with C = 0.01 accuracy 0.853 and the probabilities pulled toward the middle (Avery 0.719, Casey 0.481, Blake 0.018).
- **Circulate in Lab B for:** `scaler.fit_transform(new)` (three rows become their own mean: every scaled value is near 0 and the ranking is nonsense; ask "which numbers did the model learn to expect?"); building `new` with the columns in their own order (the ValueError of TODO 4 arrives early: good, read it).
- **Fast finishers:** `labs/module-01/l1_1-estimator-api/debug_estimator.py`: three bugs that stop the script (`KeyError: 'churn'`, a Series as X, `NotFittedError`) and one that does not (the scaler fitted on the test set). Fixed, it prints accuracy 0.841 for one feature and 0.848 for ten.

START THE LESSON PROJECT

The lesson project is the **churn-risk shortlist**: the 50 test customers most likely to churn, checked against what really happened. In the room, do Steps 1 to 3 (15 minutes).

- Read the brief aloud from the lesson page ("Pinecrest's customer-care team can make 50 retention calls this month"). Step 1 together: split the whole table so the IDs stay with their rows. **Done** = `4500 1500 ['C102112', 'C103063', 'C101159']`: the same first test ID as the lesson means the same split.
- Steps 2 and 3 on their own: baseline 0.835 and 8.3 churners expected in a random 50; logistic regression 0.848 and random forest 0.873.
- **Goes home:** Step 4 (rank and take the top 50), Step 5 (the forest's list holds 41 real churners, logistic regression's 33, against 8.3 at random) and Step 6 (`churn_shortlist.csv`, 50 rows, no outcome column). They keep this notebook: Lesson 1.2 starts from the same split.

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab:

- What does `fit` return, and why does that matter? **Answer:** the estimator itself, so `DummyClassifier(...).fit(X_train, y_train)` creates and fits in one line.
- Which attributes exist only after fitting, and how can you tell? **Answer:** the ones whose names end in an underscore: `classes_`, `coef_`, `mean_`, `scale_`.
- Why `transform` (not `fit_transform`) on the test set? **Answer:** the test rows stand in for future customers; they must be scaled with the numbers learned from training.
- Baseline 0.835, model 0.848. What else would you look at before deciding? **Answer:** how well it ranks: churn in the riskiest decile (57%) or real churners in a top-50 list.
- Why do the tree and the forest get `random_state=42` but logistic regression does not? **Answer:** the tree-based models use randomness while training; logistic regression's result does not depend on a random start.

Workbook self-check, items Q7 to Q12 (learners do them at the end of the block or at home).

Answers: **Q7** D) `NotFittedError`: the last assignment replaced the fitted model with a fresh one; **Q8** `[4.] [0. 2.449]`; **Q9** A) Each test customer's probability of churning; **Q10** C) The test rows stand in for future customers, whose numbers could not have been seen when the model was built; **Q11** B) `ValueError`: X must be a table; `customers[["days_since_last_order"]]` fixes it; **Q12** `['C', 'class_weight', 'dual']`. Full set with explanations in the Answer Key.

◆ DIFFERENTIATION

Support: give anyone lost in the names a card with the four verbs and the transformer pair, and have them say which one each demo line uses before it runs. In Lab A, let a struggling pair copy the split and scaler lines from the demo notebook: the point is to see the same five moves work on new data, not to retype them.

Stretch: the stretch cell of Lab A (a forest, 58 frauds in its top 100); then the lesson's Exercise 2 (tree depth against training and test accuracy: the first look at overfitting) and the Bonus Challenge: try cut-offs from 0.1 to 0.9 on the logistic regression's probabilities, count customers flagged and real churners among them, and choose a cut-off when an offer costs $15 and reaching a real churner is worth $60. Lesson 1.4 gives them the vocabulary.

CLOSING

◆ EXIT TICKET (3 QUESTIONS, NO NOTES, 4 MINUTES)

Put the three questions on the slide. Learners answer in the workbook's Exit Ticket box or on a sticky note and leave it on the way out. Sort the notes into "got it" and "not yet" before Session 2: the pile for question 3 tells you how much of the Session 2 warm-up goes to baselines and ranking.

1. Your prompt does not start with `(.venv)`. What will probably go wrong, and what do you type to fix it? **Answer:** Python will not find the course packages (`ModuleNotFoundError`). Activate the environment from

the project folder: `.venv\Scripts\Activate.ps1` on Windows, `source .venv/bin/activate` on macOS and Linux.

2. Name the three steps every scikit-learn model follows, and the two a transformer uses instead of predicting. Which data does the transformer learn from? **Answer:** create with settings, `fit`, then `predict` or `score`; a transformer uses `fit` and `transform`, and it is fitted on the training data only.

3. A churn model scores 0.848 accuracy; always predicting "stays" scores 0.835. Is the model useful? What would you check? **Answer:** accuracy alone cannot say. Check how it ranks customers: the riskiest tenth churned at 57% against 16.5% on average, and its top 50 hold several times as many real churners as a random 50. Useful.

HOMEWORK (ABOUT 2 TO 3 HOURS)

- **Finish both lesson projects.** The data card (Steps 3 to 6: targets and kinds, churn 16.53% and fraud 0.88%, `data_overview.csv`, `journal.md`) and the churn-risk shortlist (Steps 4 to 6: 41 of 50 for the forest, 33 for logistic regression, `churn_shortlist.csv`).
- **Lesson exercises.** 0.1: Exercises 1 to 3 (customers by store, Snow gear by month, fraud by hour); Exercise 4 is a stretch. 1.1: Exercises 1 to 3 (other Dummy strategies, tree depth, the same API for regression); Exercise 4 is a stretch.
- **Both Quick Quizzes** on the lesson pages (five questions each; answers are in the Answer Key).
- **Journal:** both entries in `journal.md`. The 1.1 prompt asks for the two numbers that tell the most honest story about their model: the baseline and the shortlist hits.
- **Before the Next Lesson (from 1.1):** keep the project notebook; rerun the project with `random_state=7` in the split and note how much the baseline and forest accuracy move; predict, then check, what `DummyClassifier(strategy="constant", constant=1)` scores.
- **If the environment check said NOT READY:** follow its numbered list, then run it again and email you the last line.
- **Optional:** `labs/module-01/l0_1-orientation/diagnostic_starter.py` for anyone who missed two or more diagnostic questions; `labs/module-01/l0_1-orientation/frame_targets_starter.py` for anyone who did not reach it.

BRING LIST FOR SESSION 2

- A laptop on which `check_environment.py` ends with READY (the same environment as today: nothing new to install).
- The `pinecrest-ml` folder with `data/` unzipped and the `labs/` folder inside it. Session 2's starters are in `labs/module-02/`.
- The Lesson 1.1 project notebook: Lesson 1.2 re-creates the same split and the same ten features in a fresh notebook, so it is a reference, not a requirement (finish the shortlist this week either way).
- The workbook and the journal. No downloads, no accounts, no internet needed.

IF YOU RUN LONG OR SHORT

RUNNING LONG: WHAT TO PROTECT AND WHAT TO CUT FIRST

- **Protect:** the environment check result for every learner; 1.1 demo Steps 3 and 8 to 10 (baseline, model, probabilities, deciles); Lab A through TODO 5; Step 4 of the missing-values lab.
- **Cut first (saves 5m):** 0.1 demo Steps 11 to 13 (the date error and the script). Say the two sentences: "dates stay text until you ask with `parse_dates`" and "a notebook to work it out, a script to keep it".
- **Cut second (saves 10m):** the 0.1 project start. Read the brief, give the two check numbers (106,746 rows; three tables with gaps), send all six steps home.
- **Cut third (saves 8m):** Lab B moves to homework; its README line and expected output are in the workbook.

RUNNING SHORT: WHAT TO ADD

- Run `frame_targets_starter.py` for everyone and debrief the fraud drift: why would a score improve while a model does nothing?
- The lesson's 1.1 Exercise 2 live: depths 2, 4, 8 and None, training against test accuracy (0.856/0.851 up to 1.000/0.821). Name the gap: overfitting, Module 6.
- Start the 1.1 Bonus Challenge together: a loop over cut-offs, and the $15 offer / $60 save trade-off.

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 2: Splits, Leakage and Cross-Validation

How to teach the second session of Machine Learning Foundations: make a score honest. Lesson 1.2 measures how much a single split wobbles, gives every part of a three-way split one job, splits by time and by group when rows are related, picks the strongest free baseline and catches a leak with a single-feature audit. Lesson 1.3 replaces the single split with cross-validation: stratified shuffled folds, preprocessing inside a pipeline, paired comparisons, out-of-fold predictions, and folds that match the question. Learners leave knowing that every careless evaluation looks better than the honest one: 0.941 with a leak against 0.848, $613 on a random split against $854, and 0.825 accuracy on pure noise.

`2 lessons` `3 hrs session` `Lessons 1.2–1.3` `Session 2 of 11`

HOW TO USE THIS GUIDE

- Same rhythm for both lessons: frame (one mental model) → live demo (you type, they type along) → guided lab (a kit lab, new practice on the lesson's skills) → start the lesson's own project. The lesson exercises, quiz and journal are homework.
- Type, do not paste. Every code block in the demo scripts below was run to produce the output printed under it (Python 3.12, pandas 3.0, scikit-learn 1.9 on 4 CPU threads); the code is the lessons' own, so learners' pages show the same numbers. Ask for a prediction before every run that prints a score: this session is built on scores that look better than they are, and a surprise only teaches if the learner committed to a guess.
- The lessons are the reference. Keep `evaluation_train_test_split.html` and `evaluation_cross_validation.html` open for the pictures (the 30-split histogram, the 60/20/20 bar, the Boise time-split chart, the leak flowchart, the 5-fold grid, the paired strip plot). Labs live in `labs/module-02/` : `_starter.py` files are handouts, `_solution.py` files are yours.
- This session is one idea told twice: **a score is only as honest as the data it was graded on.** Lesson 1.2 does it with single splits (stratify, a validation set, time and group splits, leaks); Lesson 1.3 replaces the single split with folds. Say the bridge out loud at the break: "Before the break we measured the wobble; after it we average it away."
- Teal blocks are hands-on, amber blocks are watch-outs. Lesson codes (1.2, 1.3) match the course site.

> **THE #1 RISK OF SESSION 2: A ROOM THAT HEARS "THE SCORE WENT UP" AS GOOD NEWS**
>
> Every careless evaluation in these two lessons looks BETTER than the honest one: 0.941 with the leak, $613 on a random split, 0.825 on pure noise, the best fold instead of the mean. Learners who are used to "higher is better" will cheer each time. Make the reverse reflex the session's habit: when a number jumps, the first question is "what does this column, this split or this step know that it would not know in real use?" The second risk is time: both demos are long and both lessons end in a 40-minute project. Protect the demo steps that change minds (30 seeds, the time split, the leak audit; the paired comparison and the three splitters) and cut the mechanics first (see If You Run Long).

BEFORE THE SESSION

- ☐ **Read the Session 1 exit tickets** (sorted "got it" / "not yet"). If many said a 0.848 model is "not useful", spend an extra minute of the warm-up on W1 and the ranking; today's leak demo builds on that baseline.
- ☐ **Run the kit check once:** `python3 labs/check_labs.py module-02` from this kit's folder. Every line should say PASS (8 files, about 30 seconds). The two `debug_*.py` handouts are not run by the checker: they are broken on purpose.
- ☐ **Files to hand out:** the `labs/module-02/` folder **without** the `*_solution.py` files (four starters and two `debug_*.py` handouts). Learners put it inside `pinecrest-ml/labs/`, next to `module-01/` and `labs_paths.py`.
- ☐ **Downloads:** none. Nothing new to install: the Session 1 environment runs everything today.
- ☐ **Projector machine:** `(.venv)` active in your own `pinecrest-ml`, JupyterLab started there, two empty notebooks `lesson_1_2` and `lesson_1_3`. Browser tabs: `evaluation_train_test_split.html`, `evaluation_cross_validation.html`, `data/README.md`. Run the 1.3 demo once before class: the three-splitter step (gradient boosting, 13 fits) takes 5 to 20 seconds on a typical laptop, so you know how long to talk.
- ☐ **Board, before learners arrive:** the dress rehearsal rule ("evaluate under the conditions of real use"); an empty 60/20/20 bar labeled train fits / validation chooses / test grades once; and a box headed "What will be new to the model?" for after the break.
- ☐ **Print or share:** the workbook (warm-up page, worked examples, self-check), sticky notes for help flags and the exit ticket.

KNOWN ENVIRONMENT TRAPS

SYMPTOM	CAUSE	FIX
Numbers differ from the page from the very first split, by a lot	The notebook ran cells out of order, or `X_train` came from another split (a leftover from 1.1)	Kernel ▸ Restart and run all. In the lesson's own Common Questions: compare `len(X_train)` and the first fold scores.

SYMPTOM	CAUSE	FIX
Numbers differ only in the third decimal place	A different scikit-learn or pandas version (the Session 1 check prints them)	Compare versions; the course is checked with scikit-learn 1.9 and pandas 3.0. Keep going: the conclusions do not change.
The stratify mistake prints a screen-long list of numbers	Common Mistake 2: stratifying on `units_sold`; the message lists every value that occurs once	Read the last line's first sentence only. It is the expected output of the mistake, not a crash of the machine.
A cell with `n_jobs=-1` is slow or prints warnings on Windows	Process start-up on Windows; antivirus scanning the worker processes	Remove `n_jobs=-1` (results are identical, only slower in theory) or wait: the first parallel call is the slowest.
The 1.3 three-splitter cell seems to hang	13 gradient-boosting fits on 7,488 to 9,984 rows	Wait 10 to 30 seconds. Old, slow laptops: run it once at home.
`ModuleNotFoundError` or `FileNotFoundError` in a lab	`labs/module-02/` was copied somewhere other than next to `labs_paths.py`, or the environment is not active	Put it inside `pinecrest-ml/labs/`; activate; the lab's own message names both fixes.
`TypeError` or `ValueError` mentioning `random_state` in a splitter	`KFold(..., random_state=42)` without `shuffle=True` (1.3 Common Mistake 2)	Add `shuffle=True`. For classification use `StratifiedKFold`.

SESSION 2 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Five retrieval questions on Session 1; homework check: the `random_state=7` rerun and the `constant=1` dummy.
0:10	10	**1.2** Frame and mental model	The dress rehearsal rule; why the careless evaluation always looks better.
0:20	30	**1.2** Live demo	Thirty splits, stratifying, train/validation/test, a time split and forecast baselines, a group split, the leak and the single-feature audit.
0:50	20	**1.2** Guided lab	`labs/module-02/l1_2-splits-and-leakage/demand_forecast_starter.py`: an honest dress rehearsal for next year's demand forecast.

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
1:10	15	**1.2** Start the lesson project	An honest churn evaluation, Steps 1 to 3; Steps 4 to 6 go home.
1:25	10	Break	Leave the dress rehearsal rule and the 60/20/20 bar on the board.
1:35	10	**1.3** Frame and mental model	One exam or five quizzes; cross-validation scores a recipe, not a model.
1:45	30	**1.3** Live demo	The fold loop by hand, then one line; shuffled and stratified folds; a pipeline; train vs validation; paired folds; out-of-fold predictions; time and group folds.
2:15	20	**1.3** Guided lab	`labs/module-02/l1_3-cross-validation/fraud_folds_starter.py`: which folds tell the truth about next month's payments?
2:35	15	**1.3** Start the lesson project	A fair model bake-off, Steps 1 to 3; Steps 4 to 6 go home.
2:50	10	Wrap-up	Exit ticket (3 questions), homework, bring list for Session 3.
Total	180		Ends at 3:00.

In the room. The lesson pages estimate 100 + 95 minutes for a solo learner. After the warm-up, the break and the wrap-up the room has 150, so for each lesson the room does the mental model, the live demo, one guided kit lab, and the first three steps of the lesson project.

Goes home (2 to 3 hours). Both projects' Steps 4 to 6 (the honest churn evaluation and the fair bake-off); each lesson's Hands-on Exercises 1 to 3 (Exercise 4 is a stretch); both Quick Quizzes; both journal entries; the "Before the Next Lesson" lists of both lessons. The second lab of each lesson (`fraud_validation_starter.py`, `regularization_check_starter.py`) and the two `debug_*.py` handouts are optional practice.

Protect, in this order: 1.2 demo Steps 2, 8 and 12 to 13 (the wobble, the flattering random split, the leak audit); 1.3 demo Steps 4, 6, 8 and 11 (one line, the pipeline, the paired comparison, the three splitters); TODO 1 to 3 of each guided lab. **Cut first:** 1.2 Step 7 (show the calendar table on the page instead of typing it), then 1.3 Step 10 (show the time-fold output on the page), then 1.2 Step 10 (the group split: one sentence and the two numbers), then the second half of each guided lab.

OPENING SEGMENT

0:00 Warm-up: Retrieval and Homework Check — 10 min

TIMING

5m	Five retrieval questions from Session 1 (workbook Warm-up page): learners write, then compare with a neighbor
5m	Homework check: the `random_state=7` rerun and the `constant=1` dummy; who finished the churn shortlist

FIVE RETRIEVAL QUESTIONS, WITH ANSWERS

1. **W1** On the churn table, what does always predicting "stays" score? And "never fraud" on the payments? **Answer:** 0.835 on churn (16.5% churn) and 0.9912 on fraud (0.88% fraud). Every score today is read against a do-nothing baseline.
2. **W2** Which column of `customers_churn.csv` must never be a feature, and why? **Answer:** `cancellation_reason`: it is filled in only after a customer has left, so on prediction day it is always empty. Today Lesson 1.2 measures what it does to a model (0.848 becomes 0.941).
3. **W3** `train_test_split(X, y, ...)` returns four things. In which order? **Answer:** `X_train, X_test, y_train, y_test`: the X parts, then the y parts. Today's Common Mistake 1 is the wrong order.
4. **W4** Why is the scaler fitted on the training rows only, and then used to transform both sets? **Answer:** The test rows stand in for future customers, whose numbers could not have been seen when the model was built. Today the same rule moves inside every cross-validation fold.
5. **W5** Homework: what did `DummyClassifier(strategy="constant", constant=1)` score on the 1.1 test set? **Answer:** 0.165 (0.165): it calls everyone a churner, so it is right exactly for the 16.5% who churn. Accuracy rewards the majority class.

HOMEWORK CHECK (FROM 1.1 "BEFORE THE NEXT LESSON")

Ask for hands: "Who reran the shortlist project with `random_state=7`? What moved?" The reference rerun (the 1.1 project's split, scaler, baseline and forest, seed 7 in the split only):

```
random_state=42  baseline 0.835  forest 0.873  top-50 hits 41
random_state= 7  baseline 0.835  forest 0.876  top-50 hits 45
```

- **The baseline does not move** (0.835 both times): the split is stratified, so the test churn share, and with it the baseline, is pinned. That is today's second demo step.

- **The forest moves** (0.873 to 0.876) and so does its shortlist (41 to 45 real churners in the top 50). Do not explain it; say: "How much of a score is luck? That is the first experiment after this warm-up." It is the bridge into 1.2.
- **The constant dummy** scores 0.165 (W5). Anyone who predicted 0.835 mixed up which class it always predicts: ask them to say the strategy in words.
- Shortlist not finished? It is not needed today (1.2 and 1.3 start fresh notebooks), but the habit is: ask them to finish it with the 1.2 project this week.

WATCH-OUTS

- **Keep it to 10 minutes.** The warm-up is retrieval, not reteaching. Anything longer than a sentence goes to the 1.2 demo, which revisits each point.
- **NOT READY machines from Session 1:** check them during the warm-up (the learner runs `check_environment.py` while the room writes). Still not ready: pair them for today and fix it at the break.

1.2 Splits, Baselines and Leakage

75 min

OBJECTIVES (FROM THE LESSON PAGE)

- Measure how much a test score moves from one random split to another, and explain why stratifying helps
- Make a train / validation / test split and use each part for its one job
- Split by time and by group when rows are related, and show how much a careless random split flatters a model
- Pick a baseline that fits the problem (most frequent class, mean, yesterday, same day last week)
- Detect target leakage and train-test contamination, and remove them

TIMING IN THE ROOM

10m	Frame: the dress rehearsal; the careless evaluation always looks better
30m	Live demo: 30 splits, stratify, three parts, a time split and forecast baselines, a group split, the leak and the audit
20m	Guided lab: `demand_forecast_starter.py` (a time split and three free baselines for next year's demand)
15m	Start the lesson project: an honest churn evaluation, Steps 1 to 3

MENTAL MODEL TO SAY

Say: "A theater company rehearses for weeks, then holds one **dress rehearsal** run exactly like opening night: full costumes, no stopping, no script. If the dress rehearsal is easier than opening night, a prompter whispering lines or half the audience missing, it tells you nothing about the real show."

Then the rule, written on the board and left there: **evaluate the model under the same conditions it will face in real use.** "If it will predict next month from data up to this month, the test must be later than the training data. If it will score brand-new products, the test must hold products it has never seen. If a column will not exist at prediction time, the model must not see it in testing either."

Then the warning that makes the lesson stick: "Every section today breaks the rule in a different way, and every time, the careless evaluation looks BETTER. Nobody goes looking for a bug when the score goes up. That is why these mistakes survive."

LIVE DEMO (YOU TYPE, THEY FOLLOW)

New notebook, `lesson_1_2`. The code is the lesson's own, in order, minus the plotting cells: show those charts on the page instead of typing them. Ask for a prediction before Steps 2, 5, 8, 10 and 12.

1 **Same customers, same features (1m).** "We start exactly where Lesson 1.1 ended: 6,000 customers, the ten clean numeric features, 16.5% churn."

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.dummy import DummyClassifier

customers = pd.read_csv("data/customers_churn.csv")
features = ["tenure_months", "orders_12m", "spend_12m", "avg_order_value", "da
            "returns_12m", "return_rate", "support_tickets_12m", "discount_sha
X = customers[features]
y = customers["churned"]
print(X.shape, f"churn rate {y.mean():.3f}")
```

```
(6000, 10) churn rate 0.165
```

2 **One split is one sample (3m).** "In 1.1 logistic regression scored 0.848 on the split with seed 42. Forty-two was arbitrary. Let's ask thirty other seeds." Before running: "Write down the lowest and highest score you expect."

```
def split_and_score(seed, test_size=0.25):
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=test_size, stratify=y, random_state=seed
    )
    scaler = StandardScaler().fit(X_train)
    model = LogisticRegression().fit(scaler.transform(X_train), y_train)
    return model.score(scaler.transform(X_test), y_test)

scores = pd.Series([split_and_score(seed) for seed in range(30)])
print(scores.describe().round(3))
```

```
count    30.000
mean      0.851
std       0.007
min       0.836
25%       0.849
50%       0.852
75%       0.855
max       0.866
dtype: float64
```

Show the lesson page's histogram. Say: "Same model, same data. Only the luck of which 1,500 customers landed in the test set changed, and the score moved from 0.836 to 0.866. The model's lead over the baseline is about 0.016: barely two wobbles." Write **wobble ≈ 0.007** on the board; it comes back after the break.

3 **Stratify, or the yardstick moves too (2m).** "Without `stratify=y` the churn share in the test set wobbles as well, and the baseline is one minus that share."

```
test_rates = {"random": [], "stratified": []}
for seed in range(30):
    *_, y_te = train_test_split(X, y, test_size=0.25, random_state=seed)
    test_rates["random"].append(y_te.mean())
    *_, y_te = train_test_split(X, y, test_size=0.25, stratify=y, random_state
    test_rates["stratified"].append(y_te.mean())

print(pd.DataFrame(test_rates).agg(["min", "max"]).round(3))
```

```
     random  stratified
min   0.151       0.165
max   0.181       0.165
```

`*_, y_te = ...` keeps only the last of the four results. Rule for the board: **stratify classification splits on the target, never on a feature.**

4 **Three parts, three jobs (2m).** "Comparing models on the test set again and again turns it into training data: you learn from it. So we carve a third part." Draw the 60/20/20 bar: train fits, validation chooses, test grades once.

```
X_rest, X_test, y_rest, y_test = train_test_split(X, y, test_size=0.20, strati
X_train, X_val, y_train, y_val = train_test_split(X_rest, y_rest, test_size=0.

print(len(X_train), len(X_val), len(X_test))
print(f"churn rates: {y_train.mean():.3f} {y_val.mean():.3f} {y_test.mean():.3
```

```
3600 1200 1200
churn rates: 0.166 0.165 0.165
```

Point at `stratify=y_rest` in the second call: you stratify on the target of the data you are splitting.

5 **Validation makes a real decision (3m).** "How deep should the tree be? Training accuracy will say 'deeper'. Watch validation."

```
from sklearn.tree import DecisionTreeClassifier

val_by_depth = {}
for depth in [2, 3, 4, 5, 6, 8, 12]:
    tree = DecisionTreeClassifier(max_depth=depth, random_state=42).fit(X_trai
    val_by_depth[depth] = tree.score(X_val, y_val)
    print(f"max_depth={depth:2d}  train {tree.score(X_train, y_train):.3f}  va

best_depth = max(val_by_depth, key=val_by_depth.get)
print("chosen depth:", best_depth)
```

```
max_depth= 2  train 0.858  validation 0.849
max_depth= 3  train 0.872  validation 0.864
max_depth= 4  train 0.876  validation 0.864
max_depth= 5  train 0.886  validation 0.859
max_depth= 6  train 0.898  validation 0.858
max_depth= 8  train 0.921  validation 0.848
max_depth=12  train 0.978  validation 0.823
chosen depth: 3
```

Training climbs with every level; validation peaks and falls: overfitting, used to make a choice. Depths 3 and 4 tie exactly (1037 of 1,200 correct each, checked), and `max` keeps the first: the simpler tree, which is the right tie-break.

6 **Open the envelope, once (2m).** "Now, and only now, the test set. Refit the chosen depth on train plus validation, grade it once, next to the baseline."

```
final_tree = DecisionTreeClassifier(max_depth=best_depth, random_state=42).fit
baseline = DummyClassifier(strategy="most_frequent").fit(X_rest, y_rest)

print(f"baseline   test accuracy {baseline.score(X_test, y_test):.3f}")
print(f"tree (d={best_depth}) test accuracy {final_tree.score(X_test, y_test):
```

```
baseline   test accuracy 0.835
tree (d=3) test accuracy 0.858
```

0.858 is a little below the best validation score (0.864). Ask why. (The validation winner is partly the lucky one.) "This is the number you report. If it disappoints, you do NOT go back and try more depths."

7 **A table that has a calendar (2m).** "New question: daily sales per store. On opening night the model is trained on everything up to today and asked about days that have not happened yet."

```
sales = pd.read_csv("data/daily_store_sales.csv", parse_dates=["date"])
sales["store_code"] = sales["store"].map({"Asheville": 0, "Bend": 1, "Boise":
sales["day_index"] = (sales["date"] - sales["date"].min()).dt.days
sales["day_of_week"] = sales["date"].dt.dayofweek
sales["day_of_year"] = sales["date"].dt.dayofyear
sales["closed"] = (sales["event"] == "Closed").astype(int)

calendar = ["store_code", "day_index", "day_of_week", "day_of_year", "promo",
print(sales[["date", "store", "sales"] + calendar].head(5))
```

	date	store	sales	store_code	day_index	day_of_week	day_of_y
0	2023-01-01	Asheville	4838.69	0	0	6	
1	2023-01-01	Bend	4533.55	1	0	6	
2	2023-01-01	Boise	6026.92	2	0	6	
3	2023-01-01	Flagstaff	4217.59	3	0	6	
4	2023-01-02	Asheville	2695.58	0	1	0	

Store as a code, the date as a running day count, day of week, day of year. Keep it quick: the point is the next step.

8 **Random split vs time split (3m).** "Same forest, same features, same number of test days. Once on a random split, once trained on 2023 and 2024 and tested on 2025. Predict: which one looks better?"

```
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error

def forest_mae(train, test):
    forest = RandomForestRegressor(n_estimators=200, min_samples_leaf=3, rando
    forest.fit(train[calendar], train["sales"])
    return mean_absolute_error(test["sales"], forest.predict(test[calendar]))
is_2025 = sales["date"].dt.year == 2025
past, future = sales[~is_2025], sales[is_2025]
shuffled_train, shuffled_test = train_test_split(sales, test_size=len(future),

print(f"test rows: {len(future):,}")
print(f"random split MAE: ${forest_mae(shuffled_train, shuffled_test):,.0f}")
print(f"time split MAE:   ${forest_mae(past, future):,.0f}")
```

```
test rows: 1,460
random split MAE: $613
time split MAE:   $854
```

Show the Boise train/test chart from the page. Say: "The random split claims $600 a day; reality is $850. Sales have momentum, and with a random split almost every test day has its neighbors in training. Tomorrow's neighbors are never available."

9 **Is $850 good? Ask the baselines (3m).** "For a forecast there are two baselines much tougher than the mean, and they cost nothing: yesterday, and the same day last week. Each is a shifted column."

```
from sklearn.dummy import DummyRegressor

sales = sales.sort_values(["store", "date"])
sales["yesterday"] = sales.groupby("store")["sales"].shift(1)
sales["same_day_last_week"] = sales.groupby("store")["sales"].shift(7)
future = sales[sales["date"].dt.year == 2025]

mean_model = DummyRegressor(strategy="mean").fit(past[calendar], past["sales"]
print(f"training mean       MAE ${mean_absolute_error(future['sales'], mean_mo
print(f"yesterday           MAE ${mean_absolute_error(future['sales'], future[
print(f"same day last week  MAE ${mean_absolute_error(future['sales'], future[
```

```
training mean       MAE $1,967
yesterday           MAE $1,350
same day last week  MAE $1,072
```

The forest at $854 beats the strongest free baseline ($1,072), so it earns its place. Measured dishonestly at $613 it would have looked like a triumph. Shifted columns are legal: they only look backward.

10 **Splitting by group (3m).** "Pinecrest plans to launch new products. A random split tests the model on products it already knows. `GroupShuffleSplit` sends every row of a product to one side."

```
from sklearn.model_selection import GroupShuffleSplit

demand = pd.read_csv("data/weekly_demand.csv")
numeric = ["week_of_year", "store_sqft", "list_price", "price", "discount_pct"
           "promo", "holiday_week", "avg_high_temp_f"]

row_train, row_test = train_test_split(demand, test_size=0.25, random_state=42

splitter = GroupShuffleSplit(n_splits=1, test_size=0.25, random_state=42)
train_idx, test_idx = next(splitter.split(demand, groups=demand["product_id"]))
group_train, group_test = demand.iloc[train_idx], demand.iloc[test_idx]

shared = set(group_train["product_id"]) & set(group_test["product_id"])
print(f"group split: {group_test['product_id'].nunique()} test products, {len(

for name, train, test in [("random rows", row_train, row_test), ("new products
    forest = RandomForestRegressor(n_estimators=200, min_samples_leaf=3, rando
    forest.fit(train[numeric], train["units_sold"])
    mean_guess = DummyRegressor().fit(train[numeric], train["units_sold"])
    print(f"{name:12s}  forest MAE {mean_absolute_error(test['units_sold'], fo
          f"   mean baseline MAE {mean_absolute_error(test['units_sold'], mean
```

```
group split: 6 test products, 0 shared with training
random rows   forest MAE 10.23   mean baseline MAE 17.03
new products  forest MAE 17.84   mean baseline MAE 20.87
```

About 10 units off on known products, about 18 on new ones: barely better than the mean. "Neither number is wrong. Choosing the split IS choosing the question."

11 **The leak, counted (1m).** "Session 1 warned you about `cancellation_reason`. Count it against the outcome."

```
customers["has_reason"] = customers["cancellation_reason"].notna().astype(int)
print(pd.crosstab(customers["has_reason"], customers["churned"]))
```

```
churned        0    1
has_reason
0           5008  367
1              0  625
```

A zero in the stayers' column: every one of the 625 customers with a reason churned.

12 **The leak, in a model (2m).** "Add one flag to the ten honest features and retrain the 1.1 model on the 1.1 split." Ask for a guess first.

```
def logreg_accuracy(columns):
    X_tr, X_te, y_tr, y_te = train_test_split(
        customers[columns], y, test_size=0.25, stratify=y, random_state=42
    )
    scaler = StandardScaler().fit(X_tr)
    model = LogisticRegression().fit(scaler.transform(X_tr), y_tr)
    return model.score(scaler.transform(X_te), y_te)

print(f"10 honest features:         {logreg_accuracy(features):.3f}")
print(f"10 features + has_reason:   {logreg_accuracy(features + ['has_reason']
```

```
10 honest features:         0.848
10 features + has_reason:   0.941
```

"Nothing else in this course will lift a churn model this much, and that is the clue. In real use the column is always empty."

13 **The single-feature audit (3m).** "Leaks rarely have a name like this. Two habits catch most of them: ask of every column 'known at prediction time?' (the lesson's flowchart), and fit a tiny model on each feature alone."

```
X_tr, X_te, y_tr, y_te = train_test_split(
    customers[features + ["has_reason"]], y, test_size=0.25, stratify=y, rando
)

single = {}
for column in features + ["has_reason"]:
    stump = DecisionTreeClassifier(max_depth=2, random_state=42).fit(X_tr[[col
    single[column] = stump.score(X_te[[column]], y_te)

print(pd.Series(single).sort_values(ascending=False).head(4).round(3))
print(f"baseline: {1 - y_te.mean():.3f}")
```

```
has_reason               0.941
days_since_last_order    0.842
tenure_months            0.835
spend_12m                0.835
dtype: float64
baseline: 0.835
```

Real signals are weak one at a time (recency alone reaches 0.842). One column alone at 0.941 is the alarm. "From here on, `has_reason` and `cancellation_reason` stay out of every model."

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **The split unpacked in the wrong order, or stratified on a number** (the two loud ones). `X_train, y_train, X_test, y_test = ...` fails at `fit` with `ValueError: y should be a 1d array, got an array of shape (1500, 10) instead.` . `stratify=demand["units_sold"]` fails with `ValueError: The least populated classes in y have only 1 member, which is too few. ...` and a screen of numbers. Coach: "Read the first sentence of the last line. What does it say y holds?" Fix: the X parts first; stratify only a classification target (or bins of a regression target, or split by time).
- **A whole number as the test size** (silent). `test_size=25` means 25 rows, not 25%: `5975 25` . A 25-customer test set wobbles enormously; in the kit's `debug_leaky_split.py` it even reports a perfect 1.000. Coach: "How many customers is your score based on?" Have them print `len(X_test)` after every split this week.
- **Something learns from all the rows before the split** (silent, and the one to fear). Selecting 20 of 2,000 noise columns on all the rows, then splitting: `select, then split: 0.78` , versus `split, then select: 0.47` (a coin flip). A centered 7-day average on daily sales: `time split MAE with a peeking feature: $504` with an honest time split. Coach with the board question: "Could I have computed this the evening before, from the training rows only?" Fix: fit every step that learns (scaler, imputer, selector) on the training rows; build time features from the past (`shift(1)` before `rolling`).

◆ GUIDED LAB

- **Lab A (20m, everyone):** `labs/module-02/l1_2-splits-and-leakage/demand_forecast_starter.py` ; reference `labs/module-02/l1_2-splits-and-leakage/demand_forecast_solution.py` . The planning team wants next year's weekly units for every product. Learners make the honest time split (2024 train, 2025 test), a random split of the same size, grade the same forest both ways, and then build three free baselines: the training mean, each store and product's 2024 average, and the same week last year. New practice on all four 1.2 moves (time split, random split for contrast, forecast baselines, report against the strongest), on data the lesson used only for the group split.
- **Run it first (1m), TODO 1 together (3m), TODO 2 and 3 in pairs (6m), TODO 4 and 5 (8m), debrief (2m).** TODO 1 has a planted surprise: `week_start.dt.year == 2024` holds **53** weeks (2024-12-30 is a Monday of 2024) and 2025 only 51. Ask "104 weeks, so 52 and 52?" before they print it. Checking where the line falls is part of every time split.
- **Expected:** 5,088 train rows (last week 2024-12-30), 4,896 test rows (first week 2025-01-06), 53 and 51 weeks; half the random split's training rows come from 2025; forest MAE 10.45 on the random split vs 10.27 on the time split; baselines: each product's 2024 average 12.95, same week last year 13.81, training mean 17.13; the forest beats the strongest baseline.
- **The honest surprise (debrief):** here the random split does NOT flatter the forest (10.45 vs 10.27), unlike daily sales ($613 vs $854). Weekly demand carries little momentum from one week to the next once season and price are known, so a test week's neighbors help little. Say: "You only know by measuring.

The time split is still the right dress rehearsal, because it is how the model will be used, whatever it costs." Lesson 1.3 meets the same effect with folds (9.84 vs 10.17). Second point: the strongest free baseline is each product's usual level, not last year's same week (a single noisy week); a forecast that cannot beat 12.95 is not worth deploying.

- **Circulate for:** `past` / `future` swapped (a model trained on 2025); `shift(52)` without sorting by store, product and week first (the shift then crosses products: ask them to print three rows); the per-product average computed on ALL rows, 2025 included (a leak: ask "which rows may the baseline learn from?").
- **Fast finishers:** `labs/module-02/l1_2-splits-and-leakage/fraud_validation_starter.py` (train / validation / test for a fraud review queue; reference `labs/module-02/l1_2-splits-and-leakage/fraud_validation_solution.py`) and the handout `labs/module-02/l1_2-splits-and-leakage/debug_leaky_split.py`: two bugs that stop it, two that do not.

START THE LESSON PROJECT

The lesson project is **an honest churn evaluation**: a leak audit, a sealed test set, a choice made on validation, one test score next to the baseline, and a short report (`evaluation_report.txt`). In the room, Steps 1 to 3 (15 minutes).

- Read the brief aloud ("a colleague's churn model is '94% accurate'") and ask: "Where could 94% come from?" Step 1 together: split the whole table three ways so the IDs stay lined up. **Done** = `3600 1200 1200`. From here, `test` is not touched, not even `.head()`.
- Step 2 on their own: the single-feature audit on training and validation rows. **Done** = `suspects: ['has_reason']` with `has_reason` at 0.942 against a 0.835 baseline. Step 3: drop it; **done** = `10 features kept` and baseline validation accuracy 0.835.
- **Goes home:** Step 4 (choose on validation: logistic regression 0.850, tree 0.864, forest 0.870), Step 5 (open the test set once: forest 0.866, baseline 0.835) and Step 6 (`evaluation_report.txt`). Tell them the numbers so they can check themselves.

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab:

- Why stratify on the target and never on a feature? **Answer:** it pins the class share in every part, so the baseline (one minus the churn rate) stays fixed; a feature does nothing for that, and a continuous one crashes.
- Validation said 0.864, the test set said 0.858. Which do you report, and may you now try another depth? **Answer:** report 0.858; no: after one look the test set has been used.
- You forecast tomorrow's sales. Which baseline must the model beat? **Answer:** the strongest free one, same day last week ($1,072 here), not the training mean ($1,967).

- Pinecrest will score products it has never sold. Which split? **Answer:** a group split by product: every row of a product on one side.
- A new column lifts accuracy from 0.85 to 0.94. What two things do you do before celebrating? **Answer:** score it alone against the baseline, and find out when and how it is recorded (known at prediction time?).

Workbook self-check, items Q1 to Q6 (learners do them at the end of the block or at home).

Answers: **Q1** C) A group split, `GroupShuffleSplit(..., groups=demand["product_id"])`, so that no test product appears in training; **Q2** `600 200 200`; **Q3** B) A 7-day average built with `rolling(7, center=True)`; **Q4** D) `ValueError`: `y` should be one-dimensional; the second result is the 1,500 test rows of `X`; **Q5** A) Report 0.858 and keep depth 3: the test set has spoken once; **Q6** `30.0`. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: give each pair one sentence to say before every run in the demo: "This evaluation is easier / harder / the same as real use, because ...". In the lab, let a struggling pair skip TODO 2 and 3 (the random split) and go straight to the baselines: the time split plus one baseline is the core skill.

Stretch: `fraud_validation_starter.py` (a validation set chooses between four models by frauds in the top 100; validation said 47, the test set says 41: the winner was partly lucky); then the lesson's Exercise 4, a reusable leak detector, and the Bonus Challenge: honest lag features for the daily forecast.

1.3 Cross-Validation

75 min

OBJECTIVES (FROM THE LESSON PAGE)

- Explain why one train/validation split gives a noisy score, and how k-fold cross-validation averages that noise away
- Run `cross_val_score` and `cross_validate` with a stratified, shuffled splitter, and report a mean with its spread
- Keep preprocessing inside the folds by cross-validating a `make_pipeline` instead of pre-scaled data
- Compare two models fold by fold on identical folds, and get out-of-fold predictions with `cross_val_predict`
- Choose `TimeSeriesSplit` or `GroupKFold` when rows are related by time or by group

TIMING IN THE ROOM

10m	Frame: one exam or five quizzes; cross-validation scores a recipe
30m	Live demo: the loop by hand, one line, shuffled and stratified folds, a pipeline, train vs validation, paired folds, out-of-fold predictions, time and group folds
20m	Guided lab: `fraud_folds_starter.py` (which folds tell the truth about next month's payments?)
15m	Start the lesson project: a fair model bake-off, Steps 1 to 3

MENTAL MODEL TO SAY

Say: "A teacher who grades a student on one exam learns something, but a bad night's sleep or a lucky choice of questions can swing it. Five shorter quizzes, each on different material, give a steadier picture, and the spread of the five grades tells the teacher how much to trust the average. Cross-validation gives your model five quizzes instead of one exam, and it does so **without using the final exam** at all."

Point at the wobble you wrote on the board before the break (about 0.007 for a test set, about two points between ten splits): "Before the break we measured the noise. Now we average it away, and report its size." Then the two sentences that prevent the most confusion: "Cross-validation evaluates a **recipe**, not a model: it fits five models and throws them away. When you like the recipe, you fit it once on all the training rows." And: "In scikit-learn's names, 'test' inside cross-validation means the validation fold. Our real test set stays locked."

LIVE DEMO (YOU TYPE, THEY FOLLOW)

New notebook, `lesson_1_3`. The lesson's code, in order, without the strip-plot cell (show it on the page). Ask for a prediction before Steps 5, 7, 8 and 11.

1. **One split is one opinion (2m).** "Recall the wobble from before the break. Here it is again for a depth-4 tree and ten seeds." (Say once: this loop touches the whole table, fine for a thought experiment, never in real work.)

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

customers = pd.read_csv("data/customers_churn.csv")
features = ["tenure_months", "orders_12m", "spend_12m", "avg_order_value", "da
            "returns_12m", "return_rate", "support_tickets_12m", "discount_sha
X = customers[features]
y = customers["churned"]

scores = []
for seed in range(10):
    X_a, X_b, y_a, y_b = train_test_split(X, y, test_size=0.25, stratify=y, ra
    tree = DecisionTreeClassifier(max_depth=4, random_state=42).fit(X_a, y_a)
    scores.append(tree.score(X_b, y_b))

print(np.round(scores, 3))
print(f"lowest {min(scores):.3f}   highest {max(scores):.3f}")
```

```
[0.859 0.862 0.879 0.868 0.869 0.867 0.863 0.866 0.874 0.858]
lowest 0.858   highest 0.879
```

About two points between luckiest and unluckiest. "If two models truly differ by one point, one split can crown either."

2. **Lock the test set (1m).** "Everything from here happens inside the 4,500 training customers. The 1,500 test customers are the final exam; we do not touch them."

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=42
)
print(X_train.shape, X_test.shape)
```

```
(4500, 10) (1500, 10)
```

3. **The fold loop, by hand, once (4m).** "Five folds of 900. Five rounds: hold one fold out, train on the other four, score. Every training row is validated exactly once." Draw the 5 × 5 grid from the lesson page.

```
from sklearn.model_selection import StratifiedKFold

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

fold_scores = []
for fold, (train_idx, val_idx) in enumerate(cv.split(X_train, y_train), start=
    tree = DecisionTreeClassifier(max_depth=4, random_state=42)
    tree.fit(X_train.iloc[train_idx], y_train.iloc[train_idx])
    score = tree.score(X_train.iloc[val_idx], y_train.iloc[val_idx])
    fold_scores.append(score)
    print(f"fold {fold}: train {len(train_idx)}, validate {len(val_idx)}, "
          f"churn rate {y_train.iloc[val_idx].mean():.3f}, accuracy {score:.3f

print(f"mean {np.mean(fold_scores):.3f}, std {np.std(fold_scores):.3f}")
```

```
fold 1: train 3600, validate 900, churn rate 0.164, accuracy 0.859
fold 2: train 3600, validate 900, churn rate 0.166, accuracy 0.866
fold 3: train 3600, validate 900, churn rate 0.166, accuracy 0.866
fold 4: train 3600, validate 900, churn rate 0.166, accuracy 0.873
fold 5: train 3600, validate 900, churn rate 0.166, accuracy 0.850
mean 0.863, std 0.008
```

Point at `.iloc` (positions, not labels) and at the FRESH tree inside the loop: reusing a fitted model across rounds would leak one round into the next.

4 **The same thing in one line (2m).** "`cross_val_score` clones the model for every fold, fits, scores, and hands back the five numbers."

```
from sklearn.model_selection import cross_val_score

tree = DecisionTreeClassifier(max_depth=4, random_state=42)
scores = cross_val_score(tree, X_train, y_train, cv=cv)

print(scores.round(3))
print(f"accuracy {scores.mean():.3f} +/- {scores.std():.3f}")
```

```
[0.859 0.866 0.866 0.873 0.85 ]
accuracy 0.863 +/- 0.008
```

The same five numbers as the loop, because the same splitter object made the same folds. Report the mean plus or minus the spread. Then the warning box: it fitted five models and threw them all away. **It scored a recipe.** Afterward you fit the recipe once on all 4,500 rows.

5 **Why we build the splitter ourselves (3m).** "`cv=5` works, but it does not shuffle. Watch what that does to a table that happens to be sorted by the target."

```
from sklearn.model_selection import KFold

sorted_train = X_train.assign(churned=y_train).sort_values("churned")
X_sorted, y_sorted = sorted_train[features], sorted_train["churned"]

for name, splitter in [("KFold(5)", KFold(5)),
                       ("StratifiedKFold", StratifiedKFold(5, shuffle=True, ra
    rates = [y_sorted.iloc[val].mean() for _, val in splitter.split(X_sorted,
    acc = cross_val_score(tree, X_sorted, y_sorted, cv=splitter)
    print(f"{name:16s} churn rate per fold {np.round(rates, 2)}  accuracy {acc
```

```
KFold(5)         churn rate per fold [0.   0.   0.   0.   0.83]  accuracy 0.79
StratifiedKFold  churn rate per fold [0.16 0.17 0.17 0.17 0.17]  accuracy 0.8
```

Unshuffled `KFold` builds folds with no churners and one fold of mostly churners. The course habit for the board: `StratifiedKFold(5, shuffle=True, random_state=42)` for classification, `KFold(5, shuffle=True, random_state=42)` for regression.

6 **Preprocessing goes inside the folds (3m).** "If I scale `X_train` first and then cross-validate, every validation fold helped compute the mean it was scaled with. Same leak as scaling with the test set, smaller and better hidden. `make_pipeline` glues scaler and model into one estimator that is refitted in every round."

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

logreg = make_pipeline(StandardScaler(), LogisticRegression())
print(logreg)

scores = cross_val_score(logreg, X_train, y_train, cv=cv)
print(f"logistic regression accuracy {scores.mean():.3f} +/- {scores.std():.3f
```

```
Pipeline(steps=[('standardscaler', StandardScaler()),
                ('logisticregression', LogisticRegression())])
logistic regression accuracy 0.852 +/- 0.009
```

Show the lesson's flow diagram. Here the leak would be tiny; keep the habit anyway, because steps that learn more (selection, encodings) leak a lot. Lesson 2.2 is all about pipelines.

7 **Train scores next to validation scores (3m).** "`cross_validate` gives several metrics at once and, if you ask, the score on the training folds." Introduce ROC AUC in one breath: how well the model RANKS churners above stayers, 0.5 random, 1.0 perfect; Lesson 1.4 explains it.

```
from sklearn.model_selection import cross_validate

for depth in [4, None]:
    results = cross_validate(
        DecisionTreeClassifier(max_depth=depth, random_state=42), X_train, y_t
        scoring=["accuracy", "roc_auc"], return_train_score=True,
    )
    summary = pd.DataFrame(results).drop(columns=["fit_time", "score_time"]).m
    print(f"max_depth={depth}")
    print(summary.round(3).to_string())
```

```
max_depth=4
test_accuracy     0.863
train_accuracy    0.875
test_roc_auc      0.839
train_roc_auc     0.857
max_depth=None
test_accuracy     0.815
train_accuracy    1.000
test_roc_auc      0.663
train_roc_auc     1.000
```

The unlimited tree: 1.000 on its own training folds, 0.663 AUC on validation. It memorizes. Depth 4: train and validation close. "Comparing the two columns is the cheapest overfitting check you have."

8 **A fair comparison: same folds, paired (4m).** "Two rules: the same splitter for every model, so the scores are paired; and enough rounds to see the noise. Three repeats of five folds give 15 paired scores." Predict first: which wins, by how much?

```
from sklearn.model_selection import RepeatedStratifiedKFold

rcv = RepeatedStratifiedKFold(n_splits=5, n_repeats=3, random_state=42)

models = {
    "tree (depth 4)": DecisionTreeClassifier(max_depth=4, random_state=42),
    "logistic regression": logreg,
}
auc = pd.DataFrame({name: cross_val_score(model, X_train, y_train, cv=rcv, sco
                    for name, model in models.items()})

print(auc.agg(["mean", "std"]).round(3))
diff = auc["tree (depth 4)"] - auc["logistic regression"]
print(f"tree ahead in {(diff > 0).sum()} of {len(diff)} folds; mean lead {diff
```

```
      tree (depth 4)  logistic regression
mean           0.834                0.83
std            0.017                0.02
tree ahead in 12 of 15 folds; mean lead +0.004
```

Show the strip plot on the lesson page. "12 of 15 sounds convincing. But the average lead is 0.004 and one model's folds spread over about 0.07. These two rank customers about equally well; prefer the simpler."

9 **Out-of-fold predictions (3m).** "Sometimes you want predictions, not scores. `cross_val_predict` gives every training customer a probability from the fold model that did NOT train on them."

```
from sklearn.model_selection import cross_val_predict

oof_proba = cross_val_predict(logreg, X_train, y_train, cv=cv, method="predict
print(oof_proba.shape)

oof = pd.DataFrame({"churn_probability": oof_proba, "churned": y_train.to_nump
oof["risk_decile"] = pd.qcut(oof["churn_probability"].rank(method="first"), 10
print(oof.groupby("risk_decile", observed=True)["churned"].mean().round(3).to_
```

```
(4500,)
risk_decile
1     0.022
2     0.033
3     0.033
4     0.040
5     0.056
6     0.089
7     0.167
8     0.258
9     0.380
10    0.576
```

The same decile story as 1.1's test-set chart (top decile about 58%), without touching the test set. "Lesson 1.4 picks a decision threshold from exactly these probabilities."

10 **Time-ordered folds (2m).** "Weekly demand has 96 rows per week, so we split the sorted list of 104 weeks and turn each pair of week lists into row positions. A list of (train, validation) pairs is a fine `cv=` argument."

```
from sklearn.model_selection import TimeSeriesSplit
from sklearn.ensemble import HistGradientBoostingRegressor

demand = pd.read_csv("data/weekly_demand.csv")
demand_features = ["price", "list_price", "discount_pct", "promo", "holiday_we
                   "avg_high_temp_f", "store_sqft", "week_of_year"]
X_d, y_d = demand[demand_features], demand["units_sold"]

weeks = np.sort(demand["week_start"].unique())
time_folds = []
for train_w, val_w in TimeSeriesSplit(n_splits=4, test_size=13).split(weeks):
    train_rows = np.flatnonzero(demand["week_start"].isin(weeks[train_w]))
    val_rows = np.flatnonzero(demand["week_start"].isin(weeks[val_w]))
    time_folds.append((train_rows, val_rows))
    print(f"train {weeks[train_w][0]}..{weeks[train_w][-1]}  validate {weeks[v
```

```
train 2024-01-01..2024-12-23  validate 2024-12-30..2025-03-24
train 2024-01-01..2025-03-24  validate 2025-03-31..2025-06-23
train 2024-01-01..2025-06-23  validate 2025-06-30..2025-09-22
train 2024-01-01..2025-09-22  validate 2025-09-29..2025-12-22
```

The training window grows every quarter, like a forecaster who refits each quarter. Do not dwell on the loop's mechanics; the output is the point.

11 **Three splitters, three questions (3m).** "Same model, three kinds of folds." Before running: "Which one gives the biggest error?" Start it, then talk: gradient boosting on 13 fits takes some seconds on a laptop.

```
from sklearn.model_selection import GroupKFold

hgb = HistGradientBoostingRegressor(random_state=42)
splitters = {
    "shuffled KFold": dict(cv=KFold(5, shuffle=True, random_state=42)),
    "time-ordered": dict(cv=time_folds),
    "new products": dict(cv=GroupKFold(n_splits=4), groups=demand["product_id"
}
for name, kwargs in splitters.items():
    mae = -cross_val_score(hgb, X_d, y_d, scoring="neg_mean_absolute_error", *
    print(f"{name:15s} MAE per fold {mae.round(1)}  mean {mae.mean():.2f}")
```

```
shuffled KFold  MAE per fold [ 9.7  9.9  9.9 10.   9.6]  mean 9.84
time-ordered    MAE per fold [ 7.5 10.9 11.7 10.5]  mean 10.17
new products    MAE per fold [18.7 15.  10.7 19.9]  mean 16.07
```

Shuffled 9.84, time-ordered 10.17 (little week-to-week memory in this data, so only slightly optimistic; the season shows in the per-fold errors), new products 16.07. "Ask: when this model is used for real, what will be new to it? Customers drawn from the same population: shuffled stratified folds. Next season: time-ordered. A new product, store, patient: group folds." Write that question on the board.

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **A step that learns from data runs before cross-validation** (silent). Pre-scaling `X_train` is the small version; the lesson's Common Mistake 1 is the large one: selecting 20 of 5,000 noise columns on all 200 rows, then cross-validating, gives `leaky 0.825` against `honest 0.595` honestly. Coach: "Which line learned something from the data, and did it see the validation fold?" Fix: put every learning step inside the pipeline you cross-validate.
- **Splitter and scorer errors** (loud). `KFold(n_splits=5, random_state=42)` : `ValueError: Setting a random_state has no effect since shuffle is False. You should leave random_state to its default (None), or set shuffle=True.` A misspelled scorer (`"mean_absolute_error"` , `"roc-auc"`) raises `InvalidParameterError` with a list of about 60 valid names. Coach: read the last line, then `sklearn.metrics.get_scorer_names()` ; error metrics exist only as `neg_...` scorers, so flip the sign when you print.
- **Reporting the luckiest number, or cross-validating with the test rows inside** (silent). `best fold 0.873 honest estimate 0.863 +/- 0.008` : the best fold is the single-split problem again. Cross-validating on all 6,000 rows and then "confirming" on the test set is no confirmation (the kit's `debug_cross_validation.py` makes the 0.829 estimate look like 0.850 this way). Coach: "How were

the folds made, and where were the test rows while you chose?" Fix: report mean plus or minus spread of all folds, on `X_train` only.

GUIDED LAB

- **Lab A (20m, everyone):** `labs/module-02/l1_3-cross-validation/fraud_folds_starter.py`; reference `labs/module-02/l1_3-cross-validation/fraud_folds_solution.py`. The fraud model is retrained every month and used on the next. June is locked away; on January to May learners compare what unshuffled `KFold` and `StratifiedKFold` put in each fold, cross-validate a baseline, a pipeline and a tree with training scores, compare two models fold by fold, build next-month folds by hand, and open June once. New practice on every 1.3 move, on a rare class and a time question the lesson does not cover.
- **Run it first (1m), TODO 1 together (3m), TODO 2 and 3 in pairs (8m), TODO 4 and 5 (6m), debrief (2m).** TODO 1 is the predict moment: the rows are in time order and the fraud rate drifts down (1.2 Exercise 3), so unshuffled blocks hold 83 frauds in January's block and 61 in May's.
- **Expected:** frauds per fold `[83, 84, 71, 75, 61]` (KFold) vs `[75, 75, 75, 74, 75]` (stratified); validation AUC baseline 0.500, logistic regression 0.854 ± 0.023 (train 0.857), tree 0.776 ± 0.020; the tree ahead in 0 of 5 folds (mean -0.078); next-month AUC 0.864, 0.844, 0.860, 0.881 (mean 0.862); June AUC 0.874.
- **Debrief (2m):** the time-ordered estimate (0.862) is closer to June (0.874) than the shuffled one (0.854), and both are within one spread of it: on this data the model's ranking holds up month to month. The tree loses every paired fold, a lead (here a deficit) that IS bigger than the noise. And the baseline's 0.500 is the yardstick for AUC, as 0.9912 was for accuracy.
- **Circulate for:** `cv=5` instead of `cv=strat` (works, but not shuffled: ask what is in fold 1); the scaler fitted outside the pipeline; `time_folds` with labels instead of positions (`flatnonzero` gives positions); June used before TODO 5.
- **Fast finishers:** `labs/module-02/l1_3-cross-validation/regularization_check_starter.py` (reference `labs/module-02/l1_3-cross-validation/regularization_check_solution.py`): Session 1's Lab B said C=0.01 beat C=1.0 (0.853 vs 0.848 on one test split); 15 paired folds say otherwise. Then `labs/module-02/l1_3-cross-validation/debug_cross_validation.py`: two loud bugs, three silent ones.

START THE LESSON PROJECT

The lesson project is **a fair model bake-off**: four churn models on identical repeated folds, a winner chosen on the training data alone, one look at the test set, and `cv_leaderboard.csv`. In the room, Steps 1 to 3 (15 minutes).

- Read the brief aloud ("evidence that will survive a skeptical review"). Steps 1 and 2 together: the standard split (`4500 1500`) and the four candidates; point out the scaler inside the logistic regression pipeline and `strategy="prior"` (AUC exactly 0.5: the ranking yardstick).

- Step 3 on their own: one `RepeatedStratifiedKFold`, two metrics, all four candidates. It fits 60 models (the forest dominates: 10 to 40 seconds on a laptop). **Done** = the leaderboard with random forest on top (accuracy 0.871, ROC AUC 0.867 ± 0.018) and the baseline at 0.835 / 0.500.
- **Goes home:** Step 4 (the forest ahead of the tree in 15 of 15 paired folds, mean lead +0.033), Step 5 (the test set once: forest 0.865, tree 0.841) and Step 6 (`cv_leaderboard.csv` and the two-line recommendation). **They keep this notebook: Lesson 1.4 starts from the same split and models.**

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab:

- How many models does `cross_val_score(tree, X_train, y_train, cv=5)` fit, and which one do you keep? **Answer:** five; none. You refit the recipe once on all of `X_train`.
- Why build `StratifiedKFold(5, shuffle=True, random_state=42)` instead of passing `cv=5`? **Answer:** `cv=5` does not shuffle; on rows sorted in any meaningful order the folds are distorted (0% and 83% churners).
- Why does the scaler go into `make_pipeline`? **Answer:** so it is refitted on the training folds of every round and never sees the validation fold.
- The tree beats logistic regression in 12 of 15 folds by 0.004 AUC. Verdict? **Answer:** a tie: the lead is far smaller than the fold-to-fold spread (about 0.02). Prefer the simpler or cheaper model.
- A model will forecast next quarter. What will be new to it, and which folds match? **Answer:** the future; time-ordered folds (train on the past, validate on the period right after).

Workbook self-check, items Q7 to Q12 (learners do them at the end of the block or at home).

Answers: **Q7** `8`; **Q8** B) `ValueError`: setting a `random_state` has no effect since `shuffle` is False; **Q9** D) Each validation fold helped compute the median that filled it: put the filling step inside a pipeline so it is refitted on the training folds of every round; **Q10** C) `GroupKFold(n_splits=5)` with `groups=customer_id`; **Q11** `[-3. -0.5 -3. ]`; **Q12** A) The estimate held: report 0.865 and keep the forest. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: a card with the three splitter questions ("new customers? next period? new group?") and the three function names (`cross_val_score` for one score, `cross_validate` for several plus train scores, `cross_val_predict` for predictions). In the lab, a struggling pair may stop after TODO 3; TODO 4's index-building is mechanics, and the output is what matters.

Stretch: `regularization_check_starter.py`; then the lesson's Exercise 4 (nested cross-validation) and the Bonus Challenge: 50 paired folds for forest vs logistic regression and a histogram of the differences, then a different `random_state` for the splitter. Does the winner change?

CLOSING

EXIT TICKET (3 QUESTIONS, NO NOTES, 4 MINUTES)

Put the three questions on the slide. Learners answer in the workbook's Exit Ticket box or on a sticky note. Sort the notes into "got it" and "not yet" before Session 3: question 3 tells you whether the Session 3 warm-up needs a minute on pipelines inside folds before Lesson 1.4 uses `cross_val_predict` on a pipeline.

1. Name the three parts of a three-way split and the one job of each. Which part is opened only once? **Answer: Training** fits every candidate; **validation** compares candidates and chooses; **test** grades the final choice, opened once at the very end (or cross-validation on the training rows replaces the fixed validation set).
2. A sales forecast scores MAE $613 on a random split and $854 when trained on 2023 and 2024 and tested on 2025. Which number do you report, and why is the other one lower? **Answer: $854**, the time split: it is how the forecast will be used (trained on the past, asked about the future). The random split is lower because almost every test day has its neighbors, the day before and the day after, in training, and sales have momentum; tomorrow's neighbors are never available.
3. Why do you cross-validate `make_pipeline(StandardScaler(), LogisticRegression())` instead of scaling `X_train` first and cross-validating the model? **Answer: So the scaler is refitted inside every round on the training folds only.** Pre-scaling lets every validation fold help compute the mean and standard deviation it is scaled with: a leak, tiny for a scaler and large for steps that learn more (selecting features, encoding categories).

HOMEWORK (ABOUT 2 TO 3 HOURS)

- **Finish both lesson projects.** 1.2 Steps 4 to 6 (validation chooses the forest at 0.870; test 0.866 against 0.835; `evaluation_report.txt`). 1.3 Steps 4 to 6 (15 of 15 paired folds, mean lead +0.033; test AUC 0.865 for the forest, 0.841 for the tree; `cv_leaderboard.csv`).
- **Lesson exercises.** 1.2: Exercises 1 to 3 (bigger test sets wobble less; a time split with `shuffle=False`; the June fraud rate); Exercise 4 (a leak detector) is a stretch. 1.3: Exercises 1 to 3 (3, 5 or 10 folds; the cross-validated overfitting curve; a new store with `GroupKFold`); Exercise 4 (nested cross-validation) is a stretch.
- **Both Quick Quizzes** on the lesson pages (five questions each; answers in the Answer Key).
- **Journal:** both entries in `journal.md`. 1.2: the dress rehearsal for the problem you framed in Lesson 0.1. 1.3: "one exam or five quizzes" for a colleague, and the splitter your own project needs.
- **Before the Next Lesson (1.2):** look through `data/README.md` for any column, in any file, that might be recorded after the event it would help predict, and note it in the journal; rerun the 1.2 project with `random_state=7` in both splits (does the same model win on validation, and how far does the test score move?).
- **Before the Next Lesson (1.3):** keep the bake-off notebook; rerun Step 3 with `scoring=["accuracy", "roc_auc", "f1"]` and write down how the baseline's F1 compares with its accuracy, and guess why; think

of one dataset from your own work where rows are related by time, person or place, and which splitter you would use.

- **Optional practice:** `labs/module-02/l1_2-splits-and-leakage/fraud_validation_starter.py` and `labs/module-02/l1_3-cross-validation/regularization_check_starter.py` if you did not reach them; the two `debug_*.py` handouts.

BRING LIST FOR SESSION 3

- A laptop on which `check_environment.py` ends with READY (the same environment: nothing new to install).
- The `pinecrest-ml` folder with `data/` and `labs/`. Session 3's starters are in `labs/module-03/` (handed out in class).
- **The finished Lesson 1.3 bake-off notebook** (`lesson_1_3_project`, with `cv_leaderboard.csv`): Lesson 1.4 starts from the same split and models, and the F1 rerun is its opening question. Also `evaluation_report.txt` from 1.2.
- Your journal with the `data/README.md` leak notes and the F1 guess: Lesson 1.4 explains F1 and why it can disagree with accuracy, and Lesson 2.1 brings in the columns left out so far (text and gaps).
- The workbook. No downloads, no accounts, no internet needed.

IF YOU RUN LONG OR SHORT

RUNNING LONG: WHAT TO PROTECT AND WHAT TO CUT FIRST

- **Protect:** 1.2 demo Steps 2, 8, 12 and 13; 1.3 demo Steps 4, 6, 8 and 11; TODO 1 to 3 of each guided lab; Step 1 of each project (the split that the rest depends on).
- **Cut first (saves 4m):** 1.2 Step 7 and 1.3 Step 10: show the calendar table and the time-fold output on the lesson page, do not type them.
- **Cut second (saves 3m):** 1.2 Step 10 (group split): one sentence and the two numbers (10.2 vs 17.8 units).
- **Cut third (saves 8m):** each guided lab stops after TODO 3; TODO 4 and 5 go home with the expected output from the starter.
- **Cut fourth (saves 10m):** the 1.3 project start shrinks to Steps 1 and 2; Step 3 (the 60-fit cross-validation) runs at home.

RUNNING SHORT: WHAT TO ADD

- Run `debug_leaky_split.py` together: let the room call out each bug before you run the fix. The 1.000 accuracy on 25 customers is worth the minute.
- The 1.3 Exercise 2 live: tree depths 2 to None with `cross_validate` and training scores (validation AUC peaks at depth 4, 0.839; depth 12 falls to 0.635).

- Start the 1.2 Bonus Challenge: honest lag features for the daily forecast (`shift(1)` then `rolling(7)`), and compare with the $854 calendar-only forest and the $1,072 seasonal-naive baseline.

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 3: Metrics and Preprocessing

How to teach Session 3 of Machine Learning Foundations. The first half turns "84.8% accurate" into the numbers a business can act on: the confusion matrix, precision and recall, a threshold chosen from costs on training data only, ranking metrics, and honest regression errors. The second half brings the rest of the churn table into the model: gaps filled (and remembered), units scaled, categories encoded, all in one ColumnTransformer learned from the training rows. Learners leave knowing that the same model can earn $2,280 or $3,555 depending only on its threshold, and that preparing all 16 columns lifts this linear model's ROC AUC only from 0.823 to 0.828: an honest result they could finally measure.

`2 lessons` `3 hrs session` `Lessons 1.4–2.1` `Session 3 of 11`

HOW TO USE THIS GUIDE

- Same rhythm for both lessons: frame (the goal and one mental model) → live demo (you type, they type along) → guided lab (a kit lab, new practice on the lesson's skills) → start the lesson's own project. The lesson exercises, quiz and journal are homework.
- Type, do not paste. Every code block in a demo script below was run to produce the output printed under it (Python 3.12, pandas 3.0.6, scikit-learn 1.9.1 on 4 CPU threads). The demos follow the lesson pages' own code, so the numbers match the pages exactly; ask for a prediction before every run that has a number in it.
- The lessons are the reference. Keep `evaluation_metrics.html` and `preprocessing_feature_engineering.html` open in browser tabs for the pictures (the confusion-matrix layout, the side-by-side matrices at 0.5 and 0.2, the ROC and precision-recall curves, the profit curve, the three histograms, the ColumnTransformer diagram). Labs live in `labs/module-03/`: `_starter.py` files are handouts, `_solution.py` files are yours.
- Lesson 1.4 is the densest page of Module 1 (105 minutes for a solo learner). The room does not try to cover all of it: the multiclass section is an optional demo step, and Exercises 1 to 4 go home.
- Teal blocks are hands-on, amber blocks are watch-outs. Lesson codes (1.4, 2.1) match the course site.

THE #1 RISK OF SESSION 3: THE THRESHOLD GETS CHOSEN ON THE TEST SET

Lesson 1.4 gives learners a powerful new dial, and the natural thing to do with a dial is to turn it until the test score looks best. Every time someone in the room picks a threshold from a test-set table, say it out loud: "That is tuning on the test set." The demo shows the honest routes (out-of-fold probabilities, `TunedThresholdClassifierCV`), the lab practices one, and the project is built on it. The same risk returns after the break in a quieter form: `fit_transform` on the test rows. It raises no error. The checks in Step 4 of the 2.1 project (training columns centered exactly, test columns only close to center) and the `debug_preprocess.py` handout are there to make it visible. Second risk: the pace. Two dense lessons; protect the order below and cut from the bottom of the list.

BEFORE THE SESSION

- ☐ **Run the kit check once:** `python3 labs/check_labs.py module-03` from this kit's folder. Every line should say PASS (8 files, about 30 seconds). The two `debug_*.py` handouts are deliberately broken and are not run.
- ☐ **Files to hand out:** `labs/module-03/` **without** the `*_solution.py` files (the starters plus `debug_metrics.py` and `debug_preprocess.py`). Learners put it inside `labs/` in their `pinecrest-ml` folder, next to `module-01` and `module-02`. Nothing new to install and nothing to download: everything today is scikit-learn 1.9 (`FixedThresholdClassifier`, `TunedThresholdClassifierCV` and `root_mean_squared_error` need 1.5 or newer; the environment check from Session 1 already confirmed 1.9).
- ☐ **Projector machine:** your own `pinecrest-ml`, `(.venv)` active, JupyterLab started from that folder, two empty notebooks ready: `lesson_1_4` and `lesson_2_1`. Run the 1.4 demo once before class: the forest and `TunedThresholdClassifierCV` take a few seconds each, and you want to know how long.
- ☐ **Board, before learners arrive:** the empty 2×2 confusion matrix (rows: really stayed / really churned; columns: predicted stays / predicted churns); the line "Profit = 60 × TP − 15 × (TP + FP)"; and, for after the break, three bowls: "gaps → imputer, units → scaler, words → encoder".
- ☐ **Browser tabs:** `evaluation_metrics.html` (scrolled to the side-by-side confusion matrices), `preprocessing_feature_engineering.html` (the ColumnTransformer diagram), and the 1.3 project page in case anyone needs `cross_val_predict` again.
- ☐ **Print or share:** the workbook pages for Session 3, and sticky notes for help flags and the exit ticket. Collect the Session 2 exit tickets if you have not reviewed them: the warm-up builds on them.

KNOWN ENVIRONMENT AND CODE TRAPS

SYMPTOM	CAUSE	FIX
`ImportError: cannot import name 'FixedThresholdClassifier'` or	scikit-learn older than 1.5 (usually a notebook kernel that is not	Check `sklearn.__version__`; select the `.venv` kernel. The

SYMPTOM	CAUSE	FIX
`'root_mean_squared_error'`	`.venv`)	course pins 1.9.
`AttributeError: 'float' object has no attribute 'round'`	scikit-learn 1.9 metric functions return plain Python floats	`round(x, 3)` or an f-string, not `.round(3)`.
`ValueError: y should be a 1d array, got an array of shape (1500, 2)`	Both probability columns passed to a metric	`predict_proba(X)[:, 1]`.
A precision that equals the recall you expected (no error)	Arguments swapped: every metric is `(y_true, y_pred)`	Truth first; or keyword arguments `y_true=..., y_pred=...`.
An AUC much lower than expected (0.615 instead of 0.823; no error)	AUC computed from `predict` labels	Pass scores: `predict_proba(X)[:, 1]`.
`UndefinedMetricWarning: Precision is ill-defined`	The model never predicts the positive class	Nothing is broken; pass `zero_division=0` when you expect it.
`TunedThresholdClassifierCV` cell seems to hang	It fits the model 5 times; with a forest, several seconds on a laptop	Wait. Run it once before class to know the time.
`ValueError: Pandas output does not support sparse data`	`OneHotEncoder()` default sparse output with `set_output(transform="pandas")`	`OneHotEncoder(sparse_output=False)`.
`ValueError: Cannot use median strategy with non-numeric data`	One imputer pointed at text and numbers together	Separate branches; `most_frequent` or `constant` for text.
`ValueError: Found unknown categories ['Reno'] in column 0 during transform`	Default `handle_unknown="error"`	`handle_unknown="ignore"`.

SESSION 3 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Five retrieval questions on Session 2 (splits, leakage, cross-validation), and the homework check: the baseline's F1.

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:10	10	**1.4** Frame and mental model	The smoke alarm: recall, precision, and how sensitive to set it.
0:20	35	**1.4** Live demo	Confusion matrix, precision/recall/F1, the threshold table, ROC AUC and average precision, profit and TunedThresholdClassifierCV, MAE/RMSE/R^2 on the demand forecast.
0:55	20	**1.4** Guided lab	`labs/module-03/l1_4-metrics/fraud_review_starter.py`: a fraud review queue chosen from costs.
1:15	10	**1.4** Start the lesson project	Plan the retention campaign, Steps 1 and 2; Steps 3 to 6 go home.
1:25	10	Break	Leave the confusion matrix and the profit formula on the board.
1:35	10	**2.1** Frame and mental model	Mise en place: one treatment per kind of column, learned from training rows.
1:45	30	**2.1** Live demo	Gaps and their meaning, SimpleImputer with an indicator, three scalers and a log, one-hot and unknown categories, ordered tiers, the ColumnTransformer, 16 columns against 10.
2:15	20	**2.1** Guided lab	`labs/module-03/l2_1-preprocessing/fraud_columns_starter.py`: the product category joins the fraud model.
2:35	15	**2.1** Start the lesson project	A model-ready customer table, Steps 1 to 3; Steps 4 to 6 go home.
2:50	10	Wrap-up	Exit ticket (3 questions), homework, bring list for Session 4.
Total	180		Ends at 3:00.

In the room. The lesson pages estimate 105 + 95 = 200 minutes for a solo learner, and after the warm-up, the break and the wrap-up the room has 150. So the room does, for each lesson: the mental model, the live demo, one guided kit lab, and the first steps of the lesson project. Each lesson's second kit lab and the debug handouts are for fast finishers and home.

Goes home (2 to 3 hours). The retention-campaign plan Steps 3 to 6 and the model-ready customer table Steps 4 to 6; each lesson's Hands-on Exercises 1 to 3 (Exercise 4 is a stretch); both Quick Quizzes; both journal entries; the "Before the Next Lesson" lists of 1.4 and 2.1; optionally the B labs (`store_forecast_starter.py`, `demand_categories_starter.py`) and the two debug handouts.

Protect, in this order: the 1.4 demo Steps 2 to 4 and 8 to 9 (the confusion matrix, precision and recall, the threshold, and choosing the threshold on training data: the session's main idea); the 2.1 demo Steps 2 to 4 and 11 to 13 (gaps and their meaning, the ColumnTransformer, the honest 0.823 to 0.828); TODO 4 and 5 of the fraud review

lab. **Cut first:** the optional multiclass step (it is already marked optional), then 1.4 demo Steps 6 and 10 (the brute-force AUC and the toy MAE/RMSE: one sentence each), then 2.1 demo Steps 6 and 7 (the scalers: show the table on the lesson page), then the 1.4 project start (send all six steps home with the two check numbers).

WARM-UP

0:00 Warm-up: Retrieval and Homework Check — 10 min

TIMING

6m	Five retrieval questions on Session 2 (W1 to W5): think, pair, then cold-call
4m	Homework check: who finished both Session 2 projects? Who reran the bake-off with three scorers (W5)?

SEQUENCE AND TALKING POINTS

- **Put W1 to W5 on the slide** (they are also in the workbook's Warm-up box). One minute alone, one minute with a neighbor, then cold-call gently. Do not lecture the answers; let a learner say each, then add one sentence.
- **Homework check by show of hands:** the 1.2 honest evaluation (`evaluation_report.txt`), the 1.3 bake-off (`cv_leaderboard.csv`), and the rerun with three scorers. Anyone who did not finish the 1.3 project: today's project re-creates the same split and needs `cross_val_predict`, which the demo shows again.
- **End on W5** and leave the three numbers on the board: accuracy 0.835, AUC 0.5, F1 0.0. Say: "Same model, three verdicts. Which one is right depends on the question. That is Lesson 1.4."

WARM-UP QUESTIONS AND ANSWERS

1. **W1 (Lesson 1.2).** Why does every churn split in this course pass `stratify=y`? **Answer:** It keeps the churn share (16.5%) the same in the training and test parts. Without it the class balance wobbles from split to split, and with it the baseline and every score wobble too.
2. **W2 (Lesson 1.2).** In the honest-evaluation project, a depth-2 tree on the single column `has_reason` scored 0.942 on validation against a baseline of 0.835. What do you do, and what do you check first? **Answer:** Treat it as a leak suspect: far too good for one column. Check when and how it is recorded (it is filled only when an account is closed, after the outcome), then drop it. A high score alone is not proof; the recording process is.
3. **W3 (Lesson 1.3).** Why cross-validate `make_pipeline(StandardScaler(), LogisticRegression())` instead of scaling `X_train` once and cross-validating the model on the scaled table? **Answer:** The pipeline refits the scaler on the training folds of every round. Scaling first lets each validation fold shape the mean and standard deviation: a small, hidden leak, and a big one when the preprocessing learns more (today's imputers and encoders).
4. **W4 (Lesson 1.3).** What does `cross_val_predict(model, X_train, y_train, cv=cv, method="predict_proba")[:, 1]` give you, and why is it honest? **Answer:** One churn probability

per training customer, each from the fold model that did NOT train on that customer. Today's project chooses its decision threshold from exactly these out-of-fold probabilities, so the test set stays sealed.

5. **W5 (Homework check (1.3, Before the Next Lesson)).** You reran the bake-off with `scoring=["accuracy", "roc_auc", "f1"]`. What did the baseline score on accuracy and on F1, and why so different? **Answer:** Accuracy 0.835, ROC AUC 0.5, F1 0.0. The baseline never predicts churn, so it has no true positives: recall is 0 and precision is 0/0 (scikit-learn reports 0 with a warning), so F1 is 0. Accuracy rewards the 83.5% it gets right by saying "stays". That gap is today's first topic.

The W5 numbers, from a real run of the bake-off's baseline with three scorers (`DummyClassifier(strategy="prior")`, 5 stratified folds on the 4,500 training customers):

```
{'test_accuracy': 0.835, 'test_roc_auc': 0.5, 'test_f1': 0.0}
```

READING THE ROOM

- **W2 missed** ("0.942 is great, keep it"): the leakage habit has not landed. Mention it again at Step 1 of the 2.1 demo (why `cancellation_reason` stays out) and at the ColumnTransformer's `remainder="drop"`.
- **W3 or W4 missed:** the pipeline-inside-the-folds idea is shaky, and today leans on it twice (threshold tuning, and 2.1's "fit on train only"). Slow down at 1.4 demo Step 9 and say what `TunedThresholdClassifierCV` refits in each fold.
- **W5 not attempted by most:** fine, it is answered in the first ten minutes of 1.4. Ask them to note the baseline's F1 in their journal tonight.

LESSON PLANS

1.4 Metrics for Classification and Regression

75 min

OBJECTIVES (FROM THE LESSON PAGE)

- Read a confusion matrix and compute precision, recall and F1, saying what each means for Pinecrest's retention team
- Move a classifier's decision threshold and explain the precision-recall trade-off it controls
- Use ROC AUC and average precision to judge how well a model ranks, and know when the precision-recall view is the honest one
- Choose a threshold from business costs with out-of-fold probabilities or TunedThresholdClassifierCV, without peeking at the test set
- Report MAE, RMSE and R^2 for a regression against a baseline, and avoid the MAPE trap on zero sales

TIMING IN THE ROOM

10m	Frame: the smoke alarm; precision, recall and the threshold in one picture
35m	Live demo: confusion matrix, precision/recall/F1, the threshold table and FixedThresholdClassifier, ROC AUC and average precision, profit and TunedThresholdClassifierCV, regression metrics
20m	Guided lab: `fraud_review_starter.py` (a review queue chosen from costs)
10m	Start the lesson project: plan the retention campaign, Steps 1 and 2

MENTAL MODEL TO SAY

Say: "A smoke alarm that never goes off is right on almost every day of the year, because almost every day has no fire. Nobody would call it a good alarm. What you want to know is two things. When there is a fire, does it go off? That is **recall**. When it goes off, is there really a fire? That is **precision**. And how sensitive to set it, the **threshold**, depends on what a false alarm costs compared with a missed fire."

Then point back at the warm-up's three numbers: "The baseline is the alarm that never goes off: 83.5% accurate, F1 zero." And the business frame for the whole lesson, written under the 2×2: **a wasted call costs a little, a missed churner costs a lot.** "Accuracy treats both mistakes the same. Pinecrest does not."

LIVE DEMO (YOU TYPE, THEY FOLLOW)

New notebook, `lesson_1_4`, in each learner's `pinecrest-ml`. The code is the lesson page's own, so every number matches the page. Steps 2 to 4 and 8 to 9 are the heart: go slowly there and fast elsewhere. Step 12 is optional.

1 **Same split, two models, one question (3m).** "Lesson 1.1 left us with an argument: 84.8% accurate, baseline 83.5%. Is the model any good? Today we stop arguing and count." The logistic regression gets its scaler inside a pipeline, as in Lesson 1.3.

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier

customers = pd.read_csv("data/customers_churn.csv")
features = ["tenure_months", "orders_12m", "spend_12m", "avg_order_value", "da
            "returns_12m", "return_rate", "support_tickets_12m", "discount_sha
X_train, X_test, y_train, y_test = train_test_split(
    customers[features], customers["churned"], test_size=0.25, stratify=custom
)

logreg = make_pipeline(StandardScaler(), LogisticRegression()).fit(X_train, y_
forest = RandomForestClassifier(n_estimators=200, min_samples_leaf=5, random_s

y_pred = logreg.predict(X_test)
proba = logreg.predict_proba(X_test)[:, 1]
print(f"accuracy {logreg.score(X_test, y_test):.3f}")
print(f"real churners in the test set: {y_test.sum()}, flagged by the model: {
```

```
accuracy 0.848
real churners in the test set: 248, flagged by the model: 112
```

Ask: "112 flagged, 248 real churners. How many of the 112 were right?" Nobody can say from accuracy. That is the point.

2 **Count the four cells (3m).** "Right in two ways, wrong in two ways." Draw the 2×2 on the board first (rows: what really happened; columns: what the model said), then run it and fill in the numbers.

```
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_test, y_pred)
print(cm)

tn, fp, fn, tp = cm.ravel()
print(f"TN {tn}   FP {fp}   FN {fn}   TP {tp}")
```

```
[[1206   46]
 [ 182   66]]
TN 1206   FP 46   FN 182   TP 66
```

Read it as the care team would: 112 calls, 66 reach real churners, 46 are wasted; 182 churners are missed. Then: "1,206 + 66 = 1,272 right out of 1,500: that is your 84.8%. Most of it is people we correctly left alone."

3 **Two questions, two numbers (4m).** "Precision: of the people we call, what share were really leaving? Recall: of the people who were leaving, what share did we call?" Point at the cells as you say each fraction.

```
from sklearn.metrics import precision_score, recall_score, f1_score, classific

print(f"precision {precision_score(y_test, y_pred):.3f}")
print(f"recall    {recall_score(y_test, y_pred):.3f}")
print(f"F1        {f1_score(y_test, y_pred):.3f}")
print()
print(classification_report(y_test, y_pred, target_names=["stayed", "churned"]
```

```
precision 0.589
recall    0.266
F1        0.367

              precision    recall  f1-score   support

      stayed      0.869     0.963     0.914      1252
     churned      0.589     0.266     0.367       248

    accuracy                          0.848      1500
   macro avg      0.729     0.615     0.640      1500
weighted avg      0.823     0.848     0.823      1500
```

Translate out loud: "About six calls in ten reach a real churner; we reach about one churner in four." The `classification_report` row that matters is `churned`; the macro and weighted averages come back in the multiclass part (optional, Step 12 below).

4 **Move the threshold (4m).** "`predict` says churn at 0.5. Nothing about 0.5 is special: it is a default, not a decision." Before running: "If I lower the threshold to 0.2, what happens to recall? To precision?"

```
y_true = y_test.to_numpy()
rows = []
for threshold in [0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7]:
    flagged = (proba >= threshold).astype(int)
    rows.append({"threshold": threshold, "flagged": flagged.sum(),
                 "precision": precision_score(y_true, flagged),
                 "recall": recall_score(y_true, flagged),
                 "f1": f1_score(y_true, flagged)})
print(pd.DataFrame(rows).round(3).to_string(index=False))
```

```
 threshold  flagged  precision  recall    f1
       0.1      700      0.313   0.883 0.462
       0.2      401      0.404   0.653 0.499
       0.3      250      0.484   0.488 0.486
       0.4      167      0.557   0.375 0.448
       0.5      112      0.589   0.266 0.367
       0.6       69      0.609   0.169 0.265
       0.7       30      0.767   0.093 0.165
```

No row is correct: each trades wasted calls for missed churners. F1 peaks around 0.2 to 0.3, but F1 does not know Pinecrest's costs either. Show the lesson's side-by-side confusion matrices (0.5 vs 0.2): 96 more churners reached for 193 more wasted calls.

5 **Make the threshold part of the model (2m).** "If 0.2 is the decision, save it WITH the model, so `predict` uses it."

```
from sklearn.model_selection import FixedThresholdClassifier

logreg_02 = FixedThresholdClassifier(logreg, threshold=0.2).fit(X_train, y_tra
print(logreg_02.predict(X_test).sum(), "flagged")
print(f"recall {recall_score(y_test, logreg_02.predict(X_test)):.3f}")
```

```
401 flagged
recall 0.653
```

Same 401 as the 0.2 row of the table. Say once: "Never pick the threshold by looking at the test set. That is tuning on the test set. In ten minutes I show you how to pick it on training data."

6 **Ranking quality: ROC AUC and average precision (4m).** "Two summaries that do not depend on any threshold. ROC AUC: the chance a random churner outranks a random stayer. Average precision: the precision-recall view, whose random baseline is the churn rate, 0.165, not 0.5."

```
from sklearn.dummy import DummyClassifier
from sklearn.metrics import roc_auc_score, average_precision_score

dummy = DummyClassifier(strategy="prior").fit(X_train, y_train)
for name, model in [("baseline", dummy), ("logistic regression", logreg), ("ra
    p = model.predict_proba(X_test)[:, 1]
    print(f"{name:20s} ROC AUC {roc_auc_score(y_test, p):.3f}   average precis
```

```
baseline             ROC AUC 0.500   average precision 0.165
logistic regression  ROC AUC 0.823   average precision 0.497
random forest        ROC AUC 0.865   average precision 0.603
```

Show the lesson's ROC and precision-recall curves on the projector rather than drawing them live. The forest ranks better on both.

7 **Check the meaning of AUC by brute force (2m).** "Do not take my word for it: compare every churner with every stayer and count the wins."

```
churner_p = proba[y_true == 1]
stayer_p = proba[y_true == 0]
wins = (churner_p[:, None] > stayer_p[None, :]).mean()
print(f"{len(churner_p)} x {len(stayer_p)} pairs, churner ranked higher in {wi
```

```
248 x 1252 pairs, churner ranked higher in 0.823 of them
```

0.823, the same number as `roc_auc_score`. This is the moment ROC AUC stops being a mystery.

8 **Put money on the four cells (4m).** Write on the board: offer $15 to anyone contacted; $60 for each real churner reached; nothing for customers left alone. "Profit = 60 × TP − 15 × (TP + FP)." Ask for predictions: contact everyone? The default threshold?

```
OFFER_COST = 15     # dollars per customer contacted
CHURNER_VALUE = 60  # expected dollars per real churner contacted


def campaign_profit(y_true, flagged):
    """Expected profit of contacting every flagged customer."""
    y_true, flagged = np.asarray(y_true), np.asarray(flagged)
    tp = ((flagged == 1) & (y_true == 1)).sum()
    return CHURNER_VALUE * tp - OFFER_COST * flagged.sum()


print("contact nobody   ", campaign_profit(y_test, np.zeros(len(y_test), dtype
print("contact everyone ", campaign_profit(y_test, np.ones(len(y_test), dtype=
print("threshold 0.5    ", campaign_profit(y_test, y_pred))
```

```
contact nobody    0
contact everyone  -7620
threshold 0.5     2280
```

Contacting everyone LOSES $7,620. Then: "If the probabilities were perfect, contact everyone above 15 / 60 = 0.25, the break-even. They are not perfect, so we measure."

9 **Choose the threshold honestly (5m).** "TunedThresholdClassifierCV cross-validates on the training rows only, tries 100 thresholds per fold, keeps the best average profit. `make_scorer` turns our profit function into a scorer." Warn: this cell takes a few seconds.

```
from sklearn.metrics import make_scorer
from sklearn.model_selection import StratifiedKFold, TunedThresholdClassifierC

profit_scorer = make_scorer(campaign_profit)
cv = StratifiedKFold(5, shuffle=True, random_state=42)

tuned = TunedThresholdClassifierCV(logreg, scoring=profit_scorer, cv=cv).fit(X
print(f"chosen threshold {tuned.best_threshold_:.3f}")
print(f"mean profit per validation fold (900 customers) {tuned.best_score_:,.0
print(f"test-set profit {campaign_profit(y_test, tuned.predict(X_test)):,}")
```

```
chosen threshold 0.245
mean profit per validation fold (900 customers) 2,603
test-set profit 3,555
```

0.245, next to the break-even 0.25: the logistic regression's probabilities are reasonably calibrated. Same model, same probabilities, only the decision rule changed: $3,555 instead of $2,280. The fold profit (900 customers) and the test profit (1,500) are not comparable.

10 **Regression: two error sets with the same MAE (1m).** "No confusion matrix for a number: every prediction is off by some amount."

```
steady = np.array([5, 5, 5, 5, 5])
one_big_miss = np.array([0, 0, 0, 0, 25])
for name, errors in [("steady", steady), ("one big miss", one_big_miss)]:
    print(f"{name:13s} MAE {np.abs(errors).mean():.2f}   RMSE {np.sqrt((errors
```

```
steady        MAE 5.00   RMSE 5.00
one big miss  MAE 5.00   RMSE 11.18
```

Same MAE, very different RMSE. "If one big miss hurts more than five small ones (an empty shelf on Black Friday), RMSE describes your pain."

11 **Grade the demand forecast against two baselines (3m).** "Train on 2024, forecast 2025: a time split from Lesson 1.2. Two baselines: the overall mean, and each product-and-store pair's 2024 average."

```
from sklearn.dummy import DummyRegressor
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.metrics import mean_absolute_error, root_mean_squared_error, r2_s

demand = pd.read_csv("data/weekly_demand.csv")
demand_features = ["price", "list_price", "discount_pct", "promo", "holiday_we
                   "avg_high_temp_f", "store_sqft", "week_of_year"]
train_d = demand[demand["year"] == 2024]
test_d = demand[demand["year"] == 2025]

pair_mean = train_d.groupby(["product_id", "store"])["units_sold"].mean().rena
predictions = {
    "overall mean": DummyRegressor().fit(train_d[demand_features], train_d["un
    "product-store mean": test_d.join(pair_mean, on=["product_id", "store"])["
}
hgb = HistGradientBoostingRegressor(random_state=42).fit(train_d[demand_featur
predictions["gradient boosting"] = hgb.predict(test_d[demand_features])

y_d = test_d["units_sold"]
for name, pred in predictions.items():
    print(f"{name:19s} MAE {mean_absolute_error(y_d, pred):6.2f}   RMSE {root_
          f"R2 {r2_score(y_d, pred):6.3f}")
```

```
overall mean        MAE  17.13   RMSE  26.21   R2 -0.006
product-store mean  MAE  12.95   RMSE  20.45   R2  0.388
gradient boosting   MAE  10.17   RMSE  16.19   R2  0.616
```

R^2 of the mean is about 0, as it should be. The honest claim: "about 2.8 units a week better than a spreadsheet forecast", not "R^2 0.62". Then the trap in one line: `mean_absolute_percentage_error` on these weeks prints 1,263,687,238,250,694 because 105 weeks sold 0 units (Common Mistake 5).

12 **Optional: more than two classes.** Only if you are ahead of time. Predict the loyalty tier from behavior; Gold is 8% of customers.

```
tier_features = ["days_since_last_order", "orders_12m", "discount_share", "app
                 "support_tickets_12m", "return_rate"]
Xt_train, Xt_test, yt_train, yt_test = train_test_split(
    customers[tier_features], customers["loyalty_tier"], test_size=0.25,
    stratify=customers["loyalty_tier"], random_state=42,
)
tier_model = make_pipeline(StandardScaler(), LogisticRegression()).fit(Xt_trai
print(classification_report(yt_test, tier_model.predict(Xt_test), digits=3))
```

```
              precision    recall  f1-score   support

       Basic      0.640     0.808     0.714       709
        Gold      0.789     0.126     0.217       119
      Silver      0.628     0.548     0.585       672

    accuracy                          0.637      1500
   macro avg      0.686     0.494     0.506      1500
weighted avg      0.647     0.637     0.617      1500
```

Macro F1 0.506 exposes the weak Gold class (recall 0.126); weighted F1 0.617 hides it. Otherwise one sentence: "With several classes, report the macro average and the per-class rows."

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Labels where scores belong.** `roc_auc_score(y_test, model.predict(X_test))` prints 0.615 instead of 0.823, with no error; passing both probability columns raises `ValueError: y should be a 1d array, got an array of shape (1500, 2) instead.`. Coach: "AUC measures a ranking. How many different ranks do 0s and 1s give you?" Fix: `predict_proba(X)[:, 1]`, for `average_precision_score` too.
- **Truth and prediction swapped.** `precision_score(y_pred, y_test)` quietly returns the recall (0.266 instead of 0.589). Coach: "Which argument is the truth?" Every scikit-learn metric is `(y_true, y_pred)`; keyword arguments make it impossible to get wrong. Related: text labels need `pos_label="churned"`, or `ValueError: pos_label=1 is not a valid label. It should be one of ['churned' 'stayed']`.
- **The threshold chosen on the test set.** The most natural mistake of the day: a learner scans the threshold table on the test set and "picks 0.2". Coach: "Which rows did that table use? Could you have had them

before the campaign?" Then route them to out-of-fold probabilities (the project's Step 2) or `TunedThresholdClassifierCV`.

GUIDED LAB

- **File:** `labs/module-03/l1_4-metrics/fraud_review_starter.py`; reference `labs/module-03/l1_4-metrics/fraud_review_solution.py`. The lesson chose a retention threshold from a PROFIT on churn; the lab turns it around: card fraud, framed as a COST to keep low. A reviewer checks each flagged payment ($10), and every fraud that slips through costs about $375. Same tools: confusion matrix, precision and recall, AUC against average precision, a threshold from out-of-fold training probabilities, tested once.
- **Run it first, as shipped (1m),** then TODO 1 and 2 together on the projector (6m): the confusion matrix in business words, and ROC AUC next to average precision and its baseline, the fraud rate. Ask for a prediction before TODO 2: "Which of the two will look more comfortable?"
- **TODO 3 to 6 in pairs (11m).** TODO 3 has a self-check line (`review_cost([1, 0], [0, 1])` should be 385). Circulate for: `cross_val_predict` without `method="predict_proba"` (they get 0/1 predictions and every threshold gives the same cost); thresholds searched on `proba` (the TEST probabilities) instead of `oof`; `precision_score` without `zero_division=0` on the "review nothing" policy (a warning, not an error).
- **Expected:** TN 14867, FP 2, FN 93, TP 38: precision 0.950, recall 0.290 ("40 payments go to review, 38 are fraud, 93 of 131 frauds slip through"); ROC AUC 0.848 against average precision 0.444 and a fraud rate of 0.0087; break-even 10 / 375 = 0.027; cheapest training threshold 0.04 (training cost $52,320 against $75,380 at 0.5); on the test set it reviews 390 payments, catches 58% of the fraud and costs $24,525, against $35,275 at 0.5, $49,125 reviewing nothing and $150,000 reviewing everything.
- **Debrief (2m):** "At 0.5 the model is almost always right when it flags (95% precision) and it is still the expensive choice: it misses 93 frauds at $375 each. The cheap policy accepts four wasted reviews for every fraud it catches, because a review costs $10 and a miss $375." Then the mirror: the break-even here is 0.027, in the lesson 0.25; the threshold follows the costs, not the model.
- **Fast finishers:** the Stretch line in the starter (`TunedThresholdClassifierCV` with `make_scorer(review_cost, greater_is_better=False)`: threshold 0.040, test cost $24,485; ask why a cost needs `greater_is_better=False`), then `labs/module-03/l1_4-metrics/debug_metrics.py`: two loud metric bugs and two silent ones. Lab B, `labs/module-03/l1_4-metrics/store_forecast_starter.py`, is regression metrics on daily store sales: homework.

START THE LESSON PROJECT

The lesson project is **Plan the Retention Campaign**: a threshold for the random forest chosen on training customers only, one test-set comparison of four policies, and a 150-call list. In the room, do Steps 1 and 2 only (10 minutes).

- Read the brief aloud from the lesson page ($15 per offer, $60 per churner reached, a care team that can make 150 calls). Everyone types Step 1 (split the whole table so `customer_id` stays with the test rows) and Step 2 (out-of-fold forest probabilities with `cross_val_predict`).
- **Done in the room** = `4500 1500 break-even probability 0.25` and `[0.185 0.005 0.191 0.008 0.494] mean 0.166`.
- **Goes home:** Step 3 (the profit curve: best threshold 0.26, training profit $15,690 against $11,160 at 0.5), Step 4 (test once: $5,415 at 0.26 against $3,795 at 0.5 and −$7,620 for contacting everyone), Step 5 (150 calls, precision 0.687, 103 real churners) and Step 6 (`retention_calls.csv`, IDs and probabilities only). Give them the check numbers.

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab:

- Recall or precision: "of the customers we called, how many were really leaving?" **Answer:** precision.
- You lower the threshold. Which goes up, which goes down, and what stays the same? **Answer:** recall up, precision down (usually); ROC AUC does not change, because it does not depend on any threshold.
- Why is 0.165 the baseline for average precision on churn, not 0.5? **Answer:** a random ranking has precision equal to the churn rate at every recall.
- Offer $15, churner reached $60: what is the break-even probability, and why did the tuner choose 0.245 and not exactly 0.25? **Answer:** 0.25; the tuner measured profit on out-of-fold data, and real probabilities are only roughly calibrated.
- A forecast has R^2 0.62. What else do you report? **Answer:** an error in units (MAE 10.17 units a week) and the same numbers for a naive baseline (12.95 for the product-store mean).

Workbook self-check, items Q1 to Q6 (learners do them at the end of the block or at home).

Answers: **Q1** B) precision 0.400, recall 0.667; **Q2** `0.667 0.5`; **Q3** C) The training rows, out of fold: `cross_val_predict(..., method="predict_proba")` or `TunedThresholdClassifierCV`; **Q4** A) `0.75 1.0`; **Q5** D) The customers with a churn probability of at least 0.25; **Q6** `3.5 5.2`. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: give anyone who mixes up precision and recall a card with the 2×2 and the two fractions written as questions ("of those we called...", "of those who left..."), and have them point at the cells before every calculation. In the lab, let a struggling pair skip TODO 3 by copying the lesson's `campaign_profit` and changing two lines: the idea is the cost curve, not the function.

Stretch: the lab's Stretch line, then the lesson's Exercise 4 (a $25 offer moves the threshold to about 0.44) and the Bonus Challenge (12 cost/value combinations). Experienced learners: ask them to explain why `TunedThresholdClassifierCV`'s default scorer is balanced accuracy, and when they would replace it.

2.1 Impute, Scale and Encode

75 min

OBJECTIVES (FROM THE LESSON PAGE)

- Fill missing values with SimpleImputer, and keep the fact that a value was missing with `add_indicator=True`
- Choose between StandardScaler, MinMaxScaler, RobustScaler and a log transform, and explain what each one changes
- Encode categories with OneHotEncoder (including categories never seen in training) and ordered categories with OrdinalEncoder
- Build a ColumnTransformer that sends each group of columns to the right transformer, fitted on the training data only
- Train a model on all 16 usable churn columns, not just the 10 clean numeric ones

TIMING IN THE ROOM

10m	Frame: mise en place; one treatment per kind of column; the prep is learned from training rows
30m	Live demo: gaps and their meaning, SimpleImputer and add_indicator, scalers and a log, one-hot and unknown categories, OrdinalEncoder, the ColumnTransformer, 16 columns against 10
20m	Guided lab: `fraud_columns_starter.py` (the product category joins the fraud model)
15m	Start the lesson project: a model-ready customer table, Steps 1 to 3

MENTAL MODEL TO SAY

Say: "Before a restaurant opens, the kitchen does its **mise en place**: every ingredient washed, chopped and measured into its own bowl, so the cooks work fast and the dish tastes the same every night. Preprocessing is mise en place for a model. Each kind of ingredient gets its own treatment: you peel carrots, you do not peel eggs. And the prep recipe is written down once, then followed exactly for every new order."

Point at the three bowls on the board: **gaps → imputer, units → scaler, words → encoder.** Then the part beginners miss: "The recipe is LEARNED from the training data: which value fills a gap, what the average spend is, which stores exist. Then it is applied unchanged to the test rows and to every future customer. Same rule as the scaler in Lesson 1.1, now for every column." Connect to the warm-up's W3: "and that is why, in Lesson 2.2, the whole recipe goes inside the cross-validation folds."

LIVE DEMO (YOU TYPE, THEY FOLLOW)

New notebook, `lesson_2_1`. The lesson page's own code, so the numbers match. The demo is mostly small tools; keep it moving and save your energy for Steps 2, 11 and 13. Common Mistakes 1 to 5 are typed live only if you are ahead; otherwise they are in the coaching notes below.

1 **Sixteen columns, split first (2m).** "Module 1 left most of the churn table on the shelf. Today it all comes in, except three columns that stay out on purpose: the ID, the signup date (Lesson 2.3) and the leak."

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split

pd.set_option("display.width", 150)
pd.set_option("display.max_columns", 20)

customers = pd.read_csv("data/customers_churn.csv")

numeric = ["tenure_months", "orders_12m", "spend_12m", "avg_order_value", "day
           "returns_12m", "return_rate", "support_tickets_12m", "discount_shar
           "email_opens_90d"]
categorical = ["home_store", "region", "age_group", "loyalty_tier", "preferred

X = customers[numeric + categorical]
y = customers["churned"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, stra

print(X_train.shape, X_test.shape)
print(X_train.dtypes.value_counts())
```

```
(4500, 16) (1500, 16)
int64      6
float64    5
str        5
Name: count, dtype: int64
```

Same split as every lesson, so the same 1,500 test customers. In pandas 3 text columns have the dtype `str`.

2 **Find the gaps, then ask what they mean (3m).** "Count gaps on the training rows only. Then, before filling anything: does the gap itself say something?"

```
missing = X_train.isna().sum()
print(missing[missing > 0])
print(f"rows with at least one gap: {X_train.isna().any(axis=1).sum()} of {len

email_missing = X_train["email_opens_90d"].isna()
print(y_train.groupby(email_missing).mean().round(3))
```

```
email_opens_90d    419
age_group          359
dtype: int64
rows with at least one gap: 751 of 4500

email_opens_90d
False    0.156
True     0.253
Name: churned, dtype: float64
```

`dropna()` would throw away 751 customers (one in six) and still leave no answer for a new customer with no age. Customers with no email count churned at 25.3% against 15.6%: a clue worth keeping.

3 **SimpleImputer learns one number (2m).** "A median, because email opens are a skewed count. Learned from training rows; applied to the test rows."

```
from sklearn.impute import SimpleImputer

imputer = SimpleImputer(strategy="median").set_output(transform="pandas")
imputer.fit(X_train[["email_opens_90d"]])
print("learned fill value:", imputer.statistics_)

filled = imputer.transform(X_test[["email_opens_90d"]])
print(X_test["email_opens_90d"].isna().sum(), "gaps in the test column before,
      filled["email_opens_90d"].isna().sum(), "after")
```

```
learned fill value: [6.]
118 gaps in the test column before, 0 after
```

`statistics_` has the trailing underscore of a learned attribute. The test set's own median is never computed: next month's customers arrive one at a time, and one customer has no median.

4 **Keep the clue: add_indicator (2m).** "Filling erases the fact that it was a gap. `add_indicator=True` adds a 0/1 column that remembers."

```
imputer_flag = SimpleImputer(strategy="median", add_indicator=True).set_output
flagged = imputer_flag.fit_transform(X_train[["email_opens_90d"]])

print(flagged.columns.tolist())
print(flagged[email_missing].head(3))
```

```
['email_opens_90d', 'missingindicator_email_opens_90d']
      email_opens_90d  missingindicator_email_opens_90d
3996              6.0                               1.0
426               6.0                               1.0
3846              6.0                               1.0
```

5 **Gaps in a text column (2m).** "No median of age groups. Either the most frequent value, or an honest category of its own."

```
age_imputer = SimpleImputer(strategy="constant", fill_value="unknown").set_out
ages = age_imputer.fit_transform(X_train[["age_group"]])
print(ages["age_group"].value_counts())
```

```
age_group
35-44      934
25-34      899
45-54      755
55-64      649
18-24      458
65+        446
unknown    359
Name: count, dtype: int64
```

"unknown" (359) is a real answer: "did not tell us".

6 **Three scalers, three number lines (2m).** "Each answers a slightly different question. Watch where the median lands and how far out the big spenders sit."

```
from sklearn.preprocessing import StandardScaler, MinMaxScaler, RobustScaler

spend = X_train[["spend_12m"]]
for scaler in [StandardScaler(), MinMaxScaler(), RobustScaler()]:
    scaled = scaler.fit_transform(spend)
    print(f"{type(scaler).__name__:15s} min {scaled.min():6.2f}   median {np.m
```

```
StandardScaler  min  -1.30   median -0.18   max  5.38
MinMaxScaler    min   0.00   median  0.17   max  1.00
RobustScaler    min  -0.78   median  0.00   max  3.89
```

MinMax crowds most customers near the bottom because of a few large spenders; Robust puts the median at exactly 0. None changes who spends more than whom: a change of units.

7 **Units versus shape (2m).** Ask first: "After StandardScaler, will the skew change?" Then run it. (The lesson page also draws the three histograms: show them on the projector.)

```
days = X_train[["days_since_last_order"]]
versions = {
    "raw (days)": days.to_numpy().ravel(),
    "StandardScaler": StandardScaler().fit_transform(days).ravel(),
    "log1p(days)": np.log1p(days).to_numpy().ravel(),
}
for title, values in versions.items():
    print(f"{title:15s} skew {pd.Series(values).skew():5.2f}")
```

```
raw (days)      skew  1.71
StandardScaler  skew  1.71
log1p(days)     skew -0.19
```

Only the log changes the shape. Whether it helps depends on the model: trees do not care about either; Lesson 2.2 tests the log on this data.

8 **One-hot: one 0/1 column per category (2m).** "A model cannot multiply a weight by 'Bend'." Point out `sparse_output=False`, needed for pandas output.

```
from sklearn.preprocessing import OneHotEncoder

onehot = OneHotEncoder(sparse_output=False).set_output(transform="pandas")
stores = onehot.fit_transform(X_train[["home_store"]])

print(onehot.categories_)
print(pd.concat([X_train[["home_store"]], stores], axis=1).head(4))
```

```
[array(['Asheville', 'Bend', 'Boise', 'Flagstaff', 'Online'], dtype=object)]
     home_store  home_store_Asheville  home_store_Bend  home_store_Boise  home
1556       Bend                   0.0              1.0               0.0
965   Flagstaff                   0.0              0.0               0.0
5145     Online                   0.0              0.0               0.0
3004      Boise                   0.0              0.0               1.0
```

`categories_` was learned at fit. Column names are `column_category`.

9 **A store you have never seen, and gaps as a category (3m).** "Pinecrest opens in Reno next month. What should the encoder do with a Reno customer?"

```
safe_onehot = OneHotEncoder(sparse_output=False, handle_unknown="ignore").set_
safe_onehot.fit(X_train[["home_store"]])

new_customers = pd.DataFrame({"home_store": ["Bend", "Reno"]})
print(safe_onehot.transform(new_customers))

age_onehot = OneHotEncoder(sparse_output=False).fit(X_train[["age_group"]])
print(age_onehot.get_feature_names_out())
```

```
   home_store_Asheville  home_store_Bend  home_store_Boise  home_store_Flagsta
0                   0.0              1.0               0.0                   (
1                   0.0              0.0               0.0                   (

['age_group_18-24' 'age_group_25-34' 'age_group_35-44' 'age_group_45-54'
 'age_group_55-64' 'age_group_65+' 'age_group_nan']
```

Reno becomes all zeros: "none of the stores I know". And a third way to handle the missing ages: one-hot gives them their own column, `age_group_nan`.

10 **Ordered categories: spell the order out (2m).** "Basic, then Silver, then Gold. One column of 0, 1, 2, but only if you tell it the order."

```
from sklearn.preprocessing import OrdinalEncoder

tier_encoder = OrdinalEncoder(categories=[["Basic", "Silver", "Gold"]]).set_ou
tiers = tier_encoder.fit_transform(X_train[["loyalty_tier"]])

check = pd.DataFrame({"tier": X_train["loyalty_tier"], "code": tiers["loyalty_
print(check.drop_duplicates().sort_values("code"))
```

```
        tier  code
1556   Basic   0.0
965   Silver   1.0
2088    Gold   2.0
```

Without `categories=`, the order is alphabetical (Basic, Gold, Silver) and nothing warns you: Common Mistake 5.

11 **The whole table in one object (3m).** "A list of (name, transformer, columns). Each branch fits on its own columns; the results sit side by side." Then check the scales.

```
from sklearn.compose import ColumnTransformer

complete = numeric[:-1]  # the ten numeric columns without gaps (all but email
nominal = ["home_store", "region", "age_group", "preferred_category"]

first_try = ColumnTransformer([
    ("scale", StandardScaler(), complete),
    ("email", SimpleImputer(strategy="median", add_indicator=True), ["email_op
    ("tier", OrdinalEncoder(categories=[["Basic", "Silver", "Gold"]]), ["loyal
    ("onehot", OneHotEncoder(sparse_output=False, handle_unknown="ignore"), no
]).set_output(transform="pandas")

Xt_train = first_try.fit_transform(X_train)
print(Xt_train.shape)
print(Xt_train.columns[8:16].tolist())

print(Xt_train[["scale__days_since_last_order", "email__email_opens_90d", "tie
```

```
(4500, 40)
['scale__discount_share', 'scale__app_user', 'email__email_opens_90d', 'email_

scale__days_since_last_order    1.00
email__email_opens_90d          5.21
tier__loyalty_tier              0.63
dtype: float64
```

16 raw columns became 40, each named `branch__column`. Unlisted columns are dropped (`remainder="drop"`), a safety feature. But the email column still runs in raw opens (spread 5.21) and the tier codes 0 to 2: those branches need two steps.

12 **Two steps in one branch (3m).** "Branches run side by side, not one after another. To impute THEN scale, chain them with `make_pipeline` inside the branch." (Lesson 2.2 is all about this object.)

```
from sklearn.pipeline import make_pipeline

preprocess = ColumnTransformer([
    ("scale", StandardScaler(), complete),
    ("email", make_pipeline(SimpleImputer(strategy="median", add_indicator=Tru
     ["email_opens_90d"]),
    ("tier", make_pipeline(OrdinalEncoder(categories=[["Basic", "Silver", "Gol
     ["loyalty_tier"]),
    ("onehot", OneHotEncoder(sparse_output=False, handle_unknown="ignore"), no
]).set_output(transform="pandas")

Xt_train = preprocess.fit_transform(X_train)
Xt_test = preprocess.transform(X_test)
print(Xt_train.shape, Xt_test.shape)
print("missing values left:", int(Xt_train.isna().sum().sum()), int(Xt_test.is
```

```
(4500, 40) (1500, 40)
missing values left: 0 0
```

The rhythm, out loud: `fit_transform` on train, `transform` on test.

13 **The payoff, and an honest result (4m).** "All 16 columns against Lesson 1.1's ten clean ones." Ask for a prediction first: big jump, small jump, or none?

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score

model = LogisticRegression().fit(Xt_train, y_train)
auc_all = roc_auc_score(y_test, model.predict_proba(Xt_test)[:, 1])

ten_scaler = StandardScaler().fit(X_train[complete])
model_10 = LogisticRegression().fit(ten_scaler.transform(X_train[complete]), y
auc_10 = roc_auc_score(y_test, model_10.predict_proba(ten_scaler.transform(X_t

print(f"10 clean numeric columns : ROC AUC {auc_10:.3f}")
print(f"all 16 columns, prepared : ROC AUC {auc_all:.3f}   accuracy {model.sco

coefs = pd.Series(model.coef_[0], index=Xt_train.columns)
print(coefs[["email__email_opens_90d", "email__missingindicator_email_opens_90
```

```
10 clean numeric columns : ROC AUC 0.823
all 16 columns, prepared : ROC AUC 0.828   accuracy 0.851

email__email_opens_90d                     -0.28
email__missingindicator_email_opens_90d     0.16
tier__loyalty_tier                          0.39
dtype: float64
```

0.823 to 0.828: the extra columns help, but only a little, for this linear model. You only know that because you could finally try. The missing-email indicator pushes toward churn (+0.16): the clue survived. The tier weight looks backwards because tier is computed from spend and tenure, which are already in the model.

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **A text column meets a numeric tool, or sparse meets pandas.** `ValueError: Cannot use median strategy with non-numeric data: could not convert string to float: 'Bend'` (one imputer for all columns) and `ValueError: Pandas output does not support sparse data. Set sparse_output=False to output pandas...` Coach: "Read the last line. Which branch is this column in? What kind of column is it?" Fix: separate branches; `OneHotEncoder(sparse_output=False)`.
- **The encoder meets next month's data.** `ValueError: Found unknown categories ['Reno'] in column 0 during transform` (fix: `handle_unknown="ignore"`), or `pd.get_dummies` on a batch with two stores: `ValueError: The feature names should match those that were passed during fit.` Coach: "Which object REMEMBERS the training categories?" Only a fitted encoder.
- **Silent ones:** `fit_transform` **on the test rows, and alphabetical tiers.** Neither raises anything. Coach with the project's checks: "Your scaled TEST columns have a mean of exactly 0.0. Should they?" (They should be close to 0 but not exactly: 0.055 in the project.) And: "Print `categories_` of your OrdinalEncoder. Is Gold above Silver?" (Alphabetical gives [array(['Basic', 'Gold', 'Silver'], dtype=object)] [0. 2. 1.].)

GUIDED LAB

- **File:** `labs/module-03/l2_1-preprocessing/fraud_columns_starter.py`; reference `labs/module-03/l2_1-preprocessing/fraud_columns_solution.py`. Every fraud model so far used nine numbers and left `category` (text) on the shelf. The lab brings it in with one ColumnTransformer (impute and scale the numbers, one-hot the category), judges it with Lesson 1.4's metrics, and scores next month's feed: a category the shop has never sold (Bikes) and a guest checkout with no account age.
- **Run it first, then TODO 1 together (3m):** the fraud rate by category on the training rows. Gift Cards stands out (5.5% against 0.6% to 1.1%). Ask: "Is this column worth encoding?"

- **TODO 2 to 6 in pairs (15m).** Circulate for: `OneHotEncoder()` without `sparse_output=False` (the pandas-output error: read it together); `preprocess.fit_transform(new)` in TODO 6 (the encoder would learn three categories from three rows, and the shapes break: ask "which numbers should the model expect?"); ROC AUC from `predict` (the 1.4 mistake again).
- **Expected:** (35000, 19) and (15000, 19); numbers only ROC AUC 0.848 / average precision 0.444, with the category ROC AUC 0.843 / average precision 0.463; Gift Cards gets the largest weight of all (0.95); fill value 447 days, 10 categories; Bikes becomes all zeros; P1 1.000, P2 0.001, P3 0.000.
- **Debrief (2m):** "Did the category help?" The honest answer is "it depends which number you ask": ROC AUC fell a little, average precision rose. For a rare class whose top of the list is what the reviewers see, average precision is the number that matches the job (Lesson 1.4). Then: the training data had no gaps at all, yet the imputer earned its place the day a guest checkout arrived.
- **Fast finishers:** `labs/module-03/l2_1-preprocessing/debug_preprocess.py` (two loud bugs, two silent ones; the clue is a test-set mean of exactly 0.0). Lab B, `labs/module-03/l2_1-preprocessing/demand_categories_starter.py`, is homework: how much does it help a demand forecast to know the store, the category or the product?

START THE LESSON PROJECT

The lesson project is **A Model-Ready Customer Table**: one preprocessor for all 16 usable columns, checked, used to train a model, and proven on three brand-new customers (a gap in age, a missing email count, the new Reno store). In the room, do Steps 1 to 3 (15 minutes).

- Read the brief aloud. Step 1 together: split the whole table, keeping the IDs. **Done** = `16 features; 4500 train rows; 1500 test rows`.
- Step 2 on their own: the gap audit on training rows. **Done** = email missing 419 (churn 0.253 if missing against 0.156), age missing 359 (0.159 against 0.166): "the email gap gets an indicator; the age gaps become their own one-hot category."
- Step 3: the preprocessor. **Done** = `(4500, 40) (1500, 40)`. Anyone who finishes early runs Step 4's three checks.
- **Goes home:** Step 4 (gaps left 0 and 0; scaled train columns' largest |mean| 0.0, test 0.055; 27 one-hot columns), Step 5 (baseline AUC 0.500, logistic regression 0.828, accuracy 0.851) and Step 6 (N1 0.886, N2 0.009, N3 0.467; `new_customer_scores.csv`). **Keep this notebook:** Lesson 2.2 starts from the same column lists and preprocessor.

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab:

- What exactly does a fitted SimpleImputer remember, and where? **Answer:** one fill value per column, in `statistics_`, learned from the training rows.

- Median or mean for email opens, and why? **Answer:** median: the count is right-skewed (a few very active readers pull the mean up to 7.28 against a median of 6).
- A RobustScaler and a StandardScaler: which one does a single $250,000 data-entry error distort, and why? **Answer:** StandardScaler, because one extreme value inflates the standard deviation; RobustScaler uses the median and the interquartile range.
- Why one-hot for `home_store` but OrdinalEncoder for `loyalty_tier`? **Answer:** stores have no order; tiers do (Basic < Silver < Gold), and the order must be spelled out.
- A colleague's table also contains `cancellation_reason`. What does your ColumnTransformer do with it? **Answer:** drops it: it is in no branch and the default is `remainder="drop"`.

Workbook self-check, items Q7 to Q12 (learners do them at the end of the block or at home).

Answers: **Q7** C) `[[0. 1.] [0. 0.]]`; **Q8** `[[5. 1.] [1. 0.]]`; **Q9** A) Scaling changes the units, not the shape; a log transform such as `np.log1p` changes the shape; **Q10** D) `[2. 0.]`; **Q11** B) One branch whose transformer is `make_pipeline(SimpleImputer(...), StandardScaler())`; **Q12** `(3, 3) ['num__spend', 'cat__store_Bend', 'cat__store_Boise']`. Full set with explanations in the Answer Key.

◆ DIFFERENTIATION

Support: a three-column card "problem → transformer → what it learns" (gaps → SimpleImputer → `statistics_`; units → scaler → `mean_`, `scale_`; words → encoder → `categories_`). In the lab, let a pair build the ColumnTransformer by copying the lesson's and deleting branches: the skill is choosing the branch for each column.

Stretch: the lab's Stretch idea (swap in `RobustScaler` for the numbers and compare average precision), the lesson's Exercise 4 (is the indicator worth it? 0.8270 against 0.8283) and the Bonus Challenge (`KNNImputer`). Experienced learners: "Why does `OneHotEncoder` not need an imputer for `age_group`, while OrdinalEncoder would?"

CLOSING

◆ EXIT TICKET (3 QUESTIONS, NO NOTES, 4 MINUTES)

Put the three questions on the slide. Learners answer in the workbook's Exit Ticket box or on a sticky note and leave it on the way out. Sort the notes into "got it" and "not yet" before Session 4: the pile for question 2 tells you whether to open Session 4's warm-up with the threshold again; the pile for question 3 tells you how much of 2.2's opening to spend on fit versus transform.

1. A model's confusion matrix: TN 900, FP 100, FN 30, TP 70. Compute its precision and recall, and say each in one business sentence. **Answer:** precision 0.412 recall 0.700. Of the customers we flag, about four in ten really churn; we catch seven in ten of the churners.

2. Why choose a decision threshold from out-of-fold training probabilities instead of the test set? **Answer:** the test set must stay unseen to give an honest final number; out-of-fold probabilities come from models that never saw each customer, so the choice is honest and the test set is used once.

3. Next month's customer comes from the new Reno store and did not give an age. What does each part of your fitted preprocessor do with them? **Answer:** the one-hot branch encodes Reno as all zeros (`handle_unknown="ignore"`) and the missing age as the `age_group_nan` column; the scalers apply the training means and standard deviations; nothing is refitted.

HOMEWORK (ABOUT 2 TO 3 HOURS)

- **Finish both lesson projects.** The retention-campaign plan (Steps 3 to 6: best threshold 0.26, test profit $5,415 against $3,795 at 0.5, 150 calls at precision 0.687, `retention_calls.csv`) and the model-ready customer table (Steps 4 to 6: test ROC AUC 0.828, N1 0.886 / N2 0.009 / N3 0.467, `new_customer_scores.csv`).
- **Lesson exercises.** 1.4: Exercises 1 to 3 (F1 against F2 thresholds, forecast errors by category, ROC against precision-recall on fraud); Exercise 4 is a stretch. 2.1: Exercises 1 to 3 (three fill values, one whale customer, product and store for the demand model); Exercise 4 is a stretch.
- **Both Quick Quizzes** on the lesson pages (five questions each; answers are in the Answer Key).
- **Journal:** both entries. 1.4: the smoke alarm in your own words, and the four cells of a problem you care about with a rough cost on each. 2.1: three columns, three transformers, what each learns, and what the missing email counts told you.
- **Before the Next Lesson (1.4 and 2.1):** list the churn columns you had not used before today and whether each is text, has gaps, or both (2.1 answers it: check yourself); write down the one metric for Pinecrest's churn dashboard and why; print `preprocess` in the 2.1 notebook and click through the diagram; think: to cross-validate today's model, which objects must be refitted inside every fold, and in which order?
- **Optional kit labs:** `labs/module-03/l1_4-metrics/store_forecast_starter.py` (which daily sales forecast should a store manager trust?), `labs/module-03/l2_1-preprocessing/demand_categories_starter.py` (does it help to know what is being sold?), and the two `debug_*.py` handouts.

BRING LIST FOR SESSION 4

- A laptop on which `check_environment.py` ends with READY (the same environment: nothing new to install; `joblib`, which Lesson 2.2 uses to save a pipeline, comes with scikit-learn).
- The `pinecrest-ml` folder with `data/` and the `labs/` folder inside it. Session 4's starters are in `labs/module-04/`.
- The finished Lesson 2.1 project notebook (`lesson_2_1_project`): Lesson 2.2 uses the same column lists (`complete`, `nominal`, `features`) and the same preprocessor; it rebuilds them in its first cells, so an unfinished notebook is not a blocker. Lesson 2.3 starts fresh from the course's standard split.
- The workbook and the journal. No downloads, no accounts, no internet needed.

IF YOU RUN LONG OR SHORT

RUNNING LONG: WHAT TO PROTECT AND WHAT TO CUT FIRST

- **Protect:** 1.4 demo Steps 2 to 4 and 8 to 9; TODO 4 and 5 of the fraud review lab; 2.1 demo Steps 2 to 4 and 11 to 13; the 2.1 project Step 3 (Lesson 2.2 starts from it).
- **Cut first (saves 4m):** the optional multiclass step, and 1.4 Steps 6 and 10 (brute-force AUC, toy MAE/RMSE). Say the two sentences: "AUC is the chance a churner outranks a stayer" and "RMSE punishes big misses, MAE does not".
- **Cut second (saves 4m):** 2.1 Steps 6 and 7 (the scalers and the skew). Show the lesson's table and histograms instead and say "scaling changes units, a log changes shape".
- **Cut third (saves 10m):** the 1.4 project start. Read the brief, give the two check numbers (`4500 1500 break-even probability 0.25` ; out-of-fold mean 0.166), send all six steps home.

RUNNING SHORT: WHAT TO ADD

- Run the fraud lab's Stretch for everyone: `TunedThresholdClassifierCV` with a cost scorer, and why `greater_is_better=False` .
- The 1.4 Bonus Challenge live, in a 2×2 subset: offer cost $10 or $30, churner value $40 or $80. Does the chosen threshold track cost / value?
- `store_forecast_starter.py` together: four no-model forecasts, and MAPE on all days (1.33e+17) against open days (0.168).

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 4: Pipelines and Creating Features

How to teach Session 4 of Machine Learning Foundations: seal preprocessing and a model into one Pipeline that fits, cross-validates, tunes and saves as a unit (and watch a target-based feature fall from 0.930 to 0.586 once it is fitted inside the folds); then create features that let a linear model see thresholds, U-shapes, interactions, seasons and history (churn ROC AUC 0.829 → 0.883; demand error 11.3 → 9.4 units in class).

`2 lessons` `3 hrs session` `Lessons 2.2-2.3` `Session 4 of 11`

HOW TO USE THIS GUIDE

- Same rhythm as every session: warm-up → for each lesson: frame (the goal and one mental model) → live demo (you type, they type along) → guided lab (a kit lab, new practice on the lesson's skills) → start the lesson's own project. Exercises, quizzes and journals are homework.
- Type, do not paste. Every code block in the demo scripts was run to produce the output printed under it (Python 3.12, pandas 3.0, scikit-learn 1.9.1, 4 CPU threads). The demos follow the lesson pages, so learners can find every cell again at home.
- The lessons are the reference: keep `pipelines_composite.html` and `feature_engineering.html` open for the diagrams (fit/predict flow, the feature loop, the season chart). Labs live in `labs/module-04/`: `_starter.py` files are handouts, `_solution.py` files are yours.
- Teal blocks are hands-on, amber blocks are watch-outs. Lesson codes (2.2, 2.3) match the course site.

> **THE #1 RISK OF SESSION 4: NAMES AND NESTING SWAMP THE IDEA**
>
> Lesson 2.2 is full of double-underscore names, nested branches and long error messages, and Lesson 2.3 is full of pandas. A room can spend the whole afternoon on syntax and leave without the two ideas that matter: **every step that learns from data belongs inside the pipeline** (0.930 vs 0.586 is the proof), and **a linear model only sees straight lines unless you ask better questions** (0.829 vs 0.883). Protect those two moments in the demos; when a name trips someone, show `list(pipe.get_params())` once and move on. The labs repeat the naming until it sticks.

BEFORE THE SESSION

- ☐ **Run the kit check once:** `python3 labs/check_labs.py module-04` from this kit's folder: 8 files, every line PASS (about 35 seconds). Lab 2.2 A writes `fraud_pipeline.joblib` into an `output/` folder next to the lab file; nothing else is written.
- ☐ **Downloads:** none. Same environment as Sessions 1 to 3; nothing new to install. `TargetEncoder` needs scikit-learn 1.3 or newer (the course has 1.9).
- ☐ **Projector machine:** `(.venv)` active in your `pinecrest-ml`, JupyterLab started there, font 18 pt or more, two empty notebooks `lesson_2_2` and `lesson_2_3`. Browser tabs: both lesson pages and `data/README.md`.
- ☐ **Time the slow cells on YOUR machine:** demo 2.2 Step 12 (boosting, 5-fold CV) and demo 2.3 Step 6 (boosting in the CV loop) take 10 to 30 seconds on a laptop; the grid search (Step 13) about as long. Start them, then talk. Learners' 2.2 project Step 3 has the same boosting CV.
- ☐ **Board, before learners arrive:** the fit/predict diagram (empty, you fill it in demo Step 2); a box "Leak size" with three rows: learns from y / learns from X / learns nothing (filled in by Lab 2.2 B); and the feature loop: explore train → write a function → pipeline → cross-validate → keep or drop.
- ☐ **Session 3 handoff:** read the Session 3 exit-ticket notes if you have them; the warm-up retrieves precision, recall, thresholds and the 2.1 preprocessor.

KNOWN ENVIRONMENT TRAPS

SYMPTOM	CAUSE	FIX
`ImportError: cannot import name 'TargetEncoder' from 'sklearn.preprocessing'`	An old scikit-learn (before 1.3): the wrong kernel or an old environment	Check the kernel (`import sklearn; sklearn.__version__` should be 1.9.x). Reselect the `.venv` kernel.
A cell with boosting or `GridSearchCV` seems frozen for 20 to 60 seconds	Many fits on a laptop CPU	Wait; do not re-run it (that starts over). The kernel's busy indicator is the proof it is working.
`AttributeError: Estimator ... does not provide get_feature_names_out`	A `FunctionTransformer` without `feature_names_out` is the first step	Ask a later step: `pipe.named_steps["prep"].get_feature_names_out()`, or pass `feature_names_out="one-to-one"` when the function keeps the columns.
`ValueError: Invalid parameter 'C' for estimator Pipeline(...)` (or `'model'`)	Setting without a step name, or `model__` on a `make_pipeline` pipeline	`print(list(pipe.get_params()))` and copy the name.

SYMPTOM	CAUSE	FIX
Scores in the 2.2 demo differ from the page after re-running cells	The `set_params` loop or Exercise 2 left a different `C` or fill strategy behind	Rebuild the pipeline, or Restart and run all.
`AttributeError: Can only use .dt accessor with datetimelike values`	The date column was read as text	`parse_dates=["week_start"]` in `read_csv`.
The lag table shows another store's numbers, or NaN in odd places	`shift` without `groupby`, or the table not sorted by series and date first	Sort by store, product, date; `groupby([...])[col].shift(1)`; `sort_index()` afterwards.
`InconsistentVersionWarning` when loading a `.joblib` file	The file was saved with another scikit-learn version	Load it in the environment that saved it; re-save after upgrades (Lesson 12.1).

SESSION 4 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Five retrieval questions on Session 3 (metrics, preprocessing); homework check: both projects and the 2.1 "refit" question.
0:10	10	**2.2** Frame and mental model	The coffee machine: one object, every step, the one rule known by heart.
0:20	30	**2.2** Live demo	make_pipeline, looking inside, the full churn pipeline, the signup-date leak and its honest fix, CV of the whole pipeline, step__parameter, a swap, a first grid search, FunctionTransformer, joblib.
0:50	20	**2.2** Guided lab	`labs/module-04/l2_2-pipelines/fraud_pipeline_starter.py`, then `labs/module-04/l2_2-pipelines/leak_size_starter.py` in pairs.
1:10	15	**2.2** Start the lesson project	The deployable churn pipeline, Steps 1 to 3; Steps 4 to 6 go home.
1:25	10	Break	Leave the fit/predict diagram and the three leak sizes on the board.
1:35	10	**2.3** Frame and mental model	The interviewer and the calculator: a linear model can only draw straight lines unless you ask better questions.

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
1:45	30	**2.3** Live demo	Look before you build (threshold, U-shape, interaction), five features, the CV lift, dates and cycles, category x month, lags, a training-only aggregate, two silent leaks.
2:15	20	**2.3** Guided lab	`labs/module-04/l2_3-creating-features/night_features_starter.py`, then `labs/module-04/l2_3-creating-features/daily_sales_features_starter.py` in pairs.
2:35	15	**2.3** Start the lesson project	The weekly demand forecaster, Steps 1 to 3; Steps 4 to 6 go home.
2:50	10	Wrap-up	Exit ticket (3 questions), homework, bring list for Session 5.
Total	180		Ends at 3:00.

In the room. The lesson pages estimate 100 + 100 minutes for a solo learner; after the warm-up, the break and the wrap-up the room has 150. So each lesson gets its mental model, a live demo of the lesson's central code, one guided kit lab, and the first three project steps.

Goes home (2 to 3 hours). Project Steps 4 to 6 of both lessons (the test check, saving and scoring with `churn_pipeline.joblib`; the log target, lags, boosting and `demand_scoreboard.csv`); Exercises 1 to 3 of each lesson (Exercise 4 is a stretch); both Quick Quizzes; both journal entries; both "Before the Next Lesson" lists.

Protect, in this order: the signup-date leak and its honest fix (2.2 demo Steps 7 and 8: the session's key number pair, 0.930 vs 0.586); the 2.3 lift (Steps 2 to 6: look, build, measure); Lab 2.2 A through TODO 3 and Lab 2.3 A through TODO 4. **Cut first:** 2.2 demo Steps 13 and 14 (grid search, log transform: one sentence each), then 2.3 demo Steps 9 and 10 (calendar parts and cycles: show the output on the slide), then Lab B of either lesson (to homework).

WARM-UP

0:00 Warm-up: Metrics and Preprocessing — 10 min

TIMING

6m	Five retrieval questions (workbook Warm-up box, no notes), then reveal and discuss
4m	Homework check: both Session 3 projects, exercises, quizzes, journals

QUESTIONS (ON THE SLIDE AND IN THE WORKBOOK)

W1

Pinecrest's care team calls 150 customers and 103 of them really were about to churn. Which metric is 103 / 150, and what does it answer? (Lesson 1.4)

W2

What does this print? (Lesson 1.4)

```
from sklearn.metrics import precision_score, recall_score
y_true = [1, 1, 1, 0, 0, 0, 0, 0]
y_pred = [1, 1, 0, 1, 0, 0, 0, 0]
print(round(precision_score(y_true, y_pred), 3), round(recall_score(y_true, y_
```

W3

Lesson 1.4's project chose a threshold of 0.26. Why from out-of-fold probabilities on the training rows, and never from the test set? (Lesson 1.4)

W4

What does this print? (Lesson 2.1)

```
from sklearn.preprocessing import OneHotEncoder
enc = OneHotEncoder(sparse_output=False, handle_unknown="ignore").fit([["Bend"
print(enc.transform([["Reno"], ["Boise"]]))
```

W5

Lesson 2.1 asked: to cross-validate its churn model, which objects must be refitted inside every fold, and in which order? (Lesson 2.1)

ANSWERS

1. **W1 Precision** (at 150): of the customers we contacted, how many were real churners? Recall would divide by all real churners instead.
2. **W2** `0.667 0.667`: two of the three flagged are churners (precision 2/3) and two of the three churners were flagged (recall 2/3).
3. **W3** The threshold is a setting learned from data. Choosing it on the test set lets the test set help build the model, so its score is no longer an honest preview. Out-of-fold probabilities give every training customer a score from a model that did not see them.

4. **W4** Reno is unknown, so with `handle_unknown="ignore"` it becomes a row of zeros; Boise is the second learned category. The category list was learned in `fit`.

5. **W5** The whole `ColumnTransformer` (imputer, scalers, ordinal and one-hot encoders), then the model, on the training folds only; then transform-and-predict on the validation fold. Doing that by hand is error-prone: a `Pipeline` does exactly this, which is today's first lesson.

HOMEWORK CHECK AND HOW TO READ IT

- **Show of hands, one per item:** Lesson 1.4 project finished (threshold 0.26, $5,415 test profit, `retention_calls.csv` with 150 rows); Lesson 2.1 project finished (`preprocess` turns 16 columns into 40; `new_customer_scores.csv` with N1 0.886, N2 0.009, N3 0.467); exercises 1 to 3 of each; both quizzes; both journals; the one churn-dashboard metric they chose (take two answers: most will say recall or precision at a fixed number of calls, a fair choice).
- **W5 is the bridge.** Whoever answered it well has already understood half of Lesson 2.2. Write their answer on the board under the heading "What a pipeline automates".
- **Most likely to be missed: W1 and W3.** Precision and recall swap places for many people; give the smoke-alarm sentence from 1.4 again ("of the alarms, how many were fires" vs "of the fires, how many rang"). Do not reteach thresholds: Lesson 2.5 (Session 5) returns to them.
- **Anyone without the 2.1 project:** they do not need it today. Lesson 2.2 rebuilds the same preprocessor in its first cells; Lesson 2.3 starts from the CSV.

LESSON PLANS

2.2 Pipelines

75 min

OBJECTIVES (FROM THE LESSON PAGE)

- Build a Pipeline with `make_pipeline` or with named steps, and explain what `fit` and `predict` do to each step
- Reach inside a fitted pipeline (`named_steps`, indexing, slicing) to read learned values
- Cross-validate a whole pipeline, and show with a target-based encoding why preprocessing outside the folds leaks
- Change any setting with the `step__parameter` naming, swap a whole step, and run a first small grid search
- Add a custom step with `FunctionTransformer`, and save and reload a fitted pipeline with joblib

TIMING IN THE ROOM

10m	Frame: the coffee machine; fit and predict through every step; the one rule
30m	Live demo: the steps below (protect 7 and 8)
20m	Guided lab: `fraud_pipeline_starter.py` (everyone), then `leak_size_starter.py` (pairs)
15m	Start the lesson project: the deployable churn pipeline, Steps 1 to 3

MENTAL MODEL TO SAY

Say: "You could make espresso by hand: grind, weigh, tamp, heat, time the shot. Or you use a machine that does every step in order, the same way every time. Beans in, coffee out, and nobody can skip the grinder. A **pipeline** is that machine: transformers in order, one model at the end. It has `fit` and `predict` like any single model, so everything that works with a model (cross-validation, grid search, saving to a file) works with the whole machine. And it knows the one rule by heart: learn from training data, apply to everything else."

Then the stakes, pointing at the warm-up's W5 on the board: "In Session 3 you followed the rule by hand: `fit_transform`, then `fit`, then `transform`, then `predict`. Every rule you have to remember is a rule you will one day forget, at 6 p.m., in a cell you ran twice. From today, every model that needs preprocessing is a pipeline."

LIVE DEMO (YOU TYPE, THEY FOLLOW)

New notebook `lesson_2_2`. Ask for a prediction before Steps 2, 7, 12 and 14. Step 7 is the heart of the lesson: let the 0.930 sit for a moment before you explain it.

1. **Same data, same columns as Lesson 2.1 (1m).** "Same split, same 16 columns as the 2.1 project. Nothing new until the next cell."

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
pd.set_option("display.width", 150)
customers = pd.read_csv("data/customers_churn.csv")
complete = ["tenure_months", "orders_12m", "spend_12m", "avg_order_value", "da
            "returns_12m", "return_rate", "support_tickets_12m", "discount_sha
nominal = ["home_store", "region", "age_group", "preferred_category"]
features = complete + ["email_opens_90d", "loyalty_tier"] + nominal
train, test = train_test_split(customers, test_size=0.25, stratify=customers["
X_train, y_train = train[features], train["churned"]
X_test, y_test = test[features], test["churned"]
print(X_train.shape, X_test.shape)
```

```
(4500, 16) (1500, 16)
```

2. **The smallest pipeline (3m).** "Two steps, one object. Before I run it: what accuracy will it print?" (Someone should say 0.848, Lesson 1.1's number.)

```
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
simple = make_pipeline(StandardScaler(), LogisticRegression())
simple.fit(X_train[complete], y_train)
print(simple)
print(f"test accuracy: {simple.score(X_test[complete], y_test):.3f}")
```

```
Pipeline(steps=[('standardscaler', StandardScaler()),
                ('logisticregression', LogisticRegression())])
test accuracy: 0.848
```

Say: "One `fit` replaced three lines, one `score` replaced two, and the order can no longer be wrong." Draw the fit/predict diagram from the lesson on the board: fit = `fit_transform` on every step but the last, then `fit`; predict = `transform` only, then `predict`.

3. **Looking inside (2m).** "A pipeline is a box you can open: by name, by position, or sliced."

```
print(list(simple.named_steps))
print("scaler means:", simple.named_steps["standardscaler"].mean_[:3].round(2))
print("model weights:", simple[-1].coef_.shape)
print("preprocessing only:", simple[:-1].transform(X_test[complete]).shape)
```

```
['standardscaler', 'logisticregression']
scaler means: [ 41.48   5.   594.13]
model weights: (1, 10)
preprocessing only: (1500, 10)
```

`simple[-1]` is the fitted model; `simple[:-1]` is a pipeline of everything before it: the data exactly as the model sees it.

4 **Name your own steps (1m).** "For anything bigger, choose short names. `model` survives swapping the model for another class."

```
from sklearn.pipeline import Pipeline
named = Pipeline([
    ("scale", StandardScaler()),
    ("model", LogisticRegression()),
])
named.fit(X_train[complete], y_train)
print(list(named.named_steps), round(named.score(X_test[complete], y_test), 3))
```

```
['scale', 'model'] 0.848
```

5 **The full churn pipeline (4m).** "Lesson 2.1's preprocessor is itself a transformer, so it can be a step. Sixteen raw columns, text and gaps included, in; a probability out."

```
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.metrics import roc_auc_score
from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder
preprocess = ColumnTransformer([
    ("scale", StandardScaler(), complete),
    ("email", make_pipeline(SimpleImputer(strategy="median", add_indicator=Tru
     ["email_opens_90d"]),
    ("tier", make_pipeline(OrdinalEncoder(categories=[["Basic", "Silver", "Gol
     ["loyalty_tier"]),
    ("onehot", OneHotEncoder(sparse_output=False, handle_unknown="ignore"), no
])
churn_pipe = Pipeline([
    ("prep", preprocess),
    ("model", LogisticRegression()),
])
churn_pipe.fit(X_train, y_train)
proba = churn_pipe.predict_proba(X_test)[:, 1]
print(f"test ROC AUC: {roc_auc_score(y_test, proba):.3f}")
```

```
test ROC AUC: 0.828
```

Same 0.828 as Lesson 2.1. Then the payoff at prediction time:

6 **A raw record in, a probability out (1m).** "Double brackets: a one-row table, not a Series."

```
one_customer = X_test.iloc[[0]]
print(one_customer[["days_since_last_order", "email_opens_90d", "home_store",
print("churn probability:", churn_pipe.predict_proba(one_customer)[:, 1].round
```

```
      days_since_last_order  email_opens_90d home_store age_group
2111                     53              5.0  Asheville     25-34
churn probability: [0.061]
```

7 **A leak you could write on real data (4m).** "A colleague proposes a clever feature: for each signup date, the churn rate of customers who joined that day. Prediction first: is the signup date a strong signal?" Then run it.

```
from sklearn.model_selection import StratifiedKFold, cross_val_score
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
date_rate = y_train.groupby(train["signup_date"]).mean()
X_date = pd.DataFrame({"signup_churn_rate": train["signup_date"].map(date_rate
leaky = cross_val_score(LogisticRegression(), X_date, y_train, cv=cv, scoring=
print(f"leaky CV ROC AUC: {leaky.mean():.3f}")
```

```
leaky CV ROC AUC: 0.930
```

Let the room react to 0.930, higher than the full 16-column model. Ask: "How many customers joined on a typical date?" (Most of the 2,150 dates have one or two.) "So what is 'the churn rate of your signup date' for most customers?" (Their own label.)

8 **The honest version (3m).** "`TargetEncoder` computes the same kind of rate, but as a transformer: inside a pipeline it is refitted on the training folds of every round."

```
from sklearn.preprocessing import TargetEncoder
honest_pipe = make_pipeline(TargetEncoder(cv=cv), LogisticRegression())
honest = cross_val_score(honest_pipe, train[["signup_date"]], y_train, cv=cv,
print(f"honest CV ROC AUC: {honest.mean():.3f}")
honest_pipe.fit(train[["signup_date"]], y_train)
test_auc = roc_auc_score(y_test, honest_pipe.predict_proba(test[["signup_date"
print(f"test ROC AUC     : {test_auc:.3f}")
```

```
honest CV ROC AUC: 0.586
test ROC AUC     : 0.571
```

Write on the board: **0.930 leaky / 0.586 honest CV / 0.571 test.** The honest CV score and the test agree; the leaky number was a broken measurement, not a better model. Then: "Steps that learn from y leak the most. You do not need to judge each case: put every learning step in the pipeline."

9 **Cross-validate the whole pipeline (2m).** "One call; every fold refits the imputer, both scalers, both encoders and the model."

```
from sklearn.dummy import DummyClassifier
baseline_pipe = Pipeline([("prep", preprocess), ("model", DummyClassifier(stra
for name, pipe in [("baseline", baseline_pipe), ("logistic regression", churn_
    scores = cross_val_score(pipe, X_train, y_train, cv=cv, scoring="roc_auc")
    print(f"{name:20s} CV ROC AUC {scores.mean():.3f} +/- {scores.std():.3f}")
```

```
baseline             CV ROC AUC 0.500 +/- 0.000
logistic regression  CV ROC AUC 0.833 +/- 0.021
```

`cross_val_score` clones the pipeline per fold, so the fitted `churn_pipe` is untouched.

10 **Every setting has an address (3m).** "Step name, two underscores, setting; deeper levels the same way. Read it like a file path."

```
params = churn_pipe.get_params()
for key in ["model__C", "model__max_iter", "prep__email__simpleimputer__strate
    print(f"{key:40s} {params[key]!r}")
print(len(params), "settings in total")
```

```
model__C                                 1.0
model__max_iter                          100
prep__email__simpleimputer__strategy     'median'
prep__onehot__handle_unknown             'ignore'
73 settings in total
```

Then the loop:

11 **Turn a knob from the top (2m).** "`set_params` uses the same names. The loop ends at the default, so the pipeline is back where it started."

```
for c_value in [0.01, 1.0]:
    churn_pipe.set_params(model__C=c_value)
    scores = cross_val_score(churn_pipe, X_train, y_train, cv=cv, scoring="roc
    print(f"C={c_value:<5} CV ROC AUC {scores.mean():.3f}")
```

```
C=0.01  CV ROC AUC 0.835
C=1.0   CV ROC AUC 0.833
```

12 **Swap a whole step (2m).** "A step name can take a whole new object. `clone` gives an unfitted copy, so `churn_pipe` keeps its logistic regression." (Boosting takes 10 to 30 seconds here: start it, then talk.)

```
from sklearn.base import clone
from sklearn.ensemble import HistGradientBoostingClassifier
boost_pipe = clone(churn_pipe).set_params(model=HistGradientBoostingClassifier
scores = cross_val_score(boost_pipe, X_train, y_train, cv=cv, scoring="roc_auc
print(f"gradient boosting CV ROC AUC {scores.mean():.3f} +/- {scores.std():.3f
```

```
gradient boosting CV ROC AUC 0.861 +/- 0.013
```

About 0.03 better than logistic regression on identical preprocessing: the true churn patterns are not a simple weighted sum. Module 5 investigates; Lesson 2.3, after the break, attacks it from the other side.

13 **A first grid search (2m).** "Every combination, cross-validated, best one refitted. It receives the whole pipeline, so every candidate is leak-free."

```
from sklearn.model_selection import GridSearchCV
grid = GridSearchCV(
    churn_pipe,
    param_grid={
        "model__C": [0.01, 0.1, 1.0],
        "prep__email__simpleimputer__strategy": ["median", "mean"],
    },
    cv=cv, scoring="roc_auc",
)
grid.fit(X_train, y_train)
print(grid.best_params_)
print(f"best CV ROC AUC {grid.best_score_:.3f}")
```

```
{'model__C': 0.01, 'prep__email__simpleimputer__strategy': 'median'}
best CV ROC AUC 0.835
```

Six combinations × five folds = 30 fits + 1 refit. The point: a hundredfold change in C and two fill strategies move the score in the third decimal. Knowing what does not matter is a result. Lesson 6.2 does tuning properly.

14 **A custom step, measured honestly (2m).** "`FunctionTransformer` turns a plain function into a step. Lesson 2.1's histograms looked nicer after a log. Does the model agree?"

```
from sklearn.preprocessing import FunctionTransformer
skewed = ["spend_12m", "avg_order_value", "days_since_last_order"]
others = [c for c in complete if c not in skewed]
log_prep = ColumnTransformer([
    ("log", make_pipeline(FunctionTransformer(np.log1p, feature_names_out="one
    ("scale", StandardScaler(), others),
    ("email", make_pipeline(SimpleImputer(strategy="median", add_indicator=Tru
     ["email_opens_90d"]),
    ("tier", make_pipeline(OrdinalEncoder(categories=[["Basic", "Silver", "Gol
     ["loyalty_tier"]),
    ("onehot", OneHotEncoder(sparse_output=False, handle_unknown="ignore"), no
])
log_pipe = Pipeline([("prep", log_prep), ("model", LogisticRegression())])
scores = cross_val_score(log_pipe, X_train, y_train, cv=cv, scoring="roc_auc")
print(f"with log1p on 3 skewed columns: CV ROC AUC {scores.mean():.3f}")
print(log_pipe.fit(X_train, y_train)[:-1].get_feature_names_out()[:4])
```

```
with log1p on 3 skewed columns: CV ROC AUC 0.812
['log__spend_12m' 'log__avg_order_value' 'log__days_since_last_order'
 'scale__tenure_months']
```

Worse: 0.812 against 0.833. Churn jumps near 150 days of silence and the log squeezes exactly that region. The method is the lesson: one cross-validation call settled it. Lesson 2.3 builds the threshold directly.

15 **Save it as one file (1m).** "Everything needed to score a customer (fill values, means, category lists, weights) in one file."

```
import joblib
joblib.dump(churn_pipe, "churn_pipeline.joblib")
loaded = joblib.load("churn_pipeline.joblib")
same = np.allclose(loaded.predict_proba(X_test)[:, 1], churn_pipe.predict_prob
print("identical predictions after reloading:", same)
```

```
identical predictions after reloading: True
```

Say the warning out loud: a joblib file can run code when loaded. Only load files you or your team made. Lesson 12.1 shows skops and version pinning.

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **A setting without its step name.** `ValueError: Invalid parameter 'C' for estimator Pipeline(steps=[('standardscaler', StandardS...` Coach: "The pipeline has only four settings of its own; where does C live?" Then `list(pipe.get_params())`. The variant in the self-check (Q3): `model__C` on a `make_pipeline` pipeline, whose step is called `logisticregression`.
- **The model in the middle.** `TypeError: All intermediate steps should be transformers and implement fit and transform or be the string 'pas...` (it also happens when a learner appends a scaler after the model to "scale the predictions"). Coach: "Which steps need a `transform`?" Transformers first, exactly one model last. Also expect `ValueError: Estimator names must not contain __: got ['my__scaler']` when someone names a step `my__scaler`.
- **Believing the pipeline guards the columns too.** A `make_column_selector` pointed at the whole table takes every text column, `cancellation_reason` included: CV ROC AUC 0.937, no error (the lesson's Common Mistake 5). Coach: "Which leak does a pipeline prevent: between rows, or from the future?" Rows only. The feature list is still the learner's job. A missing column at prediction time is the opposite case: `ValueError: columns are missing: {'email_opens_90d'}`, fixed by sending the column (empty if unknown).

GUIDED LAB

- **Lab A (12m, everyone):** `labs/module-04/l2_2-pipelines/fraud_pipeline_starter.py`; reference `labs/module-04/l2_2-pipelines/fraud_pipeline_solution.py`. The lesson's moves on a new problem: card fraud, nine numeric columns plus the text `category`. Build `prep` + `model`, compare with a baseline pipeline by cross-validation (ROC AUC and average precision), reach inside, turn `model__C`, test once, save to `OUTPUT`, reload, score three raw payments, one in a category Pinecrest does not sell.
- **Expected, Lab A:** baseline CV ROC AUC 0.500 / average precision 0.009; logistic regression 0.867 / 0.535; 10 categories learned, the largest weight `onehot__category_Gift Cards` 0.95; `model__C=0.01` gives 0.518, 1.0 gives 0.535; test ROC AUC 0.843, average precision 0.463, 56 frauds among the 100 riskiest; reloaded steps `['prep', 'model']`, 10 columns; probabilities `[0.999, 0.0, 0.002]`.
- **Circulate in Lab A for:** `cross_validate` results read as `res["roc_auc"]` (the key is `test_roc_auc`); `named_transformers` without the trailing underscore (the unfitted originals, no `categories_`); forgetting to put C back to 1.0. **Debrief:** "Why did the Bikes payment get a score?" `handle_unknown="ignore"` made it a row of zeros in the one-hot block. And: "Why is the test AUC (0.843) below the CV estimate (0.867)?" With 131 test frauds, a few hard ones move the number; the folds' spread is wide too. Lesson 2.5 returns to this data.
- **Lab B (8m, pairs):** `labs/module-04/l2_2-pipelines/leak_size_starter.py`; reference `labs/module-04/l2_2-pipelines/leak_size_solution.py`. Three steps, each done before cross-validation and inside a pipeline: a fraud rate per dollar amount (learns from y, few rows per value), a scaler (learns from X), `np.log1p` (learns nothing). The partner who is not typing predicts each gap out loud first.
- **Expected, Lab B:** 1,016 distinct amounts, 434 of them 5 times or fewer; target rate 0.913 before CV, 0.583 inside the pipeline, 0.568 on the test set; scaler 0.8613 both ways (difference -0.00001); log1p 0.8617 both ways (+0.00000). Fill in the board's "Leak size" box from it: huge / tiny / none.
- **Fast finishers:** `labs/module-04/l2_2-pipelines/debug_pipeline.py`: a demand pipeline with three loud bugs (`alpha` without its step name, a step called `one__hot`, a missing column at prediction time) and one silent one (the "test" rows are the training year). Fixed, it prints `2025 test MAE: 11.75 units` (11.02 while the silent bug remains).

START THE LESSON PROJECT

The lesson project is **a deployable churn pipeline**: chosen by cross-validation, tested once, saved, reloaded and used on raw records. In the room, Steps 1 to 3 (15 minutes).

- Read the brief aloud ("one file that takes raw rows and returns probabilities"). Step 1 is the demo's setup plus `cv`: done = `(4500, 16) (1500, 16)`.
- Step 2: the preprocessor once, a `churn_pipeline(model)` function, three candidates. Step 3: the same folds for all three. **Done in the room** = baseline 0.500, logistic regression 0.833 ± 0.021, gradient

boosting 0.861 ± 0.013, `chosen: gradient boosting` (the boosting CV takes up to a minute on a laptop).

- **Goes home:** Step 4 (test ROC AUC 0.863, 42 real churners in the top 50 against 8.3 at random), Step 5 (`churn_pipeline.joblib`, 16 columns expected), Step 6 (two raw records: N1 0.866, N2 0.002; `monthly_churn_scores.csv`). Tell them to KEEP the `.joblib` file: later lessons compare new models against it.

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab:

- What does `pipe.fit` call on the scaler, and what does `pipe.predict` call? **Answer:** `fit_transform` during fit (then passes the result on); `transform` only during predict.
- Why did the signup-date rate score 0.930 in CV and about 0.57 on test? **Answer:** it was computed with every training label, including each validation row's own; most dates have one or two customers, so the feature was mostly the answer.
- Which name sets C in `Pipeline([("prep", ...), ("model", LogisticRegression())])`? **Answer:** `model__C`.
- Can a pipeline stop `cancellation_reason` from leaking? **Answer:** no: it separates rows, not columns. Keeping future columns out of the feature list is your job.
- Why is `FunctionTransformer(np.log1p)` safe even outside the pipeline? **Answer:** it is stateless: it learns nothing from the data.

Workbook self-check, items Q1 to Q6 (learners do them at the end of the block or at home).

Answers: **Q1** B) `transform` only, with the means and standard deviations it learned during `fit`; **Q2** D) `prep__num__simpleimputer__strategy`; **Q3** A) `ValueError`: `Invalid parameter 'model'`: make_pipeline named the step `logisticregression`; **Q4** C) `FunctionTransformer(np.log1p)`; **Q5** `[[4.605]]`; **Q6** D) `ValueError: columns are missing: {'email_opens_90d'}`: send the column, empty (NaN) where unknown. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: give anyone lost in the names a printed `list(churn_pipe.get_params())` and have them find the address of three settings before the lab. In Lab A, let a struggling pair copy the `ColumnTransformer` from the demo notebook: the goal is the comparison by cross-validation, not retyping branches.

Stretch: the lesson's Exercise 4 (one grid that swaps model families: best boosting with learning rate 0.05, CV 0.866) and the Bonus Challenge (a `Winsorizer` class with `fit` learning percentiles). Ask them why the percentiles must be learned in `fit`, not in `transform`.

2.3 Creating Features

75 min

OBJECTIVES (FROM THE LESSON PAGE)

- Explain why a linear model cannot use a threshold, a U-shape or an interaction unless you build it as a feature
- Create flags, ratios and interaction terms with pandas, choosing them from the training data only
- Extract calendar parts and cyclical (sine and cosine) features from a date column
- Build lag and rolling-window features per group with `shift`, so no row sees its own future
- Put feature creation inside a Pipeline with `FunctionTransformer` and measure the lift with cross-validation

TIMING IN THE ROOM

10m	Frame: the interviewer and the calculator; the feature loop
30m	Live demo: the steps below (protect 2 to 6)
20m	Guided lab: `night_features_starter.py` (everyone), then `daily_sales_features_starter.py` (pairs)
15m	Start the lesson project: the weekly demand forecaster, Steps 1 to 3

MENTAL MODEL TO SAY

Say: "Think of a model as a very fast calculator with a fixed shape. Logistic regression can only add up weighted columns: more of a column always pushes the same way, by the same amount. It cannot say 'nothing much happens until 150 days, then risk jumps', and it cannot say 'tickets matter, but only without the app'. You are the **interviewer** who decides which questions the calculator gets. Ask 'days since the last order?' and it answers with a straight line. Also ask 'more than 150 days?' and it can express a jump."

Then the loop, on the board: explore the TRAINING rows → write a feature function → put it in the pipeline → cross-validate against the plain model → keep it or drop it. "Most ideas will not help. An idea that does not help is a cheap experiment, not a failure: you will record two of them today."

LIVE DEMO (YOU TYPE, THEY FOLLOW)

New notebook `lesson_2_3`. Steps 2 to 4 are three groupbys: before each, ask the room to sketch the shape they expect (a line, a step, a U). Steps 9 to 14 switch to the weekly demand table.

1. **The standard split, ten numeric columns (1m).** "Lesson 1.1's ten columns. Everything we explore today uses the training rows only."

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
pd.set_option("display.width", 150)
customers = pd.read_csv("data/customers_churn.csv")
base_features = ["tenure_months", "orders_12m", "spend_12m", "avg_order_value"
                 "returns_12m", "return_rate", "support_tickets_12m", "discoun
train, test = train_test_split(customers, test_size=0.25, stratify=customers["
X_train, y_train = train[base_features], train["churned"]
X_test, y_test = test[base_features], test["churned"]
print(X_train.shape, X_test.shape)
```

```
(4500, 10) (1500, 10)
```

2 **Is risk a straight line in recency? (3m).** "A logistic regression assumes every extra day of silence adds the same risk. Prediction: does it?"

```
recency = pd.cut(train["days_since_last_order"], bins=[-1, 30, 90, 150, 250, 3
print(train.groupby(recency, observed=True)["churned"].agg(["mean", "size"]).r
```

```
                        mean  size
days_since_last_order
(-1, 30]               0.088  1516
(30, 90]               0.096  1591
(90, 150]              0.158   646
(150, 250]             0.415   431
(250, 364]             0.560   316
```

9 to 10% for three months, 16% by 150 days, then over 40%. Something happens around five months. One weight has to compromise between the flat part and the jump.

3 **A U-shape (2m).** "Tenure."

```
tenure = pd.cut(train["tenure_months"], bins=[0, 5, 12, 36, 72, 96])
print(train.groupby(tenure, observed=True)["churned"].agg(["mean", "size"]).ro
```

```
                mean  size
tenure_months
(0, 5]         0.320   930
(5, 12]        0.109   238
(12, 36]       0.076   973
(36, 72]       0.099  1397
(72, 96]       0.215   962
```

New customers churn most, the middle is safe, veterans drift away. "The model in Lesson 1.1 gave tenure a tiny weight. It was not saying tenure does not matter; it was saying 'I cannot draw this shape'."

4 **An interaction (3m).** "Tickets, split by the app."

```
tickets = train["support_tickets_12m"].clip(upper=4)
print(train.pivot_table(index=tickets, columns="app_user", values="churned", a
```

```
app_user                 0      1
support_tickets_12m
0                    0.141  0.110
1                    0.214  0.116
2                    0.266  0.112
3                    0.331  0.120
4                    0.464  0.038
```

Without the app every ticket raises churn, to 46%; with the app, flat. One weight for tickets is the average of a strong effect and no effect.

5 **Five questions as five columns (4m).** "Each pattern becomes one column. Write them as ONE function: easy to test, reuse and put in a pipeline. `df.copy()` keeps it pure."

```
def add_churn_features(df):
    df = df.copy()
    df["lapsed_150"] = (df["days_since_last_order"] > 150).astype(int)
    df["new_customer"] = (df["tenure_months"] < 6).astype(int)
    df["veteran"] = (df["tenure_months"] > 72).astype(int)
    df["heavy_discount"] = (df["discount_share"] > 0.35).astype(int)
    df["tickets_no_app"] = df["support_tickets_12m"] * (1 - df["app_user"])
    return df
engineered = add_churn_features(X_train)
new_columns = ["lapsed_150", "new_customer", "veteran", "heavy_discount", "tic
print(engineered[["tenure_months", "new_customer", "veteran"]].head(3))
print(engineered[["support_tickets_12m", "app_user", "tickets_no_app"]].head(3
print(engineered[new_columns].mean().round(3))
```

```
      tenure_months  new_customer  veteran
1556              1             1        0
965              44             0        0
5145              4             1        0
      support_tickets_12m  app_user  tickets_no_app
1556                    3         0               3
965                     2         1               0
5145                    0         0               0
lapsed_150        0.166
new_customer      0.207
veteran           0.214
heavy_discount    0.210
tickets_no_app    0.532
dtype: float64
```

Check two rows by hand against the raw columns. The means say how common each flag is: none is almost always 0 or 1.

6 **Measure the lift, inside a pipeline (4m).** "`FunctionTransformer` wraps the function; it learns nothing, so `fit` does nothing and `transform` calls it. Prediction first: logistic regression with five more columns against boosting on the raw ones." (Boosting takes a while: start it, talk.)

```
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import FunctionTransformer, StandardScaler
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
candidates = {
    "logistic, raw features": make_pipeline(StandardScaler(), LogisticRegressi
    "logistic, + engineered": make_pipeline(FunctionTransformer(add_churn_feat
                                            LogisticRegression()),
    "boosting, raw features": HistGradientBoostingClassifier(random_state=42),
}
cv_auc = {}
for name, pipe in candidates.items():
    scores = cross_val_score(pipe, X_train, y_train, cv=cv, scoring="roc_auc")
    cv_auc[name] = scores.mean()
    print(f"{name:24s} CV ROC AUC {scores.mean():.3f} +/- {scores.std():.3f}")
```

```
logistic, raw features   CV ROC AUC 0.829 +/- 0.025
logistic, + engineered   CV ROC AUC 0.883 +/- 0.018
boosting, raw features   CV ROC AUC 0.854 +/- 0.013
```

0.829 → 0.883, a bigger step than the folds' spread, and past boosting (0.854), which had to find the same shapes alone. Same data, same model, better questions.

7 **What the model sees now (2m).** "Ask the whole pipeline for pandas output and read the weights by name."

```
churn_pipe = candidates["logistic, + engineered"].set_output(transform="pandas
churn_pipe.fit(X_train, y_train)
model = churn_pipe[-1]
weights = pd.Series(model.coef_[0], index=model.feature_names_in_)
print(weights[new_columns + ["days_since_last_order", "tenure_months", "suppor
```

```
lapsed_150               0.60
new_customer             0.92
veteran                  0.49
heavy_discount           0.73
tickets_no_app           0.64
days_since_last_order    0.58
tenure_months            0.15
support_tickets_12m     -0.07
dtype: float64
```

Every new column carries a big weight; `support_tickets_12m` itself turned slightly negative, because `tickets_no_app` took over the ticket effect. The features now say what the tables said.

8 **One test-set check (1m).** "The choice is made; now one look."

```
from sklearn.metrics import roc_auc_score
plain_pipe = candidates["logistic, raw features"].fit(X_train, y_train)
for name, pipe in [("raw features", plain_pipe), ("+ engineered", churn_pipe)]
    auc = roc_auc_score(y_test, pipe.predict_proba(X_test)[:, 1])
    print(f"{name:14s} test ROC AUC {auc:.3f}  accuracy {pipe.score(X_test, y_
```

```
raw features   test ROC AUC 0.823  accuracy 0.848
+ engineered   test ROC AUC 0.880  accuracy 0.876
```

0.823 → 0.880 on the test set, in line with cross-validation: what honest feature choice looks like.

9 **Dates are not numbers yet (2m).** "Second dataset: two years of weekly sales. `parse_dates`, then the `.dt` accessor."

```
demand = pd.read_csv("data/weekly_demand.csv", parse_dates=["week_start"])
print(demand["week_start"].dtype)
print(demand[["week_start", "store", "product_id", "category", "units_sold"]].
demand["month"] = demand["week_start"].dt.month
demand["quarter"] = demand["week_start"].dt.quarter
demand["day_of_year"] = demand["week_start"].dt.dayofyear
print(demand[["week_start", "month", "quarter", "day_of_year"]].drop_duplicate
```

```
datetime64[us]
  week_start      store product_id     category  units_sold
0 2024-01-01  Asheville   ACC-2001  Accessories          20
1 2024-01-01  Asheville   ACC-2002  Accessories          15
2 2024-01-01  Asheville   ACC-2003  Accessories          14
     week_start  month  quarter  day_of_year
0    2024-01-01      1        1            1
480  2024-02-05      2        1           36
864  2024-03-04      3        1           64
1248 2024-04-01      4        2           92
```

10 **Months on a circle (2m).** "Is December far from January? As numbers, 11 apart. On a circle, neighbors."

```
demand["month_sin"] = np.sin(2 * np.pi * demand["month"] / 12)
demand["month_cos"] = np.cos(2 * np.pi * demand["month"] / 12)
circle = demand.drop_duplicates("month").set_index("month")[["month_sin", "mon
def gap(a, b):
    return np.linalg.norm(circle.loc[a] - circle.loc[b])
print(f"Dec to Jan: number gap {12 - 1}, circle gap {gap(12, 1):.2f}")
print(f"Jun to Jul: number gap {7 - 6}, circle gap {gap(6, 7):.2f}")
```

```
Dec to Jan: number gap 11, circle gap 0.52
Jun to Jul: number gap 1, circle gap 0.52
```

Both columns are needed: sine alone gives two different months the same value (quiz question 5).

11 **Every category has its own season (3m).** "The most important table of this half."

```
season = (demand[demand["year"] == 2024]
          .groupby(["month", "category"])["units_sold"].mean()
          .unstack()[["Snow", "Tents", "Camp Kitchen", "Apparel"]])
print(season.loc[[1, 4, 7, 10, 12]].round(1))
demand["category_month"] = demand["category"] + "-" + demand["month"].astype(s
print(demand["category_month"].nunique(), "levels, for example:", demand["cate
```

```
category  Snow  Tents  Camp Kitchen  Apparel
month
1         30.3    4.9          15.0     26.6
4          3.2   15.0          29.5     18.8
7          0.2   30.4          65.6     14.8
10        10.2    9.6          23.2     27.7
12        41.8    8.8          25.5     36.3
108 levels, for example: ['Accessories-1', 'Apparel-1', 'Climbing-1', 'Footwea
```

Snow in winter, tents in summer: one shared effect per month averages opposite curves into nothing. A category × month column gives each category its own season. The project measures what it is worth (MAE 11.23 → 9.39).

12 **History: lags and rolling windows (3m).** "Last week's tents in Bend, not last week's socks in Boise: per series, past only. `shift(1)` keeps the current week out."

```
demand = demand.sort_values(["store", "product_id", "week_start"])
by_series = demand.groupby(["store", "product_id"])["units_sold"]
demand["lag_1"] = by_series.shift(1)
demand["rolling_4"] = by_series.transform(lambda s: s.shift(1).rolling(4).mean
print(demand[["week_start", "store", "product_id", "units_sold", "lag_1", "rol
demand = demand.sort_index()
```

```
    week_start      store product_id  units_sold  lag_1  rolling_4
0   2024-01-01  Asheville   ACC-2001          20    NaN        NaN
96  2024-01-08  Asheville   ACC-2001          39   20.0        NaN
192 2024-01-15  Asheville   ACC-2001          13   39.0        NaN
288 2024-01-22  Asheville   ACC-2001          21   13.0        NaN
384 2024-01-29  Asheville   ACC-2001          30   21.0      23.25
480 2024-02-05  Asheville   ACC-2001          37   30.0      25.75
```

Follow one row by hand: January 29, lag 21, rolling mean of 20, 39, 13, 21 = 23.25. Ask the question of the day: "On the day this forecast is made, would this number exist?"

13 **An aggregate from the training rows (2m).** "Same product, store and month last year: computed from 2024, looked up for 2025."

```
from sklearn.metrics import mean_absolute_error
train_2024 = demand[demand["year"] == 2024]
test_2025 = demand[demand["year"] == 2025]
keys = ["product_id", "store", "month"]
same_month_avg = train_2024.groupby(keys)["units_sold"].mean().rename("avg_sam
test_2025 = test_2025.join(same_month_avg, on=keys)
overall_mean = train_2024["units_sold"].mean()
print(f"predict the 2024 mean   MAE {mean_absolute_error(test_2025['units_sold
print(f"same month last year    MAE {mean_absolute_error(test_2025['units_sold
```

```
predict the 2024 mean   MAE 17.13
same month last year    MAE 11.36
```

One honest group average cuts the error by a third.

14 **Two silent leaks (2m).** "Neither raises an error. Both look like better features." Run the rolling window without `shift`, then the average over both years.

```
demand = demand.sort_values(["store", "product_id", "week_start"])
by_series = demand.groupby(["store", "product_id"])["units_sold"]
demand["rolling_4_leaky"] = by_series.transform(lambda s: s.rolling(4).mean())
demand = demand.sort_index()
in_2025 = demand["year"] == 2025
for col in ["rolling_4", "rolling_4_leaky"]:
    print(f"{col:16s} used as the forecast: MAE {mean_absolute_error(demand.lo
leaky_avg = demand.groupby(keys)["units_sold"].mean()
leaky_forecast = test_2025.join(leaky_avg.rename("leaky_avg"), on=keys)["leaky
print(f"average from 2024 only   MAE {mean_absolute_error(test_2025['units_sol
print(f"average from both years  MAE {mean_absolute_error(test_2025['units_sol
```

```
rolling_4        used as the forecast: MAE 12.21
rolling_4_leaky  used as the forecast: MAE 9.28
average from 2024 only   MAE 11.36
average from both years  MAE 9.87
```

12.21 → 9.28 and 11.36 → 9.87, both fake: the window contains the week being predicted, the average contains the 2025 answers. Fix: shift first, then roll; aggregate the training rows only.

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Dates read as text.** `AttributeError: Can only use .dt accessor with datetimelike values` Coach: "What is the column's dtype?" `parse_dates` when loading.
- **A ratio that divides by zero.** About 40% of customers have no support tickets, so `returns_12m / support_tickets_12m` quietly makes `inf` and NaN, and the error appears only inside the model:

`ValueError: Input X contains infinity or a value too large for dtype('float64').` Coach: "Which rows have 0 in the denominator?" Divide by something that cannot be 0 (`+ 1`), or replace on purpose.

- **A feature that knows the answer.** A rolling window without `shift(1)` (MAE 12.21 → 9.28), an average that includes the test year (11.36 → 9.87), or a threshold picked while looking at the test set. None raises anything. Coach with the one question: "On the day this prediction is made, would this number exist?" Write it on the board.

GUIDED LAB

- **Lab A (12m, everyone):** `labs/module-04/l2_3-creating-features/night_features_starter.py`; reference `labs/module-04/l2_3-creating-features/night_features_solution.py`. Closes a loop from Session 1, where the fraud model gave `hour` a backwards (negative) weight. Learners look at fraud by hour band and night × new device on the training rows, write `add_fraud_features` (night, night_new_device, log_amount), put it in the pipeline with `FunctionTransformer`, and compare by cross-validation.
- **Expected, Lab A:** night (0 to 5) 6.71% fraud, about 0.65% in every other band; night with a new device 0.425 vs 0.022 or less elsewhere; CV ROC AUC 0.867 → 0.864 (flat) but CV average precision 0.535 → 0.564; the `hour` weight -0.28 → 0.01, `night_new_device` 0.19; test average precision 0.463 → 0.504, frauds in the top 100: 56 → 61.
- **Circulate in Lab A for:** `pipe[:-1].get_feature_names_out()` raising (a `FunctionTransformer` without `feature_names_out` cannot name its output: ask the `prep` step, as the TODO says); a feature function without `df.copy()`; computing flags on `X_train` by hand outside the pipeline and then forgetting them for the test set. **Debrief:** "ROC AUC did not move. Is the feature worthless?" No: average precision and the top 100 improved, which is where a review team works; AUC averages over the whole ranking. Choosing the metric that matches the job is Lesson 1.4 again.
- **Lab B (8m, pairs):** `labs/module-04/l2_3-creating-features/daily_sales_features_starter.py`; reference `labs/module-04/l2_3-creating-features/daily_sales_features_solution.py`. Daily store sales, where the day of the week matters: calendar parts, `lag_1`, `lag_7`, a shifted `rolling_7` per store, each used as a forecast, then three Ridge pipelines; TODO 5 (a leaky window) can go home.
- **Expected, Lab B:** mean sales Mon..Sun 4,639 / 4,270 / 4,512 / 4,750 / 6,024 / 8,209 / 6,988; as forecasts `lag_1` MAE 1,350, `lag_7` 1,072, `rolling_7` 1,451; Ridge: calendar only 996, + lag_7 + rolling_7 708, + store × day + lag_1 586; the leaky window 551 (better, and useless).

START THE LESSON PROJECT

The lesson project is **the weekly demand forecaster**: a linear model for 2025 built from 2024, one idea at a time, with a scoreboard. In the room, Steps 1 to 3 (15 minutes).

- Read the brief aloud. Step 1 builds every feature on the full table before splitting (lags must reach back from January 2025 into December 2024), using only the past. **Done** = `5088 4896 0 missing lags in test`, `baseline MAE 17.13`.
- Step 2: the `evaluate` helper and two models: raw columns 11.28, + month 11.23. Ask: "Why did month do almost nothing?" (The season table: opposite seasons average out.) Step 3: + category × month **9.39**.
- **Goes home:** Step 4 (log target 7.92), Step 5 (lags: still 7.92, a genuine non-result), Step 6 (boosting on raw columns 8.61, the scoreboard chart, `demand_scoreboard.csv`).

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab:

- Why did the logistic regression give tenure a small weight? **Answer:** the effect is a U-shape; a straight line through a U is nearly flat.
- What is `support_tickets_12m * (1 - app_user)` for app users? **Answer:** 0. It gives non-app tickets their own weight: an interaction.
- Which rows may you look at to choose the 150-day threshold? **Answer:** the training rows only.
- Why do cyclical encodings need both sine and cosine? **Answer:** either one alone gives two different months the same value.
- A rolling mean improved a forecast dramatically. First question? **Answer:** does the window include the row being predicted (no `shift(1)`)?

Workbook self-check, items Q7 to Q12 (learners do them at the end of the block or at home).

Answers: **Q7** `[nan, 5.0, nan, 9.0, 7.0]`; **Q8** C) A straight line through a U-shape is nearly flat; flags such as new_customer and veteran let the model express it; **Q9** A) Give support tickets their own weight for customers without the app; **Q10** `[ 0. -1. 1.]`; **Q11** B) The 2025 weeks being predicted helped compute their own feature; **Q12** C) `[0.5, nan, inf]`. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: for learners slowed by pandas, hand them the finished `add_churn_features` from the lesson and ask them to check two rows by hand against the raw columns instead of typing it; in Lab B, do TODO 1 and 2 together on the projector.

Stretch: the lesson's Exercise 3 (`SplineTransformer` on three columns: CV 0.870 with no hand-picked cut-offs) and Exercise 4 (give boosting the same ideas: MAE 8.37 vs the log-linear 7.92), then the Bonus Challenge (temperature × category interactions).

CLOSING

EXIT TICKET (3 QUESTIONS, NO NOTES, 4 MINUTES)

Put the three questions on the slide. Learners answer in the workbook's Exit Ticket box or on a sticky note. Before Session 5, sort the notes: question 1 tells you whether to spend the Session 5 warm-up on "inside the folds" (Lesson 2.4's selection leak and Lesson 2.5's SMOTE leak are the same idea).

1. Name two things a Pipeline does that you had to remember by hand in Lesson 2.1, and one kind of leak it cannot prevent. **Answer:** it fits every step on the training rows and only transforms at prediction time; cross-validation and grid search refit every step inside each fold (and one file saves it all). It cannot stop a column from the future (`cancellation_reason`) that you put in the feature list.
2. The signup-date churn rate scored 0.930 in cross-validation and 0.571 on the test set. Why, and what fixed the measurement? **Answer:** the rates were computed from every training label before the folds, so each validation row's own label was in its feature. `TargetEncoder` inside a pipeline refits per fold: honest CV 0.586, matching the test.
3. Give one feature you built today, the shape or interaction it lets a linear model express, and how you knew it helped. **Answer (examples):** `lapsed_150` (a jump at 150 days), `new_customer` / `veteran` (a U-shape), `tickets_no_app` or `night_new_device` (an interaction), category × month (a season per category); measured with cross-validation against the plain model (0.829 → 0.883; MAE 11.23 → 9.39).

HOMEWORK (ABOUT 2 TO 3 HOURS)

- **Finish both lesson projects.** 2.2 Steps 4 to 6 (test ROC AUC 0.863, 42 of the top 50, `churn_pipeline.joblib`, `monthly_churn_scores.csv` with N1 0.866 and N2 0.002). 2.3 Steps 4 to 6 (log target 7.92, lags 7.92, boosting 8.61, `demand_scoreboard.csv`).
- **Lesson exercises.** 2.2: Exercises 1 to 3 (a value deep inside, the fill strategy, a demand pipeline with a log target); Exercise 4 is a stretch. 2.3: Exercises 1 to 3 (the discount flag, all pairwise interactions, splines); Exercise 4 is a stretch.
- **Both Quick Quizzes** on the lesson pages (answers in the Answer Key).
- **Journal:** both entries. 2.2: explain the signup-date leak to someone who has never heard of cross-validation. 2.3: the interviewer and the calculator, plus the two scoreboard lines where a business idea mattered most.
- **Before the Next Lesson (2.2 and 2.3):** keep `churn_pipeline.joblib`, `demand_scoreboard.csv` and both project notebooks; rewrite one Module 1 model as a pipeline; **write down three columns of the churn table you suspect carry the same information as another column** (Lesson 2.4 checks your guesses).
- **Optional:** Lab B of either lesson if it was not finished; `labs/module-04/l2_2-pipelines/debug_pipeline.py`.

BRING LIST FOR SESSION 5

- A laptop with the course environment (`check_environment.py` READY; nothing new to install: imbalanced-learn, which Lesson 2.5 imports as `imblearn`, is already in `requirements.txt`; check with `python -c "import imblearn"`).
- The `pinecrest-ml` folder with `data/` and `labs/`. Session 5's starters are in `labs/module-05/`.

- Your Lesson 2.3 notebook with `add_churn_features` in it: Lesson 2.4's Exercise 4 brings the five churn features back. Lessons 2.4 and 2.5 otherwise start fresh notebooks from the CSV files (`lesson_2_4_project`, `lesson_2_5_project`).
- Your list of three churn columns you suspect are redundant (2.3's Before the Next Lesson): Lesson 2.4 opens by checking them.
- Precision, recall and average precision from Lesson 1.4, fresh in mind (ten minutes with the 1.4 Key Takeaways): Lesson 2.5 leans on them.
- Keep (not needed next session, needed later): `churn_pipeline.joblib`, `demand_scoreboard.csv`. The workbook and the journal.

IF YOU RUN LONG OR SHORT

RUNNING LONG: WHAT TO PROTECT AND WHAT TO CUT FIRST

- **Protect:** 2.2 demo Steps 7 and 8 (leak and honest fix); 2.3 demo Steps 2 to 6 (look, build, measure); Lab 2.2 A through TODO 3; Lab 2.3 A through TODO 4; starting both projects (Step 1 at least: it proves the setup).
- **Cut first (saves 5m):** 2.2 demo Steps 13 and 14 (grid search and the log step): say "a hundredfold change in C moved the third decimal" and "the log made it worse, 0.812, and one CV call told us".
- **Cut second (saves 5m):** 2.3 demo Steps 9 and 10 (calendar parts and the circle): show the slide.
- **Cut third (saves 8m each):** Lab B of either lesson moves to homework; its expected output is in the starter.

RUNNING SHORT: WHAT TO ADD

- The 2.2 lesson's Common Mistake 5 live: `make_column_selector` on the whole table (0.937, and the printed column list shows why).
- 2.3 Exercise 2 live: `PolynomialFeatures` on all pairs (0.827): brute force buries the one useful product among 44 meaningless ones.
- Lab 2.2's `debug_pipeline.py` together on the projector: read each error from the start of the message.

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 5: Feature Selection and Imbalanced Classes

How to teach the fifth session of Machine Learning Foundations: feature selection (filters, wrappers, L1, and the tolerance rule that cuts Pinecrest's churn model from 16 inputs to 6) and imbalanced classes (why accuracy fails on 0.9% fraud, what class weights and samplers really do, and how to set a threshold from the price of each mistake). Both lessons share one warning: a step that looks at the labels must happen inside each training fold, or cross-validation reports 80% on pure noise and AP 0.91 on fiction.

`2 lessons` `3 hrs session` `Lessons 2.4–2.5` `Session 5 of 11`

HOW TO USE THIS GUIDE

- Same rhythm for both lessons: frame (the goal and one mental model) → live demo (you type, they type along) → guided lab (a kit lab, new practice on the lesson's skills) → start the lesson project. Exercises, the quiz and the journal are homework.
- Both lessons are about the same trap in two forms: a step that looks at the labels (choosing columns, rebalancing) must happen inside each training fold, or cross-validation lies. Say it at 0:10 and at 1:35.
- Type, do not paste, except where a step says "start the cell, then talk": forward selection and the samplers take 10 to 30 seconds on a laptop. Every code block below was run to produce the output printed under it (Python 3.12, pandas 3.0, scikit-learn 1.9, imbalanced-learn, 4 CPU threads).
- Keep `dimensionality_feature_selection.html` and `imbalanced_learning.html` open in browser tabs for the charts you show but do not type: the RFECV curve, the forward-selection path, the side-by-side sampler table and the cost curve. Labs live in `labs/module-05/`: `_starter.py` files are handouts, `_solution.py` files are yours.
- Session 6 opens with Checkpoint 1 on Modules 1 and 2. Mention it in the warm-up and again at the wrap-up so nobody is surprised.

> **THE #1 RISK OF SESSION 5: NUMBERS THAT ARE TOO GOOD**
>
> Both lessons have a moment where a wrong method produces an impressive number: about 80% accuracy on pure noise, AP 0.91 from SMOTE before cross-validation. Learners who miss the point leave thinking selection or SMOTE "improves the model". Protect the noise experiment (2.4 Step 9) and the SMOTE leak (2.5 Step 11) even if you cut elsewhere, and in both labs ask the same question: "Which number would you have reported if you had not known the truth?"

BEFORE THE SESSION

- ☐ **Run the kit check:** `python3 labs/check_labs.py module-05` from this kit's folder. Expect `8 passed, 0 failed` (four lab pairs; `debug_imbalance.py` is not run) in under a minute.
- ☐ **Copy the starters only** to the shared folder: `labs/module-05/l2_4-feature-selection/*_starter.py`, `labs/module-05/l2_5-imbalanced-classes/*_starter.py` and `debug_imbalance.py`. Keep every `*_solution.py` on your machine.
- ☐ **Downloads:** none. imbalanced-learn (`import imblearn`) is in the course requirements and `check_environment.py` already checks it.
- ☐ **Projector machine:** `(.venv)` active in your `pinecrest-ml`, JupyterLab started from that folder, two empty notebooks `lesson_2_4` and `lesson_2_5`, font 18 pt or more. Run demo Steps 6 to 7 of 2.4 and Step 7 of 2.5 once beforehand to know how long they take on your machine.
- ☐ **Board:** three habits for 2.4 (decide with CV · selection inside the pipeline · smallest set within a tolerance); for 2.5 the smoke detector, RANK | DECIDE, and `cost = $10 × flagged + $375 × missed`.
- ☐ **Print or share:** the workbook pages for Session 5 and sticky notes for the exit ticket.

KNOWN ENVIRONMENT TRAPS

SYMPTOM	CAUSE	FIX
`ModuleNotFoundError: No module named 'imblearn'`	imbalanced-learn missing from this environment, or the notebook kernel is another Python	`python -m pip install imbalanced-learn` in the activated environment (install name and import name differ, like scikit-learn / sklearn); check the kernel.
`ValueError: Solver lbfgs supports only 'l2' or None penalties, got l1 penalty.`	`l1_ratio=1` with the default solver	Add `solver="liblinear"` (or `"saga"`), as in the lesson.
`ImportError: cannot import name 'TunedThresholdClassifierCV'`	scikit-learn older than 1.5	The kernel is not the course environment: compare `sklearn.__version__` with 1.9.
Forward selection or the sampler loop seems to hang	Hundreds of fits; old laptops take 30 to 60 s	Wait; in the lab, `lean_fraud` Step 4 needs about 15 s on a modern laptop.
`FutureWarning: 'penalty' was deprecated`	Copying `penalty="l1"` from older tutorials	Use `l1_ratio=1` with `solver="liblinear"`.

SESSION 5 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Five retrieval questions on Lessons 2.2 and 2.3; homework check (the saved 2.2 pipeline and the 2.3 demand scoreboard).
0:10	10	**2.4** Frame and mental model	The ultralight backpack; filters, wrappers and embedded methods; the three habits: decide with CV, keep selection in the pipeline, prefer the smaller set.
0:20	30	**2.4** Live demo	Redundant columns by hand, F against mutual information, RFECV's plateau, forward selection, the L1 sweep, the noise experiment.
0:50	20	**2.4** Guided lab	`labs/module-05/l2_4-feature-selection/lean_fraud_starter.py` (everyone), then `labs/module-05/l2_4-feature-selection/vendor_columns_starter.py` in pairs.
1:10	15	**2.4** Start the lesson project	The lean churn model, Steps 1 to 3; Steps 4 to 6 go home.
1:25	10	Break	Leave the three habits and the smoke detector on the board.
1:35	10	**2.5** Frame and mental model	The smoke detector; ranking against deciding; the cost of each kind of mistake.
1:45	30	**2.5** Live demo	The accuracy paradox, AP against its baseline, class weights and samplers that only move the cut, cost by threshold, the formula, TunedThresholdClassifierCV, the SMOTE leak.
2:15	20	**2.5** Guided lab	`labs/module-05/l2_5-imbalanced-classes/retention_threshold_starter.py` (everyone), then `labs/module-05/l2_5-imbalanced-classes/rebalance_tree_starter.py` in pairs.
2:35	15	**2.5** Start the lesson project	The June fraud review queue, Steps 1 to 3; Steps 4 to 6 go home.
2:50	10	Wrap-up	Exit ticket (3 questions), homework, bring list for Session 6 (Checkpoint 1).
Total	180		Ends at 3:00.

In the room. The lesson pages estimate 100 + 100 minutes for a solo learner; after the warm-up, the break and the wrap-up the room has 150. So the room does, for each lesson, the mental model, the live demo, one guided kit lab and the first three steps of the lesson project.

Goes home (2 to 3 hours). The lean churn model Steps 4 to 6 (forward selection by hand, the tolerance rule, one test and `lean_features.txt`) and the June review queue Steps 4 to 6 (tune the threshold, run June once, save the queue); each lesson's Hands-on Exercises 1 to 3 (4 is a stretch); both Quick Quizzes; both journal entries; any lab part not finished; and a first look back over Modules 1 and 2 for Checkpoint 1.

Protect, in this order: the noise experiment (2.4 Step 9) and the SMOTE leak (2.5 Step 11); RANK | DECIDE with the cost table and the tuned threshold (2.5 Steps 4, 8 to 10); RFECV's plateau and the tolerance rule (2.4 Step 6). **Cut first:** 2.4 Steps 4 and 5 (filters: say one sentence each and show the table on the page), then 2.5 Step 7 (samplers: show the lesson's table), then Lab B of each lesson (to homework).

OPENING SEGMENT

0:00 Warm-up: Pipelines and Features — 10 min

TIMING

6m	Five retrieval questions (workbook, Session 5 warm-up box), answered alone, then compared with a neighbor
4m	Reveal and discuss; homework check: who has `churn_pipeline.joblib` and `demand_scoreboard.csv`?

QUESTIONS AND ANSWERS

- **W1.** A pipeline is `make_pipeline(SimpleImputer(), StandardScaler(), LogisticRegression())`. What does `pipe.predict(X_new)` do to each of the three steps? **Answer:** The imputer and the scaler only **transform** the new rows, with the medians, means and standard deviations they learned during `fit`; the model then **predicts**. Nothing is refitted. *If someone says "it refits the scaler", ask what would happen with one new customer: a scaler fitted on one row.*
- **W2.** In Lesson 2.2 the signup-date churn rate scored CV ROC AUC 0.930 when computed with pandas, and 0.586 as a `TargetEncoder` inside the pipeline. Which is honest, and why was the other so high? **Answer:** 0.586. The pandas rates used every training row's label, including the rows that later sat in each validation fold; with one or two customers per date, "the rate on your date" was mostly your own label. *This is today's selection leak in another costume: any step that looks at `y` belongs inside the folds.*
- **W3.** What does this print? (code on the slide) **Answer:** `['logisticregression__C']` *`make_pipeline` names each step after its class in lowercase; `step__parameter` reaches any setting. Today you swap a whole step the same way: `set_params(logisticregression=...)`.*
- **W4.** `add_churn_features` builds `tickets_no_app = support_tickets_12m * (1 - app_user)`. What is it for an app user with 3 tickets, and why build it? **Answer:** 0. Tickets raise churn for customers without the app but not for app users; a linear model cannot express that from the two raw columns, so the product gives non-app tickets their own weight. *Keep this pair in mind: in today's demo forward selection keeps both raw columns because they matter together.*
- **W5.** Why can `FunctionTransformer(add_churn_features)` be fitted outside the folds without leaking, while `TargetEncoder` cannot? **Answer:** The feature function is stateless: its thresholds are fixed numbers and it learns nothing from the rows it sees. `TargetEncoder` learns rates from `y`, so fitting it on validation rows hands them their own answers. *Feature selection and resampling, today's two subjects, are both "learn from y" steps.*

READING THE ROOM

- **W2 and W5 wrong** means the room does not yet feel why some steps must live inside the folds. Spend an extra minute on 2.4 Step 9 and 2.5 Step 11: both are that idea again.
- **W3 wrong** is harmless: today you will use `set_params(logisticregression=...)` and `pipe[-1]`, and the demo narrates them.
- **No saved pipeline or scoreboard?** Not needed today; both projects start from the data file. They do need their Lesson 2.3 notebook later (Exercise 4 of 2.4 reuses `add_churn_features`).
- **The redundant-column guesses** (Session 4 homework): take three or four and write them on the board. Do not judge them now: 2.4 demo Steps 2 and 3 check them (no pair is a copy by correlation, yet `spend_12m`, `orders_12m` and `avg_order_value` rebuild one another). Tick the guesses that survive.
- **The Module 1 model rewritten as a pipeline** (also Session 4 homework): one hand up is enough; ask that learner which steps went inside and why. Anyone who skipped it gets the same practice in today's labs.

2.4 Feature Selection

75 min

OBJECTIVES (FROM THE LESSON PAGE)

- Explain what feature selection buys you (simpler, cheaper, sturdier models) and what it usually does not (big accuracy gains)
- Find redundant features with correlations, exact relationships and cross-tabulations
- Use filter (`SelectKBest`), wrapper (`RFECV`, `SequentialFeatureSelector`) and embedded (L1, `SelectFromModel`) methods, and say what each can miss
- Keep selection inside a `Pipeline` so cross-validation stays honest, and show what happens when it is not
- Choose the smallest feature set whose cross-validated score is within a tolerance of the best

TIMING IN THE ROOM

10m	Frame: the ultralight backpack; three families of methods and their blind spots; the three habits
30m	Live demo: redundancy by hand, two filters, RFECV, forward selection, the L1 sweep, the noise experiment
20m	Guided lab: `lean_fraud_starter.py` (everyone), then `vendor_columns_starter.py` in pairs
15m	Start the lesson project: the lean churn model, Steps 1 to 3

MENTAL MODEL TO SAY

"Experienced hikers weigh every item before a long trip. Two stoves? One goes home: a **redundant** feature. The camp chair you never sit in? Home: an **irrelevant** feature. But the water filter stays, even though you use it twice a day. The goal is not the lightest pack; it is the lightest pack that still gets you to the end of the trail. In model terms: the smallest feature set that keeps the score."

Then the honest promise, in one breath: "Selection rarely makes a good model much more accurate. It makes it simpler to collect, explain and monitor. On Pinecrest's churn table it will cut 16 inputs to 6 at no cost." Draw three columns on the board, **Filter · Wrapper · Embedded**, with one line each: scores columns alone (fast, blind to the model and to teamwork) · tries subsets with the real model (slow, sees teamwork) · selects while training (one fit, tied to one model type).

Close with the three habits, written under the columns and left there: **decide with cross-validation · keep selection inside the pipeline · prefer the smallest set within a tolerance of the best.**

LIVE DEMO (YOU TYPE, THEY FOLLOW)

New notebook `lesson_2_4`. Ask for a prediction before every number. The two moments that matter most are Step 6 (the plateau: read the curve, not the winner) and Step 9 (80% on pure noise).

1. **Load the churn table and the standard split (2m).** "Same customers, same split as every churn lesson, but today all 11 numeric columns, including `email_opens_90d` with its gaps. Every pipeline imputes it."

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split

customers = pd.read_csv("data/customers_churn.csv")
numeric = ["tenure_months", "orders_12m", "spend_12m", "avg_order_value", "day
           "returns_12m", "return_rate", "support_tickets_12m", "discount_shar
train, test = train_test_split(customers, test_size=0.25, stratify=customers["
X_train, y_train = train[numeric], train["churned"]
X_test, y_test = test[numeric], test["churned"]
print(X_train.shape, X_test.shape)
```

```
(4500, 11) (1500, 11)
```

2. **Which columns move together? (3m).** "Cheapest check first: correlations, each pair once."

```
corr = X_train.corr()
upper = corr.where(np.triu(np.ones(corr.shape, dtype=bool), k=1)).stack()
print(upper[upper.abs() > 0.3].sort_values(key=abs, ascending=False).round(2).
```

```
returns_12m  return_rate              0.68
orders_12m   spend_12m                0.57
spend_12m    avg_order_value          0.49
orders_12m   days_since_last_order   -0.41
dtype: float64
```

Ask: "Is any pair a copy?" (No: nothing near 1.) "So no redundancy?" Take answers, then Step 3.

3. **A relationship correlation cannot see (2m).** "Lesson 2.3 found this one: spend is orders times average order value."

```
rebuilt = X_train["orders_12m"] * X_train["avg_order_value"]
print("largest gap between spend and orders x AOV:", (X_train["spend_12m"] - r
```

```
largest gap between spend and orders x AOV: 0.09
```

Nine cents of rounding. Any one of the three can be rebuilt from the other two. Say: "The fastest selection method is asking how a column was made." Mention that `region` repeats `home_store` for store shoppers and `loyalty_tier` is assigned from spend and tenure (lesson page, no need to type).

4 **Two filters disagree (3m).** "A filter scores each column alone against the target. F compares the churners' mean with the stayers'; mutual information sees any kind of dependence."

```
from sklearn.feature_selection import f_classif, mutual_info_classif
from sklearn.impute import SimpleImputer

X_filled = SimpleImputer(strategy="median").set_output(transform="pandas").fit
f_scores, _ = f_classif(X_filled, y_train)
mi_scores = mutual_info_classif(X_filled, y_train, random_state=42)
filters = pd.DataFrame({"F": f_scores, "MI": mi_scores}, index=numeric)
filters["F_rank"] = filters["F"].rank(ascending=False).astype(int)
filters["MI_rank"] = filters["MI"].rank(ascending=False).astype(int)
print(filters.sort_values("F_rank").round(4))
```

```
                            F      MI  F_rank  MI_rank
days_since_last_order  735.4249  0.0641       1        1
discount_share         196.5015  0.0118       2        6
spend_12m              174.5598  0.0190       3        4
orders_12m             156.1053  0.0206       4        3
app_user                64.2232  0.0145       5        5
support_tickets_12m     41.8948  0.0000       6       10
email_opens_90d         37.1523  0.0000       7       10
returns_12m             15.5411  0.0064       8        8
tenure_months           14.9306  0.0329       9        2
avg_order_value          8.5355  0.0106      10        7
return_rate              1.1013  0.0000      11       10
```

Point at `tenure_months`: rank 9 for F, rank 2 for MI. Ask: "Which lesson showed why?" (2.3: the U-shape; the means are close.) Then `support_tickets_12m`: MI rounds it to 0 although it matters, for non-app customers.

5 **A filter inside the pipeline (3m).** "`SelectKBest` keeps the k best; in a pipeline it is refitted on every training fold. Predict: does picking the best six beat using all eleven?"

```
from functools import partial
from sklearn.feature_selection import SelectKBest
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
mi = partial(mutual_info_classif, random_state=42)

def churn_pipe(*steps):
    return make_pipeline(SimpleImputer(strategy="median"), StandardScaler(), *

for name, pipe in [("all 11 columns", churn_pipe()),
                   ("best 6 by F", churn_pipe(SelectKBest(f_classif, k=6))),
                   ("best 6 by MI", churn_pipe(SelectKBest(mi, k=6)))]:
    scores = cross_val_score(pipe, X_train, y_train, cv=cv, scoring="roc_auc")
    print(f"{name:15s} CV ROC AUC {scores.mean():.3f}")
```

```
all 11 columns  CV ROC AUC 0.833
best 6 by F     CV ROC AUC 0.824
best 6 by MI    CV ROC AUC 0.813
```

Both lose: F by 0.009, MI by 0.020. MI keeps tenure, whose U-shape a logistic regression cannot use. "A filter knows nothing about the model that comes after it."

6 **RFECV and its plateau (4m).** "Recursive elimination drops the column with the smallest weight, refits, repeats; RFECV scores every count with cross-validation."

```
from sklearn.feature_selection import RFECV

rfecv = RFECV(LogisticRegression(), cv=cv, scoring="roc_auc")
rfe_pipe = make_pipeline(SimpleImputer(strategy="median"), StandardScaler(), r
rfe_pipe.fit(X_train, y_train)
curve = pd.Series(rfecv.cv_results_["mean_test_score"], index=rfecv.cv_results
print("RFECV keeps", rfecv.n_features_, "columns")
print(curve.round(4).to_dict())
```

```
RFECV keeps 11 columns
{1: 0.7215, 2: 0.8063, 3: 0.8116, 4: 0.8226, 5: 0.8263, 6: 0.8328, 7: 0.8328,
```

It keeps all 11, won by a ten-thousandth. Show the lesson's curve chart. Say: "Read the curve, not `n_features_`. From six on it is flat, and the folds disagree by far more than these differences." Write the rule on the board: smallest set within a tolerance of the best.

7 **Forward selection finds teamwork (4m).** "A wrapper adds, one at a time, the column that raises the cross-validated score most. `tol=0.002` stops it when the next column adds less." Start this cell, then talk while it runs (about 10 to 20 seconds on a laptop).

```
from sklearn.feature_selection import SequentialFeatureSelector

sfs = SequentialFeatureSelector(LogisticRegression(), n_features_to_select="au
                                direction="forward", cv=cv, scoring="roc_auc")
sfs_pipe = make_pipeline(SimpleImputer(strategy="median"), StandardScaler(), s
sfs_pipe.fit(X_train, y_train)
forward_six = list(sfs.get_feature_names_out())
print(forward_six)
scores = cross_val_score(churn_pipe(SequentialFeatureSelector(LogisticRegressi
                                                              tol=0.002, cv=cv
                         X_train, y_train, cv=cv, scoring="roc_auc")
print(f"forward selection inside CV: {scores.mean():.3f}")
```

```
['orders_12m', 'days_since_last_order', 'support_tickets_12m', 'discount_share
forward selection inside CV: 0.832
```

Six columns, the full model's score. It kept `support_tickets_12m` AND `app_user`, the pair that matters together (warm-up W4). Point at the second number: it cross-validates the whole selection procedure, the honest estimate.

8 **L1: selection while training (4m).** "An L1 penalty charges for the total size of the weights; the cheapest way to pay is zeros. Smaller C, heavier penalty, more zeros. `liblinear` because the default solver refuses L1."

```
rows, zeroed = [], {}
for C in [0.005, 0.01, 0.02, 0.05, 1.0]:
    l1_pipe = churn_pipe().set_params(logisticregression=LogisticRegression(l1
    auc = cross_val_score(l1_pipe, X_train, y_train, cv=cv, scoring="roc_auc")
    weights = l1_pipe.fit(X_train, y_train)[-1].coef_[0]
    zeroed[C] = [col for col, w in zip(numeric, weights) if w == 0]
    rows.append({"C": C, "kept": int((weights != 0).sum()), "cv_auc": round(au
print(pd.DataFrame(rows).to_string(index=False))
print("zero at C=0.05:", zeroed[0.05])
```

```
     C  kept  cv_auc
0.005     2  0.7955
0.010     6  0.8175
0.020     8  0.8301
0.050     8  0.8333
1.000    11  0.8334
zero at C=0.05: ['avg_order_value', 'returns_12m', 'return_rate']
```

Read from the bottom: at C = 0.05 three columns drop with no loss, and they are the redundant ones from Steps 2 and 3. At C = 0.01 useful columns start to go and the score falls.

9 **The most important experiment: pure noise (4m).** "200 rows, 5,000 random columns, a random target. Honest accuracy is a coin flip. Watch what selecting first, then cross-validating, reports."

```
def noise_experiment(seed):
    rng = np.random.default_rng(seed)
    X_noise = rng.normal(size=(200, 5000))
    y_noise = rng.integers(0, 2, size=200)
    chosen = SelectKBest(f_classif, k=20).fit(X_noise, y_noise)
    leaky = cross_val_score(LogisticRegression(), chosen.transform(X_noise), y
    honest = cross_val_score(make_pipeline(SelectKBest(f_classif, k=20), Logis
                             X_noise, y_noise, cv=cv).mean()
    return {"seed": seed, "select first, then CV": leaky, "select inside CV":

noise = pd.DataFrame([noise_experiment(seed) for seed in range(42, 47)]).set_i
print(noise.round(3))
print(noise.mean().round(3).to_string())
```

```
      select first, then CV  select inside CV
seed
42                    0.825             0.595
43                    0.780             0.545
44                    0.820             0.515
45                    0.770             0.520
46                    0.795             0.460
select first, then CV    0.798
select inside CV         0.527
```

About 80% on pure noise, five times out of five, against 53% when selection is a pipeline step. Say: "Same data, same model, one line of difference. Any choice made with the labels is part of the model."

10 **One silent mistake: L1 without a scaler (1m).** "No error, and that is the problem."

```
l1 = LogisticRegression(l1_ratio=1, solver="liblinear", C=0.02)
for name, pipe in [("scaled", make_pipeline(SimpleImputer(strategy="median"),
                   ("unscaled", make_pipeline(SimpleImputer(strategy="median")
    weights = pipe.fit(X_train, y_train)[-1].coef_[0]
    print(f"{name:9s} keeps discount_share: {weights[numeric.index('discount_s
          f"keeps avg_order_value: {weights[numeric.index('avg_order_value')]
```

```
scaled    keeps discount_share: True, keeps avg_order_value: False
unscaled  keeps discount_share: False, keeps avg_order_value: True
```

Unscaled, L1 drops `discount_share` (0 to 1, needs a big weight) and keeps redundant `avg_order_value` (dollars, tiny weight). It selects by units.

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Selecting before the split or outside the pipeline.** Symptom: a CV score that rises when they select (often with `SelectKBest(...).fit_transform(X_train, y_train)` and then `cross_val_score` on the result). Coach: "Which rows did the selector see? Did the validation folds help choose the columns?" Point at the noise experiment's two numbers.
- **L1 or RFE on unscaled columns, or the default solver.** Symptoms: `ValueError: Solver lbfgs supports only 'l2' or None penalties, got l1 penalty.`, or an L1 model that keeps `avg_order_value` and drops `discount_share`. Coach: "What does the penalty charge for? Is a weight of 3 on a 0-to-1 column the same as 3 on dollars?"
- **Reading the winner instead of the curve, and "dropped" as "irrelevant".** Symptoms: "RFECV says keep all 11", or "L1 shows average order value doesn't matter". Coach: "How far apart are the scores on the plateau, and how far apart are the folds?" and "Dropped given what else is in the model?"

GUIDED LAB

- **Lab A (12m, everyone):** `labs/module-05/l2_4-feature-selection/lean_fraud_starter.py`; reference `labs/module-05/l2_4-feature-selection/lean_fraud_solution.py`. The lesson's tools on fraud: nine raw columns plus five engineered ones (night, night on a new device, gift cards, log of the amount, brand-new accounts), judged by cross-validated average precision because fraud is rare. TODO 1 to 4 in the room; TODO 5 (tolerance rule and test) can finish at home.
- **Expected, Lab A:** `amount` and `log_amount` correlate 0.88 (the only pair above 0.5); the F-test ranks `night_new_device` first (4,344); CV AP 0.5512 ± 0.0618 with all 14, 0.5202 with the raw 9; forward selection adds `velocity_24h`, `night_new_device`, `log_amount`, `card_attempts`, ...; within 0.01 of the best: 8 columns; test AP raw 9 0.444, all 14 0.505, lean 8 0.505.
- **Circulate in Lab A for:** forward selection judged on the test set ("which rows chose the columns?"); a tolerance smaller than the fold spread treated as meaningful; surprise that the F-test's winner is not forward selection's first pick (`velocity_24h` alone ranks better; the interaction shines next to it).

Debrief: "Engineering moved AP from 0.444 to 0.505; selection then dropped six columns for free. Create generously, keep what earns its place."

- **Lab B (8m, pairs):** `labs/module-05/l2_4-feature-selection/vendor_columns_starter.py`; reference `labs/module-05/l2_4-feature-selection/vendor_columns_solution.py`. A data vendor's 1,000 extra columns (simulated as pure noise) on a 300-customer pilot. One types, one predicts each number.
- **Expected, Lab B:** CV AUC 0.745 with the 10 real columns, 0.603 with all 1,010; select the best 10 on all rows, then CV: 0.825, with only 3 real columns among the picks; select inside CV: 0.660; on the 1,500 test customers the real 10 score 0.794 and the vendor's picks 0.609.
- **Debrief Lab B (1m):** "The leaky number beat the honest model built from real columns, and only 3 of its 10 columns were real. Which pilot number would the vendor put in the brochure?"

START THE LESSON PROJECT

The lesson project is **the lean churn model**: the smallest set of the 16 candidate columns whose CV ROC AUC is within 0.005 of the best, confirmed once on the test set. In the room, Steps 1 to 3 (15 minutes).

- Read the brief from the lesson page: every input column is a data feed someone must maintain. Step 1 together: `16 candidate columns; {'email_opens_90d': 419, 'age_group': 359}`.
- Step 2 (`build_model` for any list of columns, numeric and categorical) is the step that matters: walk the room. **Done** = `all 16 columns: CV ROC AUC 0.8324`.
- Step 3 on their own: dropping the four derived columns gives `0.8305`.
- **Goes home:** Step 4 (forward selection by hand, about a minute to run, starting with `days_since_last_order`), Step 5 (tolerance rule: 6 columns) and Step 6 (test ROC AUC 0.826 full against 0.824 lean; `lean_features.txt`).

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab:

- Why can't correlation find that spend is orders times average order value? **Answer:** correlation looks at two columns and straight lines; this is a product of two columns.
- RFECV's best count is 11, but 6 scores 0.8328 against 0.8333. What do you keep? **Answer:** 6: the gap is far smaller than the spread between folds; prefer the smaller set within a tolerance.
- Why does L1 need a scaler? **Answer:** it charges for weight size, and a column's unit decides how big its weight must be.
- Name one step besides selection that must live inside the folds. **Answer:** anything that looks at `y`: target encoding (2.2), resampling (2.5), tuning (6.2).

Workbook self-check, items Q1 to Q6 (learners do them at the end of the block or at home).

Answers: **Q1** C) A filter scores each column on its own; the wrapper judges each column next to the ones already chosen, so it sees that the two matter together; **Q2** `['score']`; **Q3** B) Put the selector inside a pipeline and cross-validate the whole pipeline, expecting a lower number; **Q4** D) `discount_share` runs from 0 to 1, so it needs a large weight, and the penalty charges for the size of each weight; **Q5** A) 6; **Q6** B) Given spend and orders, average order value adds nothing new to this model. Full set with explanations in the Answer Key.

◆ DIFFERENTIATION

Support: give Lab A's TODO 1 and 2 to the whole room on the projector, then let pairs do TODO 3 and 4 with `cv_ap` already written. For anyone lost in method names, the board's three columns are the map: ask "which column is this method in?" before each demo step.

Stretch: the lesson's Exercise 2 (backward selection reaches the same six), then the Bonus Challenge (how stable is the lean set across CV shuffles and half the data?). In Lab A, ask them to try `SelectFromModel` with the L1 model at a few values of `C` and compare with their forward path.

2.5 Imbalanced Classes

75 min

OBJECTIVES (FROM THE LESSON PAGE)

- Explain why accuracy fails on rare classes and judge a model with recall, precision and average precision against the right baseline
- Apply `class_weight="balanced"` and the imbalanced-learn samplers (random under- and oversampling, SMOTE) inside a leak-proof pipeline
- Show that weights and resampling mostly move the decision threshold, and explain what they do to predicted probabilities
- Choose a threshold from the cost of each kind of mistake, by formula and with `TunedThresholdClassifierCV`
- Check who carries the burden of false alarms before a model goes live

TIMING IN THE ROOM

10m	Frame: the smoke detector; ranking against deciding; mistakes have prices
30m	Live demo: the accuracy paradox, AP, weights and samplers, cost by threshold, the formula, the tuned threshold, the SMOTE leak
20m	Guided lab: `retention_threshold_starter.py` (everyone), then `rebalance_tree_starter.py` in pairs
15m	Start the lesson project: the June review queue, Steps 1 to 3

MENTAL MODEL TO SAY

"A smoke detector that never beeps is right 364 days a year, and worthless. One that beeps every time you make toast catches every fire, and soon gets its battery pulled. A useful detector sits between the two, and where exactly depends on what a false alarm costs compared with a missed fire."

Then split the board in two: **RANK** (does the model put frauds near the top? average precision against the base rate) and **DECIDE** (where do we cut? cost, capacity or a recall target). "Today's big surprise: class weights and resampling, the fixes every tutorial reaches for, mostly change DECIDE, not RANK, and you can change DECIDE directly by moving the threshold." Write `cost = $10 × flagged + $375 × missed` under DECIDE.

LIVE DEMO (YOU TYPE, THEY FOLLOW)

New notebook `lesson_2_5`. The two predictions that matter: "what does 'all legitimate' score?" (Step 2) and "what AP does SMOTE-before-CV report?" (Step 11).

1 **How rare is fraud? (1m).** "Six months of online payments."

```
import numpy as np
import pandas as pd

tx = pd.read_csv("data/transactions.csv")
print(tx.shape)
print(tx["is_fraud"].value_counts().to_dict())
print(f"fraud rate: {tx['is_fraud'].mean():.3%}")
```

```
(50000, 13)
{0: 49562, 1: 438}
fraud rate: 0.876%
```

Fewer than one in a hundred. Write 0.876% on the board.

2 **The accuracy paradox (3m).** "Stratified split, because with 438 frauds a random split could leave the test set short. Predict: what does 'all legitimate' score?"

```
from sklearn.model_selection import train_test_split
from sklearn.dummy import DummyClassifier
from sklearn.metrics import recall_score

num_cols = ["account_age_days", "amount", "hour", "items", "device_new",
            "billing_shipping_mismatch", "country_mismatch", "card_attempts",
cat_cols = ["category"]
X = tx[num_cols + cat_cols]
y = tx["is_fraud"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, strat
print(f"train: {len(y_train):,} rows, {y_train.sum()} frauds")
print(f"test : {len(y_test):,} rows, {y_test.sum()} frauds")
dummy = DummyClassifier(strategy="most_frequent").fit(X_train, y_train)
print(f"'all legitimate' accuracy: {dummy.score(X_test, y_test):.4f}")
print(f"frauds caught: {recall_score(y_test, dummy.predict(X_test)):.0%}")
```

```
train: 35,000 rows, 307 frauds
test : 15,000 rows, 131 frauds
'all legitimate' accuracy: 0.9913
frauds caught: 0%
```

99.1% accurate, catches nothing. Draw the smoke detector that never beeps.

3 **Judge the ranking: AP against its baseline (3m).** "A pipeline from Lesson 2.2, built by a function so no two pipelines share one fitted ColumnTransformer. Average precision's baseline is the fraud rate, not

0.5."

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.pipeline import make_pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score, average_precision_score

def make_preprocess():
    return ColumnTransformer([
        ("num", StandardScaler(), num_cols),
        ("cat", OneHotEncoder(handle_unknown="ignore"), cat_cols),
    ])

plain = make_pipeline(make_preprocess(), LogisticRegression())
plain.fit(X_train, y_train)
p_plain = plain.predict_proba(X_test)[:, 1]
print(f"ROC AUC {roc_auc_score(y_test, p_plain):.3f}   average precision {aver
print(f"average precision of a random ranking: {y_test.mean():.4f}")
```

```
ROC AUC 0.843   average precision 0.463
average precision of a random ranking: 0.0087
```

0.46 against 0.009: a good ranking.

4 **Then the decisions at 0.5 (2m).** "A helper that prints what a fraud team cares about."

```
from sklearn.metrics import confusion_matrix

def report(name, y_true, y_pred):
    tn, fp, fn, tp = confusion_matrix(y_true, y_pred, labels=[0, 1]).ravel()
    print(f"{name:16s} flagged {tp + fp:5d}  caught {tp:3d}  missed {fn:3d}  "
          f"recall {tp / (tp + fn):.2f}  precision {tp / max(tp + fp, 1):.2f}"

report("plain @ 0.5", y_test, plain.predict(X_test))
```

```
plain @ 0.5      flagged    44  caught  41  missed  90  recall 0.31  precisio
```

44 flags, 41 real, 90 frauds missed. "The ranking is good; the decision rule is timid." Write two words on the board: RANK | DECIDE.

5 **Class weights (3m).** "Each fraud mistake now counts about 57 times."

```
from sklearn.utils.class_weight import compute_class_weight

print(compute_class_weight("balanced", classes=np.array([0, 1]), y=y_train).ro
weighted = make_pipeline(make_preprocess(), LogisticRegression(class_weight="b
weighted.fit(X_train, y_train)
p_weighted = weighted.predict_proba(X_test)[:, 1]
print(f"ROC AUC {roc_auc_score(y_test, p_weighted):.3f}   average precision {a
report("balanced @ 0.5", y_test, weighted.predict(X_test))
```

```
[ 0.5 57. ]
ROC AUC 0.848   average precision 0.438
balanced @ 0.5   flagged  1863  caught  90  missed  41  recall 0.69  precisio
```

Recall 0.69, but 1,863 flags for 90 frauds, and AUC and AP barely moved (AP even fell). Ask: "Did it learn fraud better, or say fraud more readily?"

6 **What the weights did to the probabilities (2m).** "Compare the average probability with the real rate."

```
from scipy.stats import spearmanr

print(f"real fraud rate          : {y_test.mean():.4f}")
print(f"mean probability, plain  : {p_plain.mean():.4f}")
print(f"mean probability, weighted: {p_weighted.mean():.4f}")
print(f"rank correlation of the two scores: {spearmanr(p_plain, p_weighted).st
```

```
real fraud rate          : 0.0087
mean probability, plain  : 0.0085
mean probability, weighted: 0.2671
rank correlation of the two scores: 0.961
```

The plain model is calibrated (0.0085 against 0.0087); the weighted one averages 0.27. Same order (rank correlation 0.96), probabilities of nothing real. "Hold that thought for Step 8."

7 **Three samplers, inside an imblearn pipeline (4m).** "Samplers change the training rows. imblearn's pipeline runs them during fit and skips them during predict, so test rows keep the real balance." Start the cell, then talk through the three samplers while it runs.

```
from imblearn.under_sampling import RandomUnderSampler
from imblearn.over_sampling import RandomOverSampler, SMOTE
from imblearn.pipeline import make_pipeline as make_imb_pipeline

samplers = {
    "undersample": RandomUnderSampler(random_state=42),
    "oversample": RandomOverSampler(random_state=42),
    "SMOTE": SMOTE(random_state=42),
}
for name, sampler in samplers.items():
    pipe = make_imb_pipeline(make_preprocess(), sampler, LogisticRegression())
    pipe.fit(X_train, y_train)
    proba = pipe.predict_proba(X_test)[:, 1]
    print(f"{name:12s} ROC AUC {roc_auc_score(y_test, proba):.3f}   AP {averag
    report(f"{name} @ 0.5", y_test, pipe.predict(X_test))
```

```
undersample  ROC AUC 0.843   AP 0.419
undersample @ 0.5 flagged  2172  caught  93  missed  38  recall 0.71  precisi
oversample   ROC AUC 0.848   AP 0.437
oversample @ 0.5 flagged  1866  caught  90  missed  41  recall 0.69  precisio
SMOTE        ROC AUC 0.845   AP 0.428
SMOTE @ 0.5      flagged  1940  caught  89  missed  42  recall 0.68  precisio
```

Show the lesson's side-by-side table: four fixes, the same place: about 1,900 flags, recall near 0.7, precision near 0.05, ranking no better. "For a linear model they shift scores across 0.5. You can do that directly."

8 **Price the mistakes (3m).** "A review costs $10 of analyst time; a missed fraud costs about $375. Cost = $10 per flag plus $375 per miss."

```
REVIEW_COST, FRAUD_LOSS = 10, 375

def fraud_cost(y_true, y_pred):
    tn, fp, fn, tp = confusion_matrix(y_true, y_pred, labels=[0, 1]).ravel()
    return int(REVIEW_COST * (tp + fp) + FRAUD_LOSS * fn)

print(f"review nothing   : ${fraud_cost(y_test, np.zeros(len(y_test), dtype=in
print(f"review everything: ${fraud_cost(y_test, np.ones(len(y_test), dtype=int
for threshold in [0.5, 0.2, 0.1, 0.05, 0.02, 0.01]:
    pred = (p_plain >= threshold).astype(int)
    print(f"threshold {threshold:<5} cost ${fraud_cost(y_test, pred):,}")
```

```
review nothing   : $49,125
review everything: $150,000
threshold 0.5   cost $34,190
threshold 0.2   cost $30,100
threshold 0.1   cost $27,455
threshold 0.05  cost $25,395
threshold 0.02  cost $27,250
threshold 0.01  cost $33,230
```

Both do-nothing options are expensive, so is 0.5. Lowering the cut to 0.05 saves almost $9,000 with the same model.

9 **The formula, and why not to pick on the test set (3m).** "Flag when p × $375 > $10: p > 10/375. That works because the plain model's probabilities are honest."

```
print(f"formula threshold: {REVIEW_COST} / {FRAUD_LOSS} = {REVIEW_COST / FRAUD
thresholds = np.geomspace(0.003, 0.9, 80)
costs = [fraud_cost(y_test, (p_plain >= t).astype(int)) for t in thresholds]
best = thresholds[int(np.argmin(costs))]
print(f"cheapest threshold on the test set: {best:.3f}  (cost ${min(costs):,})
```

```
formula threshold: 10 / 375 = 0.027
cheapest threshold on the test set: 0.043  (cost $25,025)
```

Show the lesson's cost-curve chart (log scale). Then: "0.043 was found by peeking at the test set: fine for a picture, wrong for a choice."

10 **TunedThresholdClassifierCV (3m).** "Cross-validation tries many thresholds on the training folds, keeps the cheapest, then refits."

```
from sklearn.metrics import make_scorer
from sklearn.model_selection import TunedThresholdClassifierCV

cost_scorer = make_scorer(fraud_cost, greater_is_better=False)
tuned = TunedThresholdClassifierCV(make_pipeline(make_preprocess(), LogisticRe
                                   scoring=cost_scorer, cv=5)
tuned.fit(X_train, y_train)
print(f"threshold chosen by 5-fold CV: {tuned.best_threshold_:.3f}")
print(f"test cost: ${fraud_cost(y_test, tuned.predict(X_test)):,}")
report("tuned threshold", y_test, tuned.predict(X_test))
```

```
threshold chosen by 5-fold CV: 0.040
test cost: $25,295
tuned threshold  flagged   392  caught  74  missed  57  recall 0.56  precisio
```

0.040, close to the formula, chosen without the test set: 392 flags, 74 frauds caught, $25,295 against $34,190 at 0.5.

11 **The leak, on purpose (3m).** "Resample first, then cross-validate. Predict the AP."

```
from sklearn.model_selection import StratifiedKFold, cross_val_score

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
X_train_enc = make_preprocess().fit_transform(X_train)
X_smote, y_smote = SMOTE(random_state=42).fit_resample(X_train_enc, y_train)
leaky = cross_val_score(LogisticRegression(), X_smote, y_smote, cv=cv, scoring
honest = cross_val_score(make_imb_pipeline(make_preprocess(), SMOTE(random_sta
                         X_train, y_train, cv=cv, scoring="average_precision")
print(f"SMOTE first, then CV : AP {leaky.mean():.3f}")
print(f"SMOTE inside the CV  : AP {honest.mean():.3f}")
```

```
SMOTE first, then CV : AP 0.912
SMOTE inside the CV  : AP 0.495
```

0.912 is fiction: validation folds full of synthetic near-copies, at a 50/50 balance production never has. Honest: 0.495. Then say one sentence about Mistake 1 (a sampler in a scikit-learn pipeline raises `TypeError`) and Mistake 3 (Step 6's inflated probabilities cut at the formula threshold flag all 15,000 payments, the most expensive option).

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **A sampler in a scikit-learn pipeline, or resampling before cross-validation.** Symptoms: `TypeError: All intermediate steps should be transformers and implement fit and transform or be the string 'passthrough...` ; or a CV AP near 0.9. Coach: "Which pipeline did you import? Did the validation folds keep the real fraud rate?"
- **Judging by accuracy or by recall at 0.5.** Symptoms: "the balanced model is better, recall 0.69"; `cross_val_score` without `scoring=` . Coach: "How many payments did it flag to get that recall? What did AP do?"
- **The formula threshold on weighted or resampled probabilities, or a threshold chosen on the test set.** Symptoms: a model that flags everything; a "best threshold" read off the test-set cost curve. Coach: "Are these probabilities honest? Compare their mean with the real rate." and "Which rows chose the threshold?"

GUIDED LAB

- **Lab A (12m, everyone):** `labs/module-05/l2_5-imbalanced-classes/retention_threshold_starter.py` ; reference `labs/module-05/l2_5-imbalanced-classes/retention_threshold_solution.py` . The lesson's cost reasoning on a different decision:

a $15 retention offer against $60 lost per churner nobody contacts, on the canonical churn split with 14 columns.

- **Expected, Lab A:** offer nobody $14,880, everyone $22,500; ROC AUC 0.827, AP 0.496 (random 0.165); at 0.5: 112 offers, $12,540; the sweep is cheapest at 0.2 to 0.25 ($10,830 to $10,935); formula 15/60 = 0.25; balanced weights at 0.5: 471 offers, $10,965, but their probabilities average 0.377 against a real 0.165, so the formula on them costs $14,070; tuned threshold 0.234: 343 offers, 153 of 248 churners reached, $10,845.
- **Circulate in Lab A for:** a cost function with the `ravel()` order wrong (tn, fp, fn, tp); the formula applied to the balanced model; thresholds chosen from the test sweep and then reported as the result. **Debrief:** "With churn at 16.5% the 0.5 cut still costs $1,700 more than a cut chosen from the prices. Same lesson as fraud, milder imbalance."
- **Lab B (8m, pairs):** `labs/module-05/l2_5-imbalanced-classes/rebalance_tree_starter.py`; reference `labs/module-05/l2_5-imbalanced-classes/rebalance_tree_solution.py`. Tests the lesson's claim that SMOTE can help a deep tree: a depth-6 decision tree on fraud, five candidates compared by CV AP inside imblearn pipelines.
- **Expected, Lab B:** CV AP plain 0.335, balanced 0.269, undersample 0.044, oversample 0.271, SMOTE 0.182; at 0.5 plain flags 44 (recall 0.27, precision 0.80), the others 1,517 to 2,242 (recall near 0.65, precision 0.04 to 0.06); SMOTE first, then CV: AP 0.873 on folds that are 50% fraud.
- **Debrief Lab B (1m):** "Here every rebalancing hurt the ranking, and undersampling (614 training rows) wrecked it. On this data the claim fails: try it, do not assume it. The leak still reports 0.87."
- **Fast finishers:** `labs/module-05/l2_5-imbalanced-classes/debug_imbalance.py`: two bugs that stop it (`TypeError` from a scikit-learn pipeline, `ValueError: could not convert string to float: 'Gift Cards'` from SMOTE before the encoder) and two that do not (accuracy reported as AP; the `ravel()` order). Fixed, it prints AP 0.535 and 0.495 and a cost of $25,985.

START THE LESSON PROJECT

The lesson project is **a fraud review queue for June**: train on January to May, choose model and threshold on those months only, run June once. In the room, Steps 1 to 3 (15 minutes).

- Read the brief aloud. Step 1 together: the split by date, no shuffling. **Done** = `train Jan-May: 41,738 payments, 374 frauds (0.90%)` and `test June : 8,262 payments, 64 frauds (0.77%)`.
- Step 2: the cost function and June's do-nothing costs, $24,000 and $82,620. "Any model has to beat $24,000."
- Step 3: three candidates by CV AP with SMOTE inside an imblearn pipeline: plain 0.506, balanced 0.481, SMOTE 0.472. Keep the plain model.
- **Goes home:** Step 4 (tuned threshold 0.051 against the formula's 0.027), Step 5 (June once: 166 flags, 41 of 64 caught, $10,285) and Step 6 (`fraud_review_queue.csv`, no `is_fraud` column).

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab:

- What is AP's baseline here, and why not 0.5? **Answer:** the fraud rate, about 0.009: that is what a random ranking scores.
- The balanced model's recall jumped and its AP did not. What changed? **Answer:** where the 0.5 cut falls; the ranking is about the same.
- Review $10, missed fraud $375: formula threshold, and which model may use it? **Answer:** 0.027, only a model with calibrated probabilities (the plain one).
- Why must the sampler sit inside the pipeline? **Answer:** so validation folds keep the real class balance and contain no synthetic rows.
- Who carries the cost of a false alarm? **Answer:** an honest customer whose order is held; check whether one group (new accounts) carries more than its share (Exercise 3).

Workbook self-check, items Q7 to Q12 (learners do them at the end of the block or at home).

Answers: **Q7** `[ 0.526 10. ]`; **Q8** D) 0.04; **Q9** A) 0.495: resampling first put synthetic frauds, built from frauds in the training folds, into the validation folds, at a 50/50 balance production never has; **Q10** `405`; **Q11** C) `predict` applies the tuned threshold; `predict_proba` returns the underlying model's probabilities unchanged; **Q12** B) The default score of a classifier is accuracy, which hides whether any fraud is found; pass `scoring="average_precision"` or a cost scorer. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: in Lab A, write `retention_cost` together on the projector (TODO 1 is the step most people get wrong), and give struggling pairs the threshold loop from the demo to adapt. Keep RANK | DECIDE on the board and ask "which side is this?" for every number.

Stretch: the lesson's Exercise 4 (re-tune for losses of $100, $375 and $1,000) and Bonus Challenge (night and night-on-a-new-device features: one good feature moves AP more than every sampler). In Lab A: what threshold fills a capacity of 200 calls, and what does it cost?

CLOSING

EXIT TICKET (3 QUESTIONS, NO NOTES, 4 MINUTES)

Put the questions on the slide; learners answer in the workbook's Exit Ticket box or on a sticky note. Sort the notes before Session 6: question 1 tells you how much the checkpoint will test what you think it tests.

1. Why must `SelectKBest` live inside the pipeline when you cross-validate? **Answer:** it looks at the labels; outside the pipeline the validation rows help choose the columns, and the score is optimistic (about 80% on pure noise instead of 50%).

2. A class-weighted fraud model has recall 0.69 where the plain model had 0.31, with the same AP. What changed, and what did it cost? **Answer:** mostly the cut: more payments cross 0.5 (1,863 flags instead of 44, precision 0.05), and the probabilities are inflated (mean 0.27 against a real 0.009).

3. A review costs $10 and a missed fraud $375. Which threshold, and on which model's probabilities may you use it? **Answer:** 10 / 375 ≈ 0.027, on a calibrated model (plain, no weights, no resampling); otherwise tune it with `TunedThresholdClassifierCV`.

HOMEWORK (ABOUT 2 TO 3 HOURS)

- **Finish both lesson projects.** The lean churn model (Steps 4 to 6: the forward path from `days_since_last_order`, 6 columns, test ROC AUC 0.826 full against 0.824 lean, `lean_features.txt`) and the June review queue (Steps 4 to 6: threshold 0.051, 166 flags, 41 of 64 caught, $10,285, `fraud_review_queue.csv`).
- **Lesson exercises.** 2.4: Exercises 1 to 3 (what the filters keep, backward selection, a forest as the selector); 4 is a stretch. 2.5: Exercises 1 to 3 (weights between the extremes, a fixed threshold for a capacity, who gets flagged by mistake); 4 is a stretch.
- **Both Quick Quizzes** on the lesson pages, and **both journal entries**.
- **Before the Next Lesson (both pages):** keep `lean_features.txt` and the 2.5 notebook; 2.4: rerun the noise experiment with `k=200`; 2.5: one problem from your own life with two very different mistake costs, and its formula threshold; rerun 2.5 Step 3 with `scoring="roc_auc"`.
- **Labs not finished in class:** `lean_fraud` TODO 5, `vendor_columns`, `rebalance_tree`; optional `debug_imbalance.py`.
- **Checkpoint 1 is at the start of Session 6** (30 minutes, open notebook, not graded): look back over your Module 1 and 2 notebooks, especially splits, pipelines, metrics and thresholds.

BRING LIST FOR SESSION 6

- A laptop on which `check_environment.py` ends with READY (nothing new to install).
- The `pinecrest-ml` folder with `data/` and the `labs/` folder inside it. Session 6's starters are in `labs/module-06/`, including the checkpoint starter in `labs/module-06/checkpoint-1/`.
- Your notebooks from Lessons 1.1 to 2.5: Checkpoint 1 is open notebook, and your own code is the best reference.
- The finished 2.4 and 2.5 project notebooks, `lean_features.txt`, the workbook and the journal. No downloads, no internet needed.

IF YOU RUN LONG OR SHORT

RUNNING LONG: WHAT TO PROTECT AND WHAT TO CUT FIRST

- **Protect:** the noise experiment and the SMOTE leak; 2.5 Steps 4, 8 to 10 (RANK | DECIDE, cost, tuned threshold); RFECV's plateau and the tolerance rule.

- **Cut first (saves 6m):** 2.4 Steps 4 and 5 (filters). Show the F/MI table on the lesson page and say: "Filters score columns alone: fast, blind to the model and to teamwork."
- **Cut second (saves 4m):** 2.5 Step 7 (samplers). Show the lesson's side-by-side table instead of running it.
- **Cut third (saves 8m per lesson):** Lab B of either lesson moves to homework; its expected output is in the workbook's "Expected when done" lines.

RUNNING SHORT: WHAT TO ADD

- 2.4 Exercise 3 live: a forest as the selector, and why a logistic regression does worse on the forest's choice (0.813).
- 2.5 Exercise 3 live: who gets flagged by mistake? New accounts 3.2% against 1.4%. Discuss what Pinecrest should do about it.
- Run `debug_imbalance.py` together: each loud bug read from the last line, then hunt the two silent ones.

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 6: Linear Models and Regularization

How to teach the sixth session of Machine Learning Foundations: Checkpoint 1 on Modules 1 and 2 (audit and fix a leaky churn model, then choose who gets a win-back email from the prices), then linear and logistic regression opened up (least squares by hand, coefficients in units and percent, the sigmoid, odds ratios) and regularization (bias and variance measured, Ridge and Lasso, alpha by cross-validation).

`2 lessons + checkpoint` `3 hrs session` `Lessons 3.1–3.2` `Session 6 of 11`

HOW TO USE THIS GUIDE

- This session has three parts: Checkpoint 1 (30 minutes, right after the warm-up), Lesson 3.1 and Lesson 3.2. The lesson blocks are 60 minutes each instead of 75, so demos are shorter and show charts from the pages instead of typing them.
- Same rhythm for each lesson: frame → live demo (you type, they follow) → guided lab → start the project. Every code block below was run to produce the output printed under it; the plotting lines are left out (show the charts on `regression_linear_logistic.html` and `regression_regularization.html`).
- Labs live in `labs/module-06/`; the checkpoint starter and model answer in `labs/module-06/checkpoint-1/`. Starters are handouts; solutions and the model answer are yours.
- Why the checkpoint comes first: it covers Modules 1 and 2, the room is fresh, and a lesson that runs long can never eat it.

> **THE #1 RISK OF SESSION 6: THE CLOCK, THEN THE CHECKPOINT'S TONE**
> The lesson pages estimate 100 + 100 minutes; the room has 120 for both. If the checkpoint runs over, take the minutes from the 3.1 demo (cut Steps 2, 3 and 7 to one sentence each), never from the 3.2 bias-variance idea. Say in the first minute that the checkpoint is practice for the final project, open notebook and not graded, and that the rubric is in their hands afterward.

BEFORE THE SESSION

- ☐ **Run the kit check:** `python3 labs/check_labs.py module-06`. Expect `10 passed, 0 failed` (four lab pairs and the checkpoint pair), about a minute.
- ☐ **Copy to the shared folder:** `labs/module-06/l3_1-linear-logistic/*_starter.py`, `labs/module-06/l3_2-regularization/*_starter.py` and `labs/module-06/checkpoint-`

`1/checkpoint1_starter.py`. Keep `checkpoint1_solution.py` (the model answer) and every `*_solution.py` on your machine.

- ☐ **Print the rubric** from the Answer Key (one copy per learner, handed out after the checkpoint), or plan to show it on the projector at the wrap-up.
- ☐ **Downloads:** none.
- ☐ **Projector machine:** two empty notebooks `lesson_3_1` and `lesson_3_2`; browser tabs with both lesson pages scrolled to the charts (Snow scatter, residual fan, S-curve against bands, three-panel bias-variance curves, the U chart).
- ☐ **Board:** for the checkpoint, the four verbs PLAN · AUDIT · REBUILD · REPORT and the end time; for 3.1, `ŷ = b₀ + b₁x` and `p = 1 / (1 + e^-z)`; for 3.2, `expected error = bias² + variance + noise`.

KNOWN ENVIRONMENT TRAPS

SYMPTOM	CAUSE	FIX
`TypeError: LogisticRegression.__init__() got an unexpected keyword argument 'multi_class'`	Code from an older tutorial	Delete the argument: softmax is what happens.
`TypeError: Ridge.__init__() got an unexpected keyword argument 'normalize'`	Older tutorial	Put a `StandardScaler` in front, in the pipeline.
`ConvergenceWarning` from `Lasso` or from `LogisticRegression(C=np.inf)`	Tiny penalty, many correlated columns	Raise `max_iter`, scale, and ask whether so little penalty makes sense. The churn_curves lab silences it on purpose for the unpenalized degree-3 fit.
`ValueError: Unknown label type: continuous`	A classifier given a numeric target	Use `LinearRegression` or `Ridge`.

SESSION 6 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Five retrieval questions on Lessons 2.4 and 2.5; one minute on how the checkpoint runs.
0:10	30	Checkpoint 1 (Modules 1–2)	Open notebook, alone, not graded: audit and fix a colleague's churn model, choose who gets a win-back email, three sentences. `labs/module-06/checkpoint-1/checkpoint1_starter.py`.

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:40	8	**3.1** Frame and mental model	The ruler on the scatter plot; least squares; from a line to a probability.
0:48	22	**3.1** Live demo	The Snow line by hand, a multi-feature model against baselines, the log target, the sigmoid table, odds ratios, decision_function.
1:10	20	**3.1** Guided lab	`labs/module-06/l3_1-linear-logistic/daily_sales_line_starter.py` (everyone), then `labs/module-06/l3_1-linear-logistic/fraud_odds_starter.py` in pairs.
1:30	10	**3.1** Start the lesson project	What does a discount really sell? Steps 1 to 3.
1:40	10	Break	Collect or photograph checkpoint notes if you want to give feedback.
1:50	8	**3.2** Frame and mental model	Three students and a practice exam; bias, variance and the dial.
1:58	22	**3.2** Live demo	Polynomial features, underfitting and overfitting, bias and variance measured, Ridge, the 400-row pilot, Lasso and twins, RidgeCV and LassoCV.
2:20	20	**3.2** Guided lab	`labs/module-06/l3_2-regularization/churn_curves_starter.py` (everyone), then `labs/module-06/l3_2-regularization/variance_without_truth_starter.py` in pairs.
2:40	10	**3.2** Start the lesson project	The interaction forecast, Steps 1 and 2.
2:50	10	Wrap-up	Exit ticket (3 questions), homework, bring list for Session 7.
Total	180		Ends at 3:00.

In the room. After the warm-up, the checkpoint, the break and the wrap-up, the two lessons share 120 minutes against the pages' 200. The room protects each lesson's mental model, a shorter demo, one guided lab and the first project steps.

Goes home (2 to 3 hours). The price-response project Steps 4 to 6 and the interaction forecast Steps 3 to 6; Exercises 1 to 3 of both lessons; both Quick Quizzes and journal entries; unfinished lab TODOs; and the checkpoint, finished at home with the rubric if it was not finished in the room.

Protect, in this order: the full 30 minutes of the checkpoint; 3.1 Steps 5, 6 and 8 (log target, sigmoid, odds ratios); 3.2 Steps 3, 8 and 10 (the diagnostic, the pilot, alpha by CV). **Cut first:** 3.1 Steps 2, 3 and 7; then 3.2 Step 4 (show the chart only) and the twins half of Step 9; then Lab B of each lesson.

OPENING SEGMENT

0:00 Warm-up: Selection and Imbalance — 10 min

TIMING

6m	Five retrieval questions on Lessons 2.4 and 2.5 (workbook warm-up box)
4m	Reveal; a 30-second homework check by hands (both Session 5 projects: `lean_features.txt` and `fraud_review_queue.csv`; the k=200 noise rerun; one learner's own cost-ratio threshold); then one minute on how the checkpoint runs

QUESTIONS AND ANSWERS

- **W1.** RFECV's curve is 0.8328 at 6 columns and 0.8333 at 11; the folds disagree by about 0.02. How many columns do you keep, and why? **Answer:** 6: the difference is far smaller than the noise between folds, and the smaller set is cheaper and sturdier (the tolerance rule). *Checkpoint 1 asks for honest comparisons like this one.*
- **W2.** Review $10, missed fraud $375. What does this print? (code on the slide) **Answer:** `[0 1 1 1]` *10 / 375 ≈ 0.027, so every probability from 0.04 up is flagged. Valid only for calibrated probabilities.*
- **W3.** Where must SMOTE sit when you cross-validate, and what happens otherwise? **Answer:** Inside an imblearn pipeline, after the preprocessing; resampling first put synthetic near-copies into the validation folds and reported AP 0.91 instead of about 0.49. *Same rule as feature selection: a step that learns from y lives inside the folds.*
- **W4.** Why must an L1 model be scaled before it selects columns? **Answer:** The penalty charges for the size of each weight, so without scaling it keeps columns in large units and drops columns in small units, whatever their usefulness. *Lesson 3.2 today: every penalty needs scaled features.*
- **W5.** A class-weighted model's probabilities average 0.27 when the real fraud rate is 0.009. What does that change? **Answer:** Its ranking is about the same, but the probabilities are no longer calibrated: a cost formula applied to them flags almost everything. Tune the threshold with cross-validation instead. *Today's 3.1 shows where those probabilities come from: the sigmoid of the log-odds.*

0:10 Checkpoint 1: Audit and Fix a Churn Model

30 min

THE TASK (READ IT ALOUD; IT IS ALSO IN THE WORKBOOK)

Pinecrest's marketing team will email a win-back voucher to the customers a churn model flags. An email with a voucher costs **$5**; a churner nobody reaches costs about **$50** of margin. A colleague's notebook (Part A of `labs/module-06/checkpoint-1/checkpoint1_starter.py`) reports about 95% accuracy. **Plan** what you will check; **audit** the colleague's code (one line per problem); **rebuild** an honest model in a pipeline, cross-validated against a baseline with a metric suited to a rare class, and choose who gets the email from the prices, on the training rows only; look at the test set **once**; **report** three sentences for the marketing lead.

TIMING

2m	Open `checkpoint1_starter.py`, run it once: the colleague's accuracy prints
3m	Plan, written as comments above the code
5m	Audit: one comment line per problem
15m	Rebuild: pipeline, CV against a baseline, the cost threshold, the test set once
5m	Report: three sentences

HOW TO RUN IT

- Say: "Thirty minutes, alone, open notebook: your Lesson 1.1 to 2.5 notebooks are the best reference. Not graded. Afterward you get the rubric and a model answer. Practice for the final project."
- Put the four verbs and the end time on the board. Call out time at 0:15 ("you should be auditing"), 0:20 ("rebuild now"), 0:35 ("five minutes: write your three sentences, even if the code is unfinished").
- Do not help with the audit. Do help with environment trouble (kernel, file not found) at once: that is not what is assessed.
- Watch for the four habits in the rubric: the leak named, everything that learns inside a pipeline, CV against a baseline with AUC or AP, a threshold from the prices chosen without the test set.
- At the end: "Keep your file. After the break we move on; the rubric and the model answer come at the wrap-up and in the answer key." Quick read later: the report first, then a pipeline, a baseline line and a threshold line in the code.

WHAT A SOLID ANSWER REACHES (MODEL ANSWER, REAL RUN)

CV ROC AUC 0.832 against 0.500 for the dummy, CV AP 0.528 against 0.165; threshold 0.107 (formula 0.10); test ROC AUC 0.826, 654 emails reaching 214 of 248 churners for $4,970, against $7,500 emailing everyone, $9,630 at 0.5 and $12,400 emailing nobody. Full output, rubric and weak answers in the Answer Key.

LESSON PLANS

3.1 Linear and Logistic Regression

60 min

OBJECTIVES (FROM THE LESSON PAGE)

- Explain what least squares does and fit `LinearRegression`, reading `coef_` and `intercept_` in the units of the data
- Interpret a coefficient in a multiple regression as an effect "holding the other features constant", and spot when that reading breaks down
- Use a log target with `TransformedTargetRegressor` when errors grow with the size of the prediction
- Describe how logistic regression turns a weighted sum into a probability, and read its coefficients as log-odds and odds ratios
- Fit a multiclass logistic regression and know which settings are current (`C`, `l1_ratio`) and which are gone

TIMING IN THE ROOM

8m	Frame: the ruler on the scatter plot; least squares; from a line to a probability
22m	Live demo: Steps 1 to 9 (Steps 2, 3 and 7 are one-minute steps)
20m	Guided lab: `daily_sales_line_starter.py` (everyone), then `fraud_odds_starter.py` in pairs
10m	Start the lesson project: Steps 1 to 3

MENTAL MODEL TO SAY

"Plot weekly snowshoe sales against temperature and lay a ruler across the cloud. Tilt it, slide it. Each dot sits some vertical distance from the ruler: its residual. Linear regression finds the one position where the squared residuals add up to the least. That is least squares."

Then the second picture, for yes or no: "Keep the weighted sum, but squash it through an S so it stays between 0 and 1. The sum before squashing is the log-odds; exp of a coefficient is an odds ratio." Write both formulas on the board and add the phrase every coefficient needs: **holding the other features constant.**

LIVE DEMO (YOU TYPE, THEY FOLLOW)

New notebook `lesson_3_1`.

1. **A ruler on the Snow scatter (3m).** "Snow gear, 2024 to learn from, 2025 to test: a forecast must not see the future."

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error

demand = pd.read_csv("data/weekly_demand.csv")
train, test = demand[demand["year"] == 2024], demand[demand["year"] == 2025]
snow_train = train[train["category"] == "Snow"]
snow_test = test[test["category"] == "Snow"]

line = LinearRegression()
line.fit(snow_train[["avg_high_temp_f"]], snow_train["units_sold"])
print(f"slope {line.coef_[0]:.3f} units per °F, intercept {line.intercept_:.2f
print(f"test R2 {line.score(snow_test[['avg_high_temp_f']], snow_test['units_s
print(f"test MAE {mean_absolute_error(snow_test['units_sold'], line.predict(sn
```

```
slope -0.761 units per °F, intercept 60.18
test R2 0.492
test MAE 8.64 units
```

Read it in units: about 0.76 fewer snow items per extra °F. The intercept (60 units at 0 °F) is just where the line crosses the axis. Show the lesson's scatter chart.

2. **No magic in fit (2m).** "For one feature, the slope is covariance over variance, and the line passes through the averages."

```
x, y = snow_train["avg_high_temp_f"], snow_train["units_sold"]
slope = np.cov(x, y)[0, 1] / x.var()
intercept = y.mean() - slope * x.mean()
print(f"by hand: slope {slope:.3f}, intercept {intercept:.2f}")
```

```
by hand: slope -0.761, intercept 60.18
```

3. **Where the line goes (1m).** "Predict 30, 60 and 85 °F."

```
print(line.predict(pd.DataFrame({"avg_high_temp_f": [30, 60, 85]})).round(1))
```

```
[37.4 14.5 -4.5]
```

Minus 4.5 snowshoes in summer: a straight line has no idea sales stop at zero.

4 **Many features, against two baselines (4m).** "All 24 products: product and store one-hot, three numeric features, in a pipeline. First the baselines a planner would use without a model."

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.pipeline import make_pipeline

y_train, y_test = train["units_sold"], test["units_sold"]
mean_mae = mean_absolute_error(y_test, np.full(len(y_test), y_train.mean()))
product_means = train.groupby("product_id")["units_sold"].mean()
product_mae = mean_absolute_error(y_test, test["product_id"].map(product_means
print(f"baseline, overall mean    : MAE {mean_mae:.2f}")
print(f"baseline, product's mean  : MAE {product_mae:.2f}")

num_cols = ["promo", "holiday_week", "avg_high_temp_f"]
cat_cols = ["product_id", "store"]
encode = ColumnTransformer(
    [("num", "passthrough", num_cols),
     ("cat", OneHotEncoder(handle_unknown="ignore", sparse_output=False), cat_
).set_output(transform="pandas")

linear = make_pipeline(encode, LinearRegression())
linear.fit(train[num_cols + cat_cols], y_train)
pred = linear.predict(test[num_cols + cat_cols])
print(f"linear regression         : MAE {mean_absolute_error(y_test, pred):.2f
      f"R2 {linear.score(test[num_cols + cat_cols], y_test):.3f}")
coefs = pd.Series(linear[-1].coef_, index=linear[-1].feature_names_in_)
print(coefs[["num__promo", "num__holiday_week", "num__avg_high_temp_f"]].round
```

```
baseline, overall mean    : MAE 17.13
baseline, product's mean  : MAE 13.42
linear regression         : MAE 11.77, R2 0.506
num__promo              14.003
num__holiday_week       13.997
num__avg_high_temp_f     0.191
dtype: float64
```

11.8 against 13.4 and 17.1. Promo +14 units "holding the others constant": the same product, store and temperature. Then: "One temperature slope for every product: snow and tents averaged into +0.19 that fits nobody."

5 **The fan, and the fix (3m).** "Show the residual plot on the lesson page: small sellers within a few units, big sellers off by dozens. Effects act in percent. Model the log."

```
from sklearn.compose import TransformedTargetRegressor

log_linear = TransformedTargetRegressor(
    regressor=make_pipeline(encode, LinearRegression()),
    func=np.log1p,
    inverse_func=np.expm1,
)
log_linear.fit(train[num_cols + cat_cols], y_train)
log_pred = log_linear.predict(test[num_cols + cat_cols])
print(f"log-target linear regression: MAE {mean_absolute_error(y_test, log_pre
print(f"negative predictions: {(log_pred < 0).sum()}")
log_coefs = pd.Series(log_linear.regressor_[-1].coef_, index=log_linear.regres
effects = log_coefs[["num__promo", "num__holiday_week", "num__avg_high_temp_f"
print(pd.DataFrame({"log_coef": effects, "pct_change": (np.exp(effects) - 1) *
```

```
log-target linear regression: MAE 11.16
negative predictions: 0
                      log_coef  pct_change
num__promo               0.420      52.199
num__holiday_week        0.407      50.201
num__avg_high_temp_f     0.005       0.510
```

MAE down, no negative predictions, and coefficients as percentages: promo +52% for every product. Write `exp(coef) - 1` on the board.

6 **From a line to a probability (3m).** "For yes or no, squash the weighted sum through the sigmoid. The sum is the log-odds."

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

customers = pd.read_csv("data/customers_churn.csv")
features = ["tenure_months", "orders_12m", "spend_12m", "avg_order_value", "da
            "returns_12m", "return_rate", "support_tickets_12m", "discount_sha
Xc_train, Xc_test, yc_train, yc_test = train_test_split(
    customers[features], customers["churned"], test_size=0.25, stratify=custom
)
one = LogisticRegression().fit(Xc_train[["days_since_last_order"]], yc_train)
b0, b1 = one.intercept_[0], one.coef_[0, 0]
print(f"z = {b0:.3f} + {b1:.5f} * days")
days = pd.DataFrame({"days_since_last_order": [0, 30, 90, 150, 250, 364]})
days["p_churn"] = one.predict_proba(days)[:, 1].round(3)
days["by_hand"] = (1 / (1 + np.exp(-(b0 + b1 * days["days_since_last_order"]))
print(days.to_string(index=False))
print(f"p crosses 0.5 at {-b0 / b1:.0f} days")
```

```
z = -2.594 + 0.00928 * days
 days_since_last_order  p_churn  by_hand
                     0    0.070    0.070
                    30    0.090    0.090
                    90    0.147    0.147
                   150    0.231    0.231
                   250    0.432    0.432
                   364    0.687    0.687
p crosses 0.5 at 279 days
```

By hand equals `predict_proba`. Crosses 0.5 only at 279 days, so `predict` calls few churners.

7 **What really happened (1m).** "Actual churn in 30-day bands."

```
bands = pd.cut(Xc_train["days_since_last_order"], bins=[*range(0, 361, 30), 36
actual = yc_train.groupby(bands, observed=True).mean()
print(actual.round(3).iloc[[0, 3, 4, 5, 6, 11]].to_string())
```

```
days_since_last_order
[0, 30)       0.088
[90, 120)     0.125
[120, 150)    0.207
[150, 180)    0.295
[180, 210)    0.525
[330, 360)    0.567
```

Flat, then a jump between 120 and 210 days. The S-curve can only bend gently: the straight-line-in-log-odds assumption, visible. (Lesson 2.3's 150-day flag is the fix.)

8 **Odds ratios (3m).** "All ten features, scaled, so each coefficient is per standard deviation. exp(coef) is an odds ratio."

```
from sklearn.preprocessing import StandardScaler

churn_model = make_pipeline(StandardScaler(), LogisticRegression())
churn_model.fit(Xc_train, yc_train)
print(f"test accuracy {churn_model.score(Xc_test, yc_test):.3f}")
coef = pd.Series(churn_model[-1].coef_[0], index=features)
table = pd.DataFrame({"log_odds": coef, "odds_ratio": np.exp(coef)}).sort_valu
print(table.round(2))
```

```
test accuracy 0.848
                        log_odds  odds_ratio
app_user                   -0.34        0.71
orders_12m                 -0.31        0.73
tenure_months              -0.12        0.88
spend_12m                  -0.12        0.89
returns_12m                -0.02        0.98
avg_order_value             0.00        1.00
return_rate                 0.07        1.07
support_tickets_12m         0.40        1.49
discount_share              0.67        1.96
days_since_last_order       0.88        2.42
```

One SD more days since the last order (about 87 days) multiplies the odds by 2.4; app users have 0.71 times the odds. Then the warning box: these describe the model, not causes; tenure's -0.12 hides a U.

9 **The 0.5 rule is z = 0 (2m).** "decision_function returns z."

```python
z = churn_model.decision_function(Xc_test[:3])
print("log-odds   :", z.round(3))
print("sigmoid(z) :", (1 / (1 + np.exp(-z))).round(3))
print("proba      :", churn_model.predict_proba(Xc_test[:3])[:, 1].round(3))
print("predict    :", churn_model.predict(Xc_test[:3]))
```

```
log-odds   : [-2.691  0.118 -2.145]
sigmoid(z) : [0.063 0.53  0.105]
proba      : [0.063 0.53  0.105]
predict    : [0 1 0]
```

Same three customers as Lesson 1.1. Moving the threshold (Lesson 2.5) is moving the line z = 0. If time allows, one sentence on `multi_class` being gone (`TypeError: LogisticRegression.__init__() got an unexpected keyword argument 'multi_class'`).

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Reading a coefficient without its units or its company.** "Promo is 70 times more important than temperature" (14.0 against 0.19), or "price barely matters" from the naive -0.53. Coach: "Per what? Across the feature's range? Holding which other columns constant?"
- **Probability and log-odds mixed up.** "0.88 adds 0.88 to the churn probability." Coach: "Which scale is z on? What is exp(0.88)?" Have them compute one sigmoid by hand.
- **Coefficients read as causes, or twins trusted.** "Make customers install the app and churn drops 30%"; a promo coefficient that looks absurd (Lab A). Coach: "Did anyone change the app flag and watch?" and

"Is this column a copy of another?"

GUIDED LAB

- **Lab A (12m, everyone):** `labs/module-06/l3_1-linear-logistic/daily_sales_line_starter.py`; reference `labs/module-06/l3_1-linear-logistic/daily_sales_line_solution.py`. Daily store sales: two baselines, Boise's growth line by hand, a multi-feature model with an Asheville Monday in January as the reference, a twin column, a log target.
- **Expected, Lab A:** baselines $1,670 and $1,292; Boise +$1,333 a year from $6,037 (hand and sklearn agree); linear MAE $949, Saturday +$3,509, Tuesday -$168, trend +$487 a year, promo +$2,625, Black Friday +$2,083; promo is 1 exactly on sale and holiday-rush days, and dropping it changes no prediction; log target MAE $853, Saturday +74.4%, Tuesday -5.3%, trend +8.1% a year, Black Friday +82.1%.
- **Debrief Lab A:** "Promo and the event columns are twins: the split between them is arbitrary, the predictions are not. And a log target turns Saturdays into a percentage that holds for every store."
- **Lab B (8m, pairs):** `labs/module-06/l3_1-linear-logistic/fraud_odds_starter.py`; reference `labs/module-06/l3_1-linear-logistic/fraud_odds_solution.py`. Odds ratios in an analyst's units (per flag, per $100), the sigmoid by hand, and hour as a line against a night flag.
- **Expected, Lab B:** z = -5.984 + 1.515 × velocity (odds ratio 4.55 per extra payment); odds ratios device_new 7.11, night 6.93, billing mismatch 3.91, country mismatch 8.94, per $100 1.31, per card attempt 6.31; hour -0.111 (0.90) against the night flag +2.354 (10.53); predict equals z > 0; 21 flagged at 0.5, 18 fraud; test AP 0.332.

START THE LESSON PROJECT

The lesson project asks **what a discount really sells**: a log-log price elasticity that changes as the model compares like with like. In the room, Steps 1 to 3 (10 minutes).

- Step 1 together (log columns, crossed categories). Step 2: `naive elasticity: -0.53`. Step 3: `within-product elasticity: -1.50`. Ask: "Why did it triple?"
- **Goes home:** Step 4 (season, store, promo, trend: -1.606, a 10% cut sells +18.4%), Step 5 (2024 only, test 2025: MAE 8.27 against 13.42) and Step 6 (the tent what-if: +45% units, +16% revenue).

CHECKS FOR UNDERSTANDING

- What does `ŷ = 60.18 - 0.761 × temp` predict at 85 °F, and what does that tell you? **Answer:** -4.5 units: lines extrapolate without limits.
- Log target coefficient 0.42: in words? **Answer:** about 52% more, for every product.
- decision_function returns 0: probability and prediction? **Answer:** 0.5, exactly on the boundary.

Workbook self-check, items Q1 to Q6.

Answers: **Q1** `[35.]`; **Q2** B) A promo week sells about 52% more, for every product, whatever its size; **Q3** D) A new device multiplies the odds of fraud by about 2, holding the other features constant; **Q4** A) The naive slope compared different products (cheap socks sell more units than tents), so price stood in for product type; **Q5** `0.25`; **Q6** C) No error: every distinct count becomes its own class, which is wrong and slow. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: in Lab A, do TODO 1 and 2 together on the projector; give pairs the coefficient-printing line.
Stretch: the lesson's Exercise 4 (a 150-day flag helps the line) and Bonus Challenge (an elasticity per category).

3.2 Regularization and the Bias-Variance Trade-off

60 min

OBJECTIVES (FROM THE LESSON PAGE)

- Explain underfitting and overfitting in terms of bias and variance, and recognize each from training and test errors
- Add polynomial and interaction features with `PolynomialFeatures` and predict how many columns they create
- Fit `Ridge`, `Lasso` and `ElasticNet` in a pipeline with scaling, and describe how their penalties differ
- Choose the penalty strength `alpha` by cross-validation on the training data only, with `RidgeCV` and `LassoCV`
- Build a regularized demand forecast with interactions and judge it against baselines on a later year

TIMING IN THE ROOM

8m	Frame: three students and a practice exam; the flexibility dial
22m	Live demo: Steps 1 to 10
20m	Guided lab: `churn_curves_starter.py` (everyone), then `variance_without_truth_starter.py` in pairs
10m	Start the lesson project: Steps 1 and 2

MENTAL MODEL TO SAY

"Three students prepare with last year's practice exam. One learns a single rule, 'demand is about 25', and is always a bit wrong: high bias. One memorizes every answer, typos included: perfect on the practice exam, lost on the real one, and a different practice exam would have made a different student: high variance. The third learns the patterns and ignores details too specific to trust. Regularization turns the memorizer into the third student: learn many things, but pay for every strong opinion."

Write `expected error = bias² + variance + noise` and draw the U. "Every model in this course has a dial like this: tree depth, k in kNN next week, boosting rounds. Today's dial is alpha."

LIVE DEMO (YOU TYPE, THEY FOLLOW)

New notebook `lesson_3_2`.

1. **Powers and products (2m).** "A linear model can only add columns, so give it new ones."

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import PolynomialFeatures

tiny = pd.DataFrame({"a": [1, 2, 3], "b": [10, 20, 30]})
poly = PolynomialFeatures(degree=2, include_bias=False).set_output(transform="
print(poly.fit_transform(tiny))
```

```
     a     b  a^2   a b    b^2
0  1.0  10.0  1.0  10.0  100.0
1  2.0  20.0  4.0  40.0  400.0
2  3.0  30.0  9.0  90.0  900.0
```

Squares and an interaction. Still linear in its weights.

2 **A curve where we know the truth (1m).** "A sine wave plus noise with SD 0.3: even a perfect model has MSE about 0.09."

```
rng = np.random.default_rng(42)

def true_curve(x):
    return np.sin(2 * np.pi * x)

def make_sample(n=25):
    x = rng.uniform(0, 1, n)
    return x.reshape(-1, 1), true_curve(x) + rng.normal(0, 0.3, n)

X_small, y_small = make_sample()
X_new, y_new = make_sample(500)
print(X_small.shape, X_new.shape)
```

```
(25, 1) (500, 1)
```

3 **Too simple, too flexible (3m).** "Predict which degree has the best TEST error."

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

def poly_model(degree, regressor=None):
    if regressor is None:
        regressor = LinearRegression()
    return make_pipeline(PolynomialFeatures(degree, include_bias=False), Stand

for degree in [1, 4, 15]:
    m = poly_model(degree).fit(X_small, y_small)
    train_mse = mean_squared_error(y_small, m.predict(X_small))
    test_mse = mean_squared_error(y_new, m.predict(X_new))
    print(f"degree {degree:2d}   train MSE {train_mse:.3f}   test MSE {test_ms
```

```
degree  1   train MSE 0.331   test MSE 0.283
degree  4   train MSE 0.025   test MSE 0.133
degree 15   train MSE 0.016   test MSE 0.187
```

Degree 15 has the lowest training error, below the noise floor, and a worse test error than degree 4. Write the diagnostic: high/high = too simple; low/much higher = too flexible.

4 **Bias and variance, measured (3m).** "Refit each degree on 100 fresh samples. Show the lesson's three-panel chart of 20 curves while it runs."

```
samples = [make_sample() for _ in range(100)]
x_eval = np.linspace(0.05, 0.95, 50).reshape(-1, 1)

def bias_variance(make_model):
    preds = np.array([make_model().fit(X_s, y_s).predict(x_eval) for X_s, y_s
    bias_sq = ((preds.mean(axis=0) - true_curve(x_eval.ravel())) ** 2).mean()
    variance = preds.var(axis=0).mean()
    return bias_sq, variance

rows = []
for degree in [1, 3, 4, 8, 15]:
    bias_sq, variance = bias_variance(lambda: poly_model(degree))
    rows.append({"degree": degree, "bias^2": bias_sq, "variance": variance, "e
print(pd.DataFrame(rows).set_index("degree").round(3))
```

```
        bias^2  variance  expected MSE
degree
1        0.155     0.026         0.271
3        0.004     0.015         0.108
4        0.003     0.024         0.117
8        0.055     4.050         4.194
15       6.780   307.451       314.321
```

Bias falls, variance explodes. Expected error = bias^2 + variance + noise. "You can only do this when you know the truth." (Lab B shows what you CAN measure without it.)

5 **Ridge tames the wild one (2m).** "Same 15 degrees, same 25 points, a penalty on big weights."

```
from sklearn.linear_model import Ridge

for name, regressor in [("least squares", LinearRegression()), ("ridge alpha=0
    m = poly_model(15, regressor).fit(X_small, y_small)
    weights = m[-1].coef_
    test_mse = mean_squared_error(y_new, m.predict(X_new))
    print(f"{name:16s} largest |weight| {np.abs(weights).max():9.1f}   test MS
```

```
least squares    largest |weight|    1044.5   test MSE 0.187
ridge alpha=0.1  largest |weight|       1.4   test MSE 0.132
```

Largest weight from about 1,000 to 1.4; test error at degree-4 level. The rubber-band picture.

6 **Real data: the season depends on the category (2m).** "Pinecrest's weekly demand."

```
demand = pd.read_csv("data/weekly_demand.csv")
demand["month"] = pd.to_datetime(demand["week_start"]).dt.month
train = demand[demand["year"] == 2024]
test = demand[demand["year"] == 2025]
print(train.shape, test.shape)
print(demand.groupby(["category", "month"])["units_sold"].mean().unstack().loc
```

```
(5088, 16) (4896, 16)
month       1     4     7     10    12
category
Snow      32.6   2.7   0.3  10.1  41.5
Tents      5.1  16.9  32.4  10.2   9.2
Apparel   26.3  19.3  15.6  27.7  41.7
```

Snow in July is 0.3. One weight for July and one for Snow cannot say "Snow in July".

7 **489 columns (2m).** "Products of pairs of numeric, category, store and month."

```
from sklearn.compose import ColumnTransformer, TransformedTargetRegressor
from sklearn.preprocessing import OneHotEncoder

numeric = ["price", "discount_pct", "promo", "holiday_week", "avg_high_temp_f"
grouping = ["category", "store", "month"]
columns = numeric + grouping + ["product_id"]
interactions = make_pipeline(
    ColumnTransformer([
        ("num", StandardScaler(), numeric),
        ("cat", OneHotEncoder(sparse_output=False, handle_unknown="ignore"), g
    ]),
    PolynomialFeatures(degree=2, interaction_only=True, include_bias=False),
)
features = ColumnTransformer([
    ("product", OneHotEncoder(sparse_output=False, handle_unknown="ignore"), [
    ("inter", interactions, numeric + grouping),
])
print(features.fit(train[columns]).transform(train[columns].head()).shape)
```

```
(5, 489)
```

8 **A 400-row pilot (3m).** "About as many rows as useful columns. Predict the largest forecast of least squares."

```
from sklearn.metrics import mean_absolute_error

def demand_model(regressor):
    return TransformedTargetRegressor(make_pipeline(features, regressor), func

pilot = train.sample(400, random_state=42)
for name, regressor in [("least squares", LinearRegression()), ("ridge alpha=1
                        ("ridge alpha=10", Ridge(alpha=10)), ("ridge alpha=100
    m = demand_model(regressor).fit(pilot[columns], pilot["units_sold"])
    pred = m.predict(test[columns])
    print(f"{name:16s} train MAE {mean_absolute_error(pilot['units_sold'], m.p
          f"   test MAE {mean_absolute_error(test['units_sold'], pred):6.2f}
```

```
least squares    train MAE  3.43   test MAE  42.17   largest forecast    1529
ridge alpha=1    train MAE  4.16   test MAE  11.97   largest forecast      31
ridge alpha=10   train MAE  5.58   test MAE  10.00   largest forecast      18
ridge alpha=100  train MAE  9.03   test MAE  12.43   largest forecast      13
```

15,297 units for a product that never sold more than a few hundred: variance. alpha = 10 cuts the test error by three quarters; alpha = 100 underfits again.

9 **Lasso, and twins (3m).** "Lasso charges absolute size and sets weights to exactly zero."

```
from sklearn.linear_model import Lasso

for alpha in [0.003, 0.01, 0.03]:
    m = demand_model(Lasso(alpha=alpha, max_iter=20_000)).fit(pilot[columns],
    kept = (m.regressor_[-1].coef_ != 0).sum()
    print(f"lasso alpha={alpha:<6} weights kept {kept:3d} of 489   test MAE {m

from sklearn.linear_model import ElasticNet

snow = train[train["category"] == "Snow"]
twins = snow[["avg_high_temp_f"]].assign(avg_high_temp_c=(snow["avg_high_temp_
y_snow = np.log1p(snow["units_sold"])
for name, regressor in [("ridge", Ridge(alpha=1)), ("lasso", Lasso(alpha=0.05)
                        ("elastic net", ElasticNet(alpha=0.05, l1_ratio=0.5))]
    m = make_pipeline(StandardScaler(), regressor).fit(twins, y_snow)
    print(f"{name:12s} weights (F, C): {m[-1].coef_.round(3)}")
```

```
lasso alpha=0.003  weights kept 120 of 489   test MAE  9.57
lasso alpha=0.01   weights kept  52 of 489   test MAE  9.92
lasso alpha=0.03   weights kept  19 of 489   test MAE 11.98
ridge        weights (F, C): [-0.607 -0.607]
lasso        weights (F, C): [-1.164 -0.   ]
elastic net  weights (F, C): [-0.589 -0.586]
```

Fewer columns, similar error. Then the twins: Lasso keeps one at random. "Never read 'Lasso dropped it' as 'it does not matter'."

10 **Choose alpha by cross-validation (1m).** "Choosing by the test score made the test set training data."

```
from sklearn.linear_model import LassoCV, RidgeCV
from sklearn.model_selection import KFold

cv = KFold(n_splits=5, shuffle=True, random_state=42)
searches = {
    "RidgeCV": RidgeCV(alphas=np.logspace(-1, 3, 9), cv=cv),
    "LassoCV": LassoCV(alphas=np.logspace(-3.5, -1, 11), cv=cv, max_iter=20_00
}
for name, regressor in searches.items():
    m = demand_model(regressor).fit(pilot[columns], pilot["units_sold"])
    chosen = m.regressor_[-1]
    mae = mean_absolute_error(test["units_sold"], m.predict(test[columns]))
    print(f"{name:8s} chose alpha={chosen.alpha_:.4f}   weights kept {(chosen.
```

```
RidgeCV  chose alpha=3.1623   weights kept 377   test MAE 10.35
LassoCV  chose alpha=0.0056   weights kept  83   test MAE 9.50
```

Close to the best values seen by peeking, without touching 2025.

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Judging flexibility by training error.** "Degree 15 is best, lowest error." Coach: "On which rows?" Point at the diagnostic on the board.
- **A penalty on unscaled features.** Results change with units (dollars against thousands). Coach: "What does the penalty add up? Is a weight of 3.8 per $1,000 the same as 0.0038 per dollar?"
- **Choosing alpha with the test set, or C read the wrong way.** Coach: "Which rows chose alpha?" and, for logistic regression, "C is the inverse: small C, strong penalty."

GUIDED LAB

- **Lab A (12m, everyone):** `labs/module-06/l3_2-regularization/churn_curves_starter.py`; reference `labs/module-06/l3_2-regularization/churn_curves_solution.py`. The same ideas for a classifier: polynomial churn features on a 1,000-customer pilot, train against CV AUC, a validation curve over C, then all 4,500 customers, then one test.
- **Expected, Lab A:** pilot train AUC 0.823 / 0.899 / 1.000 and CV AUC 0.807 / 0.807 / 0.706 for degrees 1 to 3; degree 3 by C: CV AUC 0.808, 0.810, 0.809, 0.776, 0.730 for C = 0.001 to 10; on 4,500 customers degree 2 reaches 0.851 with or without a penalty while degree 3 needs C = 0.1 (0.851 against 0.827); the chosen degree-2 model scores test AUC 0.853 against 0.823 for the plain model.
- **Debrief Lab A:** "Real interactions exist in churn (degree 2 wins), but a pilot cannot afford them without a penalty. More data is also a regularizer."
- **Lab B (8m, pairs):** `labs/module-06/l3_2-regularization/variance_without_truth_starter.py`; reference `labs/module-06/l3_2-`

`regularization/variance_without_truth_solution.py`. Variance needs no truth: refit on 30 pilot samples of daily sales and measure how much the same day's forecast moves.

- **Expected, Lab B:** 406 columns; spread / mean MAE over 30 pilots of 200 rows: least squares $1,217 / $1,351, alpha 1 $526 / $1,089, alpha 10 $275 / $1,122, alpha 100 $167 / $1,527; 500-row pilots: spread $486 (least squares) and $384 (alpha 1).
- **Debrief Lab B:** "The spread falls steadily with alpha (variance); the error falls, then rises (bias returns). You measured one half of the U on real data."

START THE LESSON PROJECT

The lesson project is **the interaction forecast** for 2025. In the room, Steps 1 and 2 (10 minutes).

- Step 1: baselines 17.13 (overall mean) and 13.42 (product mean). Step 2: `ridge, main effects MAE 10.62`.
- **Goes home:** Step 3 (489 columns), Step 4 (RidgeCV chooses alpha 1.0), Step 5 (2025 MAE 7.92, R^2 0.754) and Step 6 (the six strongest interactions all involve Snow; `demand_forecast_2025.csv`, 4,896 rows).

CHECKS FOR UNDERSTANDING

- Training MSE 0.016, test 0.187, noise floor 0.09: which problem? **Answer:** high variance (overfitting).
- What happens to Ridge weights as alpha grows very large? **Answer:** all shrink toward zero; predictions approach the mean.
- Why choose alpha with RidgeCV and not with the 2025 MAE? **Answer:** choosing with the test set makes it training data; the final number becomes optimistic.

Workbook self-check, items Q7 to Q12.

Answers: **Q7** C) Increase the Ridge penalty (alpha), chosen by cross-validation; **Q8** `14`; **Q9** A) Ridge shares the effect between the twins; Lasso keeps one twin and zeros the other, somewhat arbitrarily; **Q10** D) In thousands the price needs a weight 1,000 times larger, and the penalty shrinks it hard; **Q11** `[0.] 13.0`; **Q12** B) More rows reduce variance on their own, so less penalty is needed to control it. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: in Lab A give the `cross_validate` call for TODO 1 and let pairs change only the degree. **Stretch:** the lesson's Exercise 4 (more data or more penalty?) and the rolling-origin Bonus Challenge.

CLOSING

EXIT TICKET (3 QUESTIONS, NO NOTES, 4 MINUTES)

1. A churn model gives `discount_share` (scaled) a coefficient of 0.67. Say what it means in one sentence, and one thing it does not mean. **Answer:** one standard deviation more discount reliance multiplies the odds of churning by exp(0.67) ≈ 1.96, other features held constant; it does not mean discounts cause churn.
2. Training error tiny, test error large: name the problem and two fixes. **Answer:** high variance; more penalty (alpha chosen by CV), fewer columns or more data.
3. Why must features be scaled before Ridge or Lasso? **Answer:** the penalty charges for weight size, and units decide how large a weight must be.

HOMEWORK (ABOUT 2 TO 3 HOURS)

- **Finish both lesson projects.** 3.1 Steps 4 to 6 (elasticity -1.61, MAE 8.27, the tent what-if); 3.2 Steps 3 to 6 (alpha 1.0, MAE 7.92, `demand_forecast_2025.csv`).
- **Lesson exercises.** 3.1: Exercises 1 to 3 (a line for tents, odds ratios you can explain, prove the 0.5 rule); 3.2: Exercises 1 to 3 (count the columns, the validation curve, how sparse can the forecast be?). Exercise 4 of each is a stretch.
- **Both Quick Quizzes and both journal entries.**
- **Checkpoint 1:** if unfinished, finish it at home, then compare with the rubric (handed out at the wrap-up). Bring one question about it to Session 7.
- **Before the Next Lesson:** 3.1: count the project model's one-hot columns; refit churn with `C=0.01`. 3.2: rerun Step 4 with `np.logspace(-2, 3, 11)`; define bias and variance in one sentence each.
- **Labs not finished in class:** any TODO of the four labs.

BRING LIST FOR SESSION 7

- A laptop on which `check_environment.py` ends with READY (nothing new to install).
- The `pinecrest-ml` folder with `data/` and `labs/`; Session 7's starters are in `labs/module-07/`.
- Your 3.2 project notebook (k-nearest neighbors reuses the bias-variance picture) and your finished checkpoint file.
- The workbook and the journal. Lesson 4.2 uses `data/reviews.csv`, already in the data folder; no downloads.

IF YOU RUN LONG OR SHORT

RUNNING LONG: WHAT TO PROTECT AND WHAT TO CUT FIRST

- **Protect:** the checkpoint's 30 minutes; 3.1 Steps 5, 6 and 8; 3.2 Steps 3, 8 and 10.
- **Cut first (saves 5m):** 3.1 Steps 2, 3 and 7: one sentence each and the chart on the page.
- **Cut second (saves 5m):** 3.2 Step 4 and the twins half of Step 9.

- **Cut third (saves 8m per lesson):** Lab B of either lesson moves to homework.

RUNNING SHORT: WHAT TO ADD

- Go through the rubric together and let learners score their own checkpoint.
- 3.1 Exercise 4 live: the 150-day flag improves log loss.
- 3.2 Exercise 3 live: LassoCV keeps 251 of 489 columns with the same MAE as Ridge.

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 7: Nearest Neighbors and Naive Bayes

How to teach the seventh session of Machine Learning Foundations: k-nearest neighbors (distance as similarity, why scaling decides everything, the k dial, lookalikes as explanations, the curse of dimensionality) and Naive Bayes (Bayes' rule from counts, bag of words, smoothing, the variants, and a model that ranks well while its probabilities are too confident).

`2 lessons` `3 hrs session` `Lessons 4.1–4.2` `Session 7 of 11`

HOW TO USE THIS GUIDE

- Same rhythm for both lessons: frame → live demo → guided lab → start the project. Exercises, the quiz and the journal are homework.
- The two lessons are opposites worth contrasting out loud: kNN has no weights and lives on distances (scaling is everything, more columns hurt); Naive Bayes counts and multiplies (no scaling, thrives on thousands of word columns, but double-counts duplicates).
- Every code block below was run to produce the output printed under it; plotting lines are left out. Show the decision maps (4.1) and the bell-curve and calibration charts (4.2) from `classification_knn.html` and `classification_naive_bayes.html`.
- Labs live in `labs/module-07/`. Starters are handouts; solutions are yours.

> **THE #1 RISK OF SESSION 7: KNN'S LOOKALIKES AND NAIVE BAYES' PROBABILITIES TAKEN AT FACE VALUE**
> Both models produce something that looks like an explanation or a confidence: "these 15 customers" and "99% sure". Learners leave trusting both. Protect two moments: the noise-columns step (4.1 Step 10: lookalikes are only as good as the distance) and the overconfidence check (4.2 Step 10: above 90% sure, 61% churned).

BEFORE THE SESSION

- ☐ **Run the kit check:** `python3 labs/check_labs.py module-07`. Expect `8 passed, 0 failed` (`debug_knn.py` is not run), about a minute.
- ☐ **Copy the starters** (`labs/module-07/l4_1-knn/*_starter.py`, `debug_knn.py`, `labs/module-07/l4_2-naive-bayes/*_starter.py`) to the shared folder; keep the solutions.
- ☐ **Downloads:** none; `data/reviews.csv` and `data/products_ml.csv` are in the data folder.

- ☐ **Projector:** notebooks `lesson_4_1` and `lesson_4_2`; lesson tabs scrolled to the decision maps, the bell curves and the calibration chart. The 4.1 CV loop takes about 10 seconds: start it, then talk.
- ☐ **Board:** the comps picture with two questions (what counts as similar? how many comps?); Bayes' rule `P(class | clue) = P(clue | class) × P(class) / P(clue)`.
- ☐ **Checkpoint 1 feedback:** read a few checkpoint reports; pick the two most common rubric gaps for the warm-up.

KNOWN ENVIRONMENT TRAPS

SYMPTOM	CAUSE	FIX
`ValueError: Expected n_neighbors <= n_samples_fit, ...`	k larger than the rows (a tiny subset, a small fold)	Keep k well below the smallest training fold.
`ValueError: Input X contains NaN.`	`email_opens_90d` has gaps; kNN cannot measure a distance to a gap	`SimpleImputer` in the pipeline.
`ValueError: Negative values in data passed to MultinomialNB (input X).`	A scaler in front of a count model	No scaler for Multinomial/Complement/Bernoulli NB; GaussianNB for continuous features.
`TypeError: Sparse data was passed for X, but dense data is required.`	CountVectorizer output into GaussianNB	Use MultinomialNB for text.
kNN predictions take a few seconds	Every prediction searches all stored rows	Expected: kNN trains instantly and predicts slowly.

SESSION 7 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Five retrieval questions on Lessons 3.1 and 3.2; two minutes of Checkpoint 1 feedback.
0:10	10	**4.1** Frame and mental model	The real-estate agent's comps: what counts as similar, and how many comps.
0:20	30	**4.1** Live demo	A distance by hand, scaling decides everything, vote shares and lookalikes, the k dial, the curse of dimensionality.

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:50	20	**4.1** Guided lab	`labs/module-07/l4_1-knn/fraud_neighbors_starter.py` (everyone), then `labs/module-07/l4_1-knn/lookalike_days_starter.py` in pairs.
1:10	15	**4.1** Start the lesson project	Lookalike churn scoring, Steps 1 to 3.
1:25	10	Break	Leave the comps and the k dial on the board.
1:35	10	**4.2** Frame and mental model	The seasoned store manager: a prior, clues that multiply the odds, and the naive habit.
1:45	30	**4.2** Live demo	Bayes' rule from counts, bag of words, MultinomialNB, the words it counts, smoothing, the variants, GaussianNB and overconfidence.
2:15	20	**4.2** Guided lab	`labs/module-07/l4_2-naive-bayes/fraud_flags_bayes_starter.py` (everyone), then `labs/module-07/l4_2-naive-bayes/review_category_starter.py` in pairs.
2:35	15	**4.2** Start the lesson project	Review triage, Steps 1 to 3.
2:50	10	Wrap-up	Exit ticket (3 questions), homework, bring list for Session 8.
Total	180		Ends at 3:00.

In the room. The pages estimate 95 + 95 minutes; the room has 150 for both. Each lesson gets its mental model, the live demo, one guided lab and the first project steps.

Goes home (2 to 3 hours). Lookalike scoring Steps 4 to 6 and review triage Steps 4 to 6; Exercises 1 to 3 of both lessons; both Quick Quizzes and journal entries; unfinished lab TODOs; optional `debug_knn.py`.

Protect, in this order: 4.1 Steps 2 to 4 (scaling decides everything) and Step 10 (noise); 4.2 Steps 2 and 8 (Bayes from counts, smoothing) and Step 10 (overconfidence). **Cut first:** 4.1 Step 9 (say the one sentence), 4.2 Steps 3, 4 and 9, then Lab B of each lesson.

OPENING SEGMENT

0:00 Warm-up: Linear Models and Regularization — 10 min

TIMING

6m	Five retrieval questions on Lessons 3.1 and 3.2
4m	Reveal; two minutes on the two most common Checkpoint 1 gaps in the reports you read (likely: preprocessing fitted before the split, and a threshold chosen on the test set), answering one or two of the checkpoint questions learners were asked to bring

QUESTIONS AND ANSWERS

- **W1.** A log-target demand model gives `holiday_week` a coefficient of 0.405. In words? **Answer:** A holiday week sells about exp(0.405) − 1 ≈ 50% more, for every product. *Percent effects come back today as likelihood ratios that multiply.*
- **W2.** A logistic regression flag has coefficient 0.69. What does this print, and what does it mean? (code on the slide) **Answer:** `1.99` *The odds ratio: the flag about doubles the odds. Naive Bayes today also multiplies odds, one clue at a time.*
- **W3.** Degree 15 has training MSE 0.016 and test MSE 0.187 (noise floor 0.09). Which problem, and which dial helps? **Answer:** High variance (overfitting); a penalty such as Ridge's alpha, fewer features or more data. *Today's dial is k: k = 1 is the degree-15 model of nearest neighbors.*
- **W4.** Temperature in °F and °C: Ridge gives [-0.607, -0.607], Lasso [-1.164, 0]. What does Lasso's zero mean? **Answer:** Only that the column was redundant given its twin, not that it has no effect. *Naive Bayes has the opposite problem with twins: it counts both.*
- **W5.** Why must features be scaled before Ridge or Lasso? **Answer:** The penalty charges for weight size, and a feature's units decide how big its weight must be. *For kNN scaling is even more basic: it defines what "similar" means.*

LESSON PLANS

4.1 k-Nearest Neighbors

75 min

OBJECTIVES (FROM THE LESSON PAGE)

- Explain how k-nearest neighbors classifies a row, and compute Euclidean and Manhattan distances by hand
- Show why unscaled features wreck distances, and fix it with a scaler inside a pipeline
- Choose `k` with cross-validation and connect small and large `k` to variance and bias
- Recognize the curse of dimensionality and why irrelevant features hurt kNN more than a linear model
- Use `kneighbors` to show the lookalike customers behind a prediction

TIMING IN THE ROOM

10m	Frame: the real-estate agent's comps; distance and k
30m	Live demo: Steps 1 to 10
20m	Guided lab: `fraud_neighbors_starter.py` (everyone), then `lookalike_days_starter.py` in pairs
15m	Start the lesson project: Steps 1 to 3

MENTAL MODEL TO SAY

"Ask an experienced real-estate agent what a house will sell for. No formula: they pull up the comps, the five or six recently sold homes most like this one, and average their prices. That is kNN. Two questions decide whether the estimate is any good: what counts as similar (is one extra bedroom a bigger difference than ten years of age?) and how many comps (one is a gamble; fifty drag in houses that are not alike)."

Write the two dials on the board: **distance** (which columns, what scale) and **k** (variance at 1, bias at 500). "Nothing is learned in advance: fit stores the table, predict does the work. A lazy learner."

LIVE DEMO (YOU TYPE, THEY FOLLOW)

New notebook `lesson_4_1`.

1. **The churn split again (1m).** "Ten numeric features from Lesson 1.1, the canonical split."

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split

customers = pd.read_csv("data/customers_churn.csv")
features = ["tenure_months", "orders_12m", "spend_12m", "avg_order_value", "da
            "returns_12m", "return_rate", "support_tickets_12m", "discount_sha
X = customers[features]
y = customers["churned"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, stra
print(X_train.shape, X_test.shape)
```

```
(4500, 10) (1500, 10)
```

2 **A distance by hand (3m).** "Subtract, square, add, square root. Then ask where the distance comes from."

```
a, b = X_train.iloc[0], X_train.iloc[1]
diff = a - b
parts = pd.DataFrame({"customer A": a, "customer B": b, "squared diff": diff *
parts["share of total"] = parts["squared diff"] / parts["squared diff"].sum()
print(parts.round(3).iloc[[0, 1, 2, 3, 9]])
print(f"Euclidean distance: {np.sqrt((diff ** 2).sum()):.2f}")
```

```
                  customer A  customer B  squared diff  share of total
tenure_months           1.00       44.00      1849.000           0.026
orders_12m              2.00       10.00        64.000           0.001
spend_12m             630.30      578.82      2650.190           0.037
avg_order_value       315.15       57.88     66187.853           0.935
app_user                0.00        1.00         1.000           0.000
Euclidean distance: 266.00
```

94% from average order value. A brand-new customer with 2 orders and a 44-month app user with 10 count as close, because they spend about the same per order.

3 **Scaled, every column has a say (2m).** "After StandardScaler one standard deviation counts the same in every column."

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler().set_output(transform="pandas").fit(X_train)
a_s, b_s = scaler.transform(X_train.iloc[[0, 1]]).to_numpy()
share = pd.Series((a_s - b_s) ** 2, index=features)
print((share / share.sum()).round(3).sort_values(ascending=False).head(4))
print(f"Euclidean distance (scaled): {np.sqrt(((a_s - b_s) ** 2).sum()):.2f}")
```

```
avg_order_value    0.306
orders_12m         0.301
app_user           0.205
tenure_months      0.097
dtype: float64
Euclidean distance (scaled): 4.52
```

4 **Does it matter for predictions? (3m).** "Predict the AUC gap."

```
from sklearn.metrics import roc_auc_score
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import make_pipeline

for name, model in [("unscaled", KNeighborsClassifier(n_neighbors=15)),
                    ("scaled", make_pipeline(StandardScaler(), KNeighborsClass
    model.fit(X_train, y_train)
    auc = roc_auc_score(y_test, model.predict_proba(X_test)[:, 1])
    print(f"{name:9s} accuracy {model.score(X_test, y_test):.3f}   ROC AUC {au
```

```
unscaled  accuracy 0.851   ROC AUC 0.774
scaled    accuracy 0.858   ROC AUC 0.836
```

0.774 against 0.836, same algorithm, same k. Accuracy barely moves: Lesson 1.4 again.

5 **Probabilities are vote shares (2m).** "With 15 neighbors there are only 16 possible probabilities."

```
knn = make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=15)).fi
proba = knn.predict_proba(X_test)[:, 1]
print("first five churn probabilities:", proba[:5].round(3))
print("as counts out of 15:", (proba[:5] * 15).round().astype(int))
print("distinct probability values:", np.unique(proba).size)
```

```
first five churn probabilities: [0.133 0.467 0.133 0.067 0.133]
as counts out of 15: [2 7 2 1 2]
distinct probability values: 15
```

6 **Ask for the lookalikes (4m).** "kneighbors lives on the classifier, so transform the customer with the pipeline's scaler first."

```
riskiest = int(np.argmax(proba))
customer = X_test.iloc[[riskiest]]
distances, positions = knn[-1].kneighbors(knn[:-1].transform(customer))
lookalikes = X_train.iloc[positions[0]].assign(churned=y_train.iloc[positions[
                                                distance=distances[0].round(2))
print(customer[["tenure_months", "days_since_last_order", "discount_share", "a
print(lookalikes[["days_since_last_order", "discount_share", "app_user", "chur
print(f"lookalikes who churned: {lookalikes['churned'].sum()} of 15 -> probabi
```

```
      tenure_months  days_since_last_order  discount_share  app_user
4050             65                    364           0.432         0
      days_since_last_order  discount_share  app_user  churned  distance
4799                    364           0.231         0        1      1.70
4997                    220           0.484         0        1      2.00
2361                    254           0.558         0        1      2.09
2385                    364           0.110         0        1      2.10
lookalikes who churned: 14 of 15 -> probability 0.933
```

14 of 15 churned: a long gap and no app. "These 15 customers looked like her" is an explanation a retention agent understands. Privacy caveat: in a real system summarize, do not print other customers.

7 **The k dial (4m).** "Watch training and test AUC."

```
for k in [1, 5, 15, 51, 151, 501]:
    m = make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=k)).f
    train_auc = roc_auc_score(y_train, m.predict_proba(X_train)[:, 1])
    test_auc = roc_auc_score(y_test, m.predict_proba(X_test)[:, 1])
    print(f"k={k:3d}   train AUC {train_auc:.3f}   test AUC {test_auc:.3f}")
```

```
k=  1   train AUC 1.000   test AUC 0.645
k=  5   train AUC 0.932   test AUC 0.789
k= 15   train AUC 0.893   test AUC 0.836
k= 51   train AUC 0.865   test AUC 0.842
k=151   train AUC 0.848   test AUC 0.837
k=501   train AUC 0.833   test AUC 0.823
```

k = 1: training AUC 1.000 because every customer is its own nearest neighbor. Show the lesson's three decision maps (k = 1, 25, 301): islands, the 150-day cliff, broad strokes.

8 **Choose k by cross-validation (4m).** "Training rows only, as with alpha."

```
from sklearn.model_selection import StratifiedKFold, cross_val_score

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
for k in [5, 15, 25, 51, 101, 151]:
    scores = cross_val_score(make_pipeline(StandardScaler(), KNeighborsClassif
                             X_train, y_train, cv=cv, scoring="roc_auc")
    print(f"k={k:3d}   CV AUC {scores.mean():.3f} ± {scores.std():.3f}")
```

```
k=  5   CV AUC 0.778 ± 0.012
k= 15   CV AUC 0.826 ± 0.016
k= 25   CV AUC 0.834 ± 0.023
k= 51   CV AUC 0.839 ± 0.018
k=101   CV AUC 0.838 ± 0.021
k=151   CV AUC 0.835 ± 0.021
```

A flat plateau from 25 to 151: choose a range, not a magic number. Tuned kNN beats logistic regression's 0.823 on these columns, because it can follow the 150-day cliff.

9 **The curse of dimensionality (3m).** "Random points: nearest against farthest distance."

```
from sklearn.metrics import pairwise_distances

rng = np.random.default_rng(42)
for n_dims in [2, 10, 100, 1000]:
    points = rng.uniform(size=(1000, n_dims))
    d = pairwise_distances(points[:1], points[1:])[0]
    print(f"{n_dims:4d} dimensions: nearest / farthest distance = {d.min() / d
```

```
   2 dimensions: nearest / farthest distance = 0.010
  10 dimensions: nearest / farthest distance = 0.317
 100 dimensions: nearest / farthest distance = 0.715
1000 dimensions: nearest / farthest distance = 0.890
```

In 1,000 dimensions the nearest point is only 10% closer than the farthest: "nearest" stops meaning "similar".

10 **Noise hurts kNN, not the weighted model (4m).** "Add useless columns to both."

```
from sklearn.linear_model import LogisticRegression

for n_noise in [0, 5, 20, 50]:
    noise = pd.DataFrame(rng.normal(size=(len(X), n_noise)), index=X.index).ad
    X_noisy = pd.concat([X, noise], axis=1)
    Xn_train, Xn_test = X_noisy.loc[X_train.index], X_noisy.loc[X_test.index]
    aucs = []
    for clf in [KNeighborsClassifier(n_neighbors=51), LogisticRegression()]:
        m = make_pipeline(StandardScaler(), clf).fit(Xn_train, y_train)
        aucs.append(roc_auc_score(y_test, m.predict_proba(Xn_test)[:, 1]))
    print(f"{n_noise:2d} noise columns   kNN AUC {aucs[0]:.3f}   logistic AUC
```

```
 0 noise columns   kNN AUC 0.842   logistic AUC 0.823
 5 noise columns   kNN AUC 0.823   logistic AUC 0.824
20 noise columns   kNN AUC 0.790   logistic AUC 0.820
50 noise columns   kNN AUC 0.755   logistic AUC 0.816
```

Logistic regression gives noise near-zero weights; kNN cannot. "For kNN, feature selection is part of the model": the project tests a lean set for that reason.

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **No scaler, or a scaler fitted outside the pipeline.** Symptom: AUC near 0.77, neighbors that share only an order value. Coach: "Which column decides the distance? Print the shares."
- **k chosen by training AUC, or k = 5 left at the default.** Symptoms: "k = 1 is perfect"; a ranking full of ties. Coach: "On which rows? How many distinct probabilities can k = 5 give?"
- **Lookalikes searched with an unscaled row, or too many columns** (`kneighbors`). Symptom: lookalikes that look nothing like the customer. Coach: "Did the customer go through the same scaler as the stored rows?" (`debug_knn.py` has exactly this bug.)

GUIDED LAB

- **Lab A (12m, everyone):** `labs/module-07/l4_1-knn/fraud_neighbors_starter.py`; reference `labs/module-07/l4_1-knn/fraud_neighbors_solution.py`. kNN on fraud, where dollars, days and 0/1 flags meet in one distance.
- **Expected, Lab A:** account age makes 99.7% of the first two payments' unscaled distance; unscaled kNN AUC 0.659, AP 0.056; scaled AUC 0.812, AP 0.397; logistic regression AUC 0.848, AP 0.444; CV AP 0.362, 0.448, 0.468 for k = 5, 51, 201; k = 5 gives 6 distinct probabilities (14,588 test payments at 0) and test AP 0.329; k = 201 gives 43 and test AP 0.419.
- **Debrief Lab A:** "Scaling multiplied AP by seven. Tuned kNN still trails logistic regression on fraud: nine columns, some weak, every one with an equal vote."

- **Lab B (8m, pairs):** `labs/module-07/l4_1-knn/lookalike_days_starter.py`; reference `labs/module-07/l4_1-knn/lookalike_days_solution.py`. KNeighborsRegressor forecasts Boise's 2025 days from lookalike days in 2023 and 2024.
- **Expected, Lab B:** weekday mean $1,669; raw dow and day-of-year unscaled $1,889, scaled $1,327; cyclical calendar plus events: k = 1 $1,389, k = 5 $1,244, k = 15 $1,256, always under-forecasting ($+517 to $+1,016); Boise grew $6,831 → $7,821 → $8,342 a day; trained on 2024 only, k = 5: $1,202 (+$458).
- **Debrief Lab B:** "An average of past days can never forecast a level it has never seen. kNN cannot extrapolate a trend; a linear model can (Session 6)."
- **Fast finishers:** `labs/module-07/l4_1-knn/debug_knn.py`: two loud bugs (`ValueError: Expected n_neighbors <= n_samples_fit, but n_neighbors = 51, n_samples_fit = 40, n_samples = 3`; `ValueError: Input X contains NaN.`) and two silent ones (no scaler: CV AUC 0.787; `kneighbors` on an unscaled row: 39 of 51 lookalikes with a median of 364 days). Fixed: CV AUC 0.854, 3 of 51.

START THE LESSON PROJECT

The lesson project is **lookalike churn scoring**: a shortlist where every name comes with the lookalikes that justify it. In the room, Steps 1 to 3.

- Step 1: `4500 1500 test churn rate 0.165`. Step 2: baseline AUC 0.500, logistic regression 0.823.
- Step 3: features and k by CV: the lean six with k = 51 and distance weights wins (CV AUC 0.8552).
- **Goes home:** Step 4 (test AUC 0.855; 31 churners in the top 50), Step 5 (a reason per customer from a summary of lookalikes) and Step 6 (`lookalike_shortlist.csv`).

CHECKS FOR UNDERSTANDING

- Why is the k = 1 training AUC exactly 1.000? **Answer:** every training customer is its own nearest neighbor.
- What does `predict_proba` return for kNN with k = 15? **Answer:** the share of the 15 neighbors in each class.
- Name two things kNN needs that logistic regression does not. **Answer:** scaled features (distance is the model) and the stored training rows at prediction time.

Workbook self-check, items Q1 to Q6.

Answers: **Q1** `[[0.333 0.667]]`; **Q2** A) Variance falls and bias rises: predictions drift toward the overall churn rate; **Q3** C) Account age runs up to 7,030 days, so it alone decided who the neighbors were; the flags that signal fraud had no say; **Q4** B) kNN gives every scaled column an equal say in the distance; logistic regression learns near-zero weights for useless columns; **Q5** `[[5.]]`; **Q6** D) The scaler and every training customer, because each prediction searches them. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: do TODO 1 of Lab A together; keep the two dials on the board and ask which one each step touches.
Stretch: the lesson's Exercise 4 (backward elimination by hand) and the Bonus Challenge (feature weights before the distance).

4.2 Naive Bayes

75 min

OBJECTIVES (FROM THE LESSON PAGE)

- Compute a posterior probability by hand with Bayes' rule, starting from counts in a table
- Explain the "naive" independence assumption and show what it costs when features repeat each other
- Turn review text into word counts with `CountVectorizer` and classify it with `MultinomialNB` inside a pipeline
- Explain what the smoothing parameter `alpha` prevents, and choose between the Multinomial, Bernoulli, Complement and Gaussian variants
- Tell a model that ranks well apart from one whose probabilities you can take at face value

TIMING IN THE ROOM

10m	Frame: the seasoned store manager; prior, clues, posterior; the naive habit
30m	Live demo: Steps 1 to 10
20m	Guided lab: `fraud_flags_bayes_starter.py` (everyone), then `review_category_starter.py` in pairs
15m	Start the lesson project: Steps 1 to 3

MENTAL MODEL TO SAY

"A store manager has read every comment card for years. Before reading a new one, they have a hunch: three cards in four are happy. That is the prior. Then they read clue by clue. 'Returned' nudges them strongly toward unhappy; 'love' nudges them back. Each clue multiplies the odds. At the end the hunch has become a judgment: the posterior. The manager has one habit: they weigh each word on its own, as if words did not influence each other. That is the naive part. Often wrong, still surprisingly good."

Write Bayes' rule and, under it, the odds form: **posterior odds = prior odds × likelihood ratio × likelihood ratio × ...**

LIVE DEMO (YOU TYPE, THEY FOLLOW)

New notebook `lesson_4_2`.

1. **Reviews, a clean yes or no (2m).** "Drop the mixed 3-star reviews; split as always."

```
import pandas as pd
from sklearn.model_selection import train_test_split

reviews = pd.read_csv("data/reviews.csv")
print(reviews.shape)
print(reviews["rating"].value_counts().sort_index().to_dict())

labeled = reviews[reviews["rating"] != 3].copy()
labeled["positive"] = (labeled["rating"] >= 4).astype(int)
train, test = train_test_split(labeled, test_size=0.25, stratify=labeled["posi
X_train, y_train = train["text"], train["positive"]
X_test, y_test = test["text"], test["positive"]
print(len(labeled), "labeled reviews,", f"{labeled['positive'].mean():.1%} pos
print(len(train), "train,", len(test), "test")
```

```
(4000, 8)
{1: 230, 2: 558, 3: 905, 4: 1093, 5: 1214}
3095 labeled reviews, 74.5% positive
2321 train, 774 test
```

2 **Bayes' rule from counts (4m).** "Prior, likelihood, posterior, for one word."

```
has_word = X_train.str.contains(r"\breturned\b", case=False)
is_neg = y_train == 0
p_neg = is_neg.mean()
p_word_given_neg = has_word[is_neg].mean()
p_word_given_pos = has_word[~is_neg].mean()
p_word = has_word.mean()
posterior = p_word_given_neg * p_neg / p_word
print(f"prior P(negative)          {p_neg:.3f}")
print(f"P(word | negative)         {p_word_given_neg:.4f}")
print(f"P(word | positive)         {p_word_given_pos:.4f}")
print(f"posterior P(negative|word) {posterior:.3f}")
print(f"check by direct counting   {is_neg[has_word].mean():.3f}")
```

```
prior P(negative)          0.255
P(word | negative)         0.0406
P(word | positive)         0.0029
posterior P(negative|word) 0.828
check by direct counting   0.828
```

The likelihood ratio (about 14) lifts 0.25 to 0.83. With one clue, Bayes' rule equals counting. With 20 words no combination repeats: that is why we need the naive assumption.

3 **A bag of words (2m).** "CountVectorizer: one column per word, word order thrown away."

```
from sklearn.feature_extraction.text import CountVectorizer

demo = ["The tent leaked. Returned it.", "Great tent, great price."]
demo_vec = CountVectorizer()
demo_counts = demo_vec.fit_transform(demo)
print(demo_vec.get_feature_names_out())
print(demo_counts.toarray())
print(type(demo_counts).__name__)
```

```
['great' 'it' 'leaked' 'price' 'returned' 'tent' 'the']
[[0 1 1 0 1 1 1]
 [2 0 0 1 0 1 0]]
csr_matrix
```

4 **Sparse on purpose (1m).** "333 words, about 18 per review."

```
vec = CountVectorizer()
counts_train = vec.fit_transform(X_train)
print(counts_train.shape)
print(f"non-zero cells: {counts_train.nnz:,} of {counts_train.shape[0] * count
```

```
(2321, 333)
non-zero cells: 40,981 of 772,893
```

5 **MultinomialNB against the baseline (3m).** "Vectorizer and model in one pipeline, so the vocabulary is learned from training reviews only."

```
from sklearn.pipeline import make_pipeline
from sklearn.naive_bayes import MultinomialNB
from sklearn.dummy import DummyClassifier

baseline = DummyClassifier(strategy="most_frequent").fit(X_train, y_train)
nb = make_pipeline(CountVectorizer(), MultinomialNB())
nb.fit(X_train, y_train)
print(f"baseline accuracy    {baseline.score(X_test, y_test):.3f}")
print(f"Naive Bayes accuracy {nb.score(X_test, y_test):.3f}")

from sklearn.metrics import classification_report, roc_auc_score

pred = nb.predict(X_test)
print(classification_report(y_test, pred, target_names=["negative", "positive"
print(f"ROC AUC {roc_auc_score(y_test, nb.predict_proba(X_test)[:, 1]):.3f}")
```

```
baseline accuracy    0.745
Naive Bayes accuracy 0.857
              precision    recall  f1-score   support

    negative      0.692     0.787     0.736       197
    positive      0.924     0.880     0.902       577

    accuracy                          0.857       774
   macro avg      0.808     0.834     0.819       774
weighted avg      0.865     0.857     0.859       774

ROC AUC 0.921
```

Recall 0.787 for negative reviews, AUC 0.92: a strong ranking for a triage list.

6 **What it counted (3m).** "feature_log_prob_ row 1 minus row 0: each word's nudge."

```
vocab = nb.named_steps["countvectorizer"].get_feature_names_out()
mnb = nb.named_steps["multinomialnb"]
print(mnb.classes_, mnb.class_count_)
log_ratio = pd.Series(mnb.feature_log_prob_[1] - mnb.feature_log_prob_[0], ind
print("most negative:", log_ratio.head(5).round(2).to_dict())
print("most positive:", log_ratio.tail(5).round(2).to_dict())
```

```
[0 1] [ 591. 1730.]
most negative: {'save': -3.4, 'your': -3.4, 'very': -3.29, 'disappointed': -3.
most positive: {'stars': 3.43, 'from': 3.43, 'regrets': 3.46, 'no': 3.46, 'lov
```

"save" and "your" from "save your money"; "no" is POSITIVE because of "no regrets". The model knows counts, not meaning.

7 **Try your own reviews (3m).** "Predict before running."

```
new_reviews = [
    "Leaked on the first night and the zipper broke. Returned it.",
    "Light, warm and packs down small. Would buy again.",
    "Not bad at all for the price.",
]
for text, p in zip(new_reviews, nb.predict_proba(new_reviews)[:, 1]):
    print(f"P(positive) = {p:.3f}  {text}")
```

```
P(positive) = 0.063  Leaked on the first night and the zipper broke. Returned
P(positive) = 0.302  Light, warm and packs down small. Would buy again.
P(positive) = 0.745  Not bad at all for the price.
```

"Would buy again" reads negative word by word ("would not buy again" is common). Bag-of-words cannot see negation.

8 **Smoothing: never say never (4m).** "'love' never appears in a negative review."

```
love = counts_train[:, vec.vocabulary_["love"]].toarray().ravel()
print("'love' in negative reviews:", love[y_train.to_numpy() == 0].sum())
print("'love' in positive reviews:", love[y_train.to_numpy() == 1].sum())
tricky = ["Leaked at the seams, fell apart after three trips and I returned it
for alpha in [1e-10, 1.0]:
    model = make_pipeline(CountVectorizer(), MultinomialNB(alpha=alpha)).fit(X
    print(f"alpha={alpha:g}  P(negative), P(positive) = {model.predict_proba(t
```

```
'love' in negative reviews: 0
'love' in positive reviews: 106
alpha=1e-10  P(negative), P(positive) = [0. 1.]
alpha=1  P(negative), P(positive) = [0.973 0.027]
```

Without smoothing one word vetoes all the evidence. Write the formula: (count + alpha) / (total + alpha × vocabulary).

9 **Four flavors (2m).** "Counts, presence, complement."

```
from sklearn.naive_bayes import BernoulliNB, ComplementNB

variants = {
    "MultinomialNB": make_pipeline(CountVectorizer(), MultinomialNB()),
    "ComplementNB": make_pipeline(CountVectorizer(), ComplementNB()),
    "BernoulliNB": make_pipeline(CountVectorizer(binary=True), BernoulliNB()),
}
for name, model in variants.items():
    model.fit(X_train, y_train)
    auc = roc_auc_score(y_test, model.predict_proba(X_test)[:, 1])
    print(f"{name:14s} accuracy {model.score(X_test, y_test):.3f}  AUC {auc:.3
```

```
MultinomialNB  accuracy 0.857  AUC 0.921
ComplementNB   accuracy 0.835  AUC 0.921
BernoulliNB    accuracy 0.859  AUC 0.923
```

All within a couple of points on short reviews; try two or three.

10 **GaussianNB, and the cost of being naive (6m).** "A bell curve per feature per class on churn."

```
from sklearn.naive_bayes import GaussianNB

customers = pd.read_csv("data/customers_churn.csv")
features = ["tenure_months", "orders_12m", "spend_12m", "avg_order_value", "da
            "returns_12m", "return_rate", "support_tickets_12m", "discount_sha
Xc_train, Xc_test, yc_train, yc_test = train_test_split(
    customers[features], customers["churned"], test_size=0.25, stratify=custom
)
gnb = GaussianNB().fit(Xc_train, yc_train)
print("priors:", gnb.class_prior_.round(3))
learned = pd.DataFrame({"mean_stayed": gnb.theta_[0], "mean_churned": gnb.thet
                        "sd_stayed": gnb.var_[0] ** 0.5, "sd_churned": gnb.var
print(learned.loc[["days_since_last_order", "discount_share", "tenure_months"]

from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler

logreg = make_pipeline(StandardScaler(), LogisticRegression()).fit(Xc_train, y
for name, model in [("GaussianNB", gnb), ("logistic regression", logreg)]:
    auc = roc_auc_score(yc_test, model.predict_proba(Xc_test)[:, 1])
    print(f"{name:20s} accuracy {model.score(Xc_test, yc_test):.3f}  AUC {auc:

independent = [f for f in features if f not in ("spend_12m", "avg_order_value"
gnb_lean = GaussianNB().fit(Xc_train[independent], yc_train)
auc = roc_auc_score(yc_test, gnb_lean.predict_proba(Xc_test[independent])[:, 1
print(f"GaussianNB, {len(independent)} features: accuracy {gnb_lean.score(Xc_t

p_gnb = gnb.predict_proba(Xc_test)[:, 1]
confident = p_gnb > 0.9
print(f"GaussianNB says above 90% for {confident.sum()} customers; {yc_test[co
```

```
priors: [0.835 0.165]
                       mean_stayed  mean_churned  sd_stayed  sd_churned
days_since_last_order        68.98        156.94      71.21      117.90
discount_share                0.19          0.29       0.17        0.21
tenure_months                42.26         37.53      29.45       35.24
GaussianNB           accuracy 0.815  AUC 0.778
logistic regression  accuracy 0.848  AUC 0.823
GaussianNB, 6 features: accuracy 0.845  AUC 0.813
GaussianNB says above 90% for 102 customers; 60.8% of them churned
```

Churners averaged 157 days since their last order against 69; show the bell-curve chart on the page. Fewer columns, better model: duplicated evidence was counted twice. And above 90% sure, only 61% churned: rank with it, recalibrate before quoting it.

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **The wrong variant for the input.** Raw text into the model (`could not convert string to float`), a scaler in front of MultinomialNB (`Negative values`), word counts into GaussianNB (`Sparse data`). Coach: "What does this variant expect: counts, flags or measurements?"
- **alpha = 0 or tiny.** One word vetoes the rest. Coach: "What is P(word | class) for a word never seen in that class?"
- **Probabilities quoted as facts, or stop words removed by habit.** Coach: "Above 90% sure: how many really churned?" and "Is 'not' a stop word?"

GUIDED LAB

- **Lab A (12m, everyone):** `labs/module-07/l4_2-naive-bayes/fraud_flags_bayes_starter.py`; reference `labs/module-07/l4_2-naive-bayes/fraud_flags_bayes_solution.py`. Bayes' rule on fraud warning signs, the naive product checked against direct counts, BernoulliNB on nine flags.
- **Expected, Lab A:** prior 0.0088; country mismatch 0.137 against 0.0121, posterior 0.091 by both routes; likelihood ratios from 11.3 (country mismatch) to 1.9 (new account); night + new device: naive 0.280 against counted 0.425 (193 payments); big amount + gift card: naive 0.185 against 0.508 (59); BernoulliNB AUC 0.810, AP 0.439 (21 payments above 0.9, all fraud); logistic regression AUC 0.810, AP 0.435 (8 above 0.9).
- **Debrief Lab A:** "Here the naive answer is too LOW: among frauds these flags come together (night on a new device), so treating them as independent misses that they reinforce each other. Independence can fail in either direction. Nine 0/1 flags, and Naive Bayes ranks as well as logistic regression."
- **Lab B (8m, pairs):** `labs/module-07/l4_2-naive-bayes/review_category_starter.py`; reference `labs/module-07/l4_2-naive-bayes/review_category_solution.py`. Ten classes: which product category is a review about?
- **Expected, Lab B:** baseline 0.237; MultinomialNB 0.586, ComplementNB 0.592, logistic regression 0.605; CV accuracy 0.579 to 0.583 for alpha 0.1 to 3; Tents' telling words start with canopy, camp, tent; most frequent mix-ups Camp Kitchen → Accessories (20), Apparel ↔ Paddling (20, 19); lowest recall Packs 0.39.

START THE LESSON PROJECT

The lesson project is **review triage for customer care**. In the room, Steps 1 to 3.

- Step 1: `2321 774 905 ['R02194', 'R02522', 'R03559']`. Step 2: word pairs help: CV AUC 0.899 → 0.912. Step 3: alpha 0.3 (0.9123), barely different from the others.
- **Goes home:** Step 4 (test accuracy 0.868, AUC 0.934, negative recall 0.843), Step 5 (rank the 905 three-star reviews; 457 read as negative) and Step 6 (60 possible star/text mismatches; two CSV files).

CHECKS FOR UNDERSTANDING

- Prior 0.25, likelihood ratio 3: posterior? **Answer:** odds 1/3 × 3 = 1, so 0.5.

- Why does MultinomialNB refuse scaled data? **Answer:** it models counts; scaling makes negative numbers.
- AUC 0.81 but above 0.9 only 61% churned: use it how? **Answer:** for ranking; recalibrate before quoting probabilities.

Workbook self-check, items Q7 to Q12.

Answers: **Q7** D) About 0.71; **Q8** `[[1 2] [1 0]]`; **Q9** A) "love" never appeared in a negative training review, so with almost no smoothing its probability in that class is about 0, and one zero factor vetoes everything else; **Q10** C) GaussianNB, unscaled: it models continuous measurements with a bell curve per class; **Q11** B) The dropped columns repeat information already present, and Naive Bayes counts each one as a new, independent clue; **Q12** `True False`. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: do Lab A's TODO 1 on the projector with the odds form of Bayes' rule on the board. **Stretch:** the lesson's Exercise 4 (GaussianNB by hand) and the Bonus Challenge (`CalibratedClassifierCV` and the Brier score).

CLOSING

EXIT TICKET (3 QUESTIONS, NO NOTES, 4 MINUTES)

1. Why does kNN need scaled features, and why does MultinomialNB refuse them? **Answer:** kNN's distance adds raw differences, so units decide similarity; MultinomialNB models counts, and scaling creates negative values.
2. k = 1 gives training AUC 1.000 and test AUC 0.645. Name the problem and the fix. **Answer:** high variance (each customer is its own neighbor); choose a larger k by cross-validation.
3. Prior P(negative) = 0.25; a word appears in 8% of negative and 2% of positive reviews. Posterior? **Answer:** odds $1/3 \times 4 = 4/3$, so $P \approx 0.57$.

HOMEWORK (ABOUT 2 TO 3 HOURS)

- **Finish both lesson projects.** 4.1 Steps 4 to 6 (test AUC 0.855, a reason for every name, `lookalike_shortlist.csv`); 4.2 Steps 4 to 6 (accuracy 0.868, AUC 0.934; 3-star ranking; mismatches; `triage_three_star.csv`, `rating_mismatches.csv`).
- **Lesson exercises.** 4.1: Exercises 1 to 3 (distances by hand, Manhattan or Euclidean, kNN for regression); 4.2: Exercises 1 to 3 (Bayes for "regret", how small can the training set be, fool the model). Exercise 4 of each is a stretch.
- **Both Quick Quizzes and both journal entries.**

- **Before the Next Lesson:** 4.1: MinMaxScaler instead of StandardScaler; predict, then check, a duplicated recency column. 4.2: read five mismatches; predict BernoulliNB on long manuals.
- **Labs not finished in class;** optional `debug_knn.py`.

BRING LIST FOR SESSION 8

- A laptop on which `check_environment.py` ends with READY (nothing new to install).
- The `pinecrest-ml` folder with `data/` and `labs/`; Session 8's starters are in `labs/module-08/`.
- Your 4.1 project notebook and `lookalike_shortlist.csv` (Session 8 compares support vector machines and trees with today's models on churn) and your 4.2 notebook (the review split returns in Lesson 11.1).
- The workbook and the journal. No downloads.

IF YOU RUN LONG OR SHORT

RUNNING LONG: WHAT TO PROTECT AND WHAT TO CUT FIRST

- **Protect:** 4.1 Steps 2 to 4 and 10; 4.2 Steps 2, 8 and 10.
- **Cut first (saves 6m):** 4.1 Step 9 (one sentence) and 4.2 Steps 3 and 4 (show the page).
- **Cut second (saves 4m):** 4.2 Step 9 (variants: show the table).
- **Cut third (saves 8m per lesson):** Lab B moves to homework.

RUNNING SHORT: WHAT TO ADD

- 4.1 Exercise 3 live: kNN regression for snowshoes against the weather.
- 4.2 Exercise 2 live: Naive Bayes against logistic regression as the training set grows.
- Run `debug_knn.py` together.

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 8: Support Vector Machines and Decision Trees

How to teach Session 8 of Machine Learning Foundations: support vector machines (the widest street, support vectors, the C and gamma dials, the kernel trick, scaling, class weights, calibration) and decision trees (signposts and campsites, Gini, leaves as probabilities, overfitting and its controls, importances and instability). Learners leave with a tuned SVM shortlist started, a churn rule book started, and a fraud rule book of their own.

`2 lessons` `3 hrs session` `Lessons 4.3-5.1` `Session 8 of 11`

HOW TO USE THIS GUIDE

- Same rhythm for both lessons: frame (one mental model) → live demo (you type, they follow) → guided lab (kit labs, new practice) → start the lesson project. Exercises, quizzes and journals are homework.
- Every code block below was run to produce the output printed under it (Python 3.12, pandas 3.0, scikit-learn 1.9.1, 4 CPU threads). The demo code is the lesson pages' code with the plotting lines left out: show the pictures from the lesson page (the street with C = 0.01 and 100, the ring with both kernels, the moons with three gammas, the tree drawn with `plot_tree`, the depth curve).
- Keep `classification_svm.html` and `trees_decision.html` open in browser tabs. Labs live in `labs/module-08/`: hand out the `_starter.py` and `debug_*.py` files, keep the `_solution.py` files.
- Session 8 is the hinge of the course: the last of Module 4's four classifiers, then the first tree. Today's tree is the building block of Session 9's forests and boosting, so protect the 5.1 demo.

> **THE #1 RISK OF SESSION 8: AN SVM DEMO THAT DROWNS IN MATH**
>
> The lesson needs no formulas beyond the RBF similarity, and the room does not either. Teach with the street picture, the two dials (C, gamma) and the ring. If someone asks about Lagrange multipliers or the dual problem, give one sentence ("the training only ever needs the similarity of each pair of points; that is why kernels work") and offer to talk at the break. The second risk is time: the 4.3 project's grid search (45 fits) takes 30 to 60 seconds on a laptop. Start it, then talk about what the table will show while it runs.

BEFORE THE SESSION

- ☐ **Run the kit check:** `python3 labs/check_labs.py module-08` from this kit's folder. Every line PASS (8 files, about 30 seconds).

- ☐ **Downloads:** none. Everything today uses the Pinecrest CSV files already in `data/`.
- ☐ **Projector machine:** `(.venv)` active in your own `pinecrest-ml`, JupyterLab started there, two empty notebooks: `lesson_4_3` and `lesson_5_1`. Browser tabs: `classification_svm.html`, `trees_decision.html`, and the Session 7 lesson pages for the warm-up.
- ☐ **Board, before learners arrive:** a street between two neighborhoods with its two curbs (dashed) and a few circled curb houses; under it "C = price of a violation · gamma = how far each point reaches". On the right, an empty trail map with a first signpost.
- ☐ **Print or share:** the Session 8 pages of the workbook; sticky notes for the exit ticket.
- ☐ **Pre-run the 4.3 project grid once on the projector machine** so you know its time (in the kit's measurements, under a minute).

KNOWN ENVIRONMENT TRAPS

SYMPTOM	CAUSE	FIX
`AttributeError: This 'Pipeline' has no attribute 'predict_proba'`	An SVM has no probabilities	`decision_function` for ranking and AUC; `CalibratedClassifierCV(SVC(), ensemble=False)` for probabilities.
`FutureWarning` about `probability=True`	Code copied from an older tutorial	Remove `probability=True`; calibrate as above.
`ValueError: Invalid parameter 'C' for estimator Pipeline(...)`	Grid key without the step name	`svc__C`, `svc__gamma` (lowercase class name, two underscores).
A grid search that seems frozen	45 kernel-SVM fits; or `n_jobs=-1` on a busy laptop	Wait up to a minute; check that the scaler is in the pipeline (unscaled SVMs can be much slower to converge).
`InvalidParameterError: The 'max_depth' parameter ... Got 0 instead.`	max_depth=0	Use 1 or more, or None.
`ValueError: could not convert string to float: 'Bend'`	A text column given to a tree	One-hot encode it (the 5.1 project's `tree_prep`).
Tree rules that look backwards (big safe leaf labeled churned)	`class_names` not in the order of `classes_`	Build the names from `model.classes_`.
`plot_tree` text overlapping or tiny	Figure too small for the depth	`figsize=(10, 5)` and depth 2 or 3 for display; use `export_text` for bigger trees.

SESSION 8 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Five retrieval questions on Lessons 4.1 and 4.2; homework check (lookalike shortlist, review triage).
0:10	10	**4.3** Frame and mental model	The widest street between two neighborhoods; support vectors; the two dials C and gamma.
0:20	30	**4.3** Live demo	Support vectors and C on blobs, the ring lifted by hand, the RBF kernel, gamma overfitting, scaling, a fair comparison on churn, calibration, two errors.
0:50	25	**4.3** Guided lab	`labs/module-08/l4_3-svm/review_svm_starter.py`, then `labs/module-08/l4_3-svm/kernel_by_hand_starter.py` in pairs.
1:15	10	**4.3** Start the lesson project	The tuned SVM retention list, Steps 1 to 3 (the grid search runs while you talk); Steps 4 to 6 go home.
1:25	10	Break	Leave the street and the two dials on the board.
1:35	10	**5.1** Frame and mental model	Trail junction signposts: questions, trails, campsites; Gini impurity.
1:45	30	**5.1** Live demo	The depth-2 churn tree, Gini by hand, every threshold, stumps, leaf probabilities, no scaling, depth and leaf size by CV, importances, instability, two mistakes.
2:15	25	**5.1** Guided lab	`labs/module-08/l5_1-decision-trees/fraud_rules_starter.py`, then `labs/module-08/l5_1-decision-trees/sales_tree_starter.py` in pairs.
2:40	10	**5.1** Start the lesson project	The churn rule book, Steps 1 to 3; Steps 4 to 6 go home.
2:50	10	Wrap-up	Exit ticket (3 questions), homework, bring list for Session 9.
Total	180		Ends at 3:00.

In the room. The lesson pages estimate 100 + 100 minutes for a solo learner; after the warm-up, the break and the wrap-up the room has 150. So each lesson gets 75 minutes: the mental model, the live demo, the guided labs and the first steps of the project.

Goes home (2 to 3 hours). The SVM retention list Steps 4 to 6 and the churn rule book Steps 4 to 6; each lesson's Hands-on Exercises 1 to 3 (Exercise 4 is a stretch); both Quick Quizzes; both journal entries; both "Before the Next Lesson" lists; any unfinished lab.

Protect, in this order: the 4.3 demo Steps 4 to 6 (lift, kernel, gamma: the kernel idea and the dial); the 5.1 demo Steps 1, 5 and 7 (the tree as rules, leaves as probabilities, the depth curve); Lab A of each lesson. **Cut first:** 4.3 demo Steps 9 to 11 (say the three fixes), then 5.1 Steps 3, 6 and 10 to 12, then the paired Lab B of each lesson (homework).

WARM-UP

0:00 Warm-up: Nearest Neighbors and Naive Bayes — 10 min

RUN IT

Put the five questions on the slide. Two minutes alone, one minute with a neighbor, then cold-call gently. While they work, walk the room for the Session 7 homework: the `lookalike_shortlist.csv` (Lesson 4.1 project) and the review triage files (Lesson 4.2 project).

- **W1** Lesson 4.1: a kNN churn model scores a perfect 1.000 on its training data with k=1. Why, and what do you do about it? **Answer:** Each training customer is its own nearest neighbor, so k=1 is right by construction (high variance). Choose a larger k by cross-validation; the 4.1 project chose k=51. Ask for the word: overfitting. Today's two models have their own versions: gamma for SVMs, depth for trees.
- **W2** Two unscaled features: spend differs by $300 and days since last order by 120. What share of the squared distance comes from spend? Predict, then check. **Answer:** 0.86 (86%). $300^2 = 90{,}000$ against $120^2 = 14{,}400$: spend takes 86% of the distance purely because of its units. The same reason SVMs need a scaler today.
- **W3** Lesson 4.2: a word appears in 6% of negative reviews and 0.5% of positive ones; 25% of reviews are negative. After seeing the word, about how likely is the review to be negative? **Answer:** About 0.8: the odds go from 1:3 to $(1 \times 6) : (3 \times 0.5) = 6 : 1.5 = 4 : 1$. Prior odds times the likelihood ratio (12). Anyone stuck: draw the 1,000-review table.
- **W4** Why does MultinomialNB need alpha above 0? **Answer:** Without smoothing a word never seen in one class gives that class a probability of exactly 0, which vetoes all the other evidence. Then bridge: today's SVM on reviews (Lab A) needs no smoothing, but it needs scaling, in its text form (TF-IDF).
- **W5** Your Naive Bayes model ranks customers well (AUC 0.81) but its probabilities pile up near 0 and 1. Can you quote them to the care team? **Answer:** Rank with it, but recalibrate (CalibratedClassifierCV) before quoting probabilities. Today's SVM gives no probabilities at all, and the same tool fixes it.

BRIDGE TO TODAY

"kNN asked the closest past customers, Naive Bayes multiplied clues. Today: a model that draws the widest possible boundary, and a model that asks yes-or-no questions. By the end of the day you will have five classifiers and an honest way to compare them."

LESSON PLANS

4.3 Support Vector Machines

75 min

OBJECTIVES (FROM THE LESSON PAGE)

- Explain the maximum-margin idea and find the support vectors of a fitted model
- Predict how changing C and gamma changes the margin, the number of support vectors and the risk of overfitting
- Explain the kernel trick and use the RBF kernel to learn a curved boundary
- Build a scaled SVC pipeline, rank with `decision_function`, handle imbalance with `class_weight`, and tune C and gamma with cross-validation
- Decide when an SVM is a sensible choice and when its training time rules it out

TIMING IN THE ROOM

10m	Frame: the widest street; support vectors; C and gamma
30m	Live demo: Steps 1 to 12
25m	Guided lab: `review_svm_starter.py` (13m), then `kernel_by_hand_starter.py` in pairs (12m)
10m	Start the lesson project: Steps 1 to 3

MENTAL MODEL TO SAY

Point at the board: "Two neighborhoods, red and blue, and a city planner who must build a straight street between them. Many streets would work. The planner picks the **widest** one, because a wide street leaves room for error: a new house built near either edge still lands on the right side."

Circle the curb houses: "Only these decided where the street went. The houses deep inside could move and nothing would change. They are the **support vectors**, and they are all the model remembers."

Then the two extensions, one per dial: "Real neighborhoods overlap, so we allow some houses on the street, at a price: that price is **C**. Real boundaries curve, so we bend the street with a kernel; for the usual RBF kernel, **gamma** says how far each house's influence reaches."

LIVE DEMO (YOU TYPE, THEY FOLLOW)

New notebook `lesson_4_3`. Ask for a prediction before every run that prints a number: the street gets narrower as C rises, and gamma's test score collapses while its training score climbs.

1. **The widest street (3m).** "Many lines separate these two clouds. The SVM picks the widest street that fits, and only the points on its curbs decide where it goes." Show the lesson's first picture, then run.

```
import numpy as np
import pandas as pd
from sklearn.datasets import make_blobs
from sklearn.svm import SVC

X_toy, y_toy = make_blobs(n_samples=80, centers=[(-2, 0), (2, 0)], cluster_std

svm = SVC(kernel="linear", C=1).fit(X_toy, y_toy)
print("support vectors per class:", svm.n_support_)
print("margin width:", round(2 / np.linalg.norm(svm.coef_[0]), 2))
print("training accuracy:", svm.score(X_toy, y_toy))
```

```
support vectors per class: [6 5]
margin width: 1.34
training accuracy: 0.9625
```

80 points, 11 support vectors: the other 69 could move freely and nothing would change. Ask: "Why not 100% training accuracy?" (the clouds overlap; no straight line separates them).

2 **C prices the violations (3m).** "C is what a point inside the street costs. Cheap violations: a wide street held up by many points. Expensive: a narrow street that bends toward individual points."

```
for C in [0.01, 0.1, 1, 100]:
    model = SVC(kernel="linear", C=C).fit(X_toy, y_toy)
    width = 2 / np.linalg.norm(model.coef_[0])
    print(f"C={C:<6} support vectors {model.n_support_.sum():2d}   margin widt
```

```
C=0.01   support vectors 42   margin width 4.06
C=0.1    support vectors 18   margin width 2.11
C=1      support vectors 11   margin width 1.34
C=100    support vectors  7   margin width 0.75
```

Write on the board: small C = strong regularization = wide street. It is the same C as logistic regression's, the inverse of Lesson 3.2's alpha.

3 **A problem no line can solve (2m).** "One class makes a ring around the other. Predict the linear SVM's accuracy."

```
from sklearn.datasets import make_circles
from sklearn.model_selection import train_test_split

X_ring, y_ring = make_circles(n_samples=300, noise=0.08, factor=0.5, random_st
Xr_train, Xr_test, yr_train, yr_test = train_test_split(X_ring, y_ring, test_s

linear_ring = SVC(kernel="linear").fit(Xr_train, yr_train)
print(f"linear SVM, test accuracy {linear_ring.score(Xr_test, yr_test):.3f}")
```

```
linear SVM, test accuracy 0.478
```

Worse than a coin flip.

4 **Lift it by hand (3m).** "What separates them? Distance from the center. Add it as a third feature and a flat plane in 3-D separates them perfectly."

```
def add_radius(X):
    return np.column_stack([X, (X ** 2).sum(axis=1)])

lifted = SVC(kernel="linear").fit(add_radius(Xr_train), yr_train)
print(f"linear SVM on (x, y, x²+y²), test accuracy {lifted.score(add_radius(Xr
```

```
linear SVM on (x, y, x²+y²), test accuracy 1.000
```

Draw the bowl: inner points low, outer points high, a flat sheet between them. Projected back, the flat sheet is a circle.

5 **Let the kernel do it (3m).** "With ten customer features you cannot see which feature to add. The kernel trick never builds the features: it only needs how similar each pair of points is, as if they had been lifted." Write the RBF similarity: exp(-gamma × squared distance).

```
rbf_ring = SVC(kernel="rbf").fit(Xr_train, yr_train)
print(f"RBF SVM, test accuracy {rbf_ring.score(Xr_test, yr_test):.3f}, support
```

```
RBF SVM, test accuracy 1.000, support vectors 42
```

Lab B (`kernel_by_hand`) does exactly this comparison on the churn table: 66 hand-built columns against a kernel that builds none.

6 **gamma is how far each point reaches (4m).** "Small gamma: each support vector's influence spreads wide. Huge gamma: a tiny bubble around each point." Ask for a prediction of the test score at gamma=1000 before you run it.

```
from sklearn.datasets import make_moons

X_moon, y_moon = make_moons(n_samples=400, noise=0.3, random_state=42)
Xm_train, Xm_test, ym_train, ym_test = train_test_split(X_moon, y_moon, test_s

for gamma in [0.1, 1, 10, 100, 1000]:
    model = SVC(kernel="rbf", gamma=gamma).fit(Xm_train, ym_train)
    print(f"gamma={gamma:<5} train {model.score(Xm_train, ym_train):.3f}  test
          f"  support vectors {model.n_support_.sum()}")
```

```
gamma=0.1   train 0.814  test 0.908  support vectors 141
gamma=1     train 0.882  test 0.925  support vectors 96
gamma=10    train 0.921  test 0.900  support vectors 141
gamma=100   train 0.975  test 0.867  support vectors 259
gamma=1000  train 1.000  test 0.650  support vectors 280
```

Training climbs to 1.000, test collapses to 0.650, and the support-vector count rises at the end: almost every point must be remembered. Same overfitting picture as Lesson 3.2, new dial.

7 **Scaling is not optional (3m).** "The RBF kernel is distances, and Lesson 4.1 showed what unscaled distances do." Note `decision_function` in the AUC line: AUC only needs a ranking score.

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import roc_auc_score

customers = pd.read_csv("data/customers_churn.csv")
features = ["tenure_months", "orders_12m", "spend_12m", "avg_order_value", "da
            "returns_12m", "return_rate", "support_tickets_12m", "discount_sha
X_train, X_test, y_train, y_test = train_test_split(
    customers[features], customers["churned"], test_size=0.25, stratify=custom
)

for name, model in [("unscaled SVC", SVC()), ("scaled SVC", make_pipeline(Stan
    model.fit(X_train, y_train)
    auc = roc_auc_score(y_test, model.decision_function(X_test))
    print(f"{name:13s} accuracy {model.score(X_test, y_test):.3f}  ROC AUC {au
```

```
unscaled SVC  accuracy 0.839  ROC AUC 0.767
scaled SVC    accuracy 0.859  ROC AUC 0.814
```

0.767 to 0.814 from one line. The scaler goes INSIDE the pipeline (Lesson 2.2) so cross-validation never peeks.

8 **A fair comparison on churn (4m).** "Baseline, logistic regression, an RBF SVM, and the same SVM told that a missed churner costs more."

```
from sklearn.dummy import DummyClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import average_precision_score, recall_score

def report(name, model):
    model.fit(X_train, y_train)
    if hasattr(model, "decision_function"):
        scores = model.decision_function(X_test)
    else:
        scores = model.predict_proba(X_test)[:, 1]
    print(f"{name:22s} accuracy {model.score(X_test, y_test):.3f}  AUC {roc_au
          f"  AP {average_precision_score(y_test, scores):.3f}  recall {recall

report("baseline", DummyClassifier(strategy="most_frequent"))
report("logistic regression", make_pipeline(StandardScaler(), LogisticRegressi
report("SVC (RBF)", make_pipeline(StandardScaler(), SVC()))
report("SVC (RBF, balanced)", make_pipeline(StandardScaler(), SVC(class_weight
```

```
baseline               accuracy 0.835  AUC 0.500  AP 0.165  recall 0.000
logistic regression    accuracy 0.848  AUC 0.823  AP 0.497  recall 0.266
SVC (RBF)              accuracy 0.859  AUC 0.814  AP 0.540  recall 0.258
SVC (RBF, balanced)    accuracy 0.792  AUC 0.850  AP 0.495  recall 0.774
```

Three things: the plain SVM has the best accuracy and a WORSE AUC than logistic regression; `class_weight="balanced"` jumps recall from 0.258 to 0.774 and AUC to 0.850; accuracy falls because it flags many more stayers. Ask: "Which number would the care team care about?"

9 **Probabilities, when you need them (2m).** "An SVM scores distances. When the team wants a probability, for revenue at risk, calibrate."

```
from sklearn.calibration import CalibratedClassifierCV

svm_proba = make_pipeline(StandardScaler(), CalibratedClassifierCV(SVC(class_w
svm_proba.fit(X_train, y_train)
p = svm_proba.predict_proba(X_test)[:, 1]
print("first five churn probabilities:", p[:5].round(3))
print(f"average predicted churn {p.mean():.3f} vs real churn rate {y_test.mean
print(f"ROC AUC {roc_auc_score(y_test, p):.3f}")
```

```
first five churn probabilities: [0.066 0.484 0.105 0.148 0.046]
average predicted churn 0.164 vs real churn rate 0.165
ROC AUC 0.850
```

Average 0.164 against a real 0.165, AUC unchanged. Mention once: older code uses `SVC(probability=True)`; it is deprecated now (FutureWarning in scikit-learn 1.9).

10 **Error 1: asking for probabilities (1m).** "Read the last line."

```
make_pipeline(StandardScaler(), SVC()).fit(X_train, y_train).predict_proba(X_t
```

```
AttributeError: This 'Pipeline' has no attribute 'predict_proba'
```

The pipeline passes the question to its last step, hence "Pipeline".

11 **Error 2: a grid key without the step name (1m).** "The grid applies to the pipeline, and the pipeline has no C."

```
from sklearn.model_selection import GridSearchCV

GridSearchCV(make_pipeline(StandardScaler(), SVC()), {"C": [1, 10]}).fit(X_tra
```

```
ValueError: Invalid parameter 'C' for estimator Pipeline(steps=[('standardsca
```

Fix: `svc__C`. `pipe.get_params().keys()` lists every valid key. The project needs this in ten minutes.

12 **The silent one (1m).** "No error, and that is the problem."

```
silent = make_pipeline(StandardScaler(), SVC(kernel="linear")).fit(X_train, y_
print(f"accuracy {silent.score(X_test, y_test):.3f}")
print("customers predicted to churn:", (silent.predict(X_test) == 1).sum())
```

```
accuracy 0.835
customers predicted to churn: 0
```

Accuracy exactly the baseline's, zero churners predicted. Judge with AUC or recall, use class_weight, or choose a threshold. Skip the timing block live; say the rule: kernel SVMs up to about 10,000 rows, LinearSVC beyond (the lesson's table).

◆ THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Judging an SVM by accuracy on imbalanced data.** The linear SVM that predicts nobody churns scores 0.835, exactly the baseline. Coach: "How many churners did it flag?" Then AUC, recall, or `class_weight`.
- **Tuning by the training score.** gamma=10 on churn: training 0.996, test AUC 0.699, 4,491 of 4,500 customers as support vectors. Coach: "What share of the customers are support vectors? What does that say?" Choose C and gamma by CV.
- **Probabilities from an SVM.** `predict_proba` raises; `probability=True` warns. Coach: "Do you need a probability or a ranking?" Ranking: `decision_function`. Probability: calibrate.

GUIDED LAB

- **Lab A (13m, everyone):** `labs/module-08/l4_3-svm/review_svm_starter.py`; reference `labs/module-08/l4_3-svm/review_svm_solution.py`. Session 7's reviews and split, now with a linear SVM: rank with `decision_function`, "scale" word counts with TF-IDF, choose C by CV with the key `linearsvc__C`, read the weights, count reviews inside the street, calibrate.
- **Expected, Lab A:** raw counts accuracy 0.859, AUC 0.921 (worse than Naive Bayes' 0.868 / 0.934); TF-IDF 0.884 / 0.936 with 1,856 features; CV AUC peaks at C=1 (0.9269); strongest words `not` (-2.51) and `would buy` (1.93); 445 of 774 reviews inside the street, 90 on the wrong side; calibrated average 0.734 vs 0.745.
- **Circulate in Lab A for:** `roc_auc_score(y_test, svm.predict(X_test))` (0/1 labels: a much lower AUC; ask "how many distinct scores does that give?"); the key typed `LinearSVC__C` (case matters: the step name is lowercase). **Debrief:** "Raw counts lost to Naive Bayes; TF-IDF won. For text, TF-IDF is the scaler." And the odd weight: `late it` comes from "It arrived late. It ...": word pairs ignore sentence boundaries, so read weights skeptically.
- **Lab B (12m, pairs):** `labs/module-08/l4_3-svm/kernel_by_hand_starter.py`; reference `labs/module-08/l4_3-svm/kernel_by_hand_solution.py`. The ring's lift, on churn: `PolynomialFeatures(2)` builds 66 columns for a straight LinearSVC; a degree-2 polynomial kernel reaches the same AUC without building any.
- **Expected, Lab B:** LinearSVC 0.826; lift + LinearSVC 0.852; poly kernel 0.852 (2,293 support vectors); 66 / 286 / 1,001 / 3,003 columns for degrees 2 to 5; RBF 0.850. Fit times differ per machine. Stretch: the biggest lifted weights include `tenure_months^2` (+0.31): the U-shape in tenure that a straight line could not draw.
- **Fast finishers:** `labs/module-08/l4_3-svm/debug_svm.py`: two loud bugs (grid key `C`; `predict_proba`) and two silent ones (no scaler; the top 50 taken with `np.argsort(scores)[:50]`, the LEAST risky). Fixed: best C 1, CV 0.848, test 0.850, 32 churners in the top 50.

START THE LESSON PROJECT

The **tuned SVM retention list** (lesson page, "Project: A Tuned SVM Retention List"). In the room: Steps 1 to 3 (10 minutes).

- Step 1: split the whole table, keep the IDs. **Check:** `4500 1500 ['C102112', 'C103063', 'C101159']`.
- Step 2: logistic regression CV AUC 0.829 ± 0.021, the bar to beat.
- Step 3: start the grid (C × gamma, 45 fits) and talk while it runs: which corner will overfit? **Done** = best `{'svc__C': 10, 'svc__gamma': 0.01}`, CV AUC 0.854.
- **Goes home:** Steps 4 to 6 (test AUC 0.848 vs 0.823; 34 vs 33 churners in the top 50, only 21 shared; `svm_shortlist.csv`).

CHECKS FOR UNDERSTANDING

Ask these during the demo and the labs:

- You delete a training point far from the boundary and refit. What changes? **Answer:** nothing: it was not a support vector.
- Which way do you move C to get a simpler boundary? **Answer:** down: cheaper violations, wider street.
- Why does `roc_auc_score` accept `decision_function` output? **Answer:** AUC only needs scores that rank; the signed distance ranks.
- When would you NOT use a kernel SVM? **Answer:** many rows (training time grows roughly with the square of the rows): use LinearSVC, logistic regression or trees.

Workbook self-check, items Q1 to Q6.

Answers: **Q1** B) A wider street held up by many more support vectors; **Q2** `['svc__gamma']`; **Q3** D) The kernel and the margin are built on distances, and unscaled `spend_12m` and `days_since_last_order` would drown out the 0-to-1 columns; **Q4** B) `AttributeError: This 'Pipeline' has no attribute 'predict_proba'`; **Q5** A) Recall of churners jumped (0.258 to 0.774) while accuracy fell (0.859 to 0.792); **Q6** `0.67 0.0`. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: draw the street for every question; let pairs copy the pipeline lines from the demo and spend the lab time on the numbers. In Lab A, a struggling pair can stop after TODO 3.

Stretch: Lab A's RBF stretch (no gain on text: 0.928); the lesson's Exercise 2 (polynomial kernels on the rings) and the Nystroem bonus on 50,000 transactions.

5.1 Decision Trees

75 min

OBJECTIVES (FROM THE LESSON PAGE)

- Explain how a classification tree chooses each split by reducing Gini impurity, and compute the impurity of a node by hand
- Train `DecisionTreeClassifier` and `DecisionTreeRegressor`, and read their rules with `export_text` and `plot_tree`
- Explain where a tree's probabilities come from and why trees do not need scaled features
- Control overfitting with `max_depth`, `min_samples_leaf`, `max_leaf_nodes` and cost-complexity pruning, chosen by cross-validation
- Read a tree's feature importances, including what they cannot tell you

TIMING IN THE ROOM

10m	Frame: trail junction signposts; Gini
30m	Live demo: Steps 1 to 12
25m	Guided lab: `fraud_rules_starter.py` (13m), then `sales_tree_starter.py` in pairs (12m)
10m	Start the lesson project: Steps 1 to 3

MENTAL MODEL TO SAY

Draw on the trail map: "At the trailhead a signpost asks: has it been more than about 170 days since your last order? Each answer sends the customer down a different trail, to the next signpost. Every trail ends at a campsite with a sign: the share of past customers who walked that way and then left."

"Training a tree means choosing which question to post at each junction: it tries every feature and every threshold and keeps the one that makes the two piles purest. Predicting means walking a new customer down the trails and reading the campsite's sign." Write **Gini = chance that two customers picked from a node disagree** next to the map.

LIVE DEMO (YOU TYPE, THEY FOLLOW)

New notebook `lesson_5_1`. The demo uses the 1.1 split and ten features, so every number can be compared with the SVM's.

1. **A tree that asks two questions (3m).** "Every model so far did arithmetic. A tree asks one yes-or-no question, splits the pile, and asks again in each pile." Draw the trail map with its signposts while it runs.

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, export_text

customers = pd.read_csv("data/customers_churn.csv")
features = ["tenure_months", "orders_12m", "spend_12m", "avg_order_value", "da
            "returns_12m", "return_rate", "support_tickets_12m", "discount_sha
X = customers[features]
y = customers["churned"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, stra

small_tree = DecisionTreeClassifier(max_depth=2, random_state=42)
small_tree.fit(X_train, y_train)
print(export_text(small_tree, feature_names=features, show_weights=True))
```

```
|--- days_since_last_order <= 172.50
|   |--- discount_share <= 0.35
|   |   |--- weights: [2845.00, 175.00] class: 0
|   |--- discount_share >  0.35
|   |   |--- weights: [620.00, 245.00] class: 0
|--- days_since_last_order >  172.50
|   |--- tenure_months <= 5.50
|   |   |--- weights: [31.00, 127.00] class: 1
|   |--- tenure_months >  5.50
|   |   |--- weights: [260.00, 197.00] class: 0
```

Read the weights as [stayed, churned]. Ask: "Which campsite would you call first?" (lapsed and brand new: 127 of 158 churned, 80%). Nobody typed 172.5: the tree found it.

2 **Gini by hand (3m).** "Pick two customers from a node at random; Gini is the chance they disagree. A good split makes purer children."

```
def gini(counts):
    p = np.asarray(counts) / np.sum(counts)
    return 1 - np.sum(p ** 2)

root = np.bincount(y_train)                                    # [stayed,
left = np.bincount(y_train[X_train["days_since_last_order"] <= 172.5])
right = root - left
weighted = (left.sum() * gini(left) + right.sum() * gini(right)) / root.sum()

print("root", root, " left", left, " right", right)
print(f"Gini root {gini(root):.4f}  left {gini(left):.4f}  right {gini(right):
print(f"weighted Gini of the children {weighted:.4f}, decrease {gini(root) - w

t = small_tree.tree_
print("scikit-learn's values:", t.impurity[[0, t.children_left[0], t.children_
```

```
root [3756  744]  left [3465  420]  right [291 324]
Gini root 0.2760  left 0.1928  right 0.4986
weighted Gini of the children 0.2346, decrease 0.0414
scikit-learn's values: [0.276  0.1928 0.4986]
```

Your arithmetic matches `tree_.impurity` exactly. The right child is MORE mixed than the root, but it holds only 615 customers, so it counts for less.

3 **Every threshold, tried (2m).** "Why 172.5 and not 150? It tried them all."

```
recency = X_train["days_since_last_order"].to_numpy()
thresholds = np.arange(0, 364) + 0.5
decrease = []
for cut in thresholds:
    left = np.bincount(y_train[recency <= cut], minlength=2)
    right = root - left
    weighted = (left.sum() * gini(left) + right.sum() * gini(right)) / root.su
    decrease.append(gini(root) - weighted)

best = thresholds[np.argmax(decrease)]
print(f"best recency threshold: {best}  (impurity decrease {max(decrease):.4f}
```

```
best recency threshold: 172.5  (impurity decrease 0.0414)
```

Show the lesson's decrease-by-threshold chart: flat below 100 days, a peak at 172.5.

4 **Every feature competes (3m).** "The same scan for each feature; the winner becomes the root."

```
rows = []
for col in features:
    stump = DecisionTreeClassifier(max_depth=1, random_state=42).fit(X_train[[
    s = stump.tree_
    children = (s.n_node_samples[1] * s.impurity[1] + s.n_node_samples[2] * s.
    rows.append({"feature": col, "threshold": s.threshold[0], "decrease": s.im

stumps = pd.DataFrame(rows).sort_values("decrease", ascending=False)
print(stumps.round(4).head(5).to_string(index=False))
```

```
              feature  threshold  decrease
days_since_last_order   172.5000    0.0414
       discount_share     0.3505    0.0142
        tenure_months     5.5000    0.0125
            spend_12m   443.2250    0.0114
           orders_12m     6.5000    0.0108
```

Recency wins by a factor of three. Then the search repeats inside each child: that is why the two second-level questions differ (Lab A asks the same of fraud). Say "greedy": it never reconsiders a split.

5 **Probabilities come from leaves (3m).** "A probability is the churn share of the campsite the customer reaches. Four leaves, four possible numbers."

```
leaf_of = small_tree.apply(X_test)
proba = small_tree.predict_proba(X_test)[:, 1]

per_leaf = pd.DataFrame({"leaf": leaf_of, "p_churn": proba, "churned": y_test.
print(per_leaf.groupby("leaf").agg(customers=("churned", "size"),
                                   predicted=("p_churn", "first"),
                                   actual=("churned", "mean")).round(3))
```

```
      customers  predicted  actual
leaf
2          1011      0.058   0.057
3           286      0.283   0.308
5            42      0.804   0.786
6           161      0.431   0.429
```

Test churn rates match the training shares: the groups are real. A rule book is fine with four ties; a top-50 list is not.

6 **No scaling needed (2m).** "A split asks 'above this value?'. Rescaling moves the threshold, not the customers."

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler().set_output(transform="pandas").fit(X_train)
scaled_tree = DecisionTreeClassifier(max_depth=2, random_state=42).fit(scaler.

same = np.array_equal(scaled_tree.predict_proba(scaler.transform(X_test)), sma
print("identical predictions:", same)
print("root threshold, raw vs scaled:", small_tree.tree_.threshold[0], scaled_
```

```
identical predictions: True
root threshold, raw vs scaled: 172.5 1.021
```

7 **Grow too far (4m).** "Left alone, a tree splits until every leaf is pure. Watch training and validation AUC as it grows." Predict first: where does the validation AUC peak?

```
from sklearn.model_selection import StratifiedKFold, cross_validate, cross_val

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
rows = []
for depth in [1, 2, 3, 4, 5, 6, 8, 10, None]:
    res = cross_validate(DecisionTreeClassifier(max_depth=depth, random_state=
                         cv=cv, scoring="roc_auc", return_train_score=True)
    rows.append({"max_depth": str(depth), "train_auc": res["train_score"].mean

depth_table = pd.DataFrame(rows)
print(depth_table.round(3).to_string(index=False))
```

```
 max_depth  train_auc  cv_auc
         1      0.682   0.678
         2      0.791   0.783
         3      0.834   0.823
         4      0.857   0.839
         5      0.878   0.839
         6      0.897   0.834
         8      0.938   0.744
        10      0.973   0.674
      None      1.000   0.663
```

Peak at depth 4 to 5, then down to 0.663 while training reaches 1.000. Bias-variance from Lesson 3.2, in its clearest form.

8 **Leaf size is often the better control (3m).** "Require enough customers in every leaf."

```
for leaf in [1, 10, 25, 50, 100, 200]:
    model = DecisionTreeClassifier(min_samples_leaf=leaf, random_state=42)
    auc = cross_val_score(model, X_train, y_train, cv=cv, scoring="roc_auc").m
    n_leaves = model.fit(X_train, y_train).get_n_leaves()
    print(f"min_samples_leaf={leaf:3d}  leaves {n_leaves:3d}  CV AUC {auc:.3f}
```

```
min_samples_leaf=  1  leaves 523  CV AUC 0.663
min_samples_leaf= 10  leaves 186  CV AUC 0.800
min_samples_leaf= 25  leaves 100  CV AUC 0.831
min_samples_leaf= 50  leaves  58  CV AUC 0.848
min_samples_leaf=100  leaves  30  CV AUC 0.844
min_samples_leaf=200  leaves  16  CV AUC 0.828
```

50 per leaf: 58 leaves, CV AUC 0.848, the best so far. Mention `ccp_alpha` (pruning) in one sentence; the lesson page has it.

9 **What the tree used (3m).** "Importances add up each feature's impurity decreases."

```
from sklearn.metrics import roc_auc_score

pruned = DecisionTreeClassifier(min_samples_leaf=50, random_state=42).fit(X_tr
print(f"test ROC AUC {roc_auc_score(y_test, pruned.predict_proba(X_test)[:, 1]

importance = pd.Series(pruned.feature_importances_, index=features).sort_value
print(importance.round(3))
```

```
test ROC AUC 0.863, leaves 58
days_since_last_order    0.450
tenure_months            0.250
discount_share           0.169
app_user                 0.049
support_tickets_12m      0.034
return_rate              0.020
avg_order_value          0.016
spend_12m                0.012
orders_12m               0.001
returns_12m              0.000
dtype: float64
```

Then the three cautions: a low score is not "no effect" (spend overlaps with others), MDI favors many-valued columns (Lesson 5.2 shows a noise column winning), and it describes the model, not the customers.

10 **A single tree is unstable (2m).** "Three trees, three random 80% slices of the same customers."

```
rng = np.random.default_rng(42)
subsets = [rng.choice(len(X_train), size=3600, replace=False) for _ in range(3

for label, settings in [("unlimited tree", {}), ("pruned tree (min_samples_lea
    preds = []
    for rows in subsets:
        model = DecisionTreeClassifier(random_state=42, **settings).fit(X_trai
        preds.append(model.predict(X_test))
    preds = np.array(preds)
    disagree = (preds.min(axis=0) != preds.max(axis=0)).mean()
    print(f"{label:34s} the three trees disagree on {disagree:.1%} of test cus
```

```
unlimited tree                     the three trees disagree on 24.6% of test c
pruned tree (min_samples_leaf=50)  the three trees disagree on 10.3% of test c
```

One customer in four gets a different answer from an unlimited tree. Pruning helps; Lesson 5.2 turns the weakness into a strength.

11 **Mistake: class names in the wrong order (1m).** "No error. Read the first leaf: 175 churners of 3,020, labeled churned."

```
print(export_text(small_tree, feature_names=features, class_names=["churned",
```

```
|--- days_since_last_order <= 172.50
|   |--- discount_share <= 0.35
|   |   |--- class: churned
|   |--- discount_share >  0.35
|   |   |--- class: churned
|--- days_since_last_order >  172.50
|   |--- tenure_months <= 5.50
|   |   |--- class: stayed
|   |--- tenure_months >  5.50
|   |   |--- class: churned
```

Build names from `classes_`: `["stayed" if c == 0 else "churned" for c in model.classes_]`.

12 **Mistake: depth zero (1m).** "The smallest tree is a stump."

```
DecisionTreeClassifier(max_depth=0).fit(X_train, y_train)
```

```
InvalidParameterError: The 'max_depth' parameter of DecisionTreeClassifier mus
```

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Reporting the training score.** An unlimited tree: training 1.000, test 0.822, baseline 0.835. Coach: "Which customers was that 1.000 measured on?"
- **Ranking with an unpruned tree.** Its probabilities are all 0 or 1 (AUC 0.683): you cannot choose a top 50 from ties. Coach: "How many distinct probabilities does it give?" (`np.unique`). Fix: `min_samples_leaf`, or a forest next session.
- **Reading importance as truth.** `spend_12m` 0.012, yet churn falls from 24% to 8% across spend quartiles. Coach: "Did the model need it, or does it not matter?"

GUIDED LAB

- **Lab A (13m, everyone):** `labs/module-08/l5_1-decision-trees/fraud_rules_starter.py`; reference `labs/module-08/l5_1-decision-trees/fraud_rules_solution.py`. A rule book for the payments team: rules with weights, the root's Gini by hand, an unlimited tree judged honestly, size by CV average precision (fraud is rare), a segment table, a zero importance.
- **Expected, Lab A:** leaf "velocity_24h > 3.5 and account_age_days <= 366" holds 40 frauds of 42; Gini root 0.0174 (tiny: 0.88% fraud), decrease 0.0018; unlimited tree 433 leaves, training AUC 1.000, test AUC 0.684, AP 0.129; CV AP 0.178 / 0.259 / 0.287 / 0.295 for 4 / 8 / 12 / 16 leaves; the 8-rule book test AUC 0.756, AP 0.253, riskiest segment 15 of 19; `items` importance 0.000 although frauds buy more items (2.39 vs 1.89).
- **Circulate in Lab A for:** `bincount` without `minlength=2` on a child with no frauds; CV by accuracy (every size scores 0.991: ask what accuracy means at 0.88% fraud). **Debrief:** read the rule book aloud as a reviewer would, especially the night + new device + young account rule (38 of 68 fraud in training): that is an interaction a straight line cannot express.
- **Lab B (12m, pairs):** `labs/module-08/l5_1-decision-trees/sales_tree_starter.py`; reference `labs/module-08/l5_1-decision-trees/sales_tree_solution.py`. A regression tree for daily store sales: calendar features, a time split, `TimeSeriesSplit` for the depth, leaves as averages, and the trend a tree cannot follow.
- **Expected, Lab B:** baselines $1,967 / $1,670; first question `day_of_week <= 4`; biggest leaf Boise weekends from May on, $10,059; CV MAE $1,374 (depth 2) to $901 (depth 10); depth 10 test MAE $976; Boise 2025 actual $8,342 vs predicted $7,339 (2024 actual $7,821): a tree predicts averages of past days, so it cannot follow growth.
- **Fast finishers:** `labs/module-08/l5_1-decision-trees/debug_tree.py`: `max_depth=0`, a text column, class names reversed, training accuracy reported as accuracy on new customers. Fixed: `accuracy on new customers 0.851 (baseline 0.835)`.

START THE LESSON PROJECT

The **churn rule book** (lesson page, "Project: A Churn Rule Book"). In the room: Steps 1 to 3 (10 minutes).

- Step 1: fifteen columns, `cancellation_reason` left out. **Check:** `(4500, 15) (1500, 15)`, gaps in `email_opens_90d` (419) and `age_group` (359).
- Step 2: `tree_prep` passes numbers through and one-hot encodes categories. **Check:** 35 columns after encoding.
- Step 3: baselines by CV: do-nothing 0.500, logistic regression 0.832 ± 0.023.
- **Goes home:** Steps 4 to 6 (max_leaf_nodes by CV, 12 leaves at 0.834; test AUC 0.837; `churn_rules.txt`, riskiest segment 79% on test customers).

CHECKS FOR UNDERSTANDING

Ask these during the demo and the labs:

- Why do the two second-level questions differ? **Answer:** the search repeats inside each child, using only that child's customers.
- A depth-2 tree gives how many distinct probabilities? **Answer:** at most four, one per leaf.
- Does a tree need scaled features? **Answer:** no: scaling moves thresholds, not customers.
- Which tells you the right tree size: training score, test score, or CV on the training set? **Answer:** CV on the training set; the test set once at the end.

Workbook self-check, items Q7 to Q12.

Answers: **Q7** `0.375`; **Q8** C) The share of churners among the training customers in the leaf the new customer lands in; **Q9** B) `5.0`; **Q10** D) Choose a size control such as `min_samples_leaf` by cross-validation on the training set; **Q11** A) The tree did not need spend: other features it split on first carried the same information; **Q12** C) `InvalidParameterError`: max_depth must be at least 1 (or None). Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: have strugglers read `export_text` aloud as sentences ("if recency is at most 172 days and discount share at most 0.35, 6% churn"); give them the `gini` function ready-made.

Stretch: the lesson's Exercise 4 (a monotonic constraint on recency) and the Bonus Challenge (write `best_split` yourself); in Lab B, add `ccp_alpha` to the depth search.

CLOSING

EXIT TICKET (3 QUESTIONS, NO NOTES, 4 MINUTES)

Questions on the slide; answers in the workbook's Exit Ticket box or on a sticky note. Sort into "got it" and "not yet": question 1's pile tells you how much of the Session 9 warm-up goes to gamma, question 3's whether the readability-versus-ranking idea landed.

1. Your SVM scores 0.996 on the training customers and AUC 0.699 on the test set, with 4,491 of 4,500 customers as support vectors. Which dial is set wrong, and how do you choose it properly? **Answer:** gamma is far too large (each customer's influence is a tiny bubble, so the model memorized the training set; nearly every point is a support vector). Choose C and gamma together with cross-validation on the training set (keys svc__C, svc__gamma), never with the training score.

2. A node holds 40 stayers and 10 churners. What is its Gini impurity, and what churn probability does a tree give a customer who lands there? **Answer:** Gini = $1 - (0.8^2 + 0.2^2) = 0.32$. The probability is the leaf's churn share: 10 / 50 = 0.2.

3. The care team wants rules they can pin to the wall AND the best possible top-50 list. What do you give them from today's two models, and why not one model for both? **Answer:** A small tree (about 12 leaves, size chosen by CV) as the rule book: readable, but its probabilities come in a few big ties. For the list, a model that scores every customer individually: today the tuned SVM (34 of 50; AUC 0.848), and next session a forest (41 of 50). Readability and ranking pull in different directions.

HOMEWORK (ABOUT 2 TO 3 HOURS)

- **Finish both lesson projects:** the SVM retention list (Steps 4 to 6: test AUC 0.848 vs 0.823; 34 vs 33 churners in the top 50; `svm_shortlist.csv`) and the churn rule book (Steps 4 to 6: 12 leaves; test AUC 0.837; `churn_rules.txt`).
- **Lesson exercises:** 4.3 Exercises 1 to 3 (support-vector share, polynomial kernels on the rings, LinearSVC vs SVC); 5.1 Exercises 1 to 3 (impurity by hand, Gini or entropy, a regression tree for Snow gear). Exercise 4 of each is a stretch.
- **Both Quick Quizzes** and **both journal entries**.
- **Before the Next Lesson:** 4.3: predict, then check, the CV AUC with gamma=0.003. 5.1: rerun "A Tree Is Unstable" with min_samples_leaf=100, and predict (without running) what share of distinct customers a bootstrap sample of 4,500 from 4,500 holds: Session 9 opens with it.
- **Unfinished labs** from today: each starter's "Expected when done" block lets them check themselves.

BRING LIST FOR SESSION 9

- A laptop on which `check_environment.py` still ends with READY (nothing new to install: XGBoost and LightGBM are already in the course environment).
- The `pinecrest-ml` folder with `data/` and the `labs/` folder; Session 9's starters are in `labs/module-09/`.
- The finished 5.1 project notebook (`lesson_5_1_project`): Lesson 5.2's project starts from the same columns and split and compares the forest with the rule tree.
- The 4.3 project notebook: its numbers (0.848, 34 of 50) are the yardstick for forests and boosting.
- Your written prediction for the bootstrap question, the workbook and the journal.

IF YOU RUN LONG OR SHORT

RUNNING LONG: WHAT TO PROTECT AND WHAT TO CUT FIRST

- **Protect:** 4.3 Steps 4 to 6; 5.1 Steps 1, 5 and 7; Lab A of each lesson.
- **Cut first (saves 5m):** 4.3 Steps 9 to 11: say the three fixes (decision_function or calibrate; `svc__C`; judge by AUC or recall).
- **Cut second (saves 7m):** 5.1 Steps 3, 6 and 10 to 12; show the lesson's threshold chart instead of Step 3.
- **Cut third (saves 12m per lesson):** Lab B of each lesson goes home; its expected output is in the starter.

RUNNING SHORT: WHAT TO ADD

- 4.3 Exercise 4 live: a threshold that fits a 150-call budget (precision 0.600, recall 0.363).
- 5.1 Exercise 4 live: the monotonic constraint (risk drops for 80 of 200 customers unconstrained, 0 constrained, for 0.004 AUC).
- Put Session 1's logistic regression fraud list (53 in the top 100) next to Lab A's rule book and ask what a forest might do (Session 9 answers it).

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 9: Random Forests and Gradient Boosting

How to teach Session 9 of Machine Learning Foundations: bagging and random forests (the bootstrap, averaging away variance, decorrelated trees, out-of-bag validation, honest importances) and gradient boosting (small trees that correct each other, the learning rate, early stopping, native categories, XGBoost and LightGBM). Learners leave able to say which error each ensemble fixes, and with a forest call list and a demand forecaster started.

`2 lessons` `3 hrs session` `Lessons 5.2–5.3` `Session 9 of 11`

HOW TO USE THIS GUIDE

- Same rhythm as every session: frame → live demo → guided labs → start the project. Exercises, quizzes and journals are homework.
- Every code block below was run to produce the output under it (Python 3.12, scikit-learn 1.9.1, XGBoost 3.4, LightGBM 4.7, 4 CPU threads). The demos are the lesson pages' code without the plotting lines: show the pictures from `trees_random_forest.html` (bagging diagram, the trees curve, the noise-column bars) and `trees_gradient_boosting.html` (1 / 10 / 300 trees, the validation U-turn).
- XGBoost and LightGBM outputs may differ in the last digit on another machine (multithreaded floating-point sums); say so once, when you run them.
- Labs are in `labs/module-09/`. The two fraud labs continue the kit's payments thread: logistic regression found 53 frauds in its top 100 in Session 1; the eight-rule tree in Session 8 scored AP 0.253; today the forest and the boosters.

> **THE #1 RISK OF SESSION 9: TWO ENSEMBLE LESSONS BLUR INTO ONE**
>
> Learners leave saying "forests and boosting are both lots of trees". Keep the two pictures apart all day: a **panel** of independent, quirky experts whose average cancels their quirks (variance), and a **golfer** whose every putt aims at what is left (bias). Put both on the board, side by side, and make every exit-ticket answer use one of them. The second risk is CPU time: Step 2 of 5.2 fits 100 trees in a Python loop and Step 9 runs permutation importance; both take a few seconds on a laptop. Start them and talk.

BEFORE THE SESSION

- ☐ **Run the kit check:** `python3 labs/check_labs.py module-09` (8 files, about 50 seconds). Every line PASS.
- ☐ **Check the libraries on the projector machine:** `python -c "import xgboost, lightgbm; print(xgboost.__version__, lightgbm.__version__)"` (3.4 and 4.7 in the course environment). `check_environment.py` from Session 1 lists both.
- ☐ **Downloads:** none.
- ☐ **Projector machine:** `(.venv)` active in your own `pinecrest-ml`, JupyterLab started there, notebooks `lesson_5_2` and `lesson_5_3`. Browser tabs: `trees_random_forest.html`, `trees_gradient_boosting.html`.
- ☐ **Board, before learners arrive:** left, the panel of new hires with an averaging arrow and **variance of an average = $\rho\sigma^2 + (1 - \rho)\sigma^2/n$**; right, a golf green with three putts and **F = F + ν · tree**.
- ☐ **Print or share:** the Session 9 workbook pages; sticky notes for the exit ticket.

KNOWN ENVIRONMENT TRAPS

SYMPTOM	CAUSE	FIX
`TypeError: BaggingClassifier.__init__() got an unexpected keyword argument 'base_estimator'`	Old tutorial code	`estimator=`.
`ValueError: Out of bag estimation only available if bootstrap=True`	OOB with `bootstrap=False` (or ExtraTrees' default)	`bootstrap=True`.
`UserWarning: Some inputs do not have OOB scores`	Too few trees for OOB	100 trees or more.
`ValueError: could not convert string to float: 'Bend'`	Text column given to HistGradientBoosting	`X[cols] = X[cols].astype("category")` .
`ValueError: DataFrame.dtypes for data must be int, float, bool or category...`	Text column given to XGBoost	Category dtype and `enable_categorical=True`.
`TypeError: XGBClassifier.fit() got an unexpected keyword argument 'early_stopping_rounds'`	XGBoost 1.x code	Put `early_stopping_rounds` and `eval_metric` in the constructor.

SYMPTOM	CAUSE	FIX
A LightGBM deprecation warning about `eval_set`	LightGBM 4.7 renamed it	`eval_X=` and `eval_y=`.
`ModuleNotFoundError: No module named 'xgboost'`	Environment not active, or an old install	Activate; `python -m pip install -r requirements.txt`.
XGBoost numbers differ in the last digit from the page	Thread count, processor	Normal; a different first decimal means different data or settings.

SESSION 9 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Five retrieval questions on Lessons 4.3 and 5.1; homework check (SVM shortlist, rule book, the bootstrap prediction).
0:10	10	**5.2** Frame and mental model	A panel of new hires; the variance of an average: more models and less alike models.
0:20	30	**5.2** Live demo	The bootstrap, bagging by hand, BaggingClassifier, the forest's correlation trick, how many trees, OOB against CV, tuning by OOB, the noise column, three mistakes.
0:50	25	**5.2** Guided lab	`labs/module-09/l5_2-random-forests/fraud_forest_starter.py`, then `labs/module-09/l5_2-random-forests/decorrelate_starter.py` in pairs.
1:15	10	**5.2** Start the lesson project	The churn forest and call list, Steps 1 and 2; Steps 3 to 6 go home.
1:25	10	Break	Leave the variance formula and the panel picture on the board.
1:35	10	**5.3** Frame and mental model	Putting on a golf green: each stroke aims at what is left; the learning rate.
1:45	30	**5.3** Live demo	Boosting by hand on a wave, learning rates, HistGradientBoosting with native categories, the validation curve, early stopping, XGBoost and LightGBM, three mistakes.

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
2:15	25	**5.3** Guided lab	`labs/module-09/l5_3-gradient-boosting/boosting_by_hand_starter.py`, then `labs/module-09/l5_3-gradient-boosting/fraud_boosters_starter.py` in pairs.
2:40	10	**5.3** Start the lesson project	The weekly demand forecaster, Steps 1 and 2; Steps 3 to 6 go home.
2:50	10	Wrap-up	Exit ticket (3 questions), homework, bring list for Session 10.
Total	180		Ends at 3:00.

In the room. The lessons estimate 100 + 100 minutes for a solo learner; the room has 150 after the warm-up, break and wrap-up, so 75 minutes per lesson: mental model, demo, guided labs and the start of the project.

Goes home (2 to 3 hours). The churn forest and call list Steps 3 to 6, and the demand forecaster Steps 3 to 6; Hands-on Exercises 1 to 3 of each lesson (Exercise 4 is a stretch); both Quick Quizzes and journal entries; both "Before the Next Lesson" lists; unfinished labs.

Protect, in this order: 5.2 demo Steps 2 and 5 (averaging, and the worse-trees-better-forest result); 5.3 demo Steps 2, 3 and 7 (the loop, the learning rate, the validation U-turn); Lab A of each lesson. **Cut first:** 5.2 Steps 3, 4, 6 and 10 to 12; then 5.3 Steps 9 and 11 to 13 (say the three fixes); then the paired Lab B of each lesson (homework).

WARM-UP

0:00 Warm-up: Support Vector Machines and Decision Trees — 10 min

RUN IT

Five questions on the slide; two minutes alone, one with a neighbor, then cold-call. Walk the room for the Session 8 homework: `svm_shortlist.csv`, `churn_rules.txt`, and the written bootstrap prediction (W5).

- **W1** Lesson 4.3: you raise a linear SVM's C from 0.01 to 100 on overlapping data. Street width and number of support vectors? **Answer:** Narrower street, fewer support vectors (the lesson: 4.06 wide with 42 support vectors, then 0.75 with 7). C is the price of a violation.
- **W2** An RBF SVM with gamma=1000 scores 1.000 on training data and 0.650 on test data. What happened, and what would have warned you without the test set? **Answer:** Each point's influence shrank to a tiny bubble: it memorized the training set. Cross-validation would have shown it, and so would the support-vector count (nearly every point). Today's boosting has the same trap with a new dial: the number of trees.
- **W3** A tree node holds 40 stayers and 10 churners. Gini impurity? Predict, then check. **Answer:** `0.32`. 1 - (0.64 + 0.04) = 0.32. A customer landing there gets churn probability 0.2.
- **W4** Why does an SVM need StandardScaler while a decision tree does not? **Answer:** The SVM's kernel and margin are distances, so units matter. A tree only asks "above this threshold?", and rescaling moves the threshold, not the customers. Same for today's forests and boosted trees: no scaling.
- **W5** Homework check: a bootstrap sample of 4,500 rows drawn with replacement from 4,500. What share of the distinct customers does it contain? **Answer:** About 63%. Lesson 5.2's first block shows 62.9% (2,830 customers); theory 63.2%. Collect the written predictions: most people guess 100% or 50%. Today opens with exactly this.

BRIDGE TO TODAY

"Last session ended with a problem: one tree is unstable, three trees on slightly different customers disagreed about one customer in four. Today we turn that weakness into the two most successful models for tables like Pinecrest's."

5.2 Bagging and Random Forests

75 min

OBJECTIVES (FROM THE LESSON PAGE)

- Draw a bootstrap sample and explain why it contains about 63% of the original rows
- Build a bagged ensemble by hand and with `BaggingClassifier(estimator=...)`, and explain why averaging reduces variance
- Explain how a random forest decorrelates its trees with `max_features`, and tune `n_estimators`, `max_features` and `min_samples_leaf`
- Use out-of-bag predictions as a built-in validation estimate
- Compare impurity-based and permutation importance, and recognize the bias of the first

TIMING IN THE ROOM

10m	Frame: the panel of new hires; the variance of an average
30m	Live demo: Steps 1 to 12
25m	Guided lab: `fraud_forest_starter.py` (13m), then `decorrelate_starter.py` in pairs (12m)
10m	Start the lesson project: Steps 1 and 2

MENTAL MODEL TO SAY

"Remember the new hire from Lesson 1.1 studying case files? One diligent new hire who memorizes every file makes confident, quirky judgments: that is an unlimited tree. Now hire a panel of 100. Each studies their own random stack of files from the same archive, some files twice, some missing. For a new customer, everyone gives an opinion and you average."

"Each panelist is still quirky, but the quirks point in different directions and cancel out. What they agree on is the real pattern. That is **bagging**. A **random forest** adds one rule: at every decision, a panelist may only look at a random handful of the columns, which makes them disagree more usefully."

Point at the formula: "Averaging n models: $\rho\sigma^2 + (1 - \rho)\sigma^2/n$. More models shrink the second term. Only less alike models shrink the first. Bagging pulls one lever; a forest pulls both."

LIVE DEMO (YOU TYPE, THEY FOLLOW)

New notebook `lesson_5_2`, the 1.1 split and ten features. Step 2 runs 100 trees in a loop: a few seconds.

1. **The bootstrap (3m).** "Draw 4,500 customers from 4,500, putting each one back after every draw. How many distinct customers? You wrote a guess at home."

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import roc_auc_score

customers = pd.read_csv("data/customers_churn.csv")
features = ["tenure_months", "orders_12m", "spend_12m", "avg_order_value", "da
            "returns_12m", "return_rate", "support_tickets_12m", "discount_sha
X = customers[features]
y = customers["churned"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, stra

rng = np.random.default_rng(42)
n = len(X_train)
rows = rng.integers(0, n, size=n)                    # n draws with replacement
times_drawn = np.bincount(rows, minlength=n)

print(f"draws: {len(rows)}, distinct customers: {np.count_nonzero(times_drawn)
print("how many customers were drawn 0, 1, 2, ... times:", np.bincount(times_d
print(f"theory: 1 - (1 - 1/n)^n = {1 - (1 - 1 / n) ** n:.3f}")
```

```
draws: 4500, distinct customers: 2830 (62.9%)
how many customers were drawn 0, 1, 2, ... times: [1670 1626  840  281   65
theory: 1 - (1 - 1/n)^n = 0.632
```

1,670 customers were never drawn: they are out-of-bag for this sample. Write 63.2% / 36.8% on the board; it comes back in Step 7.

2 **Bagging by hand (4m).** "The worst-behaved model from Lesson 5.1, an unlimited tree, a hundred times, each on its own bootstrap sample. Average their votes."

```
single = DecisionTreeClassifier(random_state=42).fit(X_train, y_train)
print(f"one unlimited tree: test ROC AUC {roc_auc_score(y_test, single.predict

tree_probas = []
for i in range(100):
    rows = rng.integers(0, n, size=n)
    tree = DecisionTreeClassifier(random_state=42).fit(X_train.iloc[rows], y_t
    tree_probas.append(tree.predict_proba(X_test)[:, 1])
tree_probas = np.array(tree_probas)                  # 100 trees x 1,500 test c

for k in [1, 5, 10, 25, 50, 100]:
    print(f"average of {k:3d} trees: test ROC AUC {roc_auc_score(y_test, tree_
```

```
one unlimited tree: test ROC AUC 0.683
average of   1 trees: test ROC AUC 0.668
average of   5 trees: test ROC AUC 0.801
average of  10 trees: test ROC AUC 0.825
average of  25 trees: test ROC AUC 0.835
average of  50 trees: test ROC AUC 0.842
average of 100 trees: test ROC AUC 0.848
```

No single tree improved; the AUC climbs from 0.68 to 0.85 by averaging alone. The first 10 trees give most of the gain.

3. **The divided customer (2m).** "Each tree is certain. The panel is honestly unsure."

```
panel = tree_probas.mean(axis=0)
divided = int(np.argmin(np.abs(panel - 0.5)))
print(f"test customer {customers.loc[X_test.index[divided], 'customer_id']}: "
      f"{panel[divided]:.2f} average churn vote, really churned: {y_test.iloc[
print("first 20 trees say:", tree_probas[:20, divided])
print("distinct probabilities: one tree", len(np.unique(tree_probas[0])), "| t
```

```
test customer C105686: 0.50 average churn vote, really churned: 1
first 20 trees say: [1. 0. 1. 1. 0. 0. 0. 1. 1. 1. 0. 1. 0. 1. 1. 1. 0. 1. 1.
distinct probabilities: one tree 2 | the panel 99
```

And the panel gives 99 distinct probabilities where one tree gives 2: now a top 50 is possible.

4. **BaggingClassifier (2m).** "scikit-learn packages the loop. The argument is `estimator=`."

```
from sklearn.ensemble import BaggingClassifier

bagging = BaggingClassifier(estimator=DecisionTreeClassifier(), n_estimators=1
bagging.fit(X_train, y_train)
print(f"BaggingClassifier, 100 trees: test ROC AUC {roc_auc_score(y_test, bagg
print(len(bagging.estimators_), "fitted trees; first tree trained on", len(np.
```

```
BaggingClassifier, 100 trees: test ROC AUC 0.849
100 fitted trees; first tree trained on 2834 distinct customers
```

5. **The random forest's one twist (4m).** "Bagged trees are too alike: almost all ask about recency first. A forest lets each split choose among a random handful of features." Point at the formula: this attacks ρ.

```
from sklearn.ensemble import RandomForestClassifier

forest = RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=-1).
print(f"random forest, 100 trees: test ROC AUC {roc_auc_score(y_test, forest.p

def per_tree(ensemble):
    """Churn probabilities of every tree in an ensemble, one row per tree."""
    X_array = X_test.to_numpy()                        # the inner trees were f
    return np.array([t.predict_proba(X_array)[:, 1] for t in ensemble.estimato

for name, ensemble in [("bagging", bagging), ("random forest", forest)]:
    probas = per_tree(ensemble)
    one_tree = np.mean([roc_auc_score(y_test, p) for p in probas])
    rho = np.corrcoef(probas)[np.triu_indices(len(probas), k=1)].mean()
    print(f"{name:14s} average single-tree AUC {one_tree:.3f}   average correl
```

```
random forest, 100 trees: test ROC AUC 0.855
bagging        average single-tree AUC 0.666   average correlation between tre
random forest  average single-tree AUC 0.648   average correlation between tre
```

Each forest tree is WORSE (0.648 vs 0.666) and the forest is BETTER (0.855 vs 0.849): less alike trees average better. That sentence is the lesson.

6 **How many trees? (2m).** "Average the first k trees of each."

```
ks = np.arange(1, 101)
for name, ensemble in [("bagging", bagging), ("random forest", forest)]:
    probas = per_tree(ensemble)
    curve = [roc_auc_score(y_test, probas[:k].mean(axis=0)) for k in ks]
    print(f"{name:14s} AUC with 1, 10, 50, 100 trees:", [round(curve[k - 1], 3
```

```
bagging        AUC with 1, 10, 50, 100 trees: [0.677, 0.821, 0.846, 0.849]
random forest  AUC with 1, 10, 50, 100 trees: [0.649, 0.806, 0.849, 0.855]
```

Show the lesson's plateau chart. More trees never overfit; they cost time.

7 **Out-of-bag validation (4m).** "Every customer was left out by about a third of the trees. Those trees' average is an honest prediction, from one fit."

```
from sklearn.model_selection import StratifiedKFold, cross_val_score

oob_forest = RandomForestClassifier(n_estimators=300, min_samples_leaf=5, oob_
oob_forest.fit(X_train, y_train)
oob_proba = oob_forest.oob_decision_function_[:, 1]          # one OOB probabi

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
cv_auc = cross_val_score(oob_forest, X_train, y_train, cv=cv, scoring="roc_auc
print(f"OOB accuracy (oob_score_)  {oob_forest.oob_score_:.3f}")
print(f"OOB ROC AUC                {roc_auc_score(y_train, oob_proba):.3f}")
print(f"5-fold CV ROC AUC          {cv_auc:.3f}")
print(f"test ROC AUC               {roc_auc_score(y_test, oob_forest.predict_p
```

```
OOB accuracy (oob_score_)  0.872
OOB ROC AUC                0.866
5-fold CV ROC AUC          0.868
test ROC AUC               0.865
```

OOB 0.866, CV 0.868, test 0.865: OOB costs one fit instead of five. Then the limits: preprocessing that learned from the target, time or group structure.

8 **Tuning by OOB (3m).** "One fit per setting."

```
results = []
for max_features in ["sqrt", 0.5, None]:
    for leaf in [1, 5, 20]:
        model = RandomForestClassifier(n_estimators=200, max_features=max_feat
                                       oob_score=True, random_state=42, n_jobs
        results.append({"max_features": str(max_features), "min_samples_leaf":
                        "oob_auc": roc_auc_score(y_train, model.oob_decision_f

grid = pd.DataFrame(results).pivot(index="max_features", columns="min_samples_
print(grid.round(3))
```

```
min_samples_leaf     1      5      20
max_features
0.5                0.856  0.866  0.870
None               0.851  0.862  0.867
sqrt               0.857  0.865  0.870
```

Larger leaves help in every row; max_features matters little here (ten features, several redundant). Forests are forgiving to tune.

9 **A column of pure noise (4m).** "Add random numbers as a feature and ask both importance measures about it."

```
from sklearn.inspection import permutation_importance

noise_rng = np.random.default_rng(42)
X_train_noisy = X_train.assign(noise=noise_rng.normal(size=len(X_train)))
X_test_noisy = X_test.assign(noise=noise_rng.normal(size=len(X_test)))

noisy_forest = RandomForestClassifier(n_estimators=300, min_samples_leaf=5, ra
noisy_forest.fit(X_train_noisy, y_train)
perm = permutation_importance(noisy_forest, X_test_noisy, y_test, scoring="roc

importance = pd.DataFrame({"impurity (MDI)": noisy_forest.feature_importances_
                           "permutation (AUC drop)": perm.importances_mean},
                          index=X_train_noisy.columns).sort_values("impurity (
print(importance.iloc[::-1].round(3))
```

```
                       impurity (MDI)  permutation (AUC drop)
days_since_last_order           0.302                   0.161
tenure_months                   0.153                   0.048
discount_share                  0.139                   0.085
spend_12m                       0.087                  -0.002
avg_order_value                 0.075                   0.001
return_rate                     0.066                   0.003
noise                           0.065                   0.005
support_tickets_12m             0.042                   0.014
orders_12m                      0.036                   0.002
app_user                        0.025                   0.013
returns_12m                     0.010                   0.001
```

MDI ranks noise above four real features; permutation on the test set gives it 0.005. Spend and AOV also drop: busy in the trees, not needed. Lab A shows the reverse bias on a 0/1 column.

10 **Mistake: base_estimator (1m).** "Old tutorials."

```
BaggingClassifier(base_estimator=DecisionTreeClassifier(), n_estimators=50)
```

```
TypeError: BaggingClassifier.__init__() got an unexpected keyword argument 'ba
```

It is `estimator=` now.

11 **Mistake: OOB without the bootstrap (1m).**

```
RandomForestClassifier(bootstrap=False, oob_score=True, random_state=42).fit(X
```

```
ValueError: Out of bag estimation only available if bootstrap=True
```

12 **Mistake: judging a forest on its training rows (1m).** "No error."

```
train_auc = roc_auc_score(y_train, forest.predict_proba(X_train)[:, 1])
oob_check = RandomForestClassifier(n_estimators=100, oob_score=True, random_st
print(f"training AUC {train_auc:.3f}   OOB AUC {roc_auc_score(y_train, oob_che
```

```
training AUC 1.000   OOB AUC 0.851
```

Every training customer was in about 63% of the trees' samples. Never `predict` on the training rows to judge a forest (and never feed those predictions into another model: Lesson 5.4).

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Judging the forest on its training rows** (AUC 1.000 or close). Coach: "Which trees had seen this customer?" Use OOB, CV or the test set.
- **Believing more trees overfit.** Learners carry the depth lesson over. Coach with the plateau chart: more trees make the average more precise; the cost is time.
- **Reading MDI as truth.** The noise column and, in Lab A, the 0/1 column `device_new`. Coach: "Was it measured on training or unseen data, and does the column have many distinct values?"

GUIDED LAB

- **Lab A (13m, everyone):** `labs/module-09/l5_2-random-forests/fraud_forest_starter.py`; reference `labs/module-09/l5_2-random-forests/fraud_forest_solution.py`. A forest for the payments team's review queue: leaf size tuned out-of-bag by average precision, one test against logistic regression and a single tree, distinct scores, MDI against permutation importance. About 20 seconds to run.
- **Expected, Lab A:** OOB AP 0.489 / 0.511 / 0.513 / 0.503 for leaf 1 / 3 / 5 / 10; test: logistic regression AUC 0.848, AP 0.444, 53 frauds in the top 100; one tree 0.684, 0.129, 41; forest 0.842, 0.479, 57; distinct scores 2 against 10,197; `device_new` MDI 0.081 but permutation 0.122; `amount` 0.176 vs 0.058.
- **Circulate in Lab A for:** OOB tuned by accuracy (all 0.99); permutation importance with the default scoring (accuracy: tiny drops everywhere). **Debrief:** "The forest has a LOWER AUC than logistic regression and a HIGHER AP and a better top 100. For a review queue, which matters?" (AP and the top of the list.) Then the importance flip: MDI underrates a 0/1 column and overrates many-valued ones.

- **Lab B (12m, pairs):** `labs/module-09/l5_2-random-forests/decorrelate_starter.py`; reference `labs/module-09/l5_2-random-forests/decorrelate_solution.py`. The variance formula measured: as max_features goes from 1 to 10, single trees get better and more alike, and the forest is best in between.
- **Expected, Lab B:** forest AUC 0.865 equals the average of its 200 trees; one-tree AUC 0.709 → 0.747, correlation 0.357 → 0.535; forest test AUC 0.854 / 0.864 / 0.865 / 0.861 / 0.856 for max_features 1 / 2 / 3 / 5 / 10; first-k-trees 0.755 (1) to 0.866 (100); top 50 holds 41 churners and the trees disagree more about them (spread 0.284 vs 0.178).
- **Fast finishers:** `labs/module-09/l5_2-random-forests/debug_forest.py`: `base_estimator`, OOB without the bootstrap, training AUC printed as OOB, and no `random_state`. Fixed: bagging 0.849, forest OOB 0.866, test 0.865, every run.

START THE LESSON PROJECT

The **churn forest and call list** (lesson page, "Project: A Churn Forest and a Call List"). In the room: Steps 1 and 2 (10 minutes).

- Step 1: the 5.1 project's columns and split. **Check:** `(4500, 15) (1500, 15) C102112`.
- Step 2: a forest pipeline with the encoder, leaf size tuned out-of-bag. **Done** = OOB AUC 0.863 / 0.865 / 0.868 / 0.866 / 0.864 for leaf 1 / 5 / 10 / 20 / 40; pick 10.
- **Goes home:** Steps 3 to 6 (test: forest 0.863, rule tree 0.837, logistic regression 0.825; top 50: 41 / 36 / 31; permutation importance; `call_list.csv`).

CHECKS FOR UNDERSTANDING

Ask these during the demo and the labs:

- Why did averaging 100 bad trees give a good model? **Answer:** their errors partly cancel; nothing about any tree improved.
- Each forest tree is worse than a bagged tree. Why is the forest better? **Answer:** its trees are less correlated, so the average cancels more error.
- Can a forest have too many trees? **Answer:** not in accuracy; only in time and memory.
- When is the OOB score not enough? **Answer:** preprocessing that learned from the target, time- or group-structured rows; and the test set still stays locked.

Workbook self-check, items Q1 to Q6.

Answers: **Q1** `0.632`; **Q2** C) Lowering `max_features`; **Q3** A) Only the trees whose bootstrap sample did not include that customer; **Q4** D) `ValueError: Out of bag estimation only available if bootstrap=True`; **Q5** `[1.0, 0.37, 0.307]`; **Q6** B) MDI was inflated: a many-valued column wins chance splits on the training data. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: keep the panel picture in front of strugglers; in Lab A, let a pair skip TODO 1 (use leaf 5) and spend the time on the importance table.

Stretch: the lesson's Exercise 3 (Extra-Trees) and the Bonus Challenge (a MiniForest class with its own OOB score).

5.3 Gradient Boosting

75 min

OBJECTIVES (FROM THE LESSON PAGE)

- Explain how boosting adds small trees one after another, each fitted to the remaining errors, and how the learning rate controls the step size
- Train `HistGradientBoostingClassifier` and `HistGradientBoostingRegressor` on data with missing values and native categorical columns
- Use early stopping on a validation set to choose the number of trees without touching the test set
- Run the same model with `XGBClassifier` and `LGBMClassifier`, including their categorical and early-stopping options
- Choose between a random forest, scikit-learn's boosting and the external libraries for a given problem

TIMING IN THE ROOM

10m	Frame: the golf green; learning rate; boosting fixes bias
30m	Live demo: Steps 1 to 13
25m	Guided lab: `boosting_by_hand_starter.py` (13m), then `fraud_boosters_starter.py` in pairs (12m)
10m	Start the lesson project: Steps 1 and 2

MENTAL MODEL TO SAY

"A golfer rarely sinks a long putt in one stroke. The first putt gets close; the second corrects what is left; the third tidies up. Every stroke aims at the **remaining** distance."

"Boosting starts from a crude guess, the average. Tree 1 aims at the gap between that guess and the truth. Tree 2 at what is left after tree 1, and so on. The **learning rate** is how hard you hit each putt: a gentle tap needs more strokes but rarely overshoots." Then contrast with the panel on the left of the board: "The forest's trees never talk to each other; boosting's trees each fix the one before. Forests fix variance; boosting fixes bias, and keeps going until you stop it."

LIVE DEMO (YOU TYPE, THEY FOLLOW)

New notebook `lesson_5_3`. The first three steps are the toy wave; then churn with native categories; then the two external libraries.

1. **A wavy problem with a noise floor (2m).** "200 noisy points on a wave, plus 1,000 fresh points to play new data. The noise variance is 0.25: no model can beat that on new data."

```
import numpy as np
from sklearn.tree import DecisionTreeRegressor

rng = np.random.default_rng(42)

def make_wave(n):
    x = rng.uniform(0, 10, n)
    y = 3 * np.sin(x) + 0.3 * x + rng.normal(0, 0.5, n)
    return x.reshape(-1, 1), y

X_toy, y_toy = make_wave(200)
X_new, y_new = make_wave(1000)
print(X_toy.shape, X_new.shape, f"noise variance {0.5 ** 2}")
```

```
(200, 1) (1000, 1) noise variance 0.25
```

2 **Boosting by hand (5m).** "Start from the average. Fit a small tree to what is still wrong. Add a fraction of it. Repeat." Read the loop line by line; it is the whole algorithm.

```
def boost(learning_rate, n_trees, report=()):
    pred = np.full(len(y_toy), y_toy.mean())       # start: the average
    pred_new = np.full(len(y_new), y_toy.mean())
    for m in range(1, n_trees + 1):
        residual = y_toy - pred                        # what is still wrong
        tree = DecisionTreeRegressor(max_depth=2, random_state=42).fit(X_toy,
        pred += learning_rate * tree.predict(X_toy)    # a small step towards t
        pred_new += learning_rate * tree.predict(X_new)
        if m in report:
            print(f"{m:4d} trees  train MSE {np.mean((y_toy - pred) ** 2):.3f}
                  f"  new-data MSE {np.mean((y_new - pred_new) ** 2):.3f}")
    return pred_new

curve = boost(learning_rate=0.3, n_trees=300, report=(1, 3, 10, 30, 100, 300))
```

```
   1 trees  train MSE 3.055  new-data MSE 2.964
   3 trees  train MSE 1.364  new-data MSE 1.410
  10 trees  train MSE 0.353  new-data MSE 0.476
  30 trees  train MSE 0.156  new-data MSE 0.361
 100 trees  train MSE 0.075  new-data MSE 0.393
 300 trees  train MSE 0.010  new-data MSE 0.471
```

Training error falls forever, far below the 0.25 floor: memorizing. New-data error bottoms near 30 trees, then creeps up. That U-turn is why every boosting library has early stopping. Show the lesson's 1 / 10 /

300 trees pictures.

3 **The learning rate (3m).** "How hard you hit each putt." Predict first: which rate gives the best new-data error?

```
for lr in [1.0, 0.05]:
    print(f"learning rate {lr}")
    boost(learning_rate=lr, n_trees=300, report=(10, 100, 300))
```

```
learning rate 1.0
  10 trees  train MSE 0.220  new-data MSE 0.449
 100 trees  train MSE 0.006  new-data MSE 0.508
 300 trees  train MSE 0.000  new-data MSE 0.520
learning rate 0.05
  10 trees  train MSE 2.454  new-data MSE 2.398
 100 trees  train MSE 0.210  new-data MSE 0.369
 300 trees  train MSE 0.124  new-data MSE 0.354
```

Full strength memorizes fast; 0.05 needs many more trees but ends at 0.354, better than 0.3 ever did. Small rate + many trees + early stopping is the recipe.

4 **Churn, with categories as categories (3m).** "The one preparation step: `astype("category")`. No imputer, no scaler."

```
import pandas as pd
from sklearn.model_selection import train_test_split

customers = pd.read_csv("data/customers_churn.csv")
numeric = ["tenure_months", "orders_12m", "spend_12m", "avg_order_value", "day
           "returns_12m", "return_rate", "support_tickets_12m", "discount_shar
           "email_opens_90d"]
categorical = ["home_store", "region", "age_group", "loyalty_tier", "preferred

X = customers[numeric + categorical].copy()
X[categorical] = X[categorical].astype("category")
y = customers["churned"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=42
)
print(X_train.shape, X_test.shape)
print("missing in training:", X_train[["email_opens_90d", "age_group"]].isna()
```

```
(4500, 16) (1500, 16)
missing in training: {'email_opens_90d': 419, 'age_group': 359}
```

5 **HistGradientBoosting (2m).** "Hist: each feature is cut into at most 255 bins; missing values get their own bin."

```
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.metrics import roc_auc_score, average_precision_score

hgb = HistGradientBoostingClassifier(random_state=42)
hgb.fit(X_train, y_train)

p_hgb = hgb.predict_proba(X_test)[:, 1]
print(f"accuracy {hgb.score(X_test, y_test):.3f}  ROC AUC {roc_auc_score(y_tes
      f"  average precision {average_precision_score(y_test, p_hgb):.3f}")
print("trees built:", hgb.n_iter_)
print("treated as categorical:", list(X_train.columns[hgb.is_categorical_]))
```

```
accuracy 0.867  ROC AUC 0.862  average precision 0.609
trees built: 100
treated as categorical: ['home_store', 'region', 'age_group', 'loyalty_tier',
```

100 trees: with 4,500 rows, early stopping ("auto") is off.

6 **Against the baseline and logistic regression (3m).** "Zero preprocessing."

```
from sklearn.compose import make_column_transformer
from sklearn.dummy import DummyClassifier
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

linear_prep = make_column_transformer(
    (make_pipeline(SimpleImputer(strategy="median"), StandardScaler()), numeri
    (make_pipeline(SimpleImputer(strategy="most_frequent"), OneHotEncoder(hand
)
logreg = make_pipeline(linear_prep, LogisticRegression(max_iter=1000))

for name, model in [("baseline", DummyClassifier(strategy="most_frequent")),
                    ("logistic regression", logreg),
                    ("hist gradient boosting", hgb)]:
    model.fit(X_train, y_train)
    p = model.predict_proba(X_test)[:, 1]
    print(f"{name:24s} ROC AUC {roc_auc_score(y_test, p):.3f}  AP {average_pre
```

```
baseline                 ROC AUC 0.500  AP 0.165
logistic regression      ROC AUC 0.826  AP 0.493
hist gradient boosting   ROC AUC 0.862  AP 0.609
```

About +0.035 AUC and +0.12 AP over logistic regression: the threshold in recency, the U in tenure and the tickets-by-app interaction, found by the trees. Fair note: Lesson 2.3's hand-built features got logistic regression to 0.880.

7 **Is 100 trees right? (3m).** "Carve a validation set from the training rows and watch the loss after every tree."

```
from sklearn.metrics import log_loss

X_fit, X_val, y_fit, y_val = train_test_split(
    X_train, y_train, test_size=0.2, stratify=y_train, random_state=42
)
long_run = HistGradientBoostingClassifier(learning_rate=0.1, max_iter=400, ear
                                          random_state=42).fit(X_fit, y_fit)

train_loss = [log_loss(y_fit, p) for p in long_run.staged_predict_proba(X_fit)
val_loss = [log_loss(y_val, p) for p in long_run.staged_predict_proba(X_val)]
best = int(np.argmin(val_loss)) + 1
print(f"best number of trees on validation: {best}  (val loss {min(val_loss):.
print(f"after 100 trees: val loss {val_loss[99]:.4f};  after 400: {val_loss[-1
```

```
best number of trees on validation: 27  (val loss 0.2901)
after 100 trees: val loss 0.3026;  after 400: 0.4306
```

Best at 27 trees; the default 100 is already past it. Show the lesson's U-turn chart.

8 **Early stopping does it for you (3m).** "Three learning rates, a generous max_iter, early stopping on."

```
for lr in [0.3, 0.1, 0.03]:
    model = HistGradientBoostingClassifier(
        learning_rate=lr, max_iter=2000, early_stopping=True, validation_fract
        n_iter_no_change=20, random_state=42,
    ).fit(X_train, y_train)
    auc = roc_auc_score(y_test, model.predict_proba(X_test)[:, 1])
    print(f"learning_rate {lr:4}  trees used {model.n_iter_:4d}  test ROC AUC
```

```
learning_rate  0.3  trees used   29  test ROC AUC 0.846
learning_rate  0.1  trees used   56  test ROC AUC 0.867
learning_rate 0.03  trees used  149  test ROC AUC 0.870
```

0.03 uses the most trees and gives the best model. random_state now decides which rows become the internal validation set.

9 **XGBoost (2m).** "Same estimator API. Early stopping goes in the constructor; the validation set in fit. Never the test set."

```
from xgboost import XGBClassifier

xgb_model = XGBClassifier(
    n_estimators=2000, learning_rate=0.03, max_depth=4,
    enable_categorical=True, eval_metric="logloss", early_stopping_rounds=50,
    random_state=42, verbosity=0,
)
xgb_model.fit(X_fit, y_fit, eval_set=[(X_val, y_val)], verbose=False)

p_xgb = xgb_model.predict_proba(X_test)[:, 1]
print(f"XGBoost   best iteration {xgb_model.best_iteration:4d}  test ROC AUC {
```

```
XGBoost   best iteration  218  test ROC AUC 0.864
```

"Yours may vary slightly": multithreaded sums.

10 **LightGBM (2m).** "`eval_X` and `eval_y` in fit, early stopping as a callback."

```
import lightgbm as lgb
from lightgbm import LGBMClassifier

lgbm_model = LGBMClassifier(
    n_estimators=2000, learning_rate=0.03, num_leaves=15,
    random_state=42, verbose=-1,
)
lgbm_model.fit(X_fit, y_fit, eval_X=X_val, eval_y=y_val, eval_metric="binary_l
               callbacks=[lgb.early_stopping(50, verbose=False)])

p_lgbm = lgbm_model.predict_proba(X_test)[:, 1]
print(f"LightGBM  best iteration {lgbm_model.best_iteration_:4d}  test ROC AUC
```

```
LightGBM  best iteration  262  test ROC AUC 0.865
```

All three land in the same place: the learning rate and stopping matter more than the logo.

11 **Mistake: text columns (1m).**

```
HistGradientBoostingClassifier().fit(customers[["tenure_months", "home_store"]
```

```
ValueError: could not convert string to float: 'Bend'
```

`astype("category")`.

12 **Mistake: old XGBoost arguments (1m).**

```
XGBClassifier(n_estimators=500, enable_categorical=True, verbosity=0).fit(
    X_fit, y_fit, eval_set=[(X_val, y_val)], early_stopping_rounds=50
)
```

```
TypeError: XGBClassifier.fit() got an unexpected keyword argument 'early_stopp
```

Constructor, not fit.

13 **Mistake: early stopping on the test set (1m).** "No error, and that is the problem."

```
peeking = XGBClassifier(
    n_estimators=2000, learning_rate=0.03, max_depth=4, enable_categorical=Tru
    eval_metric="auc", early_stopping_rounds=50, random_state=42, verbosity=0,
)
peeking.fit(X_train, y_train, eval_set=[(X_test, y_test)], verbose=False)
print(f"stopped at {peeking.best_iteration} trees, the best point ON THE TEST
print(f"reported test ROC AUC {roc_auc_score(y_test, peeking.predict_proba(X_t
```

```
stopped at 127 trees, the best point ON THE TEST SET
reported test ROC AUC 0.872
```

Slightly better than the honest models, and you can no longer tell how much is real.

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **No early stopping, or early stopping on the test set.** Coach: "Which rows chose the number of trees?" Validation carved from training; test once.
- **Text columns.** `could not convert string to float` (HistGradientBoosting) or the long XGBoost dtype message. Coach: "What dtype is that column?" `astype("category")`.
- **Old API arguments copied from the internet.** `early_stopping_rounds` in XGBoost's `fit`; LightGBM's `eval_set`. Coach: "Read the last line; which argument does it name?" Then check the lesson's comparison table.

GUIDED LAB

- **Lab A (13m, everyone):** `labs/module-09/l5_3-gradient-boosting/boosting_by_hand_starter.py`; reference `labs/module-09/l5_3-gradient-boosting/boosting_by_hand_solution.py`. The lesson's loop, for churn: start at the log-odds of the churn rate, fit depth-3 regression trees to y − p, step 0.3, pick the number of trees on validation, test once, compare with HistGradientBoosting.
- **Expected, Lab A:** start −1.619 (back to 0.165 through the sigmoid); validation loss 0.4375 after 1 tree, 0.2954 after 100, best 275 trees (0.2892); hand-made test AUC 0.870, log loss 0.3082; HistGradientBoosting (max_depth 3, early stopping) 165 trees, AUC 0.866, log loss 0.3091. Stretch: rate 1.0 is best at 72 trees and worse after 300 (0.3072).
- **Circulate in Lab A for:** `residual = y - F` (log-odds instead of probabilities: the loss explodes); updating only `F_fit` and not `F_val`. **Debrief:** "Fifteen lines, and it matches the library. The library takes smarter steps (it uses curvature, so it needs fewer trees), but the idea is exactly this."
- **Lab B (12m, pairs):** `labs/module-09/l5_3-gradient-boosting/fraud_boosters_starter.py`; reference `labs/module-09/l5_3-gradient-boosting/fraud_boosters_solution.py`. HistGradientBoosting, XGBoost and LightGBM on fraud with a native category column and early stopping on validation by average precision; then the peeking version.

- **Expected, Lab B:** HGB defaults AUC 0.841, AP 0.415, 55 in the top 100 (only 46 trees: "auto" early stopping is on above 10,000 rows); gentle HGB 0.857 / 0.475 / 57; XGBoost 0.862 / 0.503 / 61; LightGBM 0.861 / 0.492 / 59; peeking 0.867 / 0.510 / 60. The fraud thread: logistic regression 53, forest 57, XGBoost 61.
- **Fast finishers:** `labs/module-09/l5_3-gradient-boosting/debug_boosting.py`: text columns, `early_stopping_rounds` in fit, and early stopping on the test set. Fixed: HGB 0.862, XGBoost 0.864 at 218 trees.

START THE LESSON PROJECT

The **weekly demand forecaster** (lesson page, "Project: Weekly Demand Forecaster"). In the room: Steps 1 and 2 (10 minutes).

- Step 1: split by time, 2024 against 2025. **Check:** `5088 training weeks x products x stores; 4896 test`, mean units 24.4 vs 26.3 (Pinecrest is growing). Ask: why not `train_test_split`?
- Step 2: three baselines. **Done** = MAE 17.13 (overall mean), 13.42 (per-product mean), 13.81 (same week last year).
- **Goes home:** Steps 3 to 6 (HGB squared error 8.16 with 45 negative forecasts; Poisson 8.05, smallest forecast 0.395; per-category table; `demand_forecast_2025.csv`, 40% lower error than the per-product mean). Remind them of Session 8's Lab B: a tree cannot follow growth, and neither can boosting.

CHECKS FOR UNDERSTANDING

Ask these during the demo and the labs:

- What is each new tree trained on? **Answer:** what the current model still gets wrong (residuals; y − p for classification).
- Lower the learning rate from 0.1 to 0.01 with early stopping on. What happens? **Answer:** many more trees, similar or slightly better validation score.
- Why does a forest not need early stopping but boosting does? **Answer:** averaging more trees only makes the average more precise; adding more corrections keeps fitting finer detail, including noise.
- When would you still pick a random forest? **Answer:** little time to tune, a robust baseline; boosting usually wins once tuned.

Workbook self-check, items Q7 to Q12.

Answers: **Q7** `[20.6 19.4]`; **Q8** A) Bagging averages independent deep trees and mainly reduces variance; boosting adds small trees in sequence, each fixing what is still wrong, and mainly reduces bias; **Q9** C) 100: early_stopping="auto" is off below 10,000 rows, so it builds max_iter=100; **Q10** B) `ValueError: could not convert string to float: 'Bend'`; **Q11** D) A validation set split off from the training rows; **Q12** B) Use loss="poisson", which predicts on a log scale. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: in Lab A, give strugglers the three loop lines from the docstring and ask them only to explain each in words. Pair a strong typist with a strong explainer in Lab B.

Stretch: the lesson's Exercise 3 (a monotonic constraint on recency), Exercise 4 (boosting on rare fraud) and the quantile-loss Bonus Challenge (a 90% safety-stock forecast).

CLOSING

EXIT TICKET (3 QUESTIONS, NO NOTES, 4 MINUTES)

Questions on the slide; answers in the workbook's Exit Ticket box or on a sticky note. Sort them: the pile for question 1 shows whether the panel and the golfer stayed apart; Session 10's Lesson 5.4 needs that distinction (diverse members).

1. A forest and a boosted model are both made of trees. In two sentences: how are their trees different, and which kind of error does each mainly fix? **Answer:** A forest grows many deep, independent trees on bootstrap samples and averages them: it mainly reduces variance. Boosting grows small trees one after another, each fitted to what the model still gets wrong: it mainly reduces bias (and needs early stopping).
2. Why is a forest's out-of-bag score free, and when should you not trust it? **Answer:** Each tree never saw about 37% of the training rows, so averaging only those trees gives an unseen-data prediction for every row from one fit. Do not trust it when preprocessing learned from the target, when rows are not independent (time or groups), or with too few trees.
3. A colleague early-stops XGBoost with eval_set=[(X_test, y_test)] and reports AUC 0.867. What is wrong, and what do you tell them? **Answer:** The number of trees was chosen by watching the test set, so the test score is partly a training score. Early-stop on a validation set split off the training rows (or HistGradientBoosting's built-in early stopping) and touch the test set once.

HOMEWORK (ABOUT 2 TO 3 HOURS)

- **Finish both lesson projects:** the churn forest and call list (Steps 3 to 6: forest 0.863, 41 of 50, `call_list.csv`) and the demand forecaster (Steps 3 to 6: Poisson MAE 8.05, `demand_forecast_2025.csv`).
- **Lesson exercises:** 5.2 Exercises 1 to 3 (the 63% rule, bagging a stable model, Extra-Trees); 5.3 Exercises 1 to 3 (stumps or bushes, tree size with early stopping, a monotonic constraint). Exercise 4 of each is a stretch.
- **Both Quick Quizzes** and **both journal entries**.
- **Before the Next Lesson:** 5.2: refit the project forest with 50 and 1,000 trees. 5.3: rerun the churn early-stopping block with random_state=7, and WRITE DOWN a prediction: will averaging a logistic regression's and the booster's probabilities beat the booster alone? Lesson 5.4 opens with it.
- **Start thinking about your final project:** churn, demand or fraud (Session 10 chooses; Session 11 is the studio).

BRING LIST FOR SESSION 10

- A laptop on which `check_environment.py` still ends with READY (nothing new to install).
- The `pinecrest-ml` folder with `data/` and `labs/`; Session 10's starters are in `labs/module-10/`.
- The 5.2 and 5.3 project notebooks: Lesson 5.4's bake-off puts a logistic regression, a forest and a booster on the same churn split, and your numbers are the reference.
- Your written prediction about averaging logistic regression with boosting.
- A first idea for the final project: which of churn, demand or fraud, and why. The workbook and the journal.

IF YOU RUN LONG OR SHORT

RUNNING LONG: WHAT TO PROTECT AND WHAT TO CUT FIRST

- **Protect:** 5.2 Steps 2 and 5; 5.3 Steps 2, 3 and 7; Lab A of each lesson.
- **Cut first (saves 6m):** 5.2 Steps 3, 4 and 10 to 12; say the three fixes.
- **Cut second (saves 6m):** 5.3 Steps 9, 11 to 13; run only XGBoost, say LightGBM "lands in the same place".
- **Cut third (saves 12m per lesson):** Lab B of each lesson goes home; the expected output is in each starter.

RUNNING SHORT: WHAT TO ADD

- 5.2 Exercise 2 live: bagging a logistic regression gains nothing (0.8295 vs 0.8292): bagging needs variance to remove.
- 5.3 Exercise 1 live: stumps, depth 2 and depth 5 on the wave.
- Put the whole fraud thread on the board: top-100 frauds 53 (logistic regression), 57 (forest), 61 (XGBoost); AP 0.444, 0.479, 0.503. Ask: "Worth the complexity?" (Lesson 5.4's question.)

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 10: Ensembles and Model Selection

How to teach Session 10 of Machine Learning Foundations: combine different models with voting and stacking and judge whether the team is worth its moving parts; compare models fairly on shared folds, read validation and learning curves, and open the test set once. The session ends with each learner choosing a final project (churn, weekly demand or fraud) and writing a one-paragraph plan.

`2 lessons` `3 hrs session` `Lessons 5.4–6.1` `Session 10 of 11`

HOW TO USE THIS GUIDE

- Same rhythm for both lessons: frame (the goal and one mental model) → live demo (you type, they type along) → guided lab (a kit lab, new practice on the lesson's skills) → start the lesson's own project. The lesson exercises, quiz and journal are homework.
- Type, do not paste. Every code block below was run to produce the output printed under it (Python 3.12, pandas 3.0, scikit-learn 1.9.1 on 4 CPU threads). Both demos are the lessons' own code, so learners can check every number against the page afterward.
- Keep `ensemble_methods.html` and `evaluation_model_selection.html` open for the charts (the crowd curves, the stacking diagram, the fold chart, the validation and learning curves). Labs live in `labs/module-10/`; `final-assessment.html` (kit root) describes the final project, which learners choose today.
- This session ends differently: a 20-minute final-project segment before the wrap-up. Protect it: a learner who leaves without a chosen problem and a written plan starts Session 11's studio with nothing to work on.

> **THE #1 RISK OF SESSION 10: WAITING FOR MODELS**
>
> Both demos fit many boosted and forest models: the 5.4 demo about 40 fits, the 6.1 demo about 100. On a fast laptop each cell takes seconds; on an older one, the cross-validation cell and the learning curves take a minute or more. **Before:** run both demo notebooks once at home so you know your machine's times. **In the room:** start every slow cell, then talk through the code above it while it runs (the steps below say which ones); never stand silent in front of a progress spinner. **Learners:** if a learner's machine is slow, they type along and run only the cells marked in the guided labs; the demo outputs are in the workbook's worked examples and on the lesson pages.

BEFORE THE SESSION

- ☐ **Run the kit check:** `python3 labs/check_labs.py module-10` from this kit's folder: 8 PASS lines (the four solutions take 10 to 70 seconds each).
- ☐ **Run both demos once** on the projector machine (new notebooks `lesson_5_4` and `lesson_6_1` in your `pinecrest-ml`), and note how long the cross-validation and learning-curve cells take there.
- ☐ **Downloads:** none. The environment from Session 1 has everything.
- ☐ **Print or share:** the workbook pages for Session 10 (its Final Project page has the three problem cards and the plan template), and one copy of `final-assessment.html` per table or on screen, brief and rubric only (the instructor answer notes are not for learners).
- ☐ **Board, before learners arrive:** a three-column table headed logreg / forest / boost (fill it in during the demo), the line "a crowd is wise only when its members disagree", and for 6.1 the checklist: same pipeline · same folds · one metric chosen first · baseline in the table · paired differences · test once.
- ☐ **Files to hand out:** `labs/module-10/` without the `*_solution.py` files; `labs/module-11/l6_2-hyperparameter-tuning/final_project_starter.py` for the final-project milestone 1 (homework).

KNOWN ENVIRONMENT TRAPS

SYMPTOM	CAUSE	FIX
`ValueError: Invalid 'estimators' attribute ...`	Members passed without names	A list of `("name", estimator)` pairs, names unique.
`ValueError: cross_val_predict only works for partitions`	`TimeSeriesSplit` passed as a stacking `cv`	Unshuffled `KFold` on rows sorted by time, or blend on a later period (Lab 5.4 B).
`InvalidParameterError: The 'scoring' parameter ... Got 'auc' instead.`	A scorer name that does not exist	`scoring="roc_auc"`; the full list is in `get_scorer_names()`.
`ValueError: Invalid parameter 'max_depth' for estimator Pipeline(...)`	A step parameter without the step name	`decisiontreeclassifier__max_depth`; `sorted(pipe.get_params())` lists the names.
The fraud lab sits for a minute or more	Several models on 42,000 payments, five folds each	Expected on an older laptop. Start the run, then read the next TODO. Pairs can share one machine.
Learning-curve or validation-curve numbers differ slightly from the page	An older scikit-learn, or a different thread count	The check prints versions; with scikit-learn 1.9 the numbers match to the third decimal.

SESSION 10 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Five retrieval questions on Lessons 5.2 and 5.3; homework check, including the 5.3 prediction: does averaging logistic regression with boosting beat boosting alone?
0:10	10	**5.4** Frame and mental model	The gear-testing panel: a crowd is wise only when its members make different mistakes. Voting counts hands; stacking learns whom to trust.
0:20	25	**5.4** Live demo	The crowd simulation, three member pipelines, out-of-fold correlation, soft and hard votes, a stack by hand and with StackingClassifier, the leaky in-sample stack.
0:45	20	**5.4** Guided lab	`labs/module-10/l5_4-voting-stacking/fraud_ensemble_starter.py` in pairs: does a team catch more fraud than its best member?
1:05	10	**5.4** Start the lesson project	Should Pinecrest ship the ensemble? Steps 1 to 3 (five contenders, churners in the top 150).
1:15	10	Break	Leave the correlation table and the 98-versus-98 call count on the board.
1:25	10	**6.1** Frame and mental model	Testing tents: same campsites, same nights, compare night by night. Three habits: same folds, paired differences, curves.
1:35	25	**6.1** Live demo	A four-candidate leaderboard on 10 shared folds, the per-fold table, paired differences, a validation curve, two learning curves, the test set opened once, two silent mistakes.
2:00	20	**6.1** Guided lab	`labs/module-10/l6_1-model-selection/fraud_bakeoff_starter.py`: a fair fraud bake-off decided before June is opened.
2:20	10	**6.1** Start the lesson project	The churn-model bake-off on average precision, Steps 1 to 3.
2:30	20	Final project: choose and plan	The three problems from `final-assessment.html` (churn, weekly demand, fraud); each learner picks one and writes a one-paragraph plan in the workbook.
2:50	10	Wrap-up	Exit ticket (3 questions), homework, bring list for Session 11.
Total	180		Ends at 3:00.

In the room. The lesson pages estimate 95 + 100 minutes for a solo learner; after the warm-up, the break, the final-project segment and the wrap-up the room has 130. So each lesson gets its mental model, the live demo, one guided kit lab and the first steps of the project.

Goes home (2 to 3 hours). The 5.4 project Steps 4 and 5 (the paired bootstrap and the bill) and the 6.1 project Steps 4 to 6 (paired check, learning curve, test once, `bakeoff_results.csv`); each lesson's Exercises 1 to 3 (Exercise 4 is a stretch); both Quick Quizzes; both journal entries; the final-project plan typed into the project notebook and milestone 1 (`final_project_starter.py` for their problem).

Protect, in this order: the final-project segment (everybody leaves with a problem and a plan); 5.4 demo Steps 3, 9 and 10 (correlation, the leaky stack, 98 vs 98); 6.1 demo Steps 4 and 5 (the per-fold table and paired differences).
Cut first: 6.1 demo Step 9 (the scorer name: one sentence), then 5.4 demo Step 5 and 6 (VotingClassifier and the hard vote: show the slide), then the 6.1 project start (send Steps 1 to 6 home with the check numbers).

WARM-UP

0:00 Warm-up: Bagging and Boosting — 10 min

FIVE RETRIEVAL QUESTIONS (NO NOTES; ANSWERS AFTER EACH)

1. You draw a bootstrap sample of 10,000 rows from 10,000. About how many distinct rows does it hold, and what are the others called? **Answer:** About 6,300 (63.2%); the rest are out-of-bag, and a forest's OOB score is computed from them.
2. A forest with 300 trees has an OOB ROC AUC of 0.866. What happens if you raise n_estimators to 3,000? **Answer:** About the same AUC, ten times the fitting time. More trees never overfit; the score plateaus.
3. In gradient boosting, what is each new tree trained to predict? **Answer:** What the current model still gets wrong: the residuals (more generally the negative gradient of the loss).
4. You early-stop an XGBoost or HistGradientBoosting model. Which rows may decide when it stops? **Answer:** A validation set split off from the training rows, never the test set.
5. HistGradientBoostingClassifier on churn needs which preparation for the five text columns? **Answer:** Only `astype("category")`: no one-hot encoding, no imputer, no scaler (missing values stay as they are).

HOMEWORK CHECK

- Lesson 5.3 "Before the Next Lesson" asked: will averaging a logistic regression's and the booster's probabilities beat the booster alone? Take a show of hands, write the split on the board, and leave it: demo Step 4 answers it (0.858 against 0.862: no).
- Lesson 5.3's other prediction: rerunning the early-stopping block with `random_state=7` moves the number of trees and the test AUC. Ask who saw how much. That wobble is the reason for Lesson 6.1.
- Projects: the 5.2 call list (the forest reached 41 churners in its top 50) and the 5.3 demand forecaster (Poisson boosting MAE 8.05 against 13.42 for the per-product mean). Anyone who did not finish 5.3 should do so this week: Final Project B builds on it.

LESSON PLANS

5.4 Voting and Stacking

65 min

OBJECTIVES (FROM THE LESSON PAGE)

- Explain why a group of models beats its members only when their errors differ, and measure that diversity
- Build hard and soft `VotingClassifier` ensembles from complete preprocessing pipelines
- Build a stack by hand from out-of-fold predictions, then with `StackingClassifier`, and explain why in-sample predictions leak
- Judge whether an ensemble's gain is real (bootstrap) and worth its cost in training time, latency and explainability

TIMING IN THE ROOM

10m	Frame: the gear-testing panel; voting counts hands, stacking learns whom to trust
25m	Live demo: the crowd simulation, three members, out-of-fold correlation, voting, stacking by hand and by class, the leaky stack, 98 vs 98
20m	Guided lab: `fraud_ensemble_starter.py` in pairs
10m	Start the lesson project: Steps 1 to 3

MENTAL MODEL TO SAY

"Pinecrest asks a panel of three testers whether a new jacket is waterproof enough to stock. If each is right 70% of the time and they judge independently, the majority is right more often than any one of them: their mistakes rarely line up. If all three read the same online review and repeat it, the panel is no better than the review. **A crowd is wise only when its members make different mistakes.**"

Then the two ways of combining: "Voting is the panel counting hands. Stacking is a manager who has watched the panel for a season and learned whose opinion to trust, and how much. The manager must learn from the panel's real track record, not from what they say about jackets they have already tested: that is the out-of-fold rule."

And the professional's question, written on the board for the whole lesson: **is the gain larger than the noise, and is it worth the moving parts?**

LIVE DEMO (YOU TYPE, THEY FOLLOW)

New notebook `lesson_5_4`. Steps 2, 3 and 8 fit models: start them, then talk. Ask for a prediction before Steps 1, 4 and 10.

1. **Simulate the panel (3m).** "Each voter is right 70% of the time. Watch what happens as more voters join, and as they start copying one shared opinion." Ask first: five independent 70% voters, majority vote: better or worse than one?

```
import numpy as np

rng = np.random.default_rng(42)

def crowd_accuracy(n_voters, shared, p_correct=0.7, n_cases=20_000):
    common = rng.random(n_cases) < p_correct                  # the shared opi
    own = rng.random((n_cases, n_voters)) < p_correct          # each voter's o
    copies = rng.random((n_cases, n_voters)) < shared          # who copies the
    correct = np.where(copies, common[:, None], own)
    return (correct.sum(axis=1) > n_voters / 2).mean()         # majority vote

for shared in [0.0, 0.5, 0.9]:
    accs = [crowd_accuracy(n, shared) for n in (1, 5, 25)]
    print(f"shared {shared:.1f}:  1 voter {accs[0]:.3f}   5 voters {accs[1]:.3
```

```
shared 0.0:  1 voter 0.695   5 voters 0.837   25 voters 0.982
shared 0.5:  1 voter 0.704   5 voters 0.754   25 voters 0.721
shared 0.9:  1 voter 0.703   5 voters 0.699   25 voters 0.702
```

Independent: 0.695 → 0.837 → 0.982. Half copying: barely better. 90% copying: no better than one voter. Say: "The question for any ensemble is how different their mistakes still are."

2. **Three members that see the data differently (3m).** "Each member is a complete pipeline, so it brings its own preprocessing and nothing can leak inside cross-validation." Narrate the three: a straight line, deep averaged trees, small trees in sequence.

```
import pandas as pd
from sklearn.compose import make_column_transformer
from sklearn.ensemble import HistGradientBoostingClassifier, RandomForestClass
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import average_precision_score, roc_auc_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

customers = pd.read_csv("data/customers_churn.csv")
numeric = ["tenure_months", "orders_12m", "spend_12m", "avg_order_value", "day
           "returns_12m", "return_rate", "support_tickets_12m", "discount_shar
           "email_opens_90d"]
categorical = ["home_store", "region", "age_group", "loyalty_tier", "preferred

X = customers[numeric + categorical].copy()
X[categorical] = X[categorical].astype("category")
y = customers["churned"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, stra

impute_encode = make_pipeline(SimpleImputer(strategy="most_frequent"),
                              OneHotEncoder(handle_unknown="ignore", sparse_ou
logreg = make_pipeline(
    make_column_transformer((make_pipeline(SimpleImputer(strategy="median"), S
                             (impute_encode, categorical)),
    LogisticRegression(max_iter=1000),
)
forest = make_pipeline(
    make_column_transformer((SimpleImputer(strategy="median"), numeric), (impu
    RandomForestClassifier(n_estimators=200, min_samples_leaf=5, random_state=
)
boost = HistGradientBoostingClassifier(random_state=42)

members = [("logreg", logreg), ("forest", forest), ("boost", boost)]
test_proba = pd.DataFrame(index=X_test.index)
for name, model in members:
    model.fit(X_train, y_train)
    test_proba[name] = model.predict_proba(X_test)[:, 1]
    print(f"{name:7s} ROC AUC {roc_auc_score(y_test, test_proba[name]):.3f}"
          f"  AP {average_precision_score(y_test, test_proba[name]):.3f}")
```

```
logreg  ROC AUC 0.826  AP 0.493
forest  ROC AUC 0.861  AP 0.606
boost   ROC AUC 0.862  AP 0.609
```

The two tree models are close; logistic regression is clearly weaker (0.826).

3 **How different are their mistakes? (3m).** "Measuring that on the test set would be peeking. Out-of-fold predictions: every training customer gets a prediction from a model that never saw them."

```
from sklearn.model_selection import StratifiedKFold, cross_val_predict

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
oof = pd.DataFrame(index=X_train.index)
for name, model in members:
    oof[name] = cross_val_predict(model, X_train, y_train, cv=cv, method="pred

print(oof.head(3).round(3))
print()
print("correlation of out-of-fold probabilities:")
print(oof.corr().round(3))
```

```
      logreg  forest  boost
1556   0.111   0.207  0.034
965    0.061   0.046  0.068
5145   0.014   0.115  0.087

correlation of out-of-fold probabilities:
        logreg  forest  boost
logreg   1.000   0.841  0.745
forest   0.841   1.000  0.890
boost    0.745   0.890  1.000
```

Forest and booster correlate 0.890: near-copies. Logistic regression is the odd one out, weaker but different. Say: "The most different member is often the weakest one."

4 **Voting is an average (2m).** "You already have each member's test probabilities, so a soft vote is one line."

```
soft_avg = test_proba.mean(axis=1)
print(f"average of all three   ROC AUC {roc_auc_score(y_test, soft_avg):.3f}")
for pair in [["forest", "boost"], ["logreg", "boost"], ["logreg", "forest"]]:
    print(f"average of {' + '.join(pair):15s} ROC AUC {roc_auc_score(y_test, t
```

```
average of all three   ROC AUC 0.863
average of forest + boost  ROC AUC 0.865
average of logreg + boost  ROC AUC 0.858
average of logreg + forest ROC AUC 0.853
```

Ask about the third line: averaging logistic regression with boosting (0.858) does NOT match boosting alone (0.862). That settles the 5.3 homework prediction.

5 **VotingClassifier: one object (2m).** "Same result, packaged so you can cross-validate, tune and save it."

```
from sklearn.ensemble import VotingClassifier

soft_vote = VotingClassifier(members, voting="soft").fit(X_train, y_train)
hard_vote = VotingClassifier(members, voting="hard").fit(X_train, y_train)

p_soft = soft_vote.predict_proba(X_test)[:, 1]
print("same as the hand-made average:", np.allclose(p_soft, soft_avg))
print(f"soft voting  accuracy {soft_vote.score(X_test, y_test):.3f}  ROC AUC {
print(f"hard voting  accuracy {hard_vote.score(X_test, y_test):.3f}  (no proba
```

```
same as the hand-made average: True
soft voting  accuracy 0.859  ROC AUC 0.863
hard voting  accuracy 0.859  (no probabilities, so no ROC AUC)
```

6 **Ask the hard vote for probabilities (1m).** "A hard vote only counts labels."

```
hard_vote.predict_proba(X_test)
```

```
AttributeError: This 'VotingClassifier' has no attribute 'predict_proba'
```

No probabilities means no ROC AUC, no ranking, no threshold. Prefer soft voting.

7 **A stack by hand (3m).** "Stacking learns whom to trust: the out-of-fold predictions you already have become the meta-model's features."

```
meta = LogisticRegression().fit(oof, y_train)
print("meta-model weights:", dict(zip(oof.columns, meta.coef_[0].round(2).toli
print(f"intercept: {meta.intercept_[0]:.2f}")

p_stack_hand = meta.predict_proba(test_proba)[:, 1]
print(f"hand-made stack  ROC AUC {roc_auc_score(y_test, p_stack_hand):.3f}"
      f"  AP {average_precision_score(y_test, p_stack_hand):.3f}")
```

```
meta-model weights: {'logreg': 1.03, 'forest': 5.23, 'boost': 1.55}
intercept: -3.37
hand-made stack  ROC AUC 0.864  AP 0.611
```

Read the weights as "how much the meta-model leans on each opinion, given the others". They say nothing about why customers churn.

8 **The same with StackingClassifier (2m).** "Same folds, same meta-model, one estimator."

```
from sklearn.ensemble import StackingClassifier

stack = StackingClassifier(estimators=members, final_estimator=LogisticRegress
stack.fit(X_train, y_train)

p_stack = stack.predict_proba(X_test)[:, 1]
print("meta-model weights:", stack.final_estimator_.coef_[0].round(2))
print("same as the hand-made stack:", np.allclose(p_stack, p_stack_hand))
print(f"StackingClassifier ROC AUC {roc_auc_score(y_test, p_stack):.3f}  AP {a
```

```
meta-model weights: [1.03 5.23 1.55]
same as the hand-made stack: True
StackingClassifier ROC AUC 0.864  AP 0.611
```

9 **Why out-of-fold? (2m).** "How well does each member 'predict' rows it trained on?"

```
for name, model in members:
    in_sample = roc_auc_score(y_train, model.predict_proba(X_train)[:, 1])
    print(f"{name:7s} training-row ROC AUC: in-sample {in_sample:.3f}   out-of
```

```
logreg  training-row ROC AUC: in-sample 0.842   out-of-fold 0.832
forest  training-row ROC AUC: in-sample 0.981   out-of-fold 0.867
boost   training-row ROC AUC: in-sample 0.999   out-of-fold 0.859
```

The booster looks almost perfect in-sample (0.999) and is really about 0.86.

10 **The leaky stack, on purpose (2m).** "Train the meta-model on those in-sample predictions and see what it learns." Predict first: better or worse than the honest stack?

```
in_sample = pd.DataFrame({name: model.predict_proba(X_train)[:, 1] for name, m
                          index=X_train.index)
leaky_meta = LogisticRegression().fit(in_sample, y_train)

print("weights:", dict(zip(in_sample.columns, leaky_meta.coef_[0].round(2).tol
print(f"leaky stack test ROC AUC {roc_auc_score(y_test, leaky_meta.predict_pro
```

```
weights: {'logreg': -3.25, 'forest': 2.69, 'boost': 12.66}
leaky stack test ROC AUC 0.854
```

No error, a respectable-looking number, and it is below its own best member. The booster gets 12.66 and logistic regression a negative vote. This is the most expensive mistake in the lesson because nothing warns you.

11 **Count what the care team counts (2m).** "Thousandths of AUC are hard to feel. The care team calls the top 150. How many real churners does each list reach?"

```
calls = 150
for c in ["logreg", "forest", "boost"]:
    print(f"{c:7s} churners in the top {calls}: {int(y_test.loc[test_proba[c].
print(f"stack   churners in the top {calls}: {int(y_test.loc[pd.Series(p_stack
print(f"random list: {calls * y_test.mean():.0f}")
```

```
logreg  churners in the top 150: 87
forest  churners in the top 150: 96
boost   churners in the top 150: 98
stack   churners in the top 150: 98
random list: 25
```

Stack 98, booster 98, random 25. Write "98 vs 98" on the board: the project's bootstrap makes the same point formally.

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Measuring diversity or choosing weights on the test set.** Symptom: `test_proba.corr()` or a loop over weights scored with `y_test`. Coach: "Which rows decided this choice? If the test set chose it, what is left to grade it?" Out-of-fold predictions on the training rows are the honest place for every ensemble decision.
- **Training the meta-model on in-sample predictions.** No error, a plausible number (0.854), wrong weights. Coach with the in-sample vs out-of-fold table: "Would you trust a tester's opinion of jackets they designed themselves?" Fix: `cross_val_predict` or `StackingClassifier(cv=...)`.
- **Shipping a third-decimal win.** "The stack wins, 0.864 against 0.862." Coach: "How many more churners does the care team reach?" (none: 98 vs 98) and "Would a different split agree?" The project's bootstrap answers it: the interval includes zero.

GUIDED LAB

- **Lab A (20m, pairs):** `labs/module-10/l5_4-voting-stacking/fraud_ensemble_starter.py`; reference `labs/module-10/l5_4-voting-stacking/fraud_ensemble_solution.py`. The lesson's whole move set on a new question: card fraud, trained on January to May and judged on June. Every choice (which team) is made on out-of-fold predictions; June is opened once. One partner types, the other predicts each output; swap at TODO 3.

- **Expected:** out-of-fold AP logreg 0.506, forest 0.513, boost 0.524, correlations 0.89 to 0.92; soft votes 0.541 (all three), 0.531, 0.539, so the team of three is chosen; meta weights about [5.04 4.45 5.5]; June AP 0.539 / 0.509 / 0.556, vote 0.570, stack 0.573, and every contender has 35 or 36 of June's 64 frauds in its top 100; stack minus booster +0.016, 95% interval [-0.016, +0.057].
- **Circulate for:** soft votes scored on June instead of out-of-fold (ask which rows decided); the bootstrap drawing different rows for the two models (it must be paired: the same `idx` for both); learners who stop at "the stack has the highest AP". **Debrief (2m):** "Here the team helps a little out-of-fold, unlike churn, because the three members disagree more on frauds. But on June the gain is inside the noise, and the review queue holds the same 36 frauds. Same verdict as the lesson, different data: keep one model."
- **Fast finishers:** `labs/module-10/l5_4-voting-stacking/debug_stack.py` (three loud bugs and one silent one), then `labs/module-10/l5_4-voting-stacking/demand_blend_starter.py`: blending two demand forecasters on a later period (October to December 2024), the time-aware alternative the lesson's Mistake 3 points to.

START THE LESSON PROJECT

Should Pinecrest ship the ensemble? In the room: Steps 1 to 3 (10 minutes).

- Read the brief aloud (the care team calls the riskiest 10%, a colleague proposes a stack). Step 1 rebuilds the members compactly; **done** = `(4500, 16) (1500, 16) ['logreg', 'forest', 'boost']`.
- Step 2: the five-contender table (stack 0.864, boost 0.862, soft vote 0.863). Step 3: churners in the top 150: logreg 87, forest 96, boost 98, soft vote 96, stack 98 (a random list reaches 25).
- **Goes home:** Step 4 (the paired bootstrap: stack minus boost +0.0023, 95% interval -0.0070 to +0.0124) and Step 5 (timings, which will differ, and the recommendation printed from variables: keep the single booster).

CHECKS FOR UNDERSTANDING

Ask these out loud during the demo and the lab:

- Why can averaging logistic regression with boosting score below boosting alone? **Answer:** an equal vote gives a clearly weaker member the same say; the gain from its difference does not outweigh its weakness.
- Where do a meta-model's training features come from? **Answer:** out-of-fold predictions on the training rows (`cross_val_predict`, or `StackingClassifier`'s `cv`).
- What do the meta-model's weights tell you about churn? **Answer:** nothing about customers; only how the members' opinions are combined.
- Stack 98, booster 98 churners in the top 150. What would make you ship the stack? **Answer:** a gain that survives a bootstrap or cross-validation and changes a decision (more churners called), worth 19 fits and four pieces to run.

Workbook self-check, items Q1 to Q6.

Answers: **Q1** B) About 0.86: members that make the same mistakes add almost nothing; **Q2** `[0.57 0.32] [20]`; **Q3** D) In-sample predictions are overconfident, so the meta-model over-trusts the most flexible member; **Q4** A) Build the vote with `voting="soft"`, from members that all have `predict_proba`; **Q5** C) `ValueError`: every member needs a name, as a (string, estimator) pair; **Q6** D) Keep the single booster: the gain is within the noise and changes no call, and the stack has 19 fits and four pieces to run. Full set with explanations in the Answer Key.

◆ DIFFERENTIATION

Support: give struggling pairs the TODO 1 loop from demo Step 3 to adapt (change the data names only); the point of the lab is the decision sequence, not the typing. Have them say, before each step, which rows it uses: Jan-May out-of-fold, or June.

Stretch: the lesson's Exercise 2 (a k-nearest-neighbors member earns a weight but barely moves the stack) and the Bonus Challenge (five splits: in how many does the stack win?). The workbook's worked example shows a stronger kind of diversity: a logistic regression given the shapes it cannot draw (the 150-day lapse, the tenure U, tickets without the app) scores 0.877 out-of-fold on its own.

6.1 Model Selection and Learning Curves

65 min

OBJECTIVES (FROM THE LESSON PAGE)

- Compare several models fairly with `cross_validate`: the same preprocessing pipeline, the same folds and the same metric, reporting the spread and the fit time
- Decide whether a gap between two models is bigger than the noise, using paired per-fold differences
- Draw a validation curve with `ValidationCurveDisplay` and point to where a model moves from underfitting to overfitting
- Draw a learning curve with `LearningCurveDisplay` and say whether more data or a more flexible model is the better next step
- Make a final choice you can defend, then score it once on the untouched test set

TIMING IN THE ROOM

10m	Frame: testing tents; three questions (which is best, is the gap real, what would help)
25m	Live demo: leaderboard on 10 shared folds, per-fold table, paired differences, validation curve, learning curves, test once, two mistakes
20m	Guided lab: `fraud_bakeoff_starter.py`
10m	Start the lesson project: Steps 1 to 3

MENTAL MODEL TO SAY

"Pinecrest's gear team is choosing a tent to stock. If they pitch tent A on a calm night and tent B in a storm, the result says more about the weather than the tents. A fair test pitches every tent at the same campsites on the same nights, many nights in a row, and compares them night by night." Then map it: **campsites = folds, nights = repeats, leaks = scores.**

Three questions, three tools: **Which is best?** Same pipeline, same folds, one metric chosen first, the spread next to every mean. **Is the gap real?** Paired per-fold differences. **What would help?** A validation curve (turn one knob) and a learning curve (add data). And one rule above all: the test set is opened once, after the choice.

LIVE DEMO (YOU TYPE, THEY FOLLOW)

New notebook `lesson_6_1`. Steps 3 and 7 take the longest: start them and talk through the code while they run. Learners on slow machines may skip running Step 7 and read it from the page.

1. **One recipe for every candidate (2m).** "A fair contest starts with identical inputs: same split, same features, preprocessing written once as a function so each candidate gets a fresh copy."

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import make_pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

customers = pd.read_csv("data/customers_churn.csv")
numeric = ["tenure_months", "orders_12m", "spend_12m", "avg_order_value", "day
           "returns_12m", "return_rate", "support_tickets_12m", "discount_shar
           "email_opens_90d"]
categorical = ["home_store", "region", "age_group", "loyalty_tier", "preferred

X = customers[numeric + categorical]
y = customers["churned"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, stra

def make_preprocess():
    return ColumnTransformer([
        ("num", make_pipeline(SimpleImputer(strategy="median"), StandardScaler
        ("cat", make_pipeline(SimpleImputer(strategy="most_frequent"),
                              OneHotEncoder(handle_unknown="ignore")), categor
    ])

print(X_train.shape, X_test.shape)
print(f"churn rate in training data: {y_train.mean():.3f}")
```

```
(4500, 16) (1500, 16)
churn rate in training data: 0.165
```

2 **Four candidates, one with no brain (1m).** "The DummyClassifier is there on purpose: every comparison needs a floor."

```
from sklearn.dummy import DummyClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, HistGradientBoostingClass

candidates = {
    "baseline": make_pipeline(make_preprocess(), DummyClassifier()),
    "logistic": make_pipeline(make_preprocess(), LogisticRegression(max_iter=1
    "forest": make_pipeline(make_preprocess(), RandomForestClassifier(
        n_estimators=200, min_samples_leaf=5, n_jobs=-1, random_state=42)),
    "boosting": make_pipeline(make_preprocess(), HistGradientBoostingClassifie
}
print(list(candidates))
```

```
['baseline', 'logistic', 'forest', 'boosting']
```

3 **Ten scores each, on the same folds (4m).** "cross_validate returns per-fold scores and fit times. One splitter object with a fixed random_state: the same campsites on the same nights for every tent." Start it, then talk while it runs (about half a minute).

```
from sklearn.model_selection import RepeatedStratifiedKFold, cross_validate

cv = RepeatedStratifiedKFold(n_splits=5, n_repeats=2, random_state=42)

fold_scores = {}
summary = []
for name, model in candidates.items():
    result = cross_validate(model, X_train, y_train, cv=cv, scoring="roc_auc")
    fold_scores[name] = result["test_score"]
    summary.append({"model": name,
                    "mean_auc": result["test_score"].mean(),
                    "std_auc": result["test_score"].std(),
                    "fit_seconds": result["fit_time"].mean()})

summary = pd.DataFrame(summary).set_index("model")
print(summary[["mean_auc", "std_auc"]].round(3))
```

```
          mean_auc  std_auc
model
baseline     0.500    0.000
logistic     0.832    0.017
forest       0.867    0.016
boosting     0.860    0.013
```

Ask: "Forest 0.867, boosting 0.860: is the forest better?" The standard deviations (0.013 to 0.017) are about twice the gap. Do not settle it yet.

4 **Read across a row (2m).** "Read down a column and the scores wander; read across a row and the models move together."

```
folds = pd.DataFrame(fold_scores).drop(columns="baseline")
print(folds.round(3))
```

```
   logistic  forest  boosting
0     0.801   0.841     0.848
1     0.847   0.873     0.863
2     0.850   0.882     0.879
3     0.855   0.894     0.872
4     0.809   0.850     0.846
5     0.830   0.878     0.874
6     0.844   0.871     0.867
7     0.818   0.853     0.847
8     0.826   0.854     0.843
9     0.836   0.879     0.866
```

Fold 0 is hard for everyone, fold 3 easy for everyone. That shared difficulty is most of the spread.

5 **Paired differences (3m).** "Subtract fold by fold: the shared ups and downs cancel."

```
def paired(a, b):
    diff = folds[a] - folds[b]
    print(f"{a} - {b}: mean {diff.mean():+.4f}, std {diff.std():.4f}, "
          f"{a} wins {(diff > 0).sum()} of {len(diff)} folds")

paired("boosting", "logistic")
paired("forest", "boosting")
```

```
boosting - logistic: mean +0.0289, std 0.0113, boosting wins 10 of 10 folds
forest - boosting: mean +0.0070, std 0.0078, forest wins 9 of 10 folds
```

Boosting beats logistic regression in 10 of 10 folds: real. The forest edges default boosting in 9 of 10 by less than 0.01: small but consistent. The std of the differences (0.008) is half the raw std: pairing removed noise.

6 **A validation curve: turn one knob (4m).** "In a pipeline the knob is step name, two underscores, parameter." Show the chart; then read the table.

```
from sklearn.model_selection import StratifiedKFold, ValidationCurveDisplay
from sklearn.tree import DecisionTreeClassifier

tree_pipe = make_pipeline(make_preprocess(), DecisionTreeClassifier(random_sta
cv5 = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
depths = [1, 2, 3, 4, 5, 6, 8, 10, 12, 15]

fig, ax = plt.subplots(figsize=(9, 4), layout="constrained")
vc = ValidationCurveDisplay.from_estimator(
    tree_pipe, X_train, y_train,
    param_name="decisiontreeclassifier__max_depth", param_range=depths,
    cv=cv5, scoring="roc_auc", score_name="ROC AUC", ax=ax,
)
ax.set_xlabel("max_depth")
ax.set_title("Deeper trees memorize the training data and stop generalizing")
plt.show()

curve = pd.DataFrame({"max_depth": depths,
                      "train_auc": vc.train_scores.mean(axis=1),
                      "validation_auc": vc.test_scores.mean(axis=1)})
print(curve.round(3).to_string(index=False))
```

```
 max_depth  train_auc  validation_auc
         1      0.682           0.678
         2      0.791           0.783
         3      0.834           0.823
         4      0.857           0.835
         5      0.880           0.834
         6      0.899           0.829
         8      0.941           0.732
        10      0.977           0.638
        12      0.993           0.659
        15      1.000           0.673
```

Depth 1 to 3 underfits, 4 to 5 is the sweet spot, 6 and beyond overfits (training 1.000, validation falling to 0.638 at depth 10). The legend's "Test" means the validation folds, not X_test.

7 **Learning curves: would more data help? (4m).** "Keep the model fixed and grow the training data." Start it (about a minute), then ask each half of the room to predict one model's shape.

```
from sklearn.model_selection import LearningCurveDisplay

sizes = np.linspace(0.1, 1.0, 5)
fig, axes = plt.subplots(1, 2, figsize=(10, 4), sharey=True, layout="constrain
curves = {}
for ax, name in zip(axes, ["logistic", "boosting"]):
    curves[name] = LearningCurveDisplay.from_estimator(
        candidates[name], X_train, y_train, train_sizes=sizes,
        cv=cv5, scoring="roc_auc", score_name="ROC AUC", ax=ax,
    )
    ax.set_title(name)
plt.show()

for name, lc in curves.items():
    print(name)
    print(pd.DataFrame({"train_rows": lc.train_sizes,
                        "train_auc": lc.train_scores.mean(axis=1),
                        "validation_auc": lc.test_scores.mean(axis=1)}).round(
```

```
logistic
 train_rows  train_auc  validation_auc
        360      0.861           0.790
       1170      0.839           0.818
       1980      0.838           0.827
       2790      0.838           0.830
       3600      0.843           0.832
boosting
 train_rows  train_auc  validation_auc
        360        1.0           0.818
       1170        1.0           0.834
       1980        1.0           0.846
       2790        1.0           0.852
       3600        1.0           0.861
```

Logistic regression has converged near 0.83: more rows of the same data will not help; features will. Boosting's training score is 1.0 and validation still rises: overfitting but learning; regularize it (Lesson 6.2) or add data.

8 **Make the call, then test once (2m).** "Take the forest as the current best default; fit it on all training rows and open the test set, once."

```
from sklearn.metrics import roc_auc_score

final = candidates["forest"].fit(X_train, y_train)
test_auc = roc_auc_score(y_test, final.predict_proba(X_test)[:, 1])

print(f"cross-validated AUC: {summary.loc['forest', 'mean_auc']:.3f} +/- {summ
print(f"test AUC (one look): {test_auc:.3f}")
```

```
cross-validated AUC: 0.867 +/- 0.016
test AUC (one look): 0.861
```

0.861 lands within the cross-validated range: the selection process did not fool you. Report it as is; do not go back and change the model because of it.

9 **Mistake: a scorer name that does not exist (1m).** "Scorers have exact names."

```
try:
    cross_validate(candidates["logistic"], X_train, y_train, cv=cv5, scoring="
except ValueError as err:
    message = str(err)
    print(type(err).__name__)
    print(message[:62], "...")
    print("...", message[-18:])
```

```
InvalidParameterError
The 'scoring' parameter of cross_validate must be a str among  ...
... Got 'auc' instead.
```

`roc_auc`, not `auc`. `sklearn.metrics.get_scorer_names()` lists them all.

10 **Mistake: each model on its own shuffle (2m).** "No error at all. Watch what it does to the forest-versus-boosting comparison."

```
cv_a = RepeatedStratifiedKFold(n_splits=5, n_repeats=2, random_state=0)
cv_b = RepeatedStratifiedKFold(n_splits=5, n_repeats=2, random_state=1)
forest_a = cross_validate(candidates["forest"], X_train, y_train, cv=cv_a, sco
boosting_b = cross_validate(candidates["boosting"], X_train, y_train, cv=cv_b,

mismatched = pd.Series(forest_a - boosting_b)
print(f"mismatched folds: mean {mismatched.mean():+.4f}, std {mismatched.std()
      f"forest wins {(mismatched > 0).sum()} of {len(mismatched)} folds")
```

```
mismatched folds: mean +0.0073, std 0.0133, forest wins 6 of 10 folds
```

The std of the differences jumps from 0.0078 to 0.0133 and the forest wins 6 of 10 instead of 9: the consistent edge disappears into noise.

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Crowning a winner by the third decimal.** "Forest 0.867, boosting 0.860: forest wins." Coach: "What is the spread? What do the paired differences say?" Teach the phrase "within noise" as a respectable conclusion.
- **A splitter per model, or the test set in the loop.** Neither raises anything. Coach with the checklist on the board: one splitter object, created once; `X_test` appears in exactly one cell, at the end.
- **Reading the curves backwards.** Learners say "logistic needs more data" because its score is lower. Coach: "Are the two curves still apart? Is validation still rising?" Flat and close = high bias = features or a more flexible model; a wide gap still closing = variance = regularize or add data.

GUIDED LAB

- **Lab A (20m):** `labs/module-10/l6_1-model-selection/fraud_bakeoff_starter.py`; reference `labs/module-10/l6_1-model-selection/fraud_bakeoff_solution.py`. A fair bake-off on fraud: four candidates on the same 10 folds of January to May, average precision decided in advance, paired differences, June opened once for the winner.
- **Expected:** CV AP boost_gentle 0.521 +/- 0.057, logistic 0.504, boost_default 0.446, baseline 0.009; AUC ranks logistic first (0.855 vs 0.847), so the metric changes the winner; gentle minus default +0.075 (10 of 10), gentle minus logistic +0.017 (std 0.023, 7 of 10); June AP 0.556 vs 0.539, both 36 of 64 frauds in the top 100.
- **Circulate for:** a splitter created inside the loop; sorting the leaderboard by AUC after deciding on AP; opening June for all four candidates and picking the best (the test set choosing). **Debrief (2m):** "Two real findings and one close call. Default boosting overfits 374 frauds: gentle settings win every fold by 0.075. Gentle boosting against logistic regression is close: by the lesson's rule it passes (0.017 against 0.023 / √10 = 0.007), but it wins only 7 of 10 and the June queues are identical. Either choice is defensible if the report says so; logistic regression is simpler to explain."
- **Fast finishers:** `labs/module-10/l6_1-model-selection/debug_bakeoff.py` (two loud bugs, two silent ones), then `labs/module-10/l6_1-model-selection/demand_curves_starter.py`: shuffled folds flatter a forecaster (7.60 against 8.49 for blocks of weeks, test 8.05), and a learning curve on time-respecting folds.

START THE LESSON PROJECT

The churn-model bake-off on average precision. In the room: Steps 1 to 3 (10 minutes).

- Step 1: **done** = `4500 1500`. Step 2: five candidates (adds knn_50 and tree_d5). Step 3 is the slow one: start it and read the brief while it runs.

- **Check:** forest 0.615 +/- 0.031, tree_d5 0.539, logistic 0.524, knn_50 0.518, baseline 0.165 (= the churn rate: a list in random order).
- **Goes home:** Step 4 (forest - tree_d5 +0.075, 10 of 10), Step 5 (learning curve 0.551 / 0.608 / 0.619), Step 6 (test AP 0.606 once, `bakeoff_results.csv`).

CHECKS FOR UNDERSTANDING

Ask these out loud:

- Why is the baseline's average precision 0.165? **Answer:** a random ordering finds churners at the base rate; AP of a constant score equals the positive share.
- Mean gap 0.004, std of differences 0.012, A wins 6 of 10. Conclusion? **Answer:** within noise; choose on speed, simplicity or explainability.
- What does "Test" mean in a ValidationCurveDisplay legend? **Answer:** the validation folds inside cross-validation, not X_test.
- The test score lands 0.006 below the CV mean. What do you do? **Answer:** report it as is. It is within the CV range; changing the model now would need a new test set.

Workbook self-check, items Q7 to Q12.

Answers: **Q7** B) So every candidate is scored on exactly the same folds and the scores can be compared fold by fold; **Q8** `0.01 4`; **Q9** C) Overfitting: the tree memorizes the training customers; **Q10** A) Better features or a more flexible model; **Q11** B) `ValueError`: the pipeline has no max_depth; use `decisiontreeclassifier__max_depth`; **Q12** `3`. Full set with explanations in the Answer Key.

DIFFERENTIATION

Support: hand struggling learners the lesson's `paired` function and the leaderboard loop to adapt; ask them to say aloud what each loop variable holds. Pair someone who finished Lab A with someone who has not for the debrief.

Stretch: the lesson's Exercise 4 (the one-standard-error rule) and the Bonus Challenge (the Nadeau-Bengio corrected t-test). The workbook's worked example is a sharp stretch question: a logistic regression with four shape features beats default boosting in AP in 10 of 10 folds (+0.045).

2:30 Final Project: Choose a Problem and Write the Plan

20 min

TIMING

5m	Present the brief and the three problems (slide); show the rubric rows learners will be scored on
10m	Each learner picks a problem and writes the one-paragraph plan in the workbook (template provided)
5m	Three volunteers read their plan aloud; you ask the two framing questions

THE THREE PROBLEMS (DETAILS, RUBRIC AND ANSWER NOTES IN FINAL-ASSESSMENT.HTML)

PROBLEM	DATA AND THE LEAK-SAFE SPLIT	DECISION AND METRIC	THE BAR TO BEAT
A · Churn	`customers_churn.csv`, stratified 75/25, `random_state=42` (4,500 / 1,500); never `cancellation_reason`	Who gets a retention offer: $15 per offer, $60 per churner reached; AP or ROC AUC to choose, profit to decide	Contact everyone: -$7,620; random-order AP 0.165
B · Weekly demand	`weekly_demand.csv`, train 2024 (5,088 rows) / test 2025 (4,896); folds are blocks of weeks	Units to order per product, store and week; MAE in units	Per-product mean MAE 13.42; overall mean 17.13
C · Fraud	`transactions.csv`, train January to May (41,738; 374 frauds) / test June (8,262; 64)	Which payments to review: $10 per review, $375 per missed fraud; AP to choose, cost to decide	Review nothing: $24,000; review everything: $82,620

RUNNING IT

- Say: "The final project is one supervised model, built end to end the way this course has taught: frame, leak-safe split, baseline, pipelines, fair comparison, tuning, one look at the test set, a decision in business terms, and an honest report. You choose the problem today; Session 11 is a studio for it; you submit one week after Session 11."
- **The plan template** (workbook): the decision the model informs; the target and when the prediction is made; the split (copy from the table, no inventions); the deciding metric, chosen now; the bar to beat; two candidate models; one leakage risk specific to this problem; what result would change your recommendation.

- **Two framing questions** for each plan read aloud: "On the day you make a prediction, which of your columns would you not know yet?" and "Which number decides between two models, and why that one?" Praise plans that name a leak and a metric; gently fix any plan that proposes its own random split for demand or fraud.
- **Balance:** no quota per problem. Steer learners who struggled with Sessions 5 and 6 toward churn (the best-trodden path); suggest fraud to anyone who wants the hardest evaluation and demand to anyone who enjoyed 5.3. Write each learner's choice on your class list: it decides who you visit first in the studio.

CLOSING

EXIT TICKET (3 QUESTIONS, NO NOTES, 4 MINUTES)

1. Why must a stack's meta-model learn from out-of-fold predictions? **Answer:** in-sample predictions are overconfident (the booster scores 0.999 on its own training rows, 0.859 honestly), so the meta-model would over-trust the most flexible member and fail on new data.
2. Two models: CV AUC 0.867 and 0.860; paired differences mean +0.007, std 0.008, the first wins 9 of 10 folds. Real? What else decides? **Answer:** small but consistent (wins nearly every fold; the mean is well above 0.008 / √10 = 0.0025). Then weigh cost, speed and explainability, and confirm once on the test set.
3. Your final project: which problem, which deciding metric, and what does doing nothing score? **Answer:** for example "churn, average precision, 0.165 (contact everyone: -$7,620)"; "demand, MAE, per-product mean 13.42"; "fraud, average precision then cost, review nothing $24,000".

HOMEWORK (ABOUT 2 TO 3 HOURS)

- **Finish both lesson projects:** 5.4 Steps 4 and 5 (paired bootstrap +0.0023, interval -0.0070 to +0.0124; recommendation: keep the single booster); 6.1 Steps 4 to 6 (forest - tree_d5 +0.075; learning curve 0.551 / 0.608 / 0.619; test AP 0.606; `bakeoff_results.csv`).
- **Lesson exercises:** 5.4 Exercises 1 to 3 (weights chosen out-of-fold, a k-NN member, blending by hand); 6.1 Exercises 1 to 3 (the metric changes the winner, a k-NN validation curve, the lapsed flag). Exercise 4 of each is a stretch.
- **Both Quick Quizzes and both journal entries.**
- **Before the Next Lesson (from 6.1):** keep the project notebook (Lesson 6.2 starts from the same split and `make_preprocess`); list three boosting hyperparameter names with `sorted(make_pipeline(make_preprocess(), HistGradientBoostingClassifier()).get_params())`; write down a guess for which settings would close boosting's training-validation gap.
- **Final project, milestone 1:** type the plan into the first Markdown cell of `final_project.ipynb`; run `labs/module-11/l6_2-hyperparameter-tuning/final_project_starter.py` with `PROBLEM` set to

your choice and copy its finished cells (the split, the bar to beat, a first cross-validated model) into the notebook.

- **Optional:** `demand_blend_starter.py` and `demand_curves_starter.py` if you did not reach them.

BRING LIST FOR SESSION 11

- A laptop with the course environment (nothing new to install; Optuna is already in it from Lesson 0.1).
- The `pinecrest-ml` folder with `data/` and `labs/`; Session 11's starters are in `labs/module-11/`.
- The 6.1 project notebook: Lesson 6.2 starts from the same split and the same `make_preprocess` function.
- Your final-project notebook with milestone 1 done: the plan, the split, the bar to beat and a first cross-validated score. The studio starts from there.
- The workbook and the journal.

IF YOU RUN LONG OR SHORT

RUNNING LONG: WHAT TO PROTECT AND WHAT TO CUT FIRST

- **Protect:** the final-project segment; 5.4 Steps 3, 9, 10; 6.1 Steps 4 and 5; Lab A of each lesson through its main TODOs.
- **Cut first (saves 3m):** 6.1 demo Step 9 (the scorer name). **Cut second (saves 4m):** 5.4 Steps 5 and 6 (show the slide). **Cut third (saves 10m):** the 6.1 project start: give the check numbers and send it home.
- **Never cut** the final-project plan to under 10 minutes: shorten the read-aloud to one volunteer instead.

RUNNING SHORT: WHAT TO ADD

- Run `debug_stack.py` for everyone and debrief the silent bug: the in-sample meta-model's weights.
- The 5.4 Bonus Challenge live: five splits, how often does the stack win?
- Start milestone 1 in class: everyone runs `final_project_starter.py` for their problem.

INSTRUCTOR KIT · FACILITATOR GUIDE

Session 11: Hyperparameter Tuning and Final Project Studio

How to teach the last session of Machine Learning Foundations: tune models with grid search, random search, successive halving and Optuna, and report a tuned model honestly (nested cross-validation, the test set once); then run a final-project studio on the clock and a lightning-talk showcase. The final project (churn, weekly demand or fraud) is submitted one week later and scored with the rubric in final-assessment.html.

`1 lesson + studio` `3 hrs session` `Lesson 6.2` `Session 11 of 11`

HOW TO USE THIS GUIDE

- Session 11 has two halves. The first is Lesson 6.2 in the usual rhythm (frame, live demo, guided lab); the lesson's own project goes home. The second is the final-project studio and showcase: learners work on the project they chose in Session 10, you coach.
- Every code block below was run to produce the output printed under it (Python 3.12, scikit-learn 1.9.1, Optuna 5.0, 4 CPU threads). The demo is the lesson's own code, so learners can check every number against `hyperparameter_tuning.html`.
- Keep `final-assessment.html` (kit root) open for the studio: the rubric, the deliverables and the instructor answer notes with reference numbers for all three problems. The studio coaching below gives the short version.
- This is the last session of the Foundations kit: the wrap-up closes the course (submission logistics, what comes next).

> **THE #1 RISK OF SESSION 11: THE DEMO EATS THE STUDIO**
>
> Lesson 6.2 runs several searches; on a slow laptop the grid, the random search, halving and Optuna take minutes each. The studio is the only supervised time learners get on their project, so it must start at 1:20. **Before:** run the whole demo notebook at home and keep it open: on a slow projector machine, show Optuna's and the nested search's saved output instead of rerunning them. **In the room:** learners type along only the cells marked in the timing table; they rerun the rest at home. **Studio:** a learner without milestone 1 does it first (`final_project_starter.py`, 15 minutes) before anything else.

BEFORE THE SESSION

- ☐ **Run the kit check:** `python3 labs/check_labs.py module-11` : 6 PASS lines (about two minutes).
- ☐ **Run the 6.2 demo once** in a notebook `lesson_6_2` on the projector machine and keep its outputs; note how long Optuna takes there.
- ☐ **Read the instructor answer notes** in `final-assessment.html` for all three problems (the traps and the reference numbers); print the rubric page and the scoring sheet.
- ☐ **Class list:** each learner's chosen problem from Session 10 and whether they emailed milestone 1. Plan your studio rounds: learners without milestone 1 first, then fraud (the hardest evaluation), then demand, then churn.
- ☐ **Board:** the milestone ladder (1 split and bar to beat · 2 fair leaderboard · 3 tuned on training rows · 4 test once and a decision · 5 one-page report), a column per learner for a sticker, and the showcase card (decision · bar to beat · best CV ± spread · one trap avoided · what is left).
- ☐ **Print:** peer feedback cards (two lines: one strength, one question), one per learner per talk they watch, or use sticky notes.

KNOWN ENVIRONMENT TRAPS

SYMPTOM	CAUSE	FIX
`ValueError: Invalid parameter 'C' for estimator Pipeline(...)`	Hyperparameter name without the step	`model__C` ; `sorted(pipe.get_params())` lists the names.
`TypeError: Parameter grid for parameter 'model__C' needs to be a list ...`	A single value, or a scipy distribution, in GridSearchCV	`[1.0]` , or use RandomizedSearchCV for distributions.
`ValueError: For multi-metric scoring, the parameter refit must be set ...`	Two scorers and no decider	`refit="average_precision"` (the deciding metric).
`ModuleNotFoundError: No module named 'optuna'`	The kernel is not the course environment	The Session 1 routine: activate `.venv` , start Jupyter from the project folder.
`ImportError: HalvingRandomSearchCV ... experimental`	The enabling import is missing	`from sklearn.experimental import enable_halving_search_cv` first.
Optuna's results differ from the page	Trials run in parallel (`n_jobs` in `optimize`), or no sampler seed	`TPESampler(seed=42)` and trials one at a time; parallelize inside cross-validation.

SYMPTOM	CAUSE	FIX
The fraud search takes minutes on a laptop	42,000 rows × settings × folds	The lab uses 8 settings × 3 folds; final projects: start with 10 to 15 settings, 3 folds.

SESSION 11 AGENDA (3 HOURS)

CLOCK	MINUTES	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up	Five retrieval questions on Lessons 5.4 and 6.1; milestone-1 check (every learner has a split, a bar to beat and a first CV score).
0:10	10	**6.2** Frame and mental model	Dialing in a camp stove: search space, budget, objective; four strategies that spend the fuel differently.
0:20	30	**6.2** Live demo	The default to beat, a grid for C, a 3 × 3 grid for boosting, random search, successive halving, Optuna, nested CV, the test set once, three error messages.
0:50	20	**6.2** Guided lab	`labs/module-11/l6_2-hyperparameter-tuning/tune_fraud_starter.py`: tune the fraud booster on January to May and report it honestly.
1:10	10	Break	Learners write their milestone status on the board (1 split and bar, 2 leaderboard, 3 tuning, 4 test once, 5 report).
1:20	55	Final project studio	Milestones 2 and 3 on the clock: a fair leaderboard on shared folds, then a budgeted search on the training rows (and a business threshold for churn and fraud). You coach by problem.
2:15	35	Showcase	Two-minute lightning talks: the decision, the bar to beat, the best CV score with its spread, one trap avoided, what is left. Peer cards: one strength, one question.
2:50	10	Wrap-up	Exit ticket, submission logistics (one week), closing the course.
Total	**180**		Ends at 3:00.

In the room. Lesson 6.2 (105 minutes solo) gets its mental model, the live demo and one guided lab (60 minutes in all); its project and exercises go home. The studio (55 minutes) and the showcase (35) belong to the final project.

Goes home (about 3 hours, plus the final project). The 6.2 project (tuning the retention model, `best_params.json`), Exercises 1 to 3, the Quick Quiz and the journal; then the final project's milestones 4 and 5 (the test set once, the decision, the one-page report), submitted one week after this session.

Protect, in this order: the studio start at 1:20; demo Steps 3, 7 and 8 (the overfitting cure, the nested estimate, the test once); the showcase (even if talks shrink to one minute). **Cut first:** demo Step 5 (halving: show the slide), then Step 6 (Optuna: show the saved output), then the lab's TODO 5 (saving JSON goes home).

WARM-UP

0:00 Warm-up: Ensembles and Model Selection — 10 min

FIVE RETRIEVAL QUESTIONS (ANSWERS AFTER EACH)

1. Five 70%-accurate classifiers always make exactly the same predictions. How accurate is their majority vote? **Answer:** 70%: identical voters are one voter repeated.
2. Where must a stacking meta-model's training features come from? **Answer:** Out-of-fold predictions on the training rows (`cross_val_predict`, or `StackingClassifier`'s `cv`).
3. Why pass one splitter object with a fixed random_state to every candidate? **Answer:** So all are scored on the same folds and can be compared fold by fold (paired differences).
4. A learning curve's training and validation scores meet at 0.83 and stay flat. Next step? **Answer:** Better features or a more flexible model (high bias); more rows of the same data will not help.
5. Boosting's learning curve: training 1.0, validation still rising to 0.861. What would close the gap? **Answer:** Regularization (smaller trees, larger leaves, a smaller learning rate) or more data. Today's lesson tests it: max_leaf_nodes 7 lifts CV AUC to 0.8795.

MILESTONE-1 CHECK (3 MINUTES)

- Hands up per problem: churn, demand, fraud. Then: "Who has a split, the bar to beat and a first cross-validated score?" Note the learners without it: they start the studio with `final_project_starter.py`.
- Quick sanity numbers to listen for: churn 4,500 / 1,500 and random-order AP 0.165; demand 5,088 / 4,896 and per-product mean MAE 13.42; fraud 41,738 / 8,262 with 374 / 64 frauds and review-nothing $24,000. A different split size means a different split: fix it first.

LESSON PLANS

6.2 Hyperparameter Tuning

60 min

OBJECTIVES (FROM THE LESSON PAGE)

- Tune a whole pipeline with `GridSearchCV` and `RandomizedSearchCV`, naming its hyperparameters as `step__param`, and read `cv_results_`
- Choose sensible search ranges (log scales, distributions from `scipy.stats`) and explain why random search usually beats a grid of the same size
- Speed up a search with successive halving (`HalvingRandomSearchCV`)
- Run a seeded Optuna study that learns where to search next
- Explain why `best_score_` is optimistic, estimate a tuned model honestly with nested cross-validation, and use the test set once

TIMING IN THE ROOM

10m	Frame: parameters versus hyperparameters; the camp stove; space, budget, objective
30m	Live demo: default, grid for C, 3 × 3 grid, random search, halving, Optuna, nested CV, test once, three errors (learners type Steps 1 to 4 and 8)
20m	Guided lab: `tune_fraud_starter.py`

MENTAL MODEL TO SAY

"A camp stove has two knobs: fuel flow and air. You cannot calculate the best setting from the manual; you try a setting, boil a pot, time it, and adjust. Every trial costs fuel, so a smart camper tries a few spread-out settings, notices which knob matters, and spends the remaining fuel near the best results."

Map it on the board: **a trial = one full cross-validation run; the fuel = computing time; the strategy = which settings get tried.** Grid tries every combination; random samples; halving runs a tournament on growing data; Optuna learns from past trials. Then the honesty rule: "The best-looking setting looks better than it is: the best of many noisy scores. Report a nested estimate or the test score, once."

LIVE DEMO (YOU TYPE, THEY FOLLOW)

Notebook `lesson_6_2`, same split and `make_preprocess` as 6.1 (learners can copy them from their 6.1 project notebook).

1. **The number to beat (3m).** "Lesson 6.1 left a puzzle: default boosting lost narrowly to a forest and its learning curve showed a perfect training score. Its settings control exactly that. First: what does the default score, and what are its settings called?" Short step names, `prep` and `model`, keep the names short.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_v
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline, make_pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.ensemble import HistGradientBoostingClassifier

customers = pd.read_csv("data/customers_churn.csv")
numeric = ["tenure_months", "orders_12m", "spend_12m", "avg_order_value", "day
           "returns_12m", "return_rate", "support_tickets_12m", "discount_shar
           "email_opens_90d"]
categorical = ["home_store", "region", "age_group", "loyalty_tier", "preferred
X_train, X_test, y_train, y_test = train_test_split(
    customers[numeric + categorical], customers["churned"],
    test_size=0.25, stratify=customers["churned"], random_state=42)

def make_preprocess():
    return ColumnTransformer([
        ("num", make_pipeline(SimpleImputer(strategy="median"), StandardScaler
        ("cat", make_pipeline(SimpleImputer(strategy="most_frequent"),
                              OneHotEncoder(handle_unknown="ignore")), categor
    ])

hgb = Pipeline([("prep", make_preprocess()),
                ("model", HistGradientBoostingClassifier(random_state=42))])
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

default_auc = cross_val_score(hgb, X_train, y_train, cv=cv, scoring="roc_auc")
print(f"default boosting, 5-fold AUC: {default_auc.mean():.4f} +/- {default_au
print([name for name in hgb.get_params() if name.startswith("model__")][:8])
```

```
default boosting, 5-fold AUC: 0.8615 +/- 0.0132
['model__categorical_features', 'model__class_weight', 'model__early_stopping
```

Write 0.8615 on the board: the number to beat. `model__learning_rate` is step name, two underscores, parameter.

2 **A grid for one knob (3m).** "Logistic regression has one knob that matters, C. Regularization works across factors of ten, so the values are log-spaced."

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV

logit = Pipeline([("prep", make_preprocess()), ("model", LogisticRegression(ma
logit_search = GridSearchCV(logit, {"model__C": [0.001, 0.01, 0.1, 1, 10, 100]
                            cv=cv, scoring="roc_auc")
logit_search.fit(X_train, y_train)

results = pd.DataFrame(logit_search.cv_results_)
print(results[["param_model__C", "mean_test_score", "std_test_score", "rank_te
```

```
 param_model__C  mean_test_score  std_test_score  rank_test_score
          0.001           0.8267          0.0218                6
          0.010           0.8336          0.0240                1
          0.100           0.8330          0.0235                2
          1.000           0.8324          0.0229                3
         10.000           0.8320          0.0228                4
        100.000           0.8320          0.0228                5
```

From 0.01 upward the scores differ by less than their standard deviations: logistic regression is not overfitting (its flat learning curve said so), and no C lifts it to boosting's level.

3 **A 3 × 3 grid for boosting (4m).** "Now a model where settings matter." Start it; while it runs, write 9 × 5 = 45 on the board and add a third knob with five values: 225.

```
grid = {"model__learning_rate": [0.05, 0.1, 0.2],
        "model__max_leaf_nodes": [7, 15, 31]}
grid_search = GridSearchCV(hgb, grid, cv=cv, scoring="roc_auc", n_jobs=-1)
grid_search.fit(X_train, y_train)

n_settings = len(grid_search.cv_results_["params"])
print(f"{n_settings} settings x {cv.get_n_splits()} folds = {n_settings * cv.g
print("best:", grid_search.best_params_)
print(f"best CV AUC: {grid_search.best_score_:.4f}")
```

```
9 settings x 5 folds = 45 fits, plus 1 refit
best: {'model__learning_rate': 0.05, 'model__max_leaf_nodes': 7}
best CV AUC: 0.8795
```

Small trees win (7 leaves instead of 31): the overfitting cure, and 0.8795 beats the default by well over a standard error.

4 **Random search (4m).** "Same budget idea, sampled from distributions: loguniform for rates, randint for sizes. And two more knobs a grid could not afford."

```
from scipy.stats import loguniform, randint
from sklearn.model_selection import RandomizedSearchCV

space = {"model__learning_rate": loguniform(0.01, 0.3),
         "model__max_leaf_nodes": randint(4, 64),
         "model__min_samples_leaf": randint(10, 200),
         "model__l2_regularization": loguniform(1e-3, 10)}
random_search = RandomizedSearchCV(hgb, space, n_iter=12, cv=cv, scoring="roc_
                                   random_state=42, n_jobs=-1)
random_search.fit(X_train, y_train)

print(f"best CV AUC: {random_search.best_score_:.4f}")
print({k: round(float(v), 4) if isinstance(v, float) else int(v)
       for k, v in random_search.best_params_.items()})
```

```
best CV AUC: 0.8764
{'model__l2_regularization': 0.0292, 'model__learning_rate': 0.0472, 'model__r
```

50 leaves but at least 199 customers per leaf and a slow rate: a different route to the same cure. Several settings are about equally good; that is normal.

5 **Successive halving (4m).** "A tournament: 27 candidates on 500 rows, the best third go through with three times the rows." Remind them of the experimental import line.

```
from sklearn.experimental import enable_halving_search_cv  # noqa: F401  (swit
from sklearn.model_selection import HalvingRandomSearchCV

halving = HalvingRandomSearchCV(hgb, space, n_candidates=27, factor=3, min_res
                                cv=cv, scoring="roc_auc", random_state=42, n_j
halving.fit(X_train, y_train)

rounds = pd.DataFrame(halving.cv_results_).groupby("iter").agg(
    candidates=("n_resources", "size"), rows_per_candidate=("n_resources", "fi
    best_auc=("mean_test_score", "max"))
print(rounds.round(4))
print(f"best CV AUC: {halving.best_score_:.4f}")
```

```
      candidates  rows_per_candidate  best_auc
iter
0             27                 500    0.8192
1              9                1500    0.8555
2              3                4500    0.8729
best CV AUC: 0.8729
```

40,500 rows fitted in total, the cost of 9 full candidates, yet 27 screened. The catch: a slow starter can be knocked out early.

6 **Optuna learns where to look (5m).** "Grid and random search never learn from their own results. Optuna's TPE does." On a slow machine, run it before the session and show the saved output; it is 100 fits.

```
import optuna

optuna.logging.set_verbosity(optuna.logging.WARNING)   # hide one log line per

def objective(trial):
    params = {
        "model__learning_rate": trial.suggest_float("learning_rate", 0.01, 0.3
        "model__max_leaf_nodes": trial.suggest_int("max_leaf_nodes", 4, 64),
        "model__min_samples_leaf": trial.suggest_int("min_samples_leaf", 10, 2
        "model__l2_regularization": trial.suggest_float("l2_regularization", 1
    }
    model = hgb.set_params(**params)
    return cross_val_score(model, X_train, y_train, cv=cv, scoring="roc_auc",

study = optuna.create_study(direction="maximize", sampler=optuna.samplers.TPES
study.optimize(objective, n_trials=20)

print(f"best CV AUC: {study.best_value:.4f} (trial number {study.best_trial.nu
print({k: round(v, 4) for k, v in study.best_params.items()})
```

```
best CV AUC: 0.8793 (trial number 12, counting from 0)
{'learning_rate': 0.0609, 'max_leaf_nodes': 5, 'min_samples_leaf': 183, 'l2_re
```

Level with the grid. The differences between searches are a few thousandths, far smaller than the fold spread; what they agree on matters more: every search beats the default.

7 **Nested cross-validation (3m).** "best_score_ is the best of many noisy numbers. To estimate the whole procedure honestly, cross-validate the search itself."

```
inner = RandomizedSearchCV(hgb, space, n_iter=6, scoring="roc_auc", random_sta
                           cv=StratifiedKFold(n_splits=3, shuffle=True, random
outer_cv = StratifiedKFold(n_splits=3, shuffle=True, random_state=1)
nested = cross_val_score(inner, X_train, y_train, cv=outer_cv, scoring="roc_au
print("outer-fold AUCs:", nested.round(4))
print(f"nested estimate of the whole tuning procedure: {nested.mean():.4f}")
```

```
outer-fold AUCs: [0.8609 0.8528 0.8907]
nested estimate of the whole tuning procedure: 0.8681
```

0.868: about 0.01 below the best scores, still above the default's 0.8615. The gain is real, a little smaller than best_score_ claims.

8 **The test set, once (2m).** "Build the model with Optuna's settings, fit on all training rows, and compare with the default for reference."

```
from sklearn.base import clone
from sklearn.metrics import roc_auc_score

tuned = clone(hgb).set_params(**{f"model__{k}": v for k, v in study.best_param
default = clone(hgb).set_params(model__learning_rate=0.1, model__max_leaf_node
                                model__min_samples_leaf=20, model__l2_regulari
for name, model in [("default boosting", default), ("tuned boosting", tuned)]:
    model.fit(X_train, y_train)
    print(f"{name:17s} test AUC {roc_auc_score(y_test, model.predict_proba(X_t
```

```
default boosting  test AUC 0.860
tuned boosting    test AUC 0.874
```

0.874 against 0.860: the gain survives unseen customers. `clone` matters because the Optuna objective changed `hgb` in place.

9 **Three error messages (2m).** "Read the last line each time." Run the three cells one after another.

```
GridSearchCV(logit, {"C": [0.1, 1, 10]}, cv=cv).fit(X_train, y_train)

GridSearchCV(logit, {"model__C": 1.0}, cv=cv).fit(X_train, y_train)

GridSearchCV(logit, {"model__C": [0.1, 1]}, cv=cv,
             scoring=["roc_auc", "average_precision"]).fit(X_train, y_train)
```

```
ValueError: Invalid parameter 'C' for estimator Pipeline(steps=[('prep', ... \
TypeError: Parameter grid for parameter 'model__C' needs to be a list or a nur
ValueError: For multi-metric scoring, the parameter refit must be set to a sc
```

No step name → `model__C`. A single value → `[1.0]` (and distributions belong to RandomizedSearchCV). Two metrics → `refit="average_precision"`.

THREE MISTAKES TO EXPECT, AND HOW TO COACH EACH

- **Tuning on the test set.** A loop that scores settings on `X_test`, or early stopping on it. Coach: "Which rows chose this setting? Then which rows are left to grade it?" Every search runs on the training rows only.
- **Linear grids for log-scale knobs.** `[1, 2, 3, 4, 5]` for C: the scores agree to three decimals and nothing was explored. Coach: "From the model's point of view, are 1 and 5 far apart?" Use `np.logspace` or `loguniform`.
- **Promising best_score_.** "The model scores 0.8795." Coach with the three numbers on the slide: best_score_ 0.8795, nested 0.868, test 0.874. "Which one did the search choose with?"

GUIDED LAB

- **Lab A (20m):** `labs/module-11/l6_2-hyperparameter-tuning/tune_fraud_starter.py`; reference `labs/module-11/l6_2-hyperparameter-tuning/tune_fraud_solution.py`. The lesson's moves on fraud: a written space, a budgeted random search on January to May, the leaderboard, June once for default and tuned, the settings saved as JSON. It takes about 40 seconds to run when finished: start it, then read the leaderboard code.
- **Expected:** default 3-fold AP 0.443 +/- 0.032; 24 fits; best CV AP 0.515 (learning_rate 0.0186, max_leaf_nodes 24, min_samples_leaf 170); 5 of 8 settings within one std of the best, all with small rates and large leaves, while the one rate of 0.254 collapses to 0.262 +/- 0.169; June AP 0.477 default vs 0.544 tuned, 32 vs 38 frauds in the top 100.
- **Circulate for:** a `model__` prefix on a bare booster (no pipeline here, so no prefix); scoring the search on June; reading the 0.254 row as "bad luck" (it is a rate so large that some folds blow up: the std says so). **Debrief (2m):** "Tuning helps a lot here because the default overfits 374 frauds. Same cure as churn: smaller steps and bigger leaves. And notice how many settings are about as good as the winner: what they share matters more than the exact winner."
- **Fast finishers:** `labs/module-11/l6_2-hyperparameter-tuning/winners_curse_starter.py`: tune a tree on shuffled labels and watch best_score_ climb to 0.537 on pure noise while the nested estimate stays at 0.501.

CHECKS FOR UNDERSTANDING

Ask these out loud:

- A grid of 4 × 5 × 3 values with cv=5: how many fits? **Answer:** 300, plus one refit.
- Why loguniform for the learning rate? **Answer:** it matters across orders of magnitude; loguniform samples as many values between 0.01 and 0.03 as between 0.1 and 0.3.
- What does nested cross-validation estimate? **Answer:** how well the whole procedure (search, then refit the winner) does on unseen data.
- Tuning did not beat the default on your project. Is that a failure? **Answer:** no: an honest negative result. The workbook's demand example: the best of 8 settings is worse than the default (CV 8.73 vs 8.49; test

8.20 vs 8.05). Report the default.

Workbook self-check, items Q1 to Q12 (project skills; learners do them at home).

Answers: **Q1** C) Which customers should get the $15 retention offer next quarter, judged by profit on held-out customers, against offering to everyone (-$7,620)?; **Q2** A) The per-product 2024 mean: MAE 13.42; **Q3** D) On the planning date, last week's sales are unknown for almost every 2025 week: it is a framing leak; **Q4** `60`; **Q5** B) 0.614, the one-time test score (or a nested CV estimate), with best_score_ described only as what chose the settings; **Q6** C) It tries 9 different values of the important hyperparameter instead of 3; **Q7** A) Train January to May, test June, never shuffling months together; **Q8** `[1.e-03 1.e-02 1.e-01 1.e+00 1.e+01]`; **Q9** D) 0.25; **Q10** B) Tuned boosting: test ROC AUC 0.874 against 0.860 for the default, on the same 1,500 test customers, after a search on the training rows only; the nested CV estimate of the procedure is 0.868; **Q11** C) $5,265: the threshold was chosen without the test set, so its test profit is an honest estimate; **Q12** `415`. Full set with explanations in the Answer Key.

◆ DIFFERENTIATION

Support: learners who are behind type only Steps 1, 3 and 8 and read the rest; in the lab, give them the space dict from the demo (drop the `model__` prefix) so they can focus on reading the leaderboard.

Stretch: the lesson's Exercise 4 (halving over boosting rounds) and Bonus Challenge (tune the forest with Optuna and rerun 6.1's paired comparison); `winners_curse_starter.py`.

1:20 Final Project Studio

55 min

TIMING

5m	Milestone board: every learner puts a sticker on their current milestone; learners without milestone 1 start `final_project_starter.py` now
20m	Milestone 2: a fair leaderboard on the training rows (baseline + at least two real candidates, one splitter, the deciding metric, mean ± std, paired check of the top two)
25m	Milestone 3: a budgeted search on the training rows (10 to 15 settings, log scales); churn and fraud also tune the decision threshold for the business cost with TunedThresholdClassifierCV
5m	Each learner writes their three numbers so far on the board: bar to beat, best CV ± std, what is next

HOW TO RUN THE STUDIO

- Say: "The test set stays locked today. Everything you choose, you choose on the training rows. You open the test set once, at home, after the choice, and you report what it says."
- Circulate by problem, with the coaching notes below. Spend at most four minutes per learner per round; ask questions rather than fixing code: "Which rows chose this?", "What does doing nothing score?", "Is that gap bigger than the fold spread?"
- Pair learners on the same problem for a five-minute check after Milestone 2: compare split sizes and baselines (they must match exactly) and each other's leaderboard.
- A learner who finishes Milestone 3 early: write the report skeleton (Question, Answer, Evidence, Caveats, Method) and the README, or try one feature idea from Lesson 2.3 on the same folds.

COACHING NOTES BY PROBLEM (REFERENCE NUMBERS FROM THE INSTRUCTOR'S OWN RUN; FULL NOTES IN FINAL-ASSESSMENT.HTML)

PROBLEM	WHERE LEARNERS SHOULD LAND	TRAPS TO LOOK FOR IN THE STUDIO
A · Churn	Logistic CV AP 0.528 ± 0.042, default boosting 0.608, forest 0.619; tuned boosting best CV AP 0.636 (test 0.614 once); logistic with four shape features CV AP 0.643. Threshold tuned on training ≈ 0.222, test profit $5,235 with 395 offers (everyone: -$7,620).	`cancellation_reason` in X (test AUC 0.939: too good to be true); accuracy as the metric (doing nothing scores 0.835); a threshold picked on the test set (best possible $5,535 at 0.27: optimistic); reporting best_score_.

PROBLEM	WHERE LEARNERS SHOULD LAND	TRAPS TO LOOK FOR IN THE STUDIO
B · Weekly demand	Per-product mean MAE 13.42; Poisson boosting default 8.05 on 2025; CV on blocks of 2024 weeks 8.49 ± 1.34. A 15-setting search is NOT better (CV 8.57, test 8.22): an honest negative.	A shuffled split (MAE 7.36: flattering) or shuffled CV folds (7.6); last week's sales as a feature for a start-of-year forecast; forgetting that 2024 has 53 week rows (5,088, not 4,992); ignoring that the forecast is about 6% low overall (2025 grew: 121,456 forecast vs 128,970 sold).
C · Fraud	Logistic CV AP 0.506 ± 0.054; default boosting 0.443 (overfits); gentle or tuned boosting about 0.528 to 0.529. Threshold tuned on January to May: June cost $9,900 (tuned booster) or $10,285 (logistic), against $24,000 for reviewing nothing.	A stratified random split over all months; accuracy (doing nothing scores 0.9923 in June); a threshold tuned on June (tuned booster $9,395: optimistic); claiming one model beats another by a few hundred dollars (one fraud is $375; June has only 64).

SHOWCASE

2:15 Showcase: Lightning Talks

35 min

RUNNING THE SHOWCASE

- **Format:** two minutes each, from the notebook or one slide, then 30 seconds for one question. The card on the board: the decision · the bar to beat · the best CV score with its spread · one trap avoided · what is left. Keep strict time: a phone timer on the projector.
- **Groups larger than 12:** split into two rooms or tables with a peer timekeeper, or run a gallery walk (each learner leaves the notebook open on the card's five lines; others rotate and leave feedback cards).
- **Peer cards:** each listener writes one strength and one question for two talks; the speaker keeps them for the report's Caveats and Next questions.
- **Your role:** one sentence of praise per talk, aimed at a habit ("you showed the spread", "you named the leak"). Write down one strength per learner for the written feedback. The showcase is practice and is not scored.
- **Listen for:** a test score already opened (ask whether anything was changed after it: if yes, the report must say so); "accuracy 99%" for fraud; a demand forecaster scored on shuffled folds. Raise these privately after the session, not in front of the room.

CLOSING

EXIT TICKET (3 QUESTIONS, 4 MINUTES)

1. Why is a search's best_score_ not the number to promise? **Answer:** it is the best of many noisy scores (the winner's curse), so it is biased upward; promise a nested estimate or the test score from one look.
2. Write the three numbers your report will lead with. **Answer (example):** churn: random-order AP 0.165 → CV AP 0.636 ± 0.04 → test AP once; plus the profit at a threshold chosen on training rows.
3. What is the one thing left before you submit? **Answer:** any honest answer; sort the notes to plan the follow-up emails (learners with "test set" or "report" left are on track; "leaderboard" or "split" left need a check-in this week).

HOMEWORK (ABOUT 2 TO 3 HOURS, PLUS THE FINAL PROJECT)

- **Lesson 6.2 project** (tune the retention model, Steps 1 to 6: default CV AP 0.608, best 0.636, test 0.600 → 0.614, top 50: 40 → 43 churners, `best_params.json`).
- **Lesson 6.2 exercises 1 to 3** (which knob mattered, a k-NN grid, two metrics one decider); Exercise 4 is a stretch. The Quick Quiz and the journal entry.
- **Final project:** milestones 4 and 5: open the test set once, make the decision in business terms, write the one-page `report.md` and the README; submit the folder within one week (details and rubric: `final-assessment.html`).
- **Optional:** `winners_curse_starter.py` if you did not reach it.

CLOSING THE COURSE (5 MINUTES OF THE WRAP-UP)

- Thank the group and name, out loud, the habits they now have: a baseline first, leak-safe splits, pipelines, shared folds, the test set once, decisions in dollars, honest negatives.
- **Submission logistics:** the project folder (notebook that reruns, report.md, README.md, results files) within one week, by the channel you use; written feedback within two weeks, with one strength per rubric row.
- **What next:** the Advanced Machine Learning kit starts from exactly here (Lesson 7.1 opens up a gradient-boosting churn model and asks what it learned). Keep the final-project notebook and `best_params.json`: the Advanced kit's churn case study (Lesson 13.1) tunes the same kind of booster again, so their own numbers make a useful comparison, and its capstone is the next step up from this project.
- **Bring list for the next course (if they continue):** the same laptop and environment, `pinecrest-ml` with `data/`, the final-project notebook, and the image data and pretrained weights (about 175 MB) downloaded at home before its Session 5 (`check_environment.py --download`, as its setup guide says).
- Ask for course feedback (two questions: what helped most, what to change).

IF YOU RUN LONG OR SHORT

RUNNING LONG

- **Protect:** the studio start at 1:20 and at least 20 minutes of showcase.
- **Cut first:** demo Step 5 (halving), then Step 6 (show Optuna's saved output), then the lab's TODO 3 to 5 (they go home with the expected output).
- **Showcase under time pressure:** one-minute talks with the five-line card, or a gallery walk.

RUNNING SHORT

- Run `winners_curse_starter.py` for everyone: the most memorable picture of why best_score_ is optimistic.
- Extend the studio: Milestone 4 rehearsal (write the report's first three sentences: answer, evidence, caveat).
